408 考纲知识点手册

计算机网络

计算机学科专业基础综合　分册

按 2027 年全国硕士研究生招生考试大纲编排

对应大纲	第四部分计算机网络
本册章数	6 章
知识点条目	覆盖大纲本部分全部条目
主要素材	语雀「计算机考研 408 笔记」知识库
	王道 2027 系列教材与课件
	工作区历年真题与强化笔记

本分册为依据 2027 年考试大纲整理的「计算机网络」知识点汇编，供复习检索使用。

使用说明

本手册按《2027 年全国硕士研究生招生考试计算机学科专业基础考试大纲》的条目顺序编排，覆盖大纲「考查内容」中数据结构、计算机组成原理、操作系统、计算机网络四科的全部条目。

本册范围：本册是《408 考纲知识点手册》的「计算机网络」分册，对应 2027 年考试大纲「计算机网络」部分的全部考查内容，共 6 章。其余三科另见对应分册；四科合订本见《408 考纲知识点手册》。

一、编排方式

全书分四部分，共 24 章。四部分与大纲的四科一一对应，章与该科下的一级条目一一对应，章内小节按大纲的二级、三级条目顺序展开。每一章的开头用引用块照抄本章对应的大纲原文条目，便于对照检查覆盖情况；每一章末尾固定给出「本章小结表」与「易错清单」两张速查内容。

二、阅读约定

本手册用三种提示框把不同性质的内容区分开：

- 核心结论——必须记住的定义、公式、判定条件，考前反复看的就是这些。
- 易错点——统考中反复出现的陷阱，以及容易与相邻概念混淆的地方。
- 真题考法——该知识点在统考中以什么形式出现；能确认年份与题号的，如实标注。无法确认的不标，不编造。

正文中的公式一律用数学排版；算法与代码用等宽框给出；对比性内容尽量做成表格。凡涉及计算的知识点，都给出公式、符号含义、适用范围，并配一组可以自己动手复核的数字算例。

三、素材来源

本手册的素材来自三处：

- 语雀知识库「计算机考研 408 笔记」（月暗）导出的 182 篇笔记 Markdown 全文；
- 工作区中的王道 2027 系列教材、课件、强化笔记与历年真题汇编；
- 工作区 408 计算机考研目录下的各类复习笔记与答题本。

素材在整理过程中做了三件事：一是按大纲重新组织顺序；二是把原笔记中的口语化叙述、错别字与图片占位改写成规范的知识点条目，公式按上下文重新推导书写；三是逐条核对，凡素材之间口径冲突的，以大纲用词与统考真题为准，并在正文中说明差异。

四、使用建议

第一轮复习时，按章通读，重点看「核心结论」与例题，把「易错点」留到做题后再回看；第二轮复习时只看每章的「本章小结表」，遇到想不起来的再翻回正文；考前冲刺时只看「核心结论」与「易错清单」。

说明：本手册为复习资料汇编，用于知识点检索与自查，不替代教材与真题训练。

目录

使用说明 **i**

第一章　计算机网络概述 **1**

本章考纲要求 1

一、 计算机网络基本概念 1

（一） 定义、组成与功能 1

（二） 分类 3

（三） 主要性能指标 6

二、 计算机网络体系结构 11

（一） 分层结构 11

（二） 协议、接口、服务等概念 12

（三） ISO/OSI 参考模型与 TCP/IP 参考模型 14

本章小结表 17

易错清单 19

第二章　物理层 **21**

本章考纲要求 21

一、 信道、信号、带宽、码元、波特、速率、信源与信宿 22

（一） 通信系统的基本模型 22

（二） 通信的三种方式 22

（三） 信号：模拟信号与数字信号 23

（四） 基带信号与带通信号，基带传输与频带传输 23

（五） 带宽的两种含义 24

（六） 码元、波特与速率 24

（七） 速率与传播速度的区别 25

二、 奈奎斯特定理与香农定理 26

（一） 为什么码元发得太快就会出错 26

（二） 奈奎斯特定理（奈氏准则） 26

（三） 香农定理 27

（四） 两个定理的综合对比 29

三、 编码与调制 30

（一） 数字数据编码为数字信号 30

（二） 数字数据调制为模拟信号 33

（三） 模拟数据编码为数字信号：PCM 34

（四） 模拟数据调制为模拟信号与频分复用 . . . 35
四、 电路交换、报文交换与分组交换 . . . 35
（一） 三种交换方式的工作过程 . . . 35
（二） 三种交换方式的对比 . . . 36
（三） 分组交换的时延计算 . . . 37
五、 数据报与虚电路 . . . 40
（一） 数据报方式 . . . 40
（二） 虚电路方式 . . . 40
（三） 数据报与虚电路的对比 . . . 41
六、 传输介质 . . . 41
（一） 双绞线 . . . 42
（二） 同轴电缆 . . . 42
（三） 光纤 . . . 43
（四） 无线传输介质 . . . 43
（五） 物理层接口的特性 . . . 44
七、 物理层设备 . . . 44
（一） 中继器 . . . 45
（二） 集线器 . . . 45
本章小结表 . . . 46
易错清单 . . . 48

第三章 数据链路层 **50**
本章考纲要求 . . . 50
一、 数据链路层的功能 . . . 51
（一） 链路、数据链路与帧 . . . 51
（二） 数据链路层的功能 . . . 51
（三） 封装成帧、透明传输与差错检测 . . . 51
二、 组帧 . . . 52
（一） 字符计数法 . . . 52
（二） 字符填充法 . . . 52
（三） 零比特填充法 . . . 52
（四） 违规编码法 . . . 53
三、 差错控制 . . . 53
（一） 差错与误码率 . . . 53
（二） 检错编码 . . . 54
（三） 纠错编码：海明码 . . . 57
四、 流量控制与可靠传输机制 . . . 60
（一） 流量控制、可靠传输与滑动窗口机制 . . . 60

（二） 停止-等待协议 . . . 61
（三） 后退 N 帧协议（GBN） . . . 63
（四） 选择重传协议（SR） . . . 64
（五） 三种协议的对比与信道利用率 . . . 65
五、 介质访问控制 . . . 67
（一） 信道划分介质访问控制 . . . 68
（二） 随机访问介质访问控制 . . . 70
（三） 轮询访问：令牌传递协议 . . . 77
六、 局域网 . . . 78
（一） 局域网的基本概念与体系结构 . . . 78
（二） 以太网与 IEEE 802.3 . . . 79
（三） IEEE 802.11 无线局域网 . . . 81
（四） VLAN 基本概念与基本原理 . . . 83
七、 广域网 . . . 85
（一） 广域网的基本概念 . . . 85
（二） PPP 协议 . . . 86
八、 数据链路层设备 . . . 89
（一） 网桥与以太网交换机 . . . 89
（二） 自学习算法与转发表的建立 . . . 89
（三） 转发/过滤/泛洪与冲突域、广播域的划分 . . . 90
（四） 直通式与存储转发式交换 . . . 91
（五） 交换机 vs 集线器 vs 路由器 . . . 92
本章小结表 . . . 92
易错清单 . . . 93

第四章 网络层 96
本章考纲要求 . . . 96
一、 网络层的功能 . . . 97
（一） 异构网络互连 . . . 97
（二） 路由与转发 . . . 98
（三） SDN 基本概念 . . . 98
（四） 拥塞控制 . . . 99
二、 路由算法 . . . 100
（一） 静态路由与动态路由 . . . 100
（二） 距离-向量路由算法 . . . 100
（三） 链路状态路由算法 . . . 101
（四） 层次路由 . . . 101
三、 IPv4 . . . 102

（一） IPv4 分组 . . . 102
（二） IPv4 地址与 NAT . . . 105
（三） 子网划分、路由聚集、子网掩码与 CIDR . . . 107
（四） ARP 协议、DHCP 协议与 ICMP 协议 . . . 112
四、 IPv6 . . . 115
（一） IPv6 的主要特点 . . . 115
（二） IPv6 地址 . . . 117
（三） IPv4 向 IPv6 的过渡 . . . 117
五、 路由协议 . . . 118
（一） 自治系统与域内、域间路由 . . . 118
（二） RIP 路由协议 . . . 118
（三） OSPF 路由协议 . . . 120
（四） BGP 路由协议 . . . 122
（五） 三种路由协议对比 . . . 124
六、 IP 组播 . . . 124
（一） 组播的概念 . . . 124
（二） IP 组播地址 . . . 125
（三） IGMP 与组播路由 . . . 125
七、 移动 IP . . . 126
（一） 移动 IP 的概念 . . . 126
（二） 移动 IP 通信过程 . . . 126
八、 网络层设备 . . . 127
（一） 冲突域和广播域 . . . 127
（二） 路由器的组成和功能 . . . 128
（三） 路由表与分组转发 . . . 129
本章小结表 . . . 131
易错清单 . . . 133

第五章 传输层 **135**
一、 传输层 . . . 135
（一） 传输层提供的服务 . . . 135
（二） UDP 协议 . . . 140
（三） TCP 协议 . . . 145
本章小结表 . . . 171
易错清单 . . . 173

第六章 应用层 **174**
本章考纲要求 . . . 174

一、 网络应用模型 . . . 174
（一） 客户/服务器模型 . . . 174
（二） 对等模型 . . . 175
（三） P2P 的三种组织方式 . . . 176
二、 DNS 系统 . . . 176
（一） 层次域名空间 . . . 176
（二） 域名服务器 . . . 177
（三） 域名解析过程 . . . 178
三、 FTP . . . 181
（一） FTP 协议的工作原理 . . . 181
（二） 控制连接与数据连接 . . . 183
四、 电子邮件 . . . 184
（一） 电子邮件系统的组成结构 . . . 184
（二） 电子邮件格式与 MIME . . . 185
（三） SMTP 协议与 POP3 协议 . . . 186
五、 WWW . . . 188
（一） WWW 的概念与组成结构 . . . 188
（二） HTTP 协议 . . . 189
本章小结表 . . . 196
易错清单 . . . 197

第一章计算机网络概述

本章考纲要求

第四部分计算机网络［考查目标］

1. 掌握计算机网络的基本概念、基本原理和基本方法。
2. 掌握典型计算机网络的结构、协议、应用以及典型网络设备的工作原理。
3. 能够运用计算机网络的基本概念、基本原理和基本方法进行网络系统的分析、设计和应用。

一、计算机网络概述（一）计算机网络基本概念

1. 计算机网络的定义、组成与功能
2. 计算机网络的分类
3. 计算机网络的主要性能指标

（二）计算机网络体系结构

1. 计算机网络分层结构
2. 计算机网络协议、接口、服务等概念
3. ISO/OSI 参考模型和 TCP/IP 参考模型

本章是计算机网络的入门章，也是整个 25 分的基本框架。统考对本章的考查方式非常固定：每年网络部分的第一道选择题（题号 **33** 起）几乎都落在本章，且集中在两个点上——「体系结构各层的功能与归属」和「性能指标的计算」。前者靠背结论和牢记层次顺序，后者靠把公式代进去算对。本章按考纲顺序把这两块写透，并把历年出现过的原题考法逐条标出。

需要注意本章的一个特点：概念题的选项往往只差一两个字，例如「协议的内部实现细节」「面向连接与可靠的关系」「网络体系结构与协议的区别」，判错就不是算错，而是理解边界不清。所以本章对每个术语都给出「是什么、不是什么」。

一、计算机网络基本概念

（一）定义、组成与功能

*1.*计算机网络的定义

计算机网络（Computer Network）的严格定义是：将地理位置不同的、具有独立功能的多台计算机及其外部设备，通过通信线路和通信设备连接起来，在网络操作系统、网络管理软件及网络通信协议的管理和协调下，实现资源共享和信息传递的计算机系统。

这个定义读起来长，但考试真正要抠的是三个限定词：

- 自治（**autonomous**）：每台计算机都有自己的软硬件系统，能独立运行，相互之间没有主从关系，一台计算机不能强制控制另一台。这是把「计算机网络」与「多终端分时系统」区分开的关键——分时系统里的终端没有自治能力，它只是主机的输入输出设备。

- 互连：计算机之间通过通信链路（双绞线、光纤、无线电波等）和通信设备（集线器、交换机、路由器）连接，能进行数据交换。
- 集合：至少两台，且是「计算机的集合」，而不是「计算机与终端的集合」。

最简洁的记法是：计算机网络 = 一些互相连接的、自治的计算机的集合。

计算机网络与分布式系统的区别

分布式系统（Distributed System）在用户看来像一台统一的计算机，用户提交任务时不需要指出由哪台机器完成；而计算机网络中，用户必须显式地指定使用哪一台计算机（例如要写明访问哪台服务器的地址）。分布式系统是建立在计算机网络之上的软件层，是网络的更高形态。两者不是并列的「另外一种东西」。

*2.*两个容易混淆的名词：*internet* 与 *Internet*

- **internet**（互连网）：通用名词，泛指由多个计算机网络互连而成的计算机网络。这些网络之间可以使用任意通信协议，不一定是 **TCP/IP**。
- **Internet**（互联网 / 因特网）：专用名词，特指当前全球最大的、开放的、由众多网络和路由器互连而成的特定计算机网络，它必须使用 **TCP/IP** 协议族。

统考的写法是：出现在「泛指」语境里用小写 internet，出现在「全球互联网」语境里用大写 Internet。

*3.*计算机网络的组成

考纲要求从三个不同角度说出组成，这三个角度不是互相替代的，而是三种切分方式，必须能分别复述。

（**1**）从物理组成看：硬件、软件、协议

组成部分	具体内容	地位
硬件	主机（端系统）、通信设备（集线器、交换机、路由器）、传输介质（双绞线、光纤、无线电波）	提供物理基础
软件	网络操作系统、协议软件、网络管理软件、应用软件	提供可用的功能
协议	通信双方必须共同遵守的规则、标准或约定	核心，缺了协议硬件软件无法互通

（**2**）从工作方式看：边缘部分与核心部分

这是理解因特网结构最重要的一刀。

- 边缘部分：由所有连接在因特网上的主机（端系统）组成。边缘部分由用户直接使用，用来通信（传送数据、音频、视频）和资源共享。边缘部分中两台主机之间的通信方式有两种：
  - 客户 / 服务器方式（**C/S**）：客户是服务请求方，服务器是服务提供方。客户程序必须知道服务器程序的地址，服务器程序不需要知道客户程序的地址，但需要一直运行并等待请求。
  - 对等连接方式（**P2P**）：不区分服务请求方与提供方，每一台主机既是客户又是服务器，可以平等地、直接地通信。
- 核心部分：由大量网络和连接这些网络的路由器组成。核心部分向边缘部分提供连通性和交换服务。核心部分中最关键的设备是路由器，它负责转发分组、选择路径。

边缘与核心的一句话结论

边缘部分是资源与需求的所在地，核心部分是转发与交换的枢纽。核心部分不产生信息，只为边缘部分

提供连通性和交换。判断某设备属于哪一部分，看它是「用户直接使用的主机」还是「负责转发的中间设备」。

（3）从功能看：通信子网与资源子网

- 通信子网：负责数据通信（数据的传输、交换、通信控制与差错控制），由通信控制处理机（如路由器）、通信线路和其他通信设备组成。它对应 OSI 参考模型的下三层（物理层、数据链路层、网络层）。
- 资源子网：负责数据处理（面向应用的数据处理与资源共享），由主机、终端、联网外设、软件与信息资源组成。它对应 OSI 参考模型的上四层（传输层、会话层、表示层、应用层）。

通信子网对应下三层、资源子网对应上四层

这是一个高频的「送分与送命只差一步」的对应关系：下三层 = 通信子网 = 管传输；上四层 = 资源子网 = 管处理。注意切分点落在传输层——传输层属于资源子网，因为它是端到端、面向进程的，已经介入端系统的处理。若题目把传输层划进通信子网，那是错的。

*4.*计算机网络的功能

考纲要求的四大功能，每一条都要能举出例子：

1. 数据通信：计算机网络最基本、最重要的功能，实现计算机与计算机之间各种信息的传递，例如收发电子邮件、文件传输、即时通信。
2. 资源共享：包括硬件资源（打印机、存储设备、处理器）、软件资源（应用程序、工具软件）和数据资源（数据库、文件、资料）的共享。资源共享是计算机网络最有吸引力的功能。
3. 分布式处理：当某个任务非常庞大时，可以把它拆成若干部分，交给网络中多台计算机分工协作完成，从而提高处理能力，例如大规模科学计算、分布式数据库。负载均衡也可归入这一条。
4. 提高可靠性：网络中的计算机可以互为备份。当一台计算机或一条链路出现故障时，任务可以转移到其他计算机或路径上继续运行，从而提升整个系统的可靠性。可替代性（可用性）的另一面就是容错。

有的教材把「负载均衡」单列为第五条功能，统考以考纲的四条为准：数据通信、资源共享、分布式处理、提高可靠性。

本章真题考法（一）

- **2024-33**：给出分组交换网络拓扑与各段链路带宽，问 H1 到 H2 的最大吞吐量。答案是两端接入链路的带宽（本题为 10 Mb/s），中间并行路径无论聚合带宽多大都不能突破端系统的接入瓶颈。陷阱：只看中间链路，忽略源端与目的端的公共瓶颈。
- **2010-33**：问「不属于网络体系结构所描述的内容」——答案是协议的内部实现细节。体系结构只描述层次划分、每层功能、各层使用的协议、层间接口，不描述具体实现。

（二）分类

考纲要求掌握四种分类角度，每一种都要能说出「分几类、依据是什么、各类的特征」。

*1.*按分布范围分类

类型	英文与缩写	覆盖范围	典型例子	关键技术特征
个域网	Personal Area Network, PAN	约 10 m 内	蓝牙、ZigBee、无线耳机	围绕个人设备，速率低、功耗低
局域网	Local Area Network, LAN	10 m ~ 1 km，一个房间、一栋楼、一个校园	以太网、WiFi、令牌环	传输速率高、时延小、误码率低，通常为广播式共享介质
城域网	Metropolitan Area Network, MAN	5 ~ 50 km，一个城市	城市有线电视网、城域以太网	介于 LAN 与 WAN 之间，早期用 DQDB
广域网	Wide Area Network, WAN	几十 ~ 几千 km，跨城市、国家、洲	因特网骨干、公共交换电话网	传输速率相对低、时延大，采用点对点链路与交换技术

局域网与广域网的区分依据不是「范围大小」

统考反复强调：**LAN** 与 **WAN** 的本质区别在于所使用的技术，而不是单纯的地理范围。局域网使用广播式共享介质的传输技术（早期总线型以太网、令牌环），广域网使用点对点链路加交换结点的传输技术。用一个校园网跨越两个城市，它仍是局域网技术。若选项只讲「范围大小」，那是干扰项。

补充一个与「分布范围」并列的分类角度：按传输技术可分为广播式网络与点对点网络。局域网基本采用广播式通信（一个结点发送，其余结点都能收到），广域网基本采用点对点通信。

*2.*按使用者分类

- 公用网（**Public Network**）：由电信运营商等建设、经营，只要按规定交纳费用，任何单位或个人都可以使用，例如公用电话网、公用数据网。特点是面向全社会、使用门槛低、但资源被共享。
- 专用网（**Private Network**）：由某个部门或单位为满足本单位业务需要而建设、使用，不向本单位以外的人开放，例如军队、铁路、银行、电力系统的内部网络。特点是安全性高、可控性强、但建设与维护成本高。

两者的关系：专用网租用公用网的传输线路是很常见的做法——线路是公用网的，逻辑网络是专用网（如 VPN 虚拟专用网）。

*3.*按交换技术分类

这是本章最重要的一组对比，几乎每年都会以计算或对比题的方式出现。

（1）电路交换（Circuit Switching）

通信全过程分三个阶段：建立连接 → 数据传输 → 释放连接。

- 建立连接阶段：在通信双方之间建立一条独占的物理通路，沿途结点为该连接分配通信资源（时隙、频段等）。这一阶段费时较长。
- 数据传输阶段：数据透明地沿这条通路传输，中间结点不对数据做处理，因此不具备差错控制能力。
- 释放连接阶段：归还占用的通信资源。

优点	缺点
通信时延小（通路一旦建立，传输几乎无额外时延）	建立连接时间长
数据按序传输，不会失序	线路利用率低（空闲时资源也被独占）
无冲突，实时性强	灵活性差（一方断开则连接中断，中途难以改路）
控制简单	不具备差错控制能力

电路交换适用于数据量很大且传送时间远大于呼叫（建立连接）时间的场合，典型例子是传统电话网。

（2）报文交换（Message Switching）

采用存储转发技术：整个报文先被传送到相邻结点，全部接收并存储完整后，再查找转发表把它转发到下一个结点，如此重复直到目的结点。

- 报文（Message）是用户数据加上源地址、目的地址等信息构成的完整数据块。
- 因为必须收完整个报文才能转发，无法流水线处理，所以转发时延很长；相应地，交换结点必须能缓存整个报文，缓存空间开销大；一旦出错，整个报文都要重传，出错重传开销大。

优点	缺点
无需建立连接	转发时延高
动态分配线路	缓存开销大
可靠性高	出错重传开销大
线路利用率高	不适用于实时通信

（3）分组交换（Packet Switching）

同样采用存储转发，但把较长的报文划分成若干较小的、长度固定的数据段，每段前面加上首部（分组头，含目的地址、源地址等控制信息），构成一个分组（Packet）。分组交换机根据分组中的目的地址查找自己的转发表进行转发。它解决了报文交换中「报文过长导致转发时延长、缓存开销大」的问题。

- 优点：无需建立连接；线路利用率高（不同应用的分组可以交替使用链路，实现公平共享）；可采用流水线方式加速传输；分组短小，出错概率和出错重传开销都更小。
- 缺点：仍存在转发时延；每个分组都要携带首部，分组头开销比报文交换更大；分组可能出现失序、丢失、重复。

分组交换进一步分为两种方式，都由网络层提供：

对比维度	数据报服务	虚电路服务
连接建立	不需要	必须先建立
目的地址	每个分组都携带完整目的地址	仅建立连接阶段使用，之后每个分组只用短虚电路号
路由选择	每个分组独立选路与转发	同一条虚电路的分组按同一路由转发
分组顺序	不保证有序到达	保证有序到达

对比维度	数据报服务	虚电路服务
可靠性	网络尽力转发，不承诺交付，可靠性由上层协议（如 TCP）保证	可靠性由网络自身保证
对故障的适应性	可重新选路，适应能力强	经过故障结点的全部虚电路均不能工作
差错与流量控制	由用户主机上层协议负责	可由分组交换网负责，也可由主机负责

虚电路不是电路交换

虚电路是分组交换的一种，是逻辑连接；电路交换是真正的物理线路交换。关键区别：电路交换预分配带宽，一条物理线路在通信期间被一条连接独占；虚电路采用多路复用技术，多条虚电路共享同一段物理线路的带宽，分组仍然要存储转发。另外，虚电路服务是 **OSI** 参考模型特有的，TCP/IP 模型的网络层只提供无连接的数据报服务。

*4.*按拓扑结构分类

拓扑	结构描述	优点	缺点	典型场景
星形	所有结点连到中心结点（集线器 / 交换机）	结构简单、控制简单、故障易诊断与隔离、易于扩充	中心结点是瓶颈与单点故障，可靠性差，线缆用量大	以交换机为中心的现代局域网
总线	所有结点共享一条公共总线	结构简单、成本低、扩充方便、可靠性较高（无中心）	冲突严重、介质访问控制复杂、总线故障全网瘫痪、故障难隔离	早期以太网（10BASE-5/2）
环形	结点首尾相连成闭合环，数据沿单一方向逐站传递	时延确定、无冲突、控制简单	单点故障可能使全网中断（需双环或旁路）、扩充困难	令牌环网、FDDI
网状	结点之间有多条链路相连（全连接或部分连接）	可靠性高、容错能力强、易于扩展、可绕开故障	结构复杂、成本高、路由算法复杂	广域网骨干

补充：树形拓扑（星形的层次化扩展）在教材中也常单列，其特点是根结点故障会导致整棵树分裂。

（三）主要性能指标

这一节是本章的计算核心，七个指标必须做到「听到名字就能写出定义式和单位」。

*1.*速率与带宽

速率（**Rate**）又称数据率（**Data Rate**）、比特率（**Bit Rate**），指连接到网络上的结点在单位时间内往信道上传送的数据量，单位是 **bit/s**（**b/s**、**bps**），也可写作 kbit/s、Mbit/s、Gbit/s。

带宽（**Bandwidth**）在计算机网络中有两种含义，必须分清：

1. 本来含义（模拟信号语境）：某个信号具有的频带宽度，即该信号包含的最高频率与最低频率之差，单位是赫兹（**Hz**）。例如某信道允许 300 Hz ~ 3 kHz 的信号通过，则带宽为 2700 Hz。
2. 计算机网络中的含义（数字信道语境）：某信道所能传送的最高数据率，单位是 **bit/s**。例如「这条链路的带宽是 100 Mb/s」。

带宽两种含义的识别方法

看单位。单位是 **Hz** 就是频带宽度（物理层信道特性，用于奈氏准则、香农定理）；单位是 **bit/s** 就是最高

数据率（用于吞吐量、时延带宽积、发送时延的计算）。题目给出带宽的数值时，先确认单位再代公式。

速率与存储容量换算进制的差别

速率（数据率、带宽）中的 $k=10^3$、$M=10^6$、$G=10^9$，是十进制；而存储容量（如文件大小、内存容量）中的 $1\text{KB}=2^{10}\text{B}$，是二进制。统考题目常在题干里明确写出「$1\text{M}=10^6$」或「$1\text{M}=2^{20}$」，以题干给定为准；若题干未给，网络题的速率一律按十进制，数据量按题目给出的口径。这是网络计算题最常见的失分点之一。

*2.*吞吐量

吞吐量（**Throughput**）指单位时间内通过某个网络（或信道、接口）的实际数据量，单位也是 bit/s。

- 吞吐量受网络带宽或链路带宽的限制：吞吐量的最大值不超过所经路径上最小的一段链路带宽。
- 因此求「从 A 到 B 的最大吞吐量」时的标准做法是木桶效应：把端到端路径上每一段的带宽列出来，取最小值。

用 2024-33 的结构说明：若路径为「H1 ——(10 Mb/s)——路由器——(三条并行路径：1000 Mb/s、1 Mb/s、100 Mb/s)——路由器——(10 Mb/s)——H2」，则

$$\text{最大吞吐量}=\min(10,\ 1000+1+100,\ 10)=10\ \text{Mb/s}$$

中间网络的聚合能力再大，也受两端 10 Mb/s 接入链路的夹持。默认不要自行脑补「分流、负载均衡、多路径」，除非题目明确写出。

*3.*时延

时延（**Delay / Latency**）指数据从网络（或链路）的一端传送到另一端所耗费的时间，由四部分组成：

$$\text{总时延}=\text{发送时延}+\text{传播时延}+\text{处理时延}+\text{排队时延}$$

（**1**）发送时延（传输时延）

$$\text{发送时延}=\frac{\text{数据长度（bit）}}{\text{发送速率（bit/s）}}$$

指主机或路由器把数据帧 / 分组的全部比特推送到链路上所需的时间。

- 数据长度越大，发送时延越大；发送速率越高，发送时延越小。
- 发送时延与链路的物理长度无关。
-「发送速率」由网卡与链路决定，不同链路的发送速率可能不同，此时必须分别计算。

（**2**）传播时延

$$\text{传播时延}=\frac{\text{信道长度（m）}}{\text{电磁波在信道上的传播速率（m/s）}}$$

指一个比特从链路的一端传播到另一端所需的时间。传播时延只与信道长度和介质中的传播速率有关，与数据量、与链路带宽都无关。

常见传播速率（记数量级即可）：自由空间 3.0×10^8 m/s；铜缆约 2.3×10^8 m/s；光纤约 2.0×10^8 m/s。

（3）处理时延

数据在交换结点为存储转发而进行的必要处理所花的时间，例如分析分组首部、从分组中提取数据、进行差错检验、查找转发表确定转发端口。

（4）排队时延

分组进入路由器后，先在输入队列中排队等待处理；路由器确定转发端口后，还要在输出队列中排队等待转发。队列长度取决于网络负载，因此排队时延是四者中最不确定的一项。

四个时延的核心对比

时延	决定因素	与带宽的关系	与链路长度的关系
发送时延	数据长度、发送速率	反比	无关
传播时延	信道长度、传播速率	无关	正比
处理时延	结点处理能力	弱相关	无关
排队时延	网络负载	强相关	无关

一句话：发送时延看「数据多长、发得多快」，传播时延看「路有多长、波跑多快」。

提高带宽不能减少传播时延

这是极易混的一对：「在高速链路上，瓶颈往往是传播时延而不是发送时延」。把链路带宽从 10 Mb/s 提升到 10 Gb/s，能把发送时延压到极小，但传播时延一点也不会变。卫星链路的单向传播时延约 250 ms，无论带宽多大都消不掉。所以「提高带宽就能降低总时延」是错的。

多分组经过多段链路的总时延公式

若报文被分成 m 个等长分组，端到端经过 n 个路由器（即 $n+1$ 段链路），各段链路发送速率相等、每段链路传播时延相等且忽略处理与排队时延，则：

$$T = (m+n) \times t_{\text{发送}} + (n+1) \times d_{\text{传播}}$$

理由：第一个分组要经历 $(n+1)$ 次发送与 $(n+1)$ 次传播才能到终点，其后每个分组因流水线而只额外增加一次发送时延。

完整算例一（点对点，四时延中只算两项）

主机 A 通过一条长 1000 km 的光纤链路向主机 B 连续发送 10 个长度为 1500 B 的分组，链路带宽 100 Mb/s，光纤中信号传播速率为 2×10^8 m/s，中间不经过转发设备，忽略处理与排队时延。求从 A 开始发送第一个比特到 B 收到最后一个比特所需的时间。

1. 单个分组的发送时延：$t = \dfrac{1500 \times 8}{100 \times 10^6} = \dfrac{12000}{10^8} = 120\ \mu s$。
2. 10 个分组连续发送的发送时延：$10 \times 120\ \mu s = 1200\ \mu s = 1.2$ ms。
3. 传播时延：$d = \dfrac{10^6\ \text{m}}{2 \times 10^8\ \text{m/s}} = 5 \times 10^{-3}\ \text{s} = 5$ ms。
4. 总时延 $= 1.2 + 5 = 6.2$ ms。

第 4 步要注意：发送与传播是流水进行的，第 1 个分组的比特已经在链路上跑了，所以总时间不是 $10 \times (0.12 + 5)$，而是「把 10 个分组全部推上链路的时间」加上「最后一个比特走完全程的传播时延」。

完整算例二（含路由器的多分组时延）

主机 H1 与 H2 通过一台路由器互连，两段链路带宽均为 100 Mb/s，每段链路的时延带宽积均为 1000 bit。H1 向 H2 发送一个大小为 1 MB（1M $= 10^6$）的文件，分组长度为 1000 B，忽略分组拆装时间、处理时延与排队时延。求从 H1 开始发送到 H2 收到全部数据所需的最少时间。

1. 分组总个数：$m = \dfrac{10^6\ \text{B}}{1000\ \text{B}} = 1000$。
2. 单个分组的发送时延：$t = \dfrac{1000 \times 8}{100 \times 10^6} = 80\ \mu s$。
3. 由时延带宽积求单向传播时延：$d = \dfrac{1000\ \text{bit}}{100 \times 10^6\ \text{bit/s}} = 10\ \mu s$。
4. 路由器数 $n = 1$，代入公式：$T = (m+n)t + (n+1)d = (1000+1) \times 80 + 2 \times 10 = 80080 + 20 = 80100\ \mu s = 80.10$ ms。

这道题的三个易错点：一是分组个数必须用「文件大小 ÷ 每个分组长度」，不是「÷ 每个分组的数据载荷」；二是路由器数 $n = 1$ 时公式里是 $m + n = 1001$ 次发送，不是 1000 次；三是时延带宽积要先换算成传播时延再代入。

*4.*时延带宽积

时延带宽积（**Bandwidth-Delay Product**）定义为：

$$\text{时延带宽积} = \text{传播时延} \times \text{带宽}$$

它的物理意义是：发送端发送的第一个比特即将到达终点时，发送端已经发出了多少个比特。因此时延带宽积又称「以比特为单位的链路长度」，也叫链路的「管道容量」。

- 时延带宽积的单位是比特（**bit**），不是秒，也不是 bit/s。
- 一条链路的时延带宽积表示「这条链路上同时能容纳多少个比特在途中」。

算例：链路带宽 10 Mb/s，单向传播时延 20 ms，则

$$\text{时延带宽积} = 10 \times 10^6\ \text{bit/s} \times 20 \times 10^{-3}\ \text{s} = 2 \times 10^5\ \text{bit} = 200\ \text{kb}$$

也就是说，这条链路上任意时刻最多有 200 kb 的数据在飞行途中。若发送 1000 B（8000 bit）的分组，链路上可同时容纳 $2 \times 10^5 / 8000 = 25$ 个分组。

时延带宽积揭示的结论

如果采用停止等待协议（每发一个分组就等确认），链路最多只能容纳 1 个分组，链路的利用率极低——有效数据率仅为 $\dfrac{\text{分组长度}}{\text{RTT}}$，与该链路的时延带宽积相比小得可以忽略。这正是「管道容量」这个概念的用途：它告诉你需要多少「在途数据」才能把链路喂满，也是后面滑动窗口、TCP 发送窗口大小的理论依据。

*5.*往返时间 *RTT*

往返时间（**Round-Trip Time, RTT**）指从发送端发送数据分组开始，到发送端收到接收端发回的相应确认分组为止，总共经历的时间。

- RTT 包含：数据分组的发送时延 + 往返的传播时延 + 中间结点的处理与排队时延 + 确认分组的发送时延。很多题目为了简化会说明「忽略确认分组的发送时延与传播时延之外的其他时延」。
- RTT 不等于单向时延的两倍（严格说，它比两倍的单向传播时延还要多出确认分组的发送时延与处理时延），但在多数题目中可以近似看作两倍的单向传播时延。

- 在 RTT 主导的场合（如 TCP 慢启动、停止等待协议），有效数据率 $\approx \dfrac{\text{数据长度}}{\text{RTT}}$。

算例：RTT = 20 ms，分组长度 1000 B，链路带宽 10 Mb/s，采用停止等待协议。忽略发送时延，则

$$\text{有效数据率} = \frac{1000 \times 8 \text{ bit}}{20 \times 10^{-3} \text{ s}} = 400 \text{ kb/s}$$

只有带宽 10 Mb/s 的 4%，与前面时延带宽积给出的「链路可容纳 25 个分组」完全吻合——每 25 个分组的位置上只发了 1 个。

*6.*利用率

- 信道利用率：某个信道有百分之多少的时间是有数据通过的。

$$U_{\text{信道}} = \frac{\text{有数据通过的时间}}{\text{有数据通过的时间} + \text{无数据通过的时间}}$$

- 网络利用率：全网络中所有链路的信道利用率的加权平均值。

利用率越高，看上去链路越「划算」，但：

$$D = \frac{D_0}{1 - U}$$

其中 D_0 表示网络空闲时的时延，U 为网络利用率，D 为当前网络时延。

算例：

网络利用率 U	时延倍数 $\frac{D}{D_0}$	含义
0	1	网络空闲，时延为 D_0
0.5	2	时延翻倍
0.8	5	时延是空闲时的 5 倍
0.95	20	时延是空闲时的 20 倍
$\to 1$	$\to \infty$	时延趋于无穷，网络接近拥塞崩溃

若 $D_0 = 10$ ms，$U = 0.6$，则 $D = \dfrac{10}{1 - 0.6} = 25$ ms。

利用率不是越大越好

由 $D = \dfrac{D_0}{1 - U}$ 可见，当 U 接近 1 时时延急剧上升。因此网络利用率并非越高越好，适度保留冗余才能保证时延可控。题目若说「利用率越大越好」或「提高利用率一定能提高网络性能」，都是错的。另外要分清 D_0 是「网络空闲时的时延」，不是「最小可能时延」，也不是「发送时延」。

本章真题考法（二）：性能指标计算题

- **2024-33**：多路径分组交换网络的最大吞吐量，用木桶效应取最小带宽（含两端接入链路）。
- 时延带宽积类：给出「两段链路带宽均为 100 Mb/s、时延带宽积均为 1000 bit、文件 1 MB、分组 1000 B」，求总时间（答案为 80.10 ms 量级）。
- 存储转发分组交换的时延题：所有链路传输速率 100 Mb/s、分组 1000 B（其中首部 20 B）、文件 980000 B，忽略分组拆装时间与传播时延。分组数 $= 980000/980 = 1000$，发送时延 $= 1000 \times 8/10^8 = 80\ \mu s$，总时延 $= (1000 + 1) \times 80\ \mu s = 80.08$ ms（答案对应 80.08 ms）。

- 报文交换与分组交换对比题：甲经一个路由器（存储转发）与乙互连，两段链路速率均为 10 Mb/s，报文 8 Mb（$1\text{M} = 10^6$），分组 10 kb。报文交换需 $800 + 800 = 1600$ ms；分组交换需 $(800 + 1) \times 1$ ms $= 801$ ms，即 1600 ms 与 801 ms。结论：分组交换在长报文传输上显著优于报文交换，且路由器越少、分组越短，优势越明显（但分组越短首部开销越大）。

二、计算机网络体系结构

（一）分层结构

*1.*为什么需要分层

计算机网络要解决的问题极为复杂：不同厂商的设备要互通、不同介质的链路要互联、上层应用不关心底层怎么传数据。分层的思路是把庞大而复杂的问题分解成若干个较小的、容易处理的局部问题，每一层只解决其中一部分。

分层带来的好处：

- 各层之间相互独立：上一层只需要知道下一层通过接口提供的服务，不需要知道下一层如何实现。
- 灵活性好：某一层的内部实现发生变化（例如把铜缆换成光纤），只要接口不变，其余各层都不受影响。
- 结构上可分割：各层都可以采用最合适的技术来实现。
- 易于实现和维护：整个系统被分解为若干相对独立的子系统。
- 能促进标准化：每一层的功能及其提供的服务都有了精确的说明。

分层的原则：

- 每层都实现一种相对独立的功能。
- 层间的接口自然清晰，信息量尽可能少。
- 各层功能的选择应有助于制定国际标准。

*2.*体系结构、实体与对等层

计算机网络体系结构（**Network Architecture**）是计算机网络各层及其协议的集合，也就是这个计算机网络及其部件所应完成的功能的精确定义。

体系结构不包括实现细节

体系结构是抽象的，只描述「做什么」，不描述「怎么做」。它回答的问题是：网络划分为哪些层次、每一层完成什么功能、各层使用什么协议、相邻层之间通过什么接口和服务协作。至于某个协议在某台设备上用什么数据结构、什么算法、什么代码实现，不属于体系结构的描述内容。

相关术语：

- 实体（**Entity**）：第 n 层中能够收发信息、执行协议的活动元素，可以是硬件（如网卡）也可以是软件进程（如浏览器进程）。网卡内部含有固件与控制器，既能收发比特流（物理层）又能封装数据帧（数据链路层），是一个典型的硬件实体。
- 对等实体（**Peer Entity**）：通信双方同一层次中的实体。例如 A 的网络层实体与 B 的网络层实体就是对等实体。只有对等实体之间才谈「协议」。
- 对等层通信：对等层之间的通信是「逻辑上」的，数据并不是从第 n 层直接飞到对方的第 n 层，而是逐层向下传递到物理层，经传输介质到达对方后，再逐层向上传递。这个「看起来像同层

直接对话」的效果，是靠封装与解封装实现的。

分层体系结构的三个基本构成

网络体系结构 = 层次划分 + 各层协议 + 层间接口（服务）的集合。

单向依赖关系贯穿始终：第 n 层只能使用第 $n-1$ 层提供的服务，并向第 $n+1$ 层提供服务；同一层内协议控制对等实体的通信。同一结点内相邻层之间是「服务」关系，不同结点同一层之间是「协议」关系。

*3.*协议三要素

协议（**Protocol**）是为进行网络中的数据交换而建立的规则、标准或约定，即控制对等实体之间通信的规则的集合。协议由三个要素组成：

要素	含义	通俗理解
语法（Syntax）	规定数据与控制信息的结构或格式	「话怎么说」——数据长什么样、字段怎么排
语义（Semantics）	规定需要发出何种控制信息、完成何种动作以及做出何种响应	「话是什么意思」——收到后要做什么
同步（Timing / 时序）	规定事件实现顺序的详细说明，即执行各种操作的条件与时序关系	「什么时候说」——先后次序、速度匹配

2020-33：协议三要素的图形识别

题目给出两条竖直时间轴与多条消息箭头，表示发送方与接收方之间消息发送与响应的先后关系。图中没有给出报文格式，也没有解释字段含义，因此只描述了时序（同步），答案是「仅 III」。

判图口诀：有字段排列 → 语法；有「收到后做什么」的说明 → 语义；只有箭头先后、没有格式与含义 → 时序。

在 TCP/IP 语境下，「同步」也常被写作「时序」。三者缺一不可——只规定格式而不规定什么人什么时候做出什么响应，通信就无法完成。

（二）协议、接口、服务等概念

*1.*协议、接口与服务

这三层术语的关系是本章最容易出辨析题的地方，必须从「方向」上彻底分清。

- 协议（**Protocol**）：控制对等实体之间通信的规则的集合。协议是「水平」的——它约束的是不同结点上同一层次的实体。
- 服务（**Service**）：下层（第 n 层）向上层（第 $n+1$ 层）提供的一组操作（功能调用）。服务是「垂直」的——它只在同一结点内的相邻层之间发生。
- 接口（**Interface**）/ 服务访问点（**SAP**）：同一系统中相邻两层实体交换信息的逻辑接口。上层要使用下层提供的服务，必须通过与下层交换一些命令，这些命令称为服务原语（**Service Primitive**）。

关于接口与 SAP 的关系，统考的口径是：

1. 接口是大框架，描述两层之间的交接处与交互入口；
2. **SAP** 是接口的具体接入点，用于区分不同的服务类型（考试时认为「接口 ≈ SAP」）；
3. 服务原语是在 **SAP** 上传输的控制命令。

常见的 SAP 举例：数据链路层帧的「类型」字段、IP 数据报的「协议」字段、TCP 报文段或 UDP 用户数据报的「端口号」字段、应用层的「用户界面」。

服务与协议的区别（必背）

1. 协议是水平的，是控制对等实体之间通信的规则；服务是垂直的，是下层向上层通过层间接口提供的。
2. 本层协议的实现要依靠下层提供的服务；本层通过协议向上一层提供服务。
3. 协议对上层是透明的：上层实体只看得见下层提供的服务，看不见（也不需要知道）该服务是用什么协议实现的。
4. 并非在一个层内完成的全部功能都称为服务，只有那些能被上一层所看见的功能才称为服务。
5. 协议的实现保证了能够向上一层提供服务；但「服务」不等于「协议」——同一服务可以由不同的协议实现。

*2.*面向连接服务与无连接服务

按通信前是否需要建立连接，下层向上层提供的服务分为两类：

对比项	面向连接服务	无连接服务
连接建立	必须先建立连接	不需要
三个阶段	连接建立、数据传输、连接释放	只有数据传输
资源占用	通信期间一直占用	随用随取，用完即走
数据顺序	一般保证按序到达	不保证顺序
典型协议	TCP、虚电路	UDP、IP 数据报
开销	建立与释放连接的开销	每个分组都要带完整目的地址，但无连接管理开销

「面向连接」不等于「可靠」

这是一个反复考的陷阱。面向连接只是一种连接管理方式，可靠性必须由确认、重传、序号等机制另行提供。

- TCP 是面向连接的，同时也是可靠的。
- 但 X.25 的虚电路是面向连接的，其可靠性由网络层保证；而帧中继的虚电路面向连接却是不可靠的。
- 反过来，无连接也不等于不可靠——例如 802.11 无线局域网的某些机制在无连接的基础上也提供了确认重传。

结论：「面向连接」与「可靠」是两个独立的属性，不能互相推导。同理，「无连接」与「不可靠」也不是同义词，只是在 TCP/IP 体系中恰好是对应关系。

*3.*服务数据单元 *SDU* 与协议数据单元 *PDU*

- 协议数据单元（**PDU, Protocol Data Unit**）：对等层次之间传送的数据单元。第 n 层的 PDU 由两部分构成：协议控制信息 **PCI**（首部）+ 服务数据单元 **SDU**。

$$n\text{-PDU} = n\text{-PCI} + n\text{-SDU}$$

- 服务数据单元（**SDU, Service Data Unit**）：同一系统内层与层之间交换的数据单元，也就是上一层交给本层、要求本层代为传送的数据。

两者的关系是「同一份数据在不同层次的不同称呼」。当上层把数据整体交给下层、下层不做拆分也不做合并时：

$$n\text{-SDU} = (n+1)\text{-PDU}$$

即：第 n 层的服务数据单元，就是第 $n+1$ 层的协议数据单元。

各层 PDU 的名称（必背）：

OSI 层次	**PDU** 名称（统考叫法）	说明
应用层	报文（Message）/ 数据	应用进程之间交换的数据
表示层	报文 / 数据	只做表示转换，不改变 PDU 名称
会话层	报文 / 数据	只做会话管理
传输层	报文段（Segment）；UDP 称用户数据报	TCP 的 PDU 叫报文段
网络层	分组 / 数据报（Packet / Datagram），常称 IP 数据报	加 IP 首部（至少 20 B）
数据链路层	帧（Frame）	加帧头（目的 MAC、源 MAC、类型）与帧尾（FCS）
物理层	比特（Bit）	比特流，不参与封装

PDU 与 SDU 的两个易错点

1. **PDU** 在水平方向（对等层之间）使用，**SDU** 在垂直方向（相邻层之间）使用。把 SDU 说成「对等层之间传送的数据单元」是错的。
2. 传输层的 **PDU** 叫「报文段」，不叫「报文」。「报文（Message）」是应用层的 PDU。网络层的 PDU 叫「分组 / 数据报」，教材上也写作「IP 数据报」或简称「IP 分组」，但不要写成「数据包」之外又另造名字。

本章真题考法（三）：协议、接口、服务

- **2020-33**：协议三要素的图形识别，答案为「仅 III（时序）」。
- **2010-33**：不属于体系结构描述内容的是「协议的内部实现细节」。
- 常考判断：「服务是水平的、协议是垂直的」是错的（反了）；「协议是水平的、服务是垂直的」是对的。「**IP** 数据报的协议字段是一个 **SAP**」是对的；「**SAP** 就是协议」是错的。

（三）ISO/OSI 参考模型与 TCP/IP 参考模型

1.ISO/OSI 参考模型

OSI（Open Systems Interconnection Reference Model，开放系统互连参考模型）由国际标准化组织 **ISO** 提出，是一个法律上的国际标准。它的设计思路是先有模型、后有协议：先定义好七层的功能框架，再逐层制定协议。结果是模型本身严谨完备，但过于复杂，实现困难，市场化失败。

七层名称（自下而上）：物理层、数据链路层、网络层、传输层、会话层、表示层、应用层。

记忆口诀：物、链、网、输、会、示、用。

各层的主要功能：

- 物理层（**Physical Layer**）：在传输介质上透明地传送比特流，为数据链路层提供一个传输比特的物理连接。它规定接口的机械特性、电气特性、功能特性、过程特性（如引脚数目、电压范围、信号含义、时序）。物理层不参与数据的封装，它看到的只是比特流。PDU 为比特。该层设备：中继器、集线器。

- 数据链路层（**Data Link Layer**）：在相邻结点之间无差错地传送以「帧」为单位的数据。主要功能：成帧（帧定界与同步）、差错控制、流量控制、介质访问控制。设备：网卡、网桥、交换机。相邻结点间的流量控制属于数据链路层。
- 网络层（**Network Layer**）：负责把分组从源结点转发到目的结点。主要功能：路由选择、分组转发、拥塞控制、网际互连（实现异构网络互联）。设备：路由器、三层交换机。OSI 的网络层既支持无连接方式，也支持面向连接方式（虚电路）。
- 传输层（**Transport Layer**）：提供端到端（进程到进程）的数据传输服务，是自下而上第一个提供端到端服务的层次。主要功能：复用与分用、差错控制、端到端流量控制、可靠传输、拥塞控制、连接建立与释放。OSI 的传输层只支持面向连接的方式。
- 会话层（**Session Layer**）：管理进程间的会话（为表示层实体或用户进程建立连接，并在连接上有序地传输数据）。主要功能：会话管理、同步服务（在数据流中插入检查点，实现「文件断点续传」）。
- 表示层（**Presentation Layer**）：解决不同主机上信息表示方式不一致的问题。主要功能：数据格式转换、字符编码转换、加密与解密、压缩与解压。
- 应用层（**Application Layer**）：体系结构中的最高层，直接为用户的应用进程提供服务，包含大量面向具体应用的协议（HTTP、FTP、SMTP、DNS 等）。

网络层与传输层的「端到端」区别

- 网络层常被称为「主机到主机」的通信：它负责把分组从源主机送到目的主机，中间要经过路由器，每台路由器都要参与网络层处理。
- 传输层是真正的「进程到进程」的端到端通信：只有两端的端系统才需要运行传输层，中间路由器不处理传输层数据。2009-33 问「自下而上第一个提供端到端服务的层次」，答案就是传输层。
- 另外，相邻结点间的流量控制是数据链路层的功能（**2022-33**），端到端的流量控制是传输层的功能，两者不能混。

2.TCP/IP 参考模型

TCP/IP 参考模型的设计思路与 OSI 相反：先有协议、后有模型，是事实上的国际标准。它共四层：

- 网络接口层（链路层）：对应 OSI 的物理层 + 数据链路层，负责与硬件打交道，把 IP 数据报封装成帧并通过物理网络发送。
- 网际层（**IP** 层 / 网络层）：核心是 **IP** 协议。它只提供无连接、不可靠的数据报服务（尽力而为的交付），负责逻辑寻址、路由选择与 IP 数据报转发；不提供流量控制与可靠传输。
- 传输层：提供端到端的进程间通信，同时支持面向连接的 **TCP** 和无连接的 **UDP**。
- 应用层：对应 OSI 的会话层 + 表示层 + 应用层，把三层功能合并。

五层教学模型：408 教材普遍采用一种折中的五层结构，即物理层、数据链路层、网络层、传输层、应用层——下面四层与 OSI 的下四层一一对应，最上面一层把 OSI 的会话层、表示层、应用层合并。五层模型既保留了 OSI 中「有物理介质、有链路、有网络、有端到端」的清晰划分，又避免了 OSI 上三层划分过细的问题。

*3.*两个模型的对比

对比维度	**OSI** 参考模型	**TCP/IP** 参考模型
层数	7 层	4 层（教学用 5 层）
设计顺序	先有模型，后有协议	先有协议，后有模型

对比维度	**OSI** 参考模型	**TCP/IP** 参考模型
标准性质	法律上的国际标准	事实上的国际标准
网络层提供的服务	面向连接（虚电路）+ 无连接	只有无连接（IP 数据报）
传输层提供的服务	只有面向连接	面向连接（TCP）+ 无连接（UDP）
可靠性由谁保证	由网络层保证（传输层一定可靠）	由传输层保证（网络层不可靠）
上三层划分	会话层、表示层、应用层三层分立	合并为一层应用层
网络互连能力	未充分定义	核心能力，靠 IP 实现异构网络互联
适用场景	理论完备、实现复杂、市场失败	简单实用、实现广泛、市场成功

两个模型的三条「反着记」的结论

1. **OSI** 的网络层有连接也有无连接，传输层只有面向连接；**TCP/IP** 的网络层只有无连接，传输层两种都有。（方向正好相反）
2. **OSI** 认为可靠性由网络层负责；**TCP/IP** 认为网络层不可靠，可靠性交给传输层。
3. **OSI** 是「法律上的」国际标准；**TCP/IP** 是「事实上的」国际标准。

*4.*数据在各层的封装与解封装

封装（**Encapsulation**）：发送方从应用层往下逐层加首部（数据链路层还要加帧尾）的过程。

$$\text{应用数据} \rightarrow \text{报文} \xrightarrow{+\text{TCP 首部}} \text{报文段} \xrightarrow{+\text{IP 首部}} \text{数据报/分组} \xrightarrow{+\text{帧头帧尾}} \text{帧} \xrightarrow{\text{转为信号}} \text{比特流}$$

逐层看：

1. 应用层：应用进程产生数据（Data），称为报文（Message）。
2. 传输层：加上 TCP 首部（源端口、目的端口、序号、确认号等），构成报文段（**Segment**）。
3. 网络层：加上 IP 首部（源 IP、目的 IP、协议、TTL 等，至少 20 B），构成 **IP** 数据报 / 分组。
4. 数据链路层：加上帧头（目的 MAC、源 MAC、类型）与帧尾（FCS 帧校验序列），构成帧（**Frame**）。
5. 物理层：把帧转换为比特流，在传输介质上以信号形式发送。物理层是「管道的存在」，数据链路层是「管道里的交通规则」。

解封装（**Decapsulation**）：接收方从物理层往上逐层剥去本层的首部（与帧尾），把剩下的数据交给上一层，直到应用层还原出原始数据。

2017-33：封装效率计算

OSI 参考模型的应用层欲发送 400 B 数据（无拆分），除物理层和应用层之外，其他各层在封装 PDU 时均引入 20 B 额外开销，求应用层数据传输效率。

会引入 20 B 开销的层有 5 个：表示层、会话层、传输层、网络层、数据链路层。总开销 $= 5 \times 20 = 100$ B，链路上总数据量 $= 400 + 100 = 500$ B。

$$\text{效率} = \frac{400}{500} \times 100\% = 80\%$$

两个陷阱：一是物理层与应用层不计开销，所以是 5 层不是 7 层；二是题目明确「无拆分」，因此同一层的首部开销不能重复计算（若报文被分成多个分组，每个分组都要带一次网络层与数据链路层首部）。

*5.*各层典型协议与设备归属

这张表要能默写。层 →PDU→ 协议 → 设备的对应关系，是本章选择题的主要出题方式。

层次	**PDU** 名称	典型协议	典型设备	最高工作在此层的设备
应用层	报文（Message）	HTTP、HTTPS、FTP、SMTP、POP3、DNS、DHCP、Telnet、SNMP	应用网关	网关
表示层	报文	数据格式与加密压缩规范（OSI 专有）	—	—
会话层	报文	会话管理协议（OSI 专有）	—	—
传输层	报文段（Segment）/ 用户数据报	TCP、UDP	—	—
网络层	分组 / 数据报（Packet）	IP、ICMP、IGMP、ARP、OSPF、RIP、BGP	路由器、三层交换机	路由器（第 3 层）
数据链路层	帧（Frame）	以太网 802.3、PPP、HDLC、802.11、VLAN	网卡、网桥、二层交换机	交换机（第 2 层）
物理层	比特（Bit）	RS-232、RJ-45 相关规范	中继器、集线器	集线器（第 1 层）

ARP 的层次归属

ARP（地址解析协议）的归属在不同教材中有分歧：有的把它放在网络层，有的放在网络接口层（链路层）。统考一般按网络层理解（因为 ARP 报文直接封装在以太网帧中，但它服务于 IP 地址到 MAC 地址的解析，属于网络层的配套协议）。遇到单选题时，若选项把 ARP 与 IP、ICMP、IGMP 并列放在网络层，按网络层选；若题干强调「封装在帧里」，则以题干给出的口径为准。

2016-33：设备的最高功能层

R1（路由器）、Switch（交换机）、Hub（集线器）实现的最高功能层分别是 **3**、**2**、**1**。

- 路由器工作在网络层（第 3 层），能连接不同网络；
- 交换机工作在数据链路层（第 2 层），识别 MAC 地址与帧；
- 集线器只是多端口中继器，工作在物理层（第 1 层），不识别帧、不识别地址。

变体考法：问「Hub、Switch、Router 依次工作在 OSI 的第几层」，答案相同；问「中继器/集线器属于哪一层」，答物理层。

本章小结表

考纲条目	核心内容	必须记住的结论
计算机网络的定义	地理位置不同、具有独立功能的多台计算机通过通信线路与设备连接，在协议管理下实现资源共享与信息传递	三个限定词：自治、互连、计算机集合
internet 与 Internet	前者泛指互连网（可用任意协议），后者特指全球互联网（必用 TCP/IP）	大小写不同，含义不同
组成（物理）	硬件、软件、协议	协议是核心

考纲条目	核心内容	必须记住的结论
组成（工作方式）	边缘部分（主机，C/S 与 P2P）、核心部分（网络与路由器）	核心部分只提供连通性与交换
组成（功能）	通信子网（下三层，管传输）、资源子网（上四层，管处理）	切分点在传输层：传输层属资源子网
功能	数据通信、资源共享、分布式处理、提高可靠性	考纲四条，负载均衡可归入分布式处理
分类（分布范围）	PAN / LAN / MAN / WAN	LAN 与 WAN 区别在技术，不在范围
分类（使用者）	公用网 / 专用网	专用网可租用公用网线路（VPN）
分类（交换技术）	电路交换 / 报文交换 / 分组交换	分组交换可流水线；分组交换分数据报与虚电路
分类（拓扑）	星形 / 总线 / 环形 / 网状	星形中心是瓶颈；总线冲突重；环形扩充难；网状可靠
速率	数据率、比特率，单位 bit/s	$k = 10^3$, $M = 10^6$（十进制）
带宽	模拟信号：频带宽度（Hz）；计网：最高数据率（bit/s）	看单位判断是哪种含义
吞吐量	单位时间通过某网络/信道/接口的实际数据量	最大值 $\leq$ 路径上最小带宽（木桶效应）
发送时延	数据长度（bit）÷ 发送速率（bit/s）	与链路长度无关
传播时延	信道长度（m）÷ 电磁波传播速率（m/s）	与带宽、数据量无关，提高带宽消不掉
处理时延	分析首部、差错检验、查转发表	由结点处理能力决定
排队时延	输入队列排队 + 输出队列排队	由网络负载决定，最不确定
时延带宽积	传播时延 × 带宽，单位 **bit**	又称「以比特为单位的链路长度」「管道容量」
RTT	从发出数据分组到收到确认分组的总时间	有效数据率 ≈ 数据长度 / RTT
利用率	信道利用率、网络利用率	$D = \frac{D_0}{1-U}$，利用率不是越大越好
分层结构	各层独立、灵活、易实现、促标准化	体系结构 = 层次 + 协议 + 接口，不含实现细节
协议三要素	语法、语义、同步（时序）	语法管格式、语义管动作、同步管次序
协议 / 接口 / 服务	协议对等（水平），服务下层给上层（垂直），SAP 是接口接入点	协议对上层透明；同一服务可由不同协议实现
面向连接 / 无连接	三阶段 vs 无连接；TCP/虚电路 vs UDP/IP	面向连接 $\neq$ 可靠
SDU 与 PDU	n-PDU $= n$-PCI $+ n$-SDU； n-SDU $= (n+1)$-PDU	PDU 水平用，SDU 垂直用
OSI 七层	物、链、网、输、会、示、用	传输层是自下而上第一个端到端的层次
TCP/IP 四层	网络接口层、网际层、传输层、应用层	网际层只有无连接、不可靠的数据报服务
五层教学模型	物理、数据链路、网络、传输、应用	下四层对应 OSI，上三层合并成应用层

考纲条目	核心内容	必须记住的结论
封装与解封装	报文 → 报文段 → 分组 → 帧 → 比特	每层加自己的首部（链路层还加帧尾）
层次归属	集线器 1 层、交换机 2 层、路由器 3 层	2016-33 答案 3、2、1

易错清单

1. 「数据通信」是计算机网络最基本的功能，「资源共享」是最有吸引力的功能。不要把「提高可靠性」说成最基本的功能。
2. 分时系统中的终端不是自治的，因此「主机 + 若干终端」不构成计算机网络。
3. 通信子网对应 **OSI** 下三层，资源子网对应上四层，切分点在传输层。把传输层划入通信子网是错的。
4. **LAN** 与 **WAN** 的本质区别是技术（广播式 **vs** 点对点、共享介质 **vs** 交换），不是地理范围。只讲范围的选项是干扰项。
5. 报文交换不能流水线，必须收完整个报文才能转发；分组交换可以流水线，这是分组交换时延更小的根本原因。
6. 电路交换不具备差错控制能力（数据透明传输，中间结点不处理），不要把它说成「可靠传输」。
7. 虚电路不是电路交换。电路交换预分配带宽、独占物理线路；虚电路是多路复用、逻辑连接、共享带宽，仍需存储转发。虚电路服务是 **OSI** 特有的，**TCP/IP** 网络层没有。
8. 速率中的 $k = 1000$、$M = 10^6$ 是十进制，存储容量中的 $1\textbf{KB} = 2^{10}\textbf{B}$ 是二进制。网络计算题以题干给定的换算为准。
9. 带宽有 **Hz** 和 **bit/s** 两种含义，单位是 Hz 时是频带宽度，单位是 bit/s 时是最高数据率，代公式前先确认。
10. 发送时延与链路长度无关，传播时延与带宽无关。「把带宽提高一倍，传播时延减半」是错的。
11. 总时延不是「每个分组的时延 × 分组数」。分组是流水发送的，m 个分组、n 个路由器时总时延为 $(m+n)t_{\text{发送}} + (n+1)d_{\text{传播}}$。
12. 求最大吞吐量必须用木桶效应，且不能忽略源端和目的端的接入链路——中间并行路径的聚合带宽再大也不能突破两端接入瓶颈（2024-33 的陷阱）。
13. 不要自行脑补「分流、负载均衡、多路径」，除非题目明确写出相关条件。绕远路只会增加转发时延，不会缩短总时间。
14. 时延带宽积的单位是比特（**bit**），不是秒、不是 bit/s。它表示的是「链路上同时能容纳的比特数」。
15. $\dfrac{D_0}{1-U}$ 中的 D_0 是网络空闲时的时延，不是发送时延，也不是「最小网络时延」的任意口径；利用率越大时延增长越快，利用率并非越大越好。
16. 「服务是水平的、协议是垂直的」是错的，正确说法是协议水平、服务垂直。
17. 协议对上层是透明的：上层只看得见服务，看不见协议。把这句话反过来（「服务对协议透明」）就是错的。
18. 协议的三要素是语法、语义、同步（时序），不包括「分层」「接口」「服务」；而「协议的内部实现细节」不属于网络体系结构描述的内容（2010-33）。
19. **PDU** 是对等层之间传送的数据单元，**SDU** 是同一系统相邻层之间交换的数据单元，两者的适用方向不能互换。

20. 传输层的 **PDU** 叫「报文段」，应用层的 **PDU** 叫「报文」，两者不能混用；网络层 PDU 叫「分组 / 数据报」。
21. 面向连接不等于可靠，无连接也不等于不可靠。连接管理方式与可靠性是两个独立属性，不要用前者推后者。
22. 传输层是自下而上第一个提供端到端服务的层次（2009-33）；网络层是「主机到主机」，传输层才是「进程到进程」。
23. 相邻结点间的流量控制属于数据链路层（2022-33），端到端的流量控制属于传输层；物理层只传比特。
24. **TCP/IP** 的网际层只提供无连接、不可靠的数据报服务（2011-33）；需要可靠传输时由上层 TCP 负责，不要写成「网络层提供可靠传输」。
25. **OSI** 是法律上的国际标准，**TCP/IP** 是事实上的国际标准；且 OSI「先有模型后有协议」，TCP/IP 相反，两者不要颠倒。
26. 封装效率计算中，物理层与应用层不计开销（2017-33 的 OSI 七层里只有 5 层加 20 B）；题干说「无拆分」时，同一层首部不能重复计算。
27. 设备的最高工作层次：集线器 **1**、交换机 **2**、路由器 **3**（2016-33）。集线器是多端口中继器，不识别帧，更不识别地址。
28. **ARP** 的层次归属有教材差异，统考一般按网络层理解；遇到与 IP、ICMP、IGMP 并列的选项时按网络层选，并以题干口径为准。

第二章物理层

本章考纲要求

第四部分计算机网络二、物理层（一）通信基础

1. 信道、信号、带宽、码元、波特、速率、信源与信宿等基本概念
2. 奈奎斯特定理与香农定理
3. 编码与调制
4. 电路交换、报文交换与分组交换
5. 数据报与虚电路

（二）传输介质

1. 双绞线、同轴电缆、光纤与无线传输介质
2. 物理层接口的特性

（三）物理层设备

1. 中继器
2. 集线器

物理层处于五层模型的最底层。它要解决的问题只有一句话：怎样在连接各种计算机的传输介质上，透明地传输比特流。

理解这一章必须抓住两个「不关心」：

- 物理层不关心传输介质具体是什么（双绞线、同轴电缆、光纤、无线电波都可以）。考纲把「传输介质」列在物理层之下，是因为介质是物理层的服务对象，但物理层规范本身描述的是接口，不是介质。
- 物理层不关心比特流代表什么含义。物理层对数据链路层「透明」——数据链路层看不见物理层究竟用什么方法把这串比特搬过去。

因此本章的命题重心非常集中，可以分成四块：

1. 定量计算（本章最大的得分区）：奈氏准则与香农定理、波特率与比特率的换算、PCM 数据率、分组交换时延。这四类每年必考其中一到两类，且都是「套公式就能算对、套错就必错」的题。
2. 编码波形的判读：给出波形判断编码类型，或者给出编码类型画出/读出比特串。曼彻斯特与差分曼彻斯特是绝对主力。
3. 文字辨析：三种交换方式的对比、数据报与虚电路的区别、物理层接口四种特性、中继器与集线器的性质。
4. 概念边界：带宽的两种含义、速率与传播速度的区别、模拟信号与数字信号的区分。

下面按考纲条目顺序逐个展开。凡是带公式的考点，本文都会给出推导、符号含义、单位、以及一个带数字的完整算例，考生可以自己算一遍核对。

一、信道、信号、带宽、码元、波特、速率、信源与信宿

（一）通信系统的基本模型

一个最简单的通信系统由五部分组成，源点和终点是设备，发送器和接收器是变换环节，信道是传输媒介：

环节	英文	作用	典型设备
源点（信源）	Source	产生要传输的数据，输出数字比特流	主机、麦克风
发送器	Transmitter	把数据编码/调制成适合信道传输的信号	调制器
信道	Channel	信号传输的媒介（一个方向一个信道）	双绞线、光纤、自由空间
接收器	Receiver	把收到的信号还原成数字比特流	解调器
终点（信宿）	Destination	接收并使用还原后的比特流	主机、扬声器

传输方向是单向的：源点 → 发送器 → 信道 → 接收器 → 终点。

- 源点（**Source**）：也叫信源，产生要传输的数据，输出的是数字比特流。例如一台主机、一个麦克风。
- 发送器（**Transmitter**）：把源点产生的比特流编码或调制成适合在信道上传输的信号。典型发送器就是调制器。
- 信道（**Channel**）：信号的传输媒介，可以看成一条线路的逻辑部件。一个方向上的传输构成一个信道，因此同一条物理传输介质上可以建立多条信道。
- 接收器（**Receiver**）：把从信道收到的信号还原成目的设备能处理的数字比特流。典型接收器就是解调器。
- 终点（**Destination**）：也叫信宿，从接收器取得比特流并进行处理。

信源与信宿的定义要点

信源 = 源点 = 产生并发送数据的一方；信宿 = 终点 = 接收并使用数据的一方。二者都是端设备（主机），不是中间设备。题目里问「发送器 / 调制器属于哪一部分」时，答案是发送器，不是信源——这是常见偷换。

（二）通信的三种方式

按信号传播方向与时间的关系，信道可分为三类：

通信方式	别称	信号方向	需要的信道数	典型例子
单工通信	单向通信	只能 A → B，不能反向	1 个	无线电广播、电视广播
半双工通信	双向交替通信	双方都能收发，但同一时刻只能一方发送	2 个	对讲机
全双工通信	双向同时通信	双方可以同时发送和接收	2 个	电话、以太网链路

「需要几个信道」与「需要几条介质」不是一回事

- 半双工：需要 **2** 个信道，但可以共用同一条物理介质（分时使用）。
- 全双工：需要 **2** 个信道，一般需要两条物理介质（或者用不同频段/波长在同一条光纤上实现）。

很多资料把「半双工只有一条介质」简化成「只需要一个信道」，这是错的：只要允许双向通信，方向就是两个，就是两个信道。统考按「半双工 2 信道共用 1 介质、全双工 2 信道 2 介质」理解。

（三）信号：模拟信号与数字信号

信号（**Signal**）是数据的电磁波或光脉冲表现形式。

- 模拟信号（**Analog Signal**）：信号的参量（幅度、频率、相位）连续取值，取值有无穷多个。例如电话线里的话音电流、传统的调幅广播。
- 数字信号（**Digital Signal**）：信号的参量只取有限个离散值，通常用两种电平表示二进制的 0 和 1。例如计算机网卡输出的方波。

对应的两类数据：

- 模拟数据：连续变化的物理量，如话音、温度、图像。
- 数字数据：离散取值的量，如计算机里的比特流。

数据与信号可以交叉组合，得到考纲要求的四种转换，这是本章后续所有编码/调制内容的总框架：

数据 → 信号	转换设备	采用的技术	典型场景
数字数据 → 数字信号	数字发送器（数字放大器）	编码：NRZ、NRZI、RZ、曼彻斯特、差分曼彻斯特、4B/5B	主机与网卡、局域网内部
数字数据 → 模拟信号	调制器（Modem）	调制：ASK、FSK、PSK、QAM	主机到调制解调器
模拟数据 → 数字信号	编码器（PCM 编解码器）	采样、量化、编码（PCM）	音频数字化、话音编码
模拟数据 → 模拟信号	放大器、调制器	频分复用（FDM）、放大调制	电话机到本地端局

编码与调制的一句话区分

编码（**Encoding**）= 把数据变换成数字信号；调制（**Modulation**）= 把数据变换成模拟信号。判题时先看题干给出的目标信号是数字还是模拟，再决定用「编码」还是「调制」的词。

（四）基带信号与带通信号，基带传输与频带传输

- 基带信号（**Baseband Signal**）：来自信源的、没有经过调制的原始电信号，其频谱从零频（直流）附近开始，占据从 0 到某个截止频率的一段低通频带。计算机内部、网卡输出的方波就是基带数字信号。
- 带通信号（**Bandpass Signal**）：把基带信号调制到某一载波频率附近后形成的信号，其频谱集中在一个以载波频率为中心的有限频带内，不包含直流分量。
- 基带传输：把数字数据编码成数字信号后直接在信道上传输，不经过载波调制。例如 10Base5 同轴电缆、以太网双绞线上的曼彻斯特码。
- 频带传输（又称宽带传输）：把数字数据调制到载波上变成模拟信号再传输。例如 ADSL、有线电视网络。

名称里的字面线索

以太网标准名「10BaseT / 100BaseT」里，`Base` 就是 **Baseband**（基带），`T` 是 **Twisted pair**（双绞线），`100` 是速率 100 Mb/s。而 `100BaseFX` 的 `F` 是光纤。看到 `Base` 就能断定用的是基带传输，这是判断「某种以太网用什么介质、什么传输方式」的快速入口。

（五）带宽的两种含义

带宽（**Bandwidth**）在不同语境下含义不同，这是概念题的高频陷阱：

语境	含义	单位	说明
信号 / 信道（模拟口径）	该信号或信道能通过的频率范围，即最高频率与最低频率之差	Hz	例如话音信道 300 ～ 3400 Hz，带宽 3100 Hz；奈氏准则、香农定理里的 W 就是这个
网络（数字口径）	单位时间内从网络某一点到另一点所能通过的最高数据率	b/s（bit/s）	例如「该链路带宽为 100 Mb/s」

Hz 与 b/s 不能混用

在奈氏准则与香农定理的公式里，W 的单位必须是 **Hz**，答案 C 的单位是 b/s。见到题干写「带宽 3 kHz」，代进 W 的是 3×10^3 Hz；见到「带宽 100 Mb/s」，那就不是 W，不能往定理里代，它描述的是链路的发送能力。

（六）码元、波特与速率

码元（**Symbol**，又称信号码元）：一个固定时长的信号波形，用来表示一个 N 进制数。这个固定时长叫码元宽度，码元是数字通信中承载信息的基本单位。

一个码元可以取多少种不同的状态，取决于编码/调制方式用了多少种可区分的波形。设码元的离散状态数为 N：

- 若 $N=2$，一个码元就是「一个二进制位」，码元与比特一一对应；
- 若 $N=4$，一个码元可以表示 4 种情况，恰好对应 2 个二进制位；
- 一般地，N 必须是 2 的整数次幂（或取最接近的较大值），一个码元携带的比特数为 $\log_2 N$。

码元与比特的关系（必背）

$$k=\log_2 N \quad \text{(bit/码元)}$$

其中 N 是一个码元可取的离散状态数（电平种类数、相位种类数、频率种类数，或它们的组合数）。

状态数 N 必须取 **2** 的幂。如果题目给出「3 种幅值」这种非 2 的幂，实际一个码元只能可靠区分 $2^{\lfloor\log_2 3\rfloor}=2$ 种状态，即携带 1 bit——统考一般不会这样出，但知道了不吃亏。

波特（**Baud**）与波特率：

- 码元传输速率（又称调制速率、波特率、符号率）：单位时间内数字通信系统传输的码元个数（脉冲个数或信号变化次数），单位是波特（**Baud**），记作 Baud 或 B。
- 信息传输速率（又称比特率、数据传输速率、速率）：单位时间内数字通信系统传输的比特数，单位是比特/秒（**b/s**）。

两者由码元携带的比特数 k 联系起来：

$$R_b=R_B\times k=R_B\times\log_2 N$$

其中 R_b 是比特率（b/s），R_B 是波特率（Baud），$k = \log_2 N$ 是每个码元携带的比特数。

反之：

$$R_B = \frac{R_b}{\log_2 N}$$

三条换算结论，考场直接用

1. $R_b = R_B \log_2 N$；$N = 2$ 时才相等（$R_b = R_B$）。
2. 波特率是每秒码元数，比特率是每秒比特数，两者只在每码元恰好 **1 bit** 时数值相等。
3. 曼彻斯特、差分曼彻斯特编码中，每个比特需要用 **2** 个码元（两个半位）表示，因此 $R_B = 2R_b$，即波特率是比特率的 **2** 倍。

算例 **1**（比特率换算）：某通信链路的数据传输速率为 2400 b/s，采用 4 相位调制，求波特率。

4 相位表示码元有 $N = 4$ 种状态，每个码元携带 $\log_2 4 = 2$ bit，因此

$$R_B = \frac{2400}{2} = 1200 \text{ Baud}$$

算例 **2**（反向验证）：某信道的波特率为 2400 Baud，每个码元携带 3 bit，求信息传输速率。

$$R_b = 2400 \times 3 = 7200 \text{ b/s}$$

算例 **3**（状态数）：一个码元有 16 种离散状态，求一个码元可表示的比特数。

$$\log_2 16 = 4 \text{ bit}$$

（七）速率与传播速度的区别

这是 2014-35 直接考过的辨析：「下列因素中，不会影响信道数据传输速率的是——信噪比、频率带宽、调制速度、信号传播速度」，答案是信号传播速度。

- 传输速率（数据传输速率）：每秒能发送多少比特，由带宽、信噪比、调制所能区分的状态数、发送速率共同决定。它只关乎「发送端多快能把比特推到线路上」。
- 传播速度（信号传播速度）：电磁波或光在介质中每秒前进多少米，决定传播时延：

$$T_p = \frac{\text{信道长度}}{\text{信号传播速度}}$$

它只影响「比特从一端跑到另一端要多久」，不改变信道每秒能发出多少比特。

传播时延 ≠ 发送时延

- 发送时延 $T_t = \dfrac{\text{分组长度 (b)}}{\text{发送速率 (b/s)}}$，取决于分组大小与链路速率。
- 传播时延 $T_p = \dfrac{\text{信道长度 (m)}}{\text{传播速率 (m/s)}}$，取决于距离与介质，与带宽无关。

  光纤中光速约为 2×10^8 m/s，铜缆中约为 2.3×10^8 m/s。「提高带宽能降低传播时延」是错的，带宽再大也不能让信号跑得更快。

本章真题分布（据 **2009—2024** 年统考选择题）

- 奈氏/香农与调制状态数：2009-34、2014-35、2016-34、2017-34、2022-34、2023-34。
- 波特率与比特率换算：2011-34。
- 编码波形判读：2013-34（曼彻斯特）、2015-34（NRZ 与曼彻斯特）、2021-34（差分曼彻斯特）。
- 分组交换时延与吞吐量：2010-34、2013-35、2023-33、2024-33。
- 数字调制方式：2024-34（FSK）。
- 数据报与虚电路：2020-34。
- 传输介质：2019-34。物理层接口特性：2012-34、2018-34。

二、奈奎斯特定理与香农定理

这两个定理回答同一个问题：一条信道的极限数据传输速率是多少？区别在于有没有考虑噪声。这是本章最核心、分值最稳定的考点。

（一）为什么码元发得太快就会出错

信道中存在各种干扰（噪声）。如果发送端单位时间内发送的码元个数太多，即码元宽度太窄，那么每个码元在接收端造成的波形会「拖尾」并侵入相邻码元的时段，接收端就分不清相邻码元的界限，这种现象叫码间串扰（**Inter-Symbol Interference，ISI**）。

于是有两条互不相同的上限：

- 码元传输速率有上限——发得太快就会码间串扰。这条上限由奈氏准则给出，与噪声无关，只与带宽有关。
- 信息传输速率有上限——即使码元速率没超限，噪声也会让接收端分辨不清码元处在哪个状态。这条上限由香农定理给出。

> **奈氏准则限的是「码元速率」，不是「比特率」**
>
> 奈奎斯特定理没有直接限制数据传输速率的上限；它限制的是码元传输速率的上限为 $2W$ **Baud**。要想提高比特率，只能让每个码元携带更多比特（增大 N），而 N 又受噪声限制——这时就要用香农定理。这句话是 2017-34 的解题核心。

（二）奈奎斯特定理（奈氏准则）

内容：在理想低通信道（无噪声、带宽有限）中，为了避免码间串扰，极限码元传输速率为

$$R_{B,\max} = 2W \quad \text{(Baud)}$$

最大数据传输速率为

$$C_{\max} = 2W \log_2 N \quad \text{(b/s)}$$

符号含义与单位：

符号	含义	单位	说明
W	信道的带宽（频率范围宽度）	Hz	必须是频率口径的带宽
N	一个码元的离散状态数（电平/相位/频率/振幅组合的种类数）	无单位	由调制方式决定，$N = 2^k$
$\log_2 N$	每个码元携带的比特数	bit/码元	$N = 2$ 时为 1

符号	含义	单位	说明
$R_{B,\max}$	极限码元传输速率	Baud	等于 $2W$
$C_{\max}$	极限（最大）数据传输速率	b/s	奈氏条件下能达到的上限

使用前提（三个条件缺一不可）：① 理想低通信道；② 无噪声；③ W 用 **Hz** 计。

奈氏准则的三个常见坑

1. W 是 **Hz**，不是 **b/s**。把「链路速率 100 Mb/s」当成 W 代进去，必然全错。
2. N 是「状态数的乘积」而不是「相加」。「采用 4 个相位，每个相位有 4 种振幅」的 QAM，$N = 4 \times 4 = 16$，不是 $4 + 4 = 8$。2009-34 的易错点正是这里。
3. N 不是码元数，是状态数。不要把「每秒钟发多少个码元」与「一个码元有几种状态」搞混。

算例 1（**2009-34** 原题）：在无噪声情况下，某链路带宽 3 kHz，采用 4 个相位、每个相位 4 种振幅的 QAM 调制，求最大数据传输速率。

状态数 $N = 4 \times 4 = 16$，每个码元携带 $\log_2 16 = 4$ bit：

$$C_{\max} = 2 \times 3 \times 10^3 \times 4 = 24 \times 10^3 \text{ b/s} = 24 \text{ kb/s}$$

算例 2（**2022-34** 原题）：带宽 200 kHz 的无噪声信道，采用 4 个幅值的 ASK 调制，求最大数据传输速率。

$N = 4$，$\log_2 4 = 2$ bit：

$$C_{\max} = 2 \times 200 \times 10^3 \times 2 = 800 \times 10^3 \text{ b/s} = 800 \text{ kb/s}$$

算例 3（反求状态数，**2023-34** 原题）：某无噪声理想信道带宽 4 MHz，采用 QAM 调制，最大数据传输率是 48 Mb/s，求采用的 QAM 方案。

由 $C_{\max} = 2W \log_2 N$ 得

$$48 \times 10^6 = 2 \times 4 \times 10^6 \times \log_2 N \implies \log_2 N = 6 \implies N = 2^6 = 64$$

即 **QAM-64**，每个码元携带 6 bit。

算例 4（反求码元速率）：带宽 4 kHz 的无噪声信道，采用 8 电平 PAM 调制，求极限码元速率与数据率。

$$R_{B,\max} = 2W = 2 \times 4 \times 10^3 = 8000 \text{ Baud}, \qquad R_b = 8000 \times \log_2 8 = 8000 \times 3 = 24 \text{ kb/s}$$

（三）香农定理

内容：在带宽受限且有高斯白噪声干扰的信道中，信道的极限数据传输速率（信道容量）为

$$C = W \log_2\left(1 + \frac{S}{N}\right) \quad \text{(b/s)}$$

符号含义与单位：

符号	含义	单位	说明
W	信道带宽	Hz	频率口径
S	信道内所传信号的平均功率	W	—
N	信道内部的高斯噪声功率	W	注意这个 N 与奈氏准则里的状态数 N 是两个完全不同的量
S/N	信噪比（signal-to-noise ratio）	无单位（倍数）	必须换算成纯比值再代入
C	信道的极限数据传输速率（信道容量）	b/s	有噪声时的上限

信噪比的两种表示与换算（必考）：

- 比值形式：S/N，无单位，例如 $S/N = 1000$。
- 分贝形式：单位为 dB，定义为

$$\text{信噪比 (dB)} = 10\log_{10}\left(\frac{S}{N}\right)$$

反过来，由 dB 求倍数：

$$\frac{S}{N} = 10^{\frac{\text{dB}}{10}}$$

常用换算建议直接背下来：

dB	**10**	**20**	**30**	**40**
S/N	10	100	1000	10000

香农定理的四条要点

1. 公式里的 N 是噪声功率，与奈氏准则的状态数 N 同名不同义，写题时最好改记成 S/N 整体。
2. C 的单位是 b/s，且 C 与码元状态数 N 无关——香农定理不关心你用什么调制方式，它给出的是物理上限。
3. W 越大、S/N 越大，C 越大；但 C 随 S/N 是对数增长，靠加大功率提升速率收益极低。
4. 香农定理给出的上限不能突破。任何实际的编码与调制方案，其数据传输速率都只能在 C 以下（含）实现。

两个定理的适用条件绝不能互换

- 无噪声、理想低通信道 → 用奈氏：$C_{\max} = 2W\log_2 N$。题干里出现「无噪声」「理想信道」就锁定它。
- 有噪声信道 → 用香农：$C = W\log_2(1 + S/N)$。题干里给出「信噪比」「噪声」，或者问「极限速率」同时给了信噪比，就锁定它。
- 同时给出噪声和调制方式怎么办？两个都算，取小者作为实际可达到的最大可靠速率。这是「综合题」的标准套路。
- 一句话记牢：奈氏管码元速率上限（不考虑噪声），香农管信息速率上限（考虑噪声）。

算例 **5**（**2016-34** 原题）：某链路带宽 8 kHz，信噪比 30 dB，实际数据传输速率约为理论最大值的 50%，求实际速率。

先把 dB 换成倍数：$S/N = 10^{30/10} = 1000$。代入香农定理：

$$C = 8 \times 10^3 \times \log_2(1 + 1000) = 8 \times 10^3 \times \log_2 1001 \approx 8 \times 10^3 \times 9.97 \approx 79.7 \text{ kb/s}$$

取 50%：

$$79.7 \times 0.5 \approx 39.85 \text{ kb/s} \approx 40 \text{ kb/s}$$

算例 **6**（**2017-34** 原题）：若信道在无噪声情况下的极限数据传输速率不小于信噪比为 30 dB 条件下的极限数据传输速率，求信号状态数至少是多少。

香农上限（带宽记为 W）：

$$C_S = W \log_2(1 + 1000) = W \log_2 1001 \approx 9.97W$$

要求奈氏上限不小于它：

$$2W \log_2 N \ \geq \ 9.97W \implies \log_2 N \ \geq \ 4.985 \implies N \ \geq \ 2^{4.985} \approx 31.7$$

可选状态数中最小的是 **32**。

算例 **7**（反求信噪比）：某信道频率范围为 2 kHz 到 10 kHz，要求按香农定理算出的极限速率不低于 40 kb/s，求信噪比至少是多少。

带宽 $W = 10 - 2 = 8$ kHz（注意：给频率范围要先相减得带宽）：

$$40 \times 10^3 = 8 \times 10^3 \times \log_2(1 + S/N) \implies \log_2(1 + S/N) = 5 \implies 1 + S/N = 2^5 = 32$$

$$\frac{S}{N} = 31$$

算例 **8**（反求带宽）：信噪比 20 dB，要求信道容量达到 20 kb/s，求所需带宽至少是多少。

$S/N = 10^{20/10} = 100$，$\log_2(1 + 100) \approx 6.66$：

$$20 \times 10^3 = W \times 6.66 \implies W \approx 3003 \text{ Hz} \approx 3 \text{ kHz}$$

算例 **9**（两定理同时出现，取小者）：某信道带宽 4 kHz，所受高斯噪声使信噪比为 15，若采用 16 电平的调制方式，求该信道实际能达到的最大数据传输速率。

- 奈氏上限：$C_{\max} = 2 \times 4000 \times \log_2 16 = 8000 \times 4 = 32$ kb/s
- 香农上限：$C = 4000 \times \log_2(1 + 15) = 4000 \times 4 = 16$ kb/s

实际最大可靠速率受更紧的那个约束决定，取两者较小值：

$$R_{\max} = \min(32, 16) = 16 \text{ kb/s}$$

（四）两个定理的综合对比

对比项	奈奎斯特定理（奈氏准则）	香农定理
考虑噪声	不考虑（理想低通信道，无噪声）	考虑高斯白噪声
限制对象	码元传输速率上限 $2W$ Baud	信息传输速率上限
公式	$C_{\max} = 2W \log_2 N$	$C = W \log_2(1 + S/N)$
影响因素	带宽 W、码元状态数 N	带宽 W、信噪比 S/N
与调制方式的关系	有关（N 由调制方式决定）	无关（不给 N 留位置）
与信噪比的关系	无关	直接相关，对数增长
核心用途	求「无噪声时能达到多快」、反求 N	求「有噪声时的物理上限」、反求 W 或 S/N
典型真题	2009-34、2017-34、2022-34、2023-34	2016-34、2017-34

计算题的四种提问方式

1. 给 W 和调制方式，求最大速率 → 奈氏。注意 N 的乘法组合。
2. 给 W 和 **dB**，求最大速率 → 香农。先 $10^{\mathrm{dB}/10}$ 换算。
3. 给速率和 W（或给信噪比），求 N 或带宽 → 反向套公式，注意取满足不等式的最小 2 的幂。
4. 给「实际速率是理论值的 x%」 → 先算理论值再乘比例，别忘了这一步（2016-34 就考了 50%）。

三、编码与调制

（一）数字数据编码为数字信号

编码（**Encoding**）把数字数据变换成数字信号，得到的通常是基带信号。这一类编码共有六种要掌握，核心是记住每种编码的跳变规则、自同步能力、以及波特率与数据率的关系。

*1.*归零编码（*RZ*）

- 规则：用高电平表示 **1**、低电平表示 **0**（或相反），并且每个码元的中间时刻信号都回归到零电平。
- 接收方只要在信号归零之后的时刻采样即可判断电平，因此自带同步机制，称为自同步信号。
- 代价：每个码元都有一段「归零」时间不携带信息，大部分带宽被浪费用于传输归零，编码效率低。
- 波特率与数据率：一个码元携带 1 bit，但需在 1 个码元时隙内完成「有电平 + 归零」两个阶段，因此波特率是数据率的 **2** 倍（带宽效率约 0.5 bit/Hz 量级）。

*2.*不归零编码（*NRZ*）

- 规则：整个码元时间内的电平保持不变，用高低电平直接表示 1 和 0（例如高电平 =1、低电平 =0），中间不归零。
- 优点：一个时钟周期全部用来传数据，编码效率最高，波特率等于数据率（每码元 1 bit）。
- 缺点：没有自同步能力。如果连续出现多个相同的比特（例如一长串 0），电平长期不变，接收方无法判断每个比特的边界在哪里，收发双方必须另接一根时钟线。
- 统考波形题里，NRZ 的特征是「连续相同比特时电平保持不动」。

*3.*反向不归零编码（*NRZI*）

- 规则：用电平的跳变表示 **0**，用电平保持不变表示 **1**（统考与多数教材口径；也有教材定义相反，以题目给出的约定为准）。整个码元时间内不归零。
- 特点：跳变本身是一种通知机制，它把时钟信息嵌进了数据里。因此 NRZI 既能传输时钟信号实现自同步，又尽量不损失系统带宽，是 RZ 与 NRZ 优点的折中。
- 应用：USB 2.0 就采用 NRZI 编码。
- 波特率与数据率：每码元 1 bit，波特率 = 数据率。

*4.*曼彻斯特编码（*Manchester*）

- 规则：每个码元的中间必定发生一次电平跳变，这次跳变既充当时钟信号（用于接收方同步），又表示数据。
- 数据含义由跳变方向决定。两种约定并存，统考按题图约定判断：

约定	位中间由低 → 高	位中间由高 → 低
IEEE 802.3 常用约定	表示 0	表示 1
相反约定（部分教材）	表示 1	表示 0

- 关键的波形描述：每个比特周期被中点跳变劈成两个相等的半位，前半位与后半位电平必定相反。因此曼彻斯特波形中绝不可能出现连续两个相同电平跨越一个完整比特周期的情况。
- 优点：自同步（跳变频繁，时钟容易提取）；抗干扰能力强，因为数据由跳变方向而非绝对电平决定，直流分量小。
- 代价：每个比特需要 **2** 个码元，波特率是数据率的 **2** 倍，编码效率只有 **0.5 bit/Hz**，所需带宽是 **NRZ** 的 **2** 倍。
- 应用：传统 10BaseT 以太网（双绞线以太网）就采用曼彻斯特编码。

*5.*差分曼彻斯特编码（*Differential Manchester*）

- 规则：每个码元的中间仍然必定发生一次电平跳变，但这个中间跳变只表示时钟，不表示数据。
- 数据由每个码元开始处是否发生跳变表示：

位起始处	表示的数据
有跳变	0
无跳变（电平延续上一位的后半位）	1

- 波形描述与判读步骤（这是解题的固定动作）：
  1. 先用题目给的比特周期虚线（或码元宽度）把波形切成若干段；
  2. 观察每一段的中点：必然有一次跳变（用来对齐时钟）；
  3. 观察每一段的起始边界：与上一段末尾电平比较，变了就是 **0**，没变就是 **1**；
  4. 逐段编号即得比特串。
- 优点：既有曼彻斯特的全部自同步与抗干扰优点，又因为数据由「是否跳变」的相对关系决定，抗干扰能力比曼彻斯特更强（极性反转不影响判读）。
- 波特率与数据率：与曼彻斯特相同，波特率 = **2** × 数据率。

- 应用：IEEE 802.5 令牌环网。

曼彻斯特 vs 差分曼彻斯特，只看一个位置

- 曼彻斯特：看位中间的跳变方向（上跳还是下跳）。
- 差分曼彻斯特：看位开始处是否跳变（跳 = 0，不跳 = 1）。

两者位中间都必变。所以看到「位中间必变」不能立刻断定是曼彻斯特——还要看它是否复用位起始处的跳变来表示数据。2015-34 的判断依据就是：编码 1 全周期电平不动 → NRZ；编码 2 位中间必变且用方向表示数据 → 曼彻斯特。

6.4B/5B 编码

- 规则：把每 **4 bit** 数据映射为 **5 bit** 的码组再发送。5 bit 共 32 种组合，从中挑选 16 种「跳变足够多」的码组用于表示 16 种 4 bit 数据，其余码组作控制码或禁用（保证不会出现长串连续相同电平）。
- 目的：保证信号中有足够的电平跳变，便于接收方提取时钟（解决 NRZ 类编码的同步问题），同时把编码开销控制在可接受范围。
- 代价：每 4 bit 数据要发 5 bit，额外开销 **25%**，即

$$R_B = R_b \times \frac{5}{4} = 1.25 R_b$$

- 应用：100BaseFX 快速以太网。100 Mb/s 的数据率对应 $1.25 \times 10^8 = 125$ MBaud 的线路波特率。

*7.*六种编码对比汇总

编码	位中间是否必变	数据如何表示	自同步	每比特码元数	波特率：数据率	主要缺点
归零 RZ	是（归零）	高电平 =1，低电平 =0	有	2	2 : 1	浪费带宽，效率低
不归零 NRZ	否	高电平 =1，低电平 =0	无	1	1 : 1	需另设时钟线
反向不归零 NRZI	否	跳变 **=0**，不变 **=1**	有	1	1 : 1	仍可能出现长串不变
曼彻斯特	是	位中间跳变方向	有	2	2 : 1	带宽开销加倍
差分曼彻斯特	是	位起始处是否跳变（跳 =0，不跳 =1）	有（更强）	2	2 : 1	带宽开销加倍
4B/5B	设计保证有	查表映射	有	1.25	5 : 4	25% 额外开销

编码类真题的三问

统考在这一块的出题形式固定为两种：

1. 给波形判编码类型：看位中间动不动 → 动则排除 NRZ/NRZI；看数据是由「绝对电平」还是「跳变方向/有无跳变」决定 → 区分曼彻斯特、差分曼彻斯特、NRZI。2015-34 就是这个套路。
2. 给编码类型读比特串：2013-34（10BaseT 曼彻斯特，答案 `00110110`）、2021-34（差分曼彻斯特，答案 `10111001`）都是此类。解题时务必先按虚线切分比特周期，再做判断。

（二）数字数据调制为模拟信号

调制（**Modulation**）把数字数据变换成模拟信号，使用载波（**Carrier**）的三个参量之一或多个来携带信息：幅度、频率、相位。对应的三种基本调制方式如下。

幅移键控（**Amplitude Shift Keying**，**ASK**）：改变载波的幅度表示 0 和 1（有载波为 1、无载波为 0，或两个不同幅度）。实现简单，但抗干扰能力差，受幅度噪声影响大。

频移键控（**Frequency Shift Keying**，**FSK**）：改变载波的频率表示 0 和 1，需要两个不同频率的载波。抗干扰能力强于 ASK，但占用带宽较大。

相移键控（**Phase Shift Keying**，**PSK**）：改变载波的相位表示数据。分为：

- 绝对相移（**PSK**）：用相位的绝对值表示数据；
- 相对相移（**DPSK**，差分相移键控）：用相邻码元的相位差表示数据，不需要绝对相位参考，抗干扰与实现上更实用。

正交幅度调制（**Quadrature Amplitude Modulation**，**QAM**）：同时改变载波的幅度和相位，是 ASK 与 PSK 的组合。若有 m 种相位、n 种振幅，理论上可组合出 $m \times n$ 个状态点（星座点）。

各种调制方式的状态数与比特率：

调制方式	改变的载波参量	状态数 N	每码元比特数	备注
2ASK	幅度	2	1	—
4ASK	幅度	4	2	4 个幅值
2FSK	频率	2	1	需 2 个频率载波（2024-34 考点）
4FSK	频率	4	2	4 个频率
2PSK / BPSK	相位	2	1	二相
4PSK / QPSK	相位	4	2	四相
8PSK	相位	8	3	八相
QAM-16	幅度 + 相位	16	4	4×4
QAM-32	幅度 + 相位	32	5	星座图非矩形排列
QAM-64	幅度 + 相位	64	6	8×8，2023-34 答案
QAM-128	幅度 + 相位	128	7	—

QAM 状态数的两种算法要分清

- 题干说「m 个相位，每个相位 n 种振幅」$\rightarrow N = m \times n$（乘法组合）。
- 题干直接说「QAM-64」$\rightarrow N = 64$。
- 不要把 **QAM-32** 强行写成 4×8 之外的东西；只要题目给了「方案名」，N 就是名字里的数字；只有题目给了相位数与振幅数时才去乘。

算例 **1**（调制方式与数据率）：某信道采用 QAM-16 调制，波特率为 2400 Baud，求数据率。

$N = 16$，每码元 $\log_2 16 = 4$ bit：$R_b = 2400 \times 4 = 9600$ b/s。

算例 **2**（由速率推调制方案）：带宽 4 MHz 的无噪声信道要达到 48 Mb/s，求最小 N。

$$\log_2 N = \frac{48 \times 10^6}{2 \times 4 \times 10^6} = 6 \implies N = 64$$

算例 **3**（比较）：同样 3000 Baud 的波特率下，2ASK、QPSK、QAM-64 的数据率分别是 3000 b/s、6000 b/s、18000 b/s。波特率相同，比特率取决于每码元比特数——这就是提高数据率的两条路径之一（另一条是提高波特率，但受奈氏上限约束）。

（三）模拟数据编码为数字信号：PCM

把模拟数据（话音、音频、图像）变换成数字信号，最典型的方法是脉冲编码调制（**Pulse Code Modulation**，**PCM**），分三步：

第一步：采样（**Sampling**）。按固定的时间间隔 T_s 取出模拟信号的瞬时值，把连续时间信号变成离散时间信号。采样频率 $f_s = 1/T_s$。

采样定理（又称奈奎斯特定理，注意与信道容量的奈氏准则同名但内容不同）：

$$f_s \geq 2f_{\max}$$

其中 $f_{\max}$ 是原始模拟信号中最高频率分量的频率。$\boxed{2f_{\max}}$ 称为奈奎斯特采样频率。

第二步：量化（**Quantization**）。把采样得到的瞬时幅度值四舍五入到有限个预设的离散电平上。若量化级数为 Q（即分成 Q 个等级），则每个样本需要

$$n = \log_2 Q \quad \text{(bit)}$$

位二进制编码表示。量化会引入量化误差，量化级数越多误差越小、所需位数越多。

第三步：编码（**Encoding**）。把每个量化后的电平值用 n 位二进制码表示，得到数字比特流。

PCM 的数据率：

$$R_b = f_s \times n = f_s \times \log_2 Q \quad \text{(b/s)}$$

$$\text{所需带宽（无噪声时按奈氏反推）} \quad W \geq \frac{R_b}{2\log_2 N}$$

其中 f_s 是采样频率（Hz），n 是每个样本的二进制位数。

PCM 的四条易错

1. 采样定理说的是 $f_s \geq 2f_{\max}$，比的是「最高频率分量」，不是信号的频率范围或带宽。若给出频率范围 $0 \sim 4$ kHz，则 $f_{\max} = 4$ kHz，$f_s \geq 8$ kHz。
2. 采样频率与信道带宽是两回事。千万级采样率（如 CD 的 44.1 kHz）不需要同量级带宽的「信道容量」，PCM 数据率才是要传输的比特率。
3. 「采样一次 = 得到一个样本 = 得到一个码元」。若每个样本编成 n 位，则这 n 位合起来才是一个码元所承载的信息，此时码元状态数 $N = 2^n$。
4. 量化级数 Q 必须是 **2** 的幂。若题目给「8 级」「256 级」，n 分别是 3 和 8。

算例 **1**（**PCM** 数据率）：话音信号最高频率 4 kHz，采样频率取 8 kHz，量化级数 256，求 PCM 数据率。

$n = \log_2 256 = 8$ bit：

$$R_b = 8000 \times 8 = 64 \text{ kb/s}$$

这就是传统数字电话的 64 kb/s 标准速率（DS0）。

算例 **2**（经典综合题）：有一个无噪声的 **8 kHz** 信道，每个信号包含 8 级量化，每秒采样 24k 次，求可以获得的最大传输速率。

- 直接按「每秒采样次数 × 每样本位数」计算：$n = \log_2 8 = 3$ bit，$R_b = 24 \times 10^3 \times 3 = 72$ kb/s。
- 但还要用奈氏准则检查信道是否支持：$C_{\max} = 2 \times 8 \times 10^3 \times \log_2 8 = 48$ kb/s。

两者取小，实际最大传输速率为 **48 kb/s**。这道题的设计意图正是考查「PCM 数据率可能超过信道容量，必须取小」这一判断。

算例 **3**（反求量化级数）：某信道带宽 6 kHz，无噪声，采用 PCM 传输一个最高频率 3 kHz 的模拟信号，要求数据率尽可能高，求最大量化级数。

- 采样定理要求 $f_s \geq 2 \times 3000 = 6000$ Hz，取 $f_s = 6$ kHz 时恰好满足。
- 奈氏上限：$C_{\max} = 2 \times 6000 \times \log_2 N = 12000 \log_2 N$。
- 而 PCM 数据率 $R_b = 6000 \times \log_2 N$，恒小于 $C_{\max}$，所以带宽不构成约束；量化级数由题目给定的位数上限决定。这说明当 $f_s = 2W$ 时，**PCM** 数据率天然在信道容量之内，这一点在综合题中常被用来排除错误的约束条件。

（四）模拟数据调制为模拟信号与频分复用

模拟数据本身已是模拟信号，若要在适合长距离传输的频段上传送，需要把它的频谱搬移到载波附近，这叫模拟数据调制为模拟信号，常用手段仍是调幅（AM）、调频（FM）、调相（PM），对应设备是放大器与调制器。典型场景是电话机到本地端局之间的传输。

频分复用（**Frequency Division Multiplexing**，**FDM**）：把信道的总频带划分成若干互不重叠的子频带，每个子频带构成一个子信道，供一路信号使用。所有用户同时占用不同的频率资源。

- 每个子信道之间需要留保护频带（**guard band**），防止相邻子信道互相干扰。
- 特点：各路信号在时间上重叠、在频率上分离；所有用户同时在用。
- 应用：有线电视、传统模拟载波电话系统。

复用技术的一句话区分（为后续章节铺垫）

- 频分复用 **FDM**：按频率分，大家同时用；用于模拟信号，如载波电话、有线电视。
- 时分复用 **TDM**：按时间分，大家轮流用；用于数字信号。
- 波分复用 **WDM**：本质就是光的频分复用，在一根光纤中同时传多个不同波长的光载波。
- 码分复用 **CDM**：用正交的码片序列区分不同用户，大家同时同频使用。

四、电路交换、报文交换与分组交换

（一）三种交换方式的工作过程

*1.*电路交换（*Circuit Switching*）

电话网是典型代表。通信过程分为三个明确的阶段：

1. 建立连接：主叫方向网络请求，网络沿途为这次通话分配一条独占的物理通路（包括链路与交换机时隙/带宽资源）。这一阶段有呼叫建立时延。
2. 传输数据：数据沿已经建好的通路透明地从源传到目的，中间结点不对数据做任何处理（不存储、不检查、不转发决策）。整条通路被这次通信独占。

3. 释放连接：通信结束后归还所占资源。

特点：

- 传输期间时延小且稳定（通路已建立，没有每跳的存储转发开销），适合实时性要求高的业务，如话音。
- 资源利用率低：独占通路，空闲时别人也用不了。
- 不具备差错检验能力：中间结点不处理数据，可靠传输要靠上层或端设备负责。
- 建立连接本身有额外时延；若连接建立失败，通信无法进行。

*2.*报文交换（*Message Switching*）

报文（**Message**）是用户数据加上源地址、目的地址等信息构成的完整数据块。

工作过程：发送方把整个报文发给相邻交换结点；交换结点必须把整个报文完整接收下来（存储），再查转发表找出下一跳，然后把整个报文转发出去（转发）。

特点：

- 采用存储转发（**Store-and-Forward**）机制：结点要收完整个报文才能开始转发，因此无法流水线化。
- 需要的缓存空间很大（要能装下整个报文），转发时延很长（转发时延 = 排队 + 处理 + 传输时延）。
- 线路利用率高于电路交换（多个报文可以分时共享链路），但远低于分组交换。
- 报文长度不受限，长报文容易长时间占用链路。

*3.*分组交换（*Packet Switching*）

分组（**Packet**）是把报文划分成若干等长的数据段，每段前面加上首部（分组头）构成的数据单元。首部中含目的地址等控制信息。

工作过程：发送方构造分组并发送；交换结点缓存收到的分组，按分组中的目的地址查转发表，决定从哪个端口转发出去；接收方接收分组并还原成原报文。

特点：

- 仍然采用存储转发，但以「分组」为单位：结点收完一个短分组就可以开始转发，下一个分组可以紧跟着到达并转发，从而实现流水线（**pipelining**）传输，把端到端时延压缩到「最后一段链路的收尾加上第一段的启动」。
- 缓存需求小，链路利用率最高，多路分组可以交替复用同一条链路，公平共享。
- 代价是引入了新的问题：分组可能失序、丢失；无端到端预留资源，各分组独立路由、动态竞争链路资源，负载过大时结点队列无限堆积会造成网络拥塞。
- 分组过小会降低有效载荷比例（首部开销占比上升），分组过大则流水线收益下降；实际系统按最大传送单元 **MTU** 权衡。

三种交换「存储转发」的区别（一句话）

电路交换：不存储转发，通路独占。报文交换：存储转发，单位是整个报文，不可流水线。分组交换：存储转发，单位是短分组，可以流水线。后者正是分组交换能把时延压到 $(p+k-1)T_t$ 的原因。

（二）三种交换方式的对比

对比维度	电路交换	报文交换	分组交换
是否需要建立连接	需要（先建后传，再释放）	不需要	不需要（数据报方式）；虚电路方式需要
资源占用	独占通路，全程占用	不独占，逐跳存储转发	不独占，逐跳存储转发，统计复用
中间结点是否处理数据	不处理，透明传输	处理（收完整个报文再转发）	处理（按分组首部查表转发）
存储转发单位	无	整个报文	分组
能否流水线传输	不涉及	不能	能
结点缓存需求	小	很大	小
传输时延	小且稳定（通路已建立）	最大（收完才转发）	介于两者之间，随负载波动（排队时延）
建立时延	有（呼叫建立时间）	无	无
线路利用率	最低	中等	最高
差错控制	不具差错检验能力	由结点和端系统共同处理	数据报不保证，由上层（如 TCP）保证；虚电路可由网络保证
灵活性	差（通路固定，故障即中断）	较好	好（分组可独立选路）
适用场景	话音等实时、长时、连续业务	早期电报、传真等短报文业务	数据通信的主流，互联网的基础
典型例子	传统电话网 PSTN	早期电报网	因特网、以太网中的 IP 转发

最容易混淆的三组对比

1. 电路交换「时延小」指的是传输阶段，不是全过程。加上建立连接时延后，短报文用电路交换反而可能更慢（建链成本摊不开）。
2. 报文交换与分组交换都用存储转发，区别在「单位」。说「报文交换不可流水线、分组交换可流水线」才是得分点。
3. 「分组交换时延最小」是错的。传输阶段时延最小的仍是电路交换；分组交换的优势在于线路利用率高、灵活、适合突发数据。真题常把「时延小」作为电路交换的优势来考，把「利用率高」作为分组交换的优势来考，不要对调。

（三）分组交换的时延计算

这是本章计算量最大、最容易失分的部分。先建立符号体系。

符号约定：

符号	含义	说明
L	一个分组的长度	单位 bit（若给 B 要 $\times 8$）
R	链路的数据传输速率（发送速率）	单位 b/s
T_t	单段链路上一个分组的发送时延	$T_t = L/R$
T_p	单段链路上的传播时延	$T_p =$ 链路长度/传播速度
p	分组的数量	由报文总长度除以每分组数据部分得到

符号	含义	说明
k	从源到目的经过的链路段数（段数 = 经过的路由器数 + 1）	无单位
h	每个分组的首部长度	计算有效载荷时要减掉
D	端到端总时延	—

基本推导（以 $k=3$、p 个分组为例）：

- 第 1 个分组从第 **1** 段链路发出需要一个 T_t，接下来它还要在第 **2**、**3** 段链路各被发送一次，共需 $3T_t$ 才能到达终点。
- 但第 **2** 个分组不必等第 1 个分组走完全程：当第 1 个分组离开第 1 段链路后，第 2 个分组立刻开始占用第 1 段链路。于是各分组在各级链路上像流水线一样首尾相接。
- 因此，最后一个分组在最后一段链路上「收尾」的时刻，就等于「第 1 个分组走完全部 k 段」+「其余 $p-1$ 个分组依次占用最后一段链路」，即

$$\boxed{D=(p+k-1)\,T_t}$$

- 若还要计入传播时延（题目给信道长度或时延带宽积），需在每一段链路上各累加一次：

$$\boxed{D=(p+k-1)\,T_t+k\,T_p}$$

变体公式：

- 若各段链路速率不同，T_t 要分段计算，不能简单乘以 k：$D=\sum_{i=1}^{k}T_{t,i}+(p-1)\,T_{t,k}+k\,T_p$（最后一个分组在最后一段的发送时延单独计）。
- 若还要计入结点处理时延 T_{proc} 与排队时延 T_q，逐跳累加：$D=(p+k-1)T_t+k(T_p+T_{proc}+T_q)$。
- 报文交换的对应公式（不可流水线，每跳都要收完整个报文）：

$$D_{\text{报文}}=k\times\frac{M}{R}+k\,T_p$$

其中 M 是报文总长度（bit）。

「+1 还是 -1」的判断法

流水线总时延可以这样记：先算「第 **1** 个分组从头走到尾」的时间，再加上「剩余分组一个个跟上」的时间。

$$D=\underbrace{k\,T_t}_{\text{第 1 个分组走完全程}}+\underbrace{(p-1)T_t}_{\text{其余分组逐个补上}}=(p+k-1)T_t$$

只要抓住「最后一段链路上要连续发送 p 个分组，而在它开始发第 **1** 个分组之前，已经过去了 $(k-1)$ 个 T_t」，就是 $(k-1)T_t+pT_t=(p+k-1)T_t$。

时延计算的五个坑

1. 分组数量按「有效载荷」算，不是按分组总长算。2010-34 中分组长 1000 B、首部 20 B，则每个分组只带 980 B 数据，$p=980000/980=1000$ 个；若误按 1000 B 算会得到 980 个，时延随之算错。
2. **B** 与 **bit** 的换算。分组长度给 B 要乘 8；但有些题目（如 2013-35）直接给 bit（10 kb），这时不要再乘 **8**。
3. 链路段数 k = 路由器数 + **1**。图里画了 2 个路由器就是 3 段链路。
4. 传播时延要乘 k（每段各一次），不是乘 **1**。2023-33 的 0.02 ms 就是 2×0.01 ms。

5. 不要自行脑补「分流、负载均衡、多路径」。除非题目明确画出并行路径并问最大吞吐量，否则一律按单一路径串行计算——长路径只会增加转发时延。

算例 1（2010-34 原题）：采用存储转发分组交换，所有链路速率 100 Mb/s，分组大小 1000 B，其中分组头 20 B。主机 H1 向 H2 发送 980 000 B 的文件。忽略分组拆装时间和传播时延，求从 H1 开始发送到 H2 接收完毕所需的最少时间。

- 每分组有效数据：$1000-20=980$ B；
- 分组数：$p=980000/980=1000$ 个；
- 单分组发送时延：$T_t=\dfrac{1000\times 8}{100\times 10^6}=80\ \mu\text{s}=0.08$ ms；
- 图中 H1 经两个分组交换结点到 H2，链路段数 $k=3$；
- 取 3 段中一条最短路径（题目问「至少」），按流水线公式：

$$D=(p+k-1)T_t=(1000+3-1)\times 0.08=80.16\ \text{ms}$$

对比：只算第一段链路的 $1000\times 0.08=80$ ms 会漏掉流水线启动开销 0.16 ms。

算例 2（2013-35 原题）：主机甲经 1 个路由器（存储转发）与主机乙互联，两段链路速率均为 10 Mb/s。甲分别采用报文交换和分组大小为 **10 kb** 的分组交换向乙发送 1 个大小为 8 Mb 的报文。忽略传播时延、分组头开销和分组拆装时间，求两种方式的总时间。

报文交换（$k=2$ 段链路，每段都要发完整个 8 Mb 报文）：

$$D_{\text{报文}}=2\times\frac{8\times 10^6}{10\times 10^6}=2\times 0.8\ \text{s}=1600\ \text{ms}$$

分组交换：

- 分组数 $p=\dfrac{8\ \text{Mb}}{10\ \text{kb}}=\dfrac{8\times 10^6}{10\times 10^3}=800$ 个；
- 单分组发送时延 $T_t=\dfrac{10\times 10^3}{10\times 10^6}=1$ ms；
- $k=2$ 段链路：

$$D_{\text{分组}}=(800+2-1)\times 1=801\ \text{ms}$$

（对照：若用报文交换公式错套成 $800\times 2=1600$ ms，或漏掉传播，都会选错。）

算例 3（2023-33 原题）：H1 和 H2 通过路由器互联，2 段链路速率均为 100 Mb/s，每段链路的时延带宽积（单向传播时延 × 带宽）均为 1000 bit。H1 向 H2 发送 1 个大小 1 MB（$M=10^6$）的文件，分组长度 1000 B，求从 H1 开始发送到 H2 收到全部数据所需的最少时间。

- 分组数：$p=\dfrac{10^6\ \text{B}}{1000\ \text{B}}=1000$ 个；
- 单分组发送时延：$T_t=\dfrac{1000\times 8}{100\times 10^6}=80\ \mu\text{s}=0.08$ ms；
- 单向传播时延（由时延带宽积反求）：$T_p=\dfrac{1000}{100\times 10^6}=10\ \mu\text{s}=0.01$ ms；
- 链路段数 $k=2$：

$$D=(p+k-1)T_t+k\,T_p=(1000+2-1)\times 0.08+2\times 0.01=80.08+0.02=80.10\ \text{ms}$$

算例 4（三跳、含首部开销）：源主机经 3 个路由器向目的主机发送 1000 个分组，每段链路速率 10 Mb/s，分组长度 1500 B，每段链路传播时延 5 ms。求总时延。

链路段数 $k=3+1=4$。$T_t=\dfrac{1500\times 8}{10\times 10^6}=1.2$ ms。

$$D=(1000+4-1)\times 1.2+4\times 5=1203.6+20=1223.6\text{ ms}$$

算例 5（反求分组数）：某文件 5000 bit，分组长度固定 100 bit（含 20 bit 首部），链路速率 R，经过 3 段链路，求端到端发送时延的表达式。

每分组有效载荷 80 bit，$p=\lceil 5000/80\rceil=63$ 个（不足一个分组也要发一个，注意向上取整），$T_t=100/R$：

$$D=(63+3-1)\times\frac{100}{R}=\frac{6500}{R}$$

三点提醒

1. 题目问「至少」通常意味着可以走最短路径（段数最少的路径），也可能意味着「不考虑排队和处理时延的乐观情形」。
2. 吞吐量题用「木桶原理」：端到端最大吞吐量取路径上所有链路的最小值。2024-33 中，中间三条并行路径虽有大带宽，但两端公共链路都只有 10 Mb/s，故最大吞吐量 = $\min(10,1000,1,100,10)=10$ Mb/s。
3. 时延带宽积 = 传播时延 × 带宽，单位是 bit，表示「链路上正在传播的比特数」（以比特为单位的链路长度）。它常被用来反求传播时延，是 2023-33 的关键一步。

五、数据报与虚电路

分组交换可以进一步分成两种工作方式：无连接的数据报（**Datagram**）方式与面向连接的虚电路（**Virtual Circuit**，**VC**）方式。它们都由网络层提供，是网络层向上提供服务的两种风格。

（一）数据报方式

- 每个分组独立携带完整的源地址和目的地址。
- 每个分组独立地进行路由选择，同一报文的不同分组可能走不同的路径。
- 中间结点不维护任何关于这次通信的状态（无状态转发）。
- 分组可能失序到达，也可能丢失，网络只提供「尽力而为（**best-effort**）」的转发服务，不承诺交付结果。
- 可靠性、顺序保证、流量控制由端主机上的上层协议（如 **TCP**）负责。
- 对网络故障的适应能力强：某条链路或结点故障后，后续分组可以重新选路绕过。
- 典型代表：**IP** 网络（因特网）。

（二）虚电路方式

- 数据传输之前必须先建立一条逻辑连接，即虚电路。建立阶段进行路由选择，确定沿途经过的结点与端口。
- 连接建立后，沿途每个结点都记录一条虚电路表项，每个分组只需携带一个很短的虚电路号（**VCID**），结点按 VCID 查表转发。
- 同一条虚电路上的所有分组沿同一条路径转发，因此保证分组按序到达。
- 差错处理与流量控制可以由分组交换网自身负责，也可以由用户主机负责。
- 对网络故障的适应能力差：所有经过故障结点的虚电路都会中断。
- 典型代表：帧中继、**ATM**、**X.25**。

虚电路 ≠ 电路交换，「预分配带宽」是高频错项

- 电路交换是真正的物理线路交换（如电话网），它预分配带宽，资源被独占。
- 虚电路只是逻辑上的连接，本质是一种多路复用技术：一条物理链路可以承载多条虚电路，多条虚电路共享链路带宽，通常采用统计复用。
- 因此「虚电路需要为每条连接预分配带宽」是错的（2020-34 的答案正是这一句）。只有需要特定服务质量保证时才可能做资源预留。

另外注意：虚电路服务是 **OSI** 参考模型的特色，**TCP/IP** 模型中没有虚电路服务——TCP/IP 的网络层（IP）只提供数据报服务。

（三）数据报与虚电路的对比

对比维度	数据报服务	虚电路服务
连接的建立	不需要	必须建立
目的地址	每个分组都带完整目的地址	仅在建立阶段使用；之后每分组用较短的虚电路号 **VCID**
路由选择	每个分组独立进行路由选择和转发	同一条虚电路的分组按同一路由转发
分组顺序	不保证有序到达	保证有序到达
可靠性	网络只负责尽力转发，不承诺交付结果，可靠性由上层协议保证	可靠性由网络自身保证
差错处理与流量控制	由用户主机上的上层协议（如 TCP）负责	可由分组交换网负责，也可由用户主机负责
对网络故障的适应性	强，可寻找新路径转发	弱，所有经过故障结点的虚电路都不能工作
结点状态	无状态（不维护通信上下文）	有状态（维护虚电路表）
典型网络	因特网 IP	帧中继、ATM、X.25

一句话总结设计哲学：数据报服务用「结点的无状态」换取网络的简单与健壮；虚电路服务用「结点的状态记忆」换取结点的控制能力。

常考的三个判断

1. 「虚电路可以确保数据分组传输顺序」→ 对。
2. 「虚电路建立时需要进行路由选择」→ 对（路由选择发生在建立阶段）。
3. 「虚电路需要为每条虚电路预分配带宽」→ 错（2020-34 答案）。
4. 「一条虚电路只能为一个进程服务 / 端系统之间只能有一条虚电路」→ 错。两个端系统之间可以有多条虚电路，为不同进程服务。

六、传输介质

传输介质是数据传输系统中发送器与接收器之间的物理路径，分为导向传输介质（**guided media**）与非导向传输介质（**unguided media**）两大类：前者是固体介质（双绞线、同轴电缆、光纤），信号沿介质传播；后者是自由空间（无线电波、微波、红外、激光）。

（一）双绞线

构成：把两根相互绝缘的铜导线按一定规则并排绞合（**twist**）在一起，通常再把多对这样的线对捆成一根电缆。

为什么要绞合：两根导线在空间上不断交换位置，使得两根线感受到的外部电磁干扰、以及自身向外辐射的电磁场尽可能相互抵消，从而减少相邻导线之间的电磁干扰和串扰。绞合越密（绞距越小），抗干扰能力越强，可支持的速率越高。

分类：

类型	全称	特点
UTP	非屏蔽双绞线（Unshielded Twisted Pair）	无金属屏蔽层，价格便宜、安装方便，最常用
STP	屏蔽双绞线（Shielded Twisted Pair）	带金属丝编织的屏蔽层，抗干扰能力更强，价格更高、更笨重、需良好接地

按电气性能分为不同类别（**Category**，**Cat**）：

类别	支持的典型速率	说明
Cat3	10 Mb/s	早期电话与 10BaseT
Cat5	100 Mb/s	100BaseT
Cat5e（超五类）	1 Gb/s（100 m 内）	增强型，串扰指标改善，千兆以太网常用
Cat6（六类）	1 Gb/s（100 m）；10 Gb/s（约 55 m 内）	内部有十字骨架分隔四对线，进一步降低串扰
Cat6a（超六类）	10 Gb/s（100 m）	屏蔽结构，更高频率

适用场景：局域网布线（最主流的介质）与传统电话网，模拟传输与数字传输都适用。

（二）同轴电缆

构成（由内到外）：内导体（铜芯）→ 绝缘层 → 网状编织的屏蔽层（外导体）→ 塑料外层。

特点：外导体屏蔽层具有良好的抗干扰特性，带宽比双绞线更大，传输速率更高、传输距离更远，但价格较贵、体积大、布线不便。

按特性阻抗分类：

类型	特性阻抗	传输的信号	典型应用
基带同轴电缆	50 Ω	基带数字信号	早期局域网（10Base5、10Base2）
宽带同轴电缆	75 Ω	宽带（频带）信号	有线电视系统（CATV）

适用场景：早期以太网（总线型局域网）、有线电视、宽带接入（HFC 混合光纤同轴）。

（三）光纤

构成：由纤芯（**core**）与包层（**cladding**）构成，利用光导纤维传递光脉冲来通信。纤芯折射率高于包层。

基本原理——光的全反射：当光从折射率大的纤芯射向折射率小的包层，且入射角大于临界角时，光不会折射进包层，而是全部被反射回纤芯，如此在纤芯内不断反射着向前传播，实现低损耗长距离传输。

分类：

类型	纤芯直径	光源	传输模式	距离与带宽	应用
多模光纤（MMF）	较粗（如 50 μm、62.5 μm）	发光二极管 **LED**	光以多种模式（多条路径）在纤芯中传播	距离较近、带宽较低；存在模式色散，脉冲会展宽	近距离传输，如楼内局域网
单模光纤（SMF）	很细（如 9 μm 左右）	半导体激光器 **ILD/LD**	光沿单一模式（近似直线）传播	距离远、带宽极高，衰减小	长距离、大容量骨干网

光纤相对于铜缆的优势：

- 带宽极大：一根光纤的潜在带宽远高于双绞线，是现代骨干网的基础。
- 传输损耗小、中继距离长：可长距离传输而无需频繁中继。
- 抗电磁干扰能力强：光信号不受电磁干扰，也不产生电磁辐射，保密性好（不易被窃听）。
- 体积小、重量轻、耐腐蚀。

缺点：价格相对较高；光纤对接与分路需要专用设备，施工难度大；光纤本身脆弱，机械强度不如铜缆。

单模/多模的三个判断要点

1. 单模纤芯细、多模纤芯粗。巧记：「单薄、多厚」。
2. 单模用激光（**LD**），多模用 **LED**。
3. 单模传得远、带宽高；多模传得近。看到题目问「长距离骨干网用什么」，答单模；「楼内局域网、近距离」答多模。
4. **10BaseT / 100BaseT** 用的是双绞线（T = Twisted pair），2019-34 的答案就是双绞线。

（四）无线传输介质

介质	频段/传播方式	是否需视线	特点与应用
无线电波	低频到超高频，向各个方向传播	否	穿透力强、传播距离远，可全向发射；频段拥挤，如广播、移动电话
微波	超高频（GHz 级），直线（视距）传播	是	需在视距内，地球曲率与障碍物会阻挡，因此要高塔架设 + 中继站转发；带宽大，如微波接力、卫星通信
红外	红外光，直线传播，不能穿透固体	是	短距离、同一房间内使用，不会穿透墙壁，天然隔离、抗窃听；如遥控器、早期 IrDA

介质	频段/传播方式	是否需视线	特点与应用
激光	激光，直线传播	是	需精确对准，方向性极强，短距离点对点；受天气影响大

微波为什么要中继：微波是直线传播的，而地球是曲面，且微波难以穿透障碍物。因此微波通信要求发送端与接收端在视距（**line of sight**）范围内；远距离传输时必须每隔一定距离（例如几十公里）建一个中继站，把信号接收后放大再转发，形成微波接力。

无线介质的共同点与区别

- 共同点：都是非导向介质，靠自由空间传播。
- 区别的判据：是否需要视线对准。无线电波不需要（可全向、可穿透）；微波、红外、激光都需要（直线/视距传播）。
- 高频段（微波及更高）虽然带宽大，但方向性强、绕射能力弱，因此对视线和天线对准要求高。

（五）物理层接口的特性

物理层接口规范（如 RS-232、USB 接口标准）从四个特性来描述：

特性	描述内容	记忆口诀
机械特性	接口所用的接线器形状、尺寸、引脚数目与排列、固定和锁定装置	看外形
电气特性	接口电缆各条线上出现的电压范围、阻抗、信号电平、传输速率、距离限制	看电压电流
功能特性	某条线上出现的某一电平的电压代表什么意义，即每根引脚/信号线完成什么功能	看引脚干什么
过程特性（又称规程特性）	对于不同功能的各种可能事件的出现顺序，即信号线上一组操作过程的时序关系	看动作先后

四个特性的判断口诀

- 出现「形状、尺寸、引脚数目、排列」→ 机械特性。
- 出现「电压范围、电平、阻抗、速率」→ 电气特性。
- 出现「某条线的作用、电平的意义」→ 功能特性。
- 出现「事件发生顺序、时序关系、操作过程」→ 过程特性。

2012-34 问「描述完成每种功能的事件发生顺序」，答案是过程特性。2018-34 问「不属于物理层接口规范定义范畴的是」，答案是物理地址（MAC 地址属于数据链路层）。接口形状属机械、引脚功能属功能、信号电平属电气，都在物理层范畴内。

与计算机组成原理的联动

物理层接口四特性与计算机组成原理中总线特性（机械、电气、功能、时间）几乎一一对应，只是计组里叫「时间特性」。复习时可以放在一起记，注意计组用「时间特性」而计网用「过程特性」。

七、物理层设备

物理层设备工作在比特级，只对信号做放大、整形、转发，不识别帧、**MAC** 地址或 **IP** 地址。

（一）中继器

作用：对接收到的数字信号进行整形、放大并转发，以扩大网络的传输距离。

为什么需要中继器：信号在传输介质中传输时会衰减，同时会因干扰而失真。当传输距离超过一定限度后，接收端收到的信号可能已经无法正确识别（既达不到判决门限，波形也发生了畸变），此时必须用中继器把已经衰减、畸变的信号再生（**regenerate**）成标准的数字波形再继续发送。

与放大器的区别（高频辨析点）：放大器放大的是模拟信号，它把模拟波形连同噪声一起线性放大；中继器再生的是数字信号，它重新产生干净的数字波形，不放大噪声。

中继器的性质：

- 不能分割冲突域，也不能分割广播域。中继器连接起来的所有网段同属一个冲突域，也同属一个广播域。
- 中继器工作的两端网段在物理层规范上必须相同，尤其数据传输速率必须相同（这也是「中继器不能连接两个速率不同的以太网」的原因）。介质可以不同（例如双绞线接光纤），但物理层协议与速率必须一致。
- 中继器没有存储转发能力，收到就转发，因此会引入很小的转发时延。

5-4-3 规则：因为信号的延迟是有规定的，中继器不能无限延长网络。在早期的 10BASE5 以太网规范中规定：

- 最多允许 **4** 个中继器串联；
- 这 4 个中继器连接 **5** 段通信介质；
- 其中只有 **3** 段可以挂接计算机（其余 2 段只能用作延长链路的中继段，不能连接主机）。

合起来即「**5** 段介质、**4** 个中继器、**3** 段可挂主机」。

5-4-3 的准确含义

- **5**：5 个网段（网段 = 中继器之间的介质段）。由 4 个中继器串联自然产生 5 段。
- **4**：4 个中继器。这是上限，不能是 5 个。
- **3**：只有 3 个网段可以连接计算机（通常是两端各一段加中间一段），另 2 段只能做纯延长段。

> 不要把「3」误记为「3 个中继器」。这个规则是 10BASE5 的规范，是历史遗留标准，但统考仍会考其数字。

（二）集线器

集线器（**Hub**）本质上是一个多端口的中继器，工作在物理层。

工作原理：集线器从某一个端口收到信号后，整形、放大，并向除该端口外的所有其他端口广播转发。它不认识 **MAC** 地址，不做任何路径选择，也没有交换表。

关键性质：

性质	说明
带宽共享	若集线器总带宽为 M，接入 N 台主机且同时工作，每台主机实际可用带宽约为 M/N
半双工	集线器采用半双工工作方式，同一时刻只能有一台主机成功发送（否则冲突）
冲突域	连接在集线器上的所有主机同属一个冲突域（一个冲突就是全网冲突）

性质	说明
广播域	也同属一个广播域（转发本来就是广播的）
转发方式	广播（向除源端口外的所有端口）
逻辑拓扑	物理连接是星状拓扑，但逻辑连接是总线状拓扑（因为共享同一条介质、会互相冲突）

吞吐量与冲突：集线器所有端口共享总带宽，任意时刻最多一个站能成功发送，因此多台主机同时发送必然冲突，需要用 CSMA/CD 来协调（这部分在数据链路层详述）。

中继器 / 集线器与交换机 / 路由器的对比：

对比项	中继器 / 集线器	网桥 / 交换机	路由器
工作层次	物理层	数据链路层	网络层
能否分割冲突域	不能	能	能
能否分割广播域	不能	不能	能
识别地址	不识别任何地址	识别 **MAC** 地址	识别 **IP** 地址
转发依据	无（广播）	交换表（自学习建立）	路由表 / 转发表
转发形式	广播（除源端口）	单播或广播（除源端口）	按路由表逐跳转发
带宽	N 台主机共享 M，每台约 M/N	N 个端口各 M，全双工，总带宽 $N \times M$	各接口独立带宽
能否互联异构网络	介质可不同，速率必须相同	直通式要求端口速率相同；存储转发式可不同	可互联异构网络

集线器三个必考结论

1. 集线器不分割冲突域，也不分割广播域。一个集线器上的所有主机在同一个冲突域里，任意两台同时发数据必然冲突。这是相对交换机最本质的区别。
2. 集线器上的主机共享总带宽。10 Mb/s 的集线器接 10 台主机，每台平均只有约 1 Mb/s；而 10 Mb/s 的交换机每个端口独享 10 Mb/s，总带宽 100 Mb/s。
3. 集线器收到数据帧会向除源端口外的所有端口转发，因此所有连接在该集线器上的其他主机都会收到该帧——这不同于交换机按 MAC 地址的单播转发。

物理层设备的真题角度

- 「集线器的拓扑结构」：物理上是星状，逻辑上是总线状。
- 「中继器连接的两个网段必须相同的是什么」：数据传输速率（不是网络层协议、不是 MAC 地址格式、不是逻辑拓扑）。
- 「10BASE-T 以太网中使用集线器，某主机发送数据时其他主机能否收到」：能收到全部，因为集线器是广播式转发。
- 「中继器/集线器能否分割冲突域或广播域」：都不能。

本章小结表

考点	核心结论 / 公式	关键记忆点
信源与信宿	信源 = 源点（产生数据），信宿 = 终点（接收数据）	发送器/接收器是变换环节，不是源/宿
通信方式	单工 1 信道；半双工 2 信道共用 1 介质；全双工 2 信道 2 介质	半双工「同一时刻只能一方发」
信号分类	模拟信号参量连续；数字信号参量取有限离散值	数字信号通常只取两个电平
带宽两义	模拟口径为频率范围（Hz）；网络口径为最高数据率（b/s）	定理里的 W 必须取 Hz
码元与比特	$k = \log_2 N$（bit/码元），N 为码元离散状态数	N 取 2 的幂
波特与速率	$R_b = R_B \log_2 N$；$R_B = R_b / \log_2 N$	$N = 2$ 时二者才相等
曼彻斯特系	波特率 = 2 × 比特率（每比特 2 个码元）	效率 0.5 bit/Hz
4B/5B	$R_B = 1.25 R_b$，开销 25%	100 Mb/s 数据 → 125 MBaud
基带 vs 频带	基带 = 数字信号直接传；频带 = 调制到载波再传	`Base` 就是基带
奈氏准则	$C_{\max} = 2W \log_2 N$（无噪声、理想低通）	限的是码元速率上限 $2W$ Baud
香农定理	$C = W \log_2(1 + S/N)$（有噪声）	与调制方式无关
dB 换算	$S/N = 10^{\mathrm{dB}/10}$；30 dB → 1000	10/20/30/40 dB → 10/100/1000/10000
两定理取舍	都给出时取 min（更紧的约束）	奈氏管码元，香农管信息
RZ / NRZ / NRZI	RZ 位中归零；NRZ 无自同步；NRZI 跳变 =0、不变 =1	NRZ 波特率 = 比特率
曼彻斯特	位中间必变，看跳变方向	IEEE 802.3：下跳 =1、上跳 =0
差分曼彻斯特	位中间必变（仅时钟），位起始处跳 =0、不跳 =1	抗干扰比曼彻斯特更强
数字调制	ASK 调幅、FSK 调频（需 2 个频率）、PSK 调相、QAM 调幅 + 调相	QAM-N 每码元 $\log_2 N$ bit
PCM 三步	采样 → 量化 → 编码；$f_s \geq 2 f_{\max}$	电话：8 kHz × 8 bit = 64 kb/s
PCM 数据率	$R_b = f_s \times \log_2 Q$	若超信道容量则取小
FDM	按频率划分，各用户同时使用，需保护频带	模拟信号复用
电路交换	建立连接 → 传输（独占、透明）→ 释放连接	时延小但利用率低，无差错检验
报文交换	存储转发整个报文，不可流水线	缓存需求大、时延大
分组交换	存储转发短分组，可流水线	利用率最高，可能失序丢失
分组交换时延	$D = (p + k - 1)T_t$；含传播 $D = (p + k - 1)T_t + kT_p$	k = 链路段数 = 路由器数 + 1
报文交换时延	$D = k \times M/R$	每跳都要收完整个报文
吞吐量	端到端取路径上各链路带宽的最小值（木桶原理）	不要脑补分流
分组数	$p = \lceil$文件大小/每分组有效载荷$\rceil$	有效载荷 = 分组长 − 首部
数据报	无连接、每分组带完整地址、独立选路、不保证顺序、故障适应强	网络只尽力而为
虚电路	面向连接、建立时选路、按 VCID 转发、保证顺序、故障适应弱	不预分配带宽

考点	核心结论 / 公式	关键记忆点
双绞线	绞合抵消干扰；UTP/STP；Cat5e 千兆、Cat6 万兆（短距）	最常用、最便宜
同轴电缆	屏蔽好、速率高、距离远；50 Ω 基带、75 Ω 宽带	75 Ω 用于有线电视
光纤	全反射原理；单模（细芯 + 激光 + 远）、多模（粗芯 +LED+ 近）	带宽大、抗干扰、保密好
无线介质	无线电波不需视线；微波/红外/激光需视线	微波直线传播要中继
接口四特性	机械（形状）、电气（电压）、功能（引脚作用）、过程（时序）	问「事件顺序」答过程特性
中继器	再生数字信号、扩大距离；不分割冲突域/广播域	放大器放大的是模拟信号
5-4-3 规则	5 段介质、4 个中继器、3 段可挂主机	10BASE5 规范
集线器	多端口中继器；共享带宽、半双工、一个冲突域	物理星状、逻辑总线状

易错清单

1. 奈氏准则限的是码元传输速率（上限 $2W$ **Baud**），不是比特率。说「奈氏准则限制了数据传输速率」不准确——它限制码元速率，比特率还要乘 $\log_2 N$。2017-34 的解题关键正是这一点。
2. 香农定理与调制方式无关。改变调制方式提高每码元比特数，在香农极限下不能提升信道容量；香农只由 W 和 S/N 决定。所以「在有噪声信道中换更高级的调制方式就能突破香农极限」是错的。
3. **dB** 换算别忘指数。$S/N = 10^{\mathrm{dB}/10}$，不是 $\mathrm{dB}/10$。30 dB 是 1000 倍不是 3 倍；40 dB 是 10000 倍。
4. **QAM** 的状态数是相位数与振幅数的乘积。「4 个相位 × 4 种振幅」是 16 种状态（4 bit），不是 8 种。2009-34 的干扰项就这么设计的。
5. 「带宽 **3 kHz**」代进公式时是 3×10^3 **Hz**。若题干给的是「链路速率 100 Mb/s」，那不是带宽，不能代进奈氏/香农。
6. 传播速度不影响数据传输速率。2014-35 的答案就是「信号传播速度」。它只影响传播时延 $T_p =$ 距离/速度。
7. 发送时延与传播时延算法不同。发送时延 = 分组长度/发送速率；传播时延 = 距离/传播速度。前者与带宽有关，后者无关。
8. 曼彻斯特编码看位中间的跳变方向，差分曼彻斯特看位起始处是否跳变。两者位中间都必变，不能只凭「位中间必变」就断定是曼彻斯特。
9. 曼彻斯特和差分曼彻斯特的波特率是比特率的 **2** 倍，编码效率只有 0.5 bit/Hz，所需带宽是 NRZ 的 2 倍。这是「为什么早期以太网只用 10 Mb/s」的原因之一。
10. **NRZI** 的规则是「跳变表示 **0**、不变表示 **1**」，与 NRZ「高电平 =1、低电平 =0」完全不同。2015-34 中若把它当成 NRZ 就会选错。
11. 归零编码（**RZ**）自同步但浪费带宽；**NRZ** 效率最高但没有自同步。这两句经常被互换来出干扰项。
12. **PCM** 采样定理比较的是最高频率分量 $f_{\max}$，不是带宽。频率范围 $0 \sim 4$ kHz 意味着 $f_{\max} = 4$ kHz，$f_s \geq 8$ kHz，而不是 ≥ 8 kHz 之外的别的数；若频率范围是 $2 \sim 10$ kHz，则 $f_{\max} = 10$ kHz，$f_s \geq 20$ kHz。
13. **PCM** 数据率超过信道容量时必须取小。「每秒采样 24k 次、每样本 3 bit」算得 72 kb/s，但 8

kHz 无噪声信道只能到 48 kb/s，答案取 48 kb/s。

14. 分组时延公式是 $(p+k-1)T_t$，不是 $p \times k \times T_t$。后者是「每个分组在每段链路上都从头算」的错误叠加，会大出约 k 倍。
15. k 是链路段数 = 路由器数 + **1**。 题目画 2 个路由器就是 3 段；画 1 个路由器就是 2 段。
16. 传播时延要按段数累加（$\times k$），不是只加一次。 2023-33 的 80.10 ms 就是 $80.08 + 2 \times 0.01$。
17. 分组数要按有效载荷算。分组长 1000 B、首部 20 B 时，每分组只带 980 B；文件 980000 B 对应 1000 个分组而不是 980 个。
18. 报文交换每跳都要收完整个报文，所以时延是 $k \times M/R$；而分组交换可以流水线，二者速率差可达数量级。
19. 电路交换的「时延小」仅指传输阶段。全过程还要加上建立连接时延；短报文场景下电路交换未必更快。
20. 虚电路不预分配带宽。虚电路是逻辑连接，多条虚电路共享链路带宽，采用统计复用；预分配带宽的是电路交换。2020-34 的错误选项正是这一句。
21. 一条虚电路不等于一条物理线路：同一物理链路上可以承载多条虚电路；两个端系统之间也可以有多条虚电路为不同进程服务。
22. **TCP/IP** 模型没有虚电路服务。虚电路服务是 OSI 参考模型的特色；IP 网络提供的是数据报服务。
23. 集线器不分割冲突域也不分割广播域。只有网桥/交换机分割冲突域，只有路由器分割广播域。
24. 集线器所有主机共享带宽：N 台主机接在带宽 M 的集线器上同时工作，每台约 M/N；而交换机每个端口独享 M。
25. 集线器的物理拓扑是星状，逻辑拓扑是总线状。两个「拓扑」不要对调。
26. 中继器连接的两个网段必须速率相同（物理层规范相同），速率可以不同是存储转发式交换机的特性，不是中继器的。
27. 中继器再生数字信号，放大器放大模拟信号。二者不通用。
28. 中继器不能无限延长网络，10BASE5 的 5-4-3 规则是「5 段介质、4 个中继器、3 段可挂主机」，不是「3 个中继器」。
29. 物理层接口四特性与「物理地址」无关。 MAC 地址属于数据链路层，不是物理层接口规范的范畴（2018-34）。
30. 单模光纤纤芯细、用激光、传得远；多模光纤纤芯粗、用 **LED**、传得近。「单模用 LED」是错的，「多模传得比单模远」也是错的。
31. 同轴电缆按阻抗分：**50** Ω 传基带数字信号，**75** Ω 传宽带信号（有线电视）。两个数字不要互换。
32. 光纤靠全反射传光，条件是光从折射率大的纤芯射向折射率小的包层且入射角大于临界角；纤芯折射率大于包层。
33. 微波、红外、激光都需要视线（直线）传播；无线电波不需要。微波远距离通信必须设中继站接力。
34. 吞吐量题用木桶原理，取路径上最小带宽；不要因为图中有并行大带宽路径就去做加法或脑补分流。
35. 「时延带宽积 = 传播时延 × 带宽」，单位是比特，不是「带宽 × 发送时延」。它常被用来反求传播时延。

第三章数据链路层

本章对应 2027 年 408 考纲「第四部分计算机网络 → 三、数据链路层」，是计算机网络部分分值最重、计算题最密集的一章。

本章考纲要求

（一）数据链路层的功能

（二）组帧

（三）差错控制

1. 检错编码
2. 纠错编码

（四）流量控制与可靠传输机制

1. 流量控制、可靠传输与滑动窗口机制
2. 停止-等待协议
3. 后退 N 帧协议（GBN）
4. 选择重传协议（SR）

（五）介质访问控制

1. 信道划分：频分多路复用、时分多路复用、波分多路复用、码分多路复用的概念和基本原理
2. 随机访问：ALOHA 协议，CSMA 协议，CSMA/CD 协议，CSMA/CA 协议
3. 轮询访问：令牌传递协议

（六）局域网

1. 局域网的基本概念与体系结构
2. 以太网与 IEEE 802.3
3. IEEE 802.11 无线局域网
4. VLAN 基本概念与基本原理

（七）广域网

1. 广域网的基本概念
2. PPP 协议

（八）数据链路层设备：以太网交换机及其工作原理

一、数据链路层的功能

（一）链路、数据链路与帧

链路（**Link**）又称物理链路，是指从一个结点到相邻结点的一段物理线路（有线或无线），中间没有其他任何交换结点。网络中各主机之间的通信路径一般由多段链路构成。

数据链路（**Data Link**）又称逻辑链路，是在链路的基础上加上实现通信协议的硬件（网络适配器，俗称网卡）和软件（驱动程序）所构成的。也就是说：

$$\text{数据链路} = \text{物理链路} + \text{实现数据链路层协议的硬件和软件}$$

帧（**Frame**）是数据链路层对等实体之间在水平方向进行逻辑通信的协议数据单元 **PDU**。发送方的数据链路层给网络层交付下来的分组添加首部和尾部，封装成帧后交付物理层发送；接收方的数据链路层从物理层交付上来的比特流中解封出分组，再上交给网络层。

从数据链路层看，主机 H1 到 H2 的通信可以看成是由若干段链路上的通信拼接而成的。相邻两段链路可能采取不同的数据链路层协议，这正是” 逐段链路、逐跳处理” 的含义。

链路 ≠ 数据链路

考试常在这里设陷阱：问” 链路” 就是纯物理线路，问” 数据链路” 必须包含协议软硬件。另外” 物理地址”（MAC 地址）虽带” 物理” 二字，却属于数据链路层范畴，不是物理层。物理层是” 管道的存在”，数据链路层是” 管道里的交通规则”。

（二）数据链路层的功能

参考教材与考纲，数据链路层的功能可归纳为五条：

- 为网络层提供服务：主要有无确认的无连接服务、有确认的无连接服务、有确认的面向连接服务三类。存在确认就必须存在连接（2013 年考过）。
- 链路管理：连接的建立、维持和释放（面向连接服务才需要）。
- 组帧（封装成帧）：把网络层的分组封装成帧，解决帧定界问题。
- 差错控制：检测传输过程中产生的比特差错，决定丢弃或重传。
- 流量控制与可靠传输：协调收发双方的发送/接收速率，必要时通过确认与重传实现可靠传输。

一道常见选择题

” 在 OSI 参考模型中，数据链路层为网络层提供服务，下列哪种说法是错误的？” A. 可以提供无确认的无连接服务　B. 可以提供有确认的面向连接服务 C. 必须提供可靠的、无差错的数据传输　D. 服务质量取决于具体的链路层协议答案选 C。是否存在可靠传输服务由具体协议决定：以太网（IEEE 802.3）提供不可靠服务，802.11 无线局域网提供可靠服务。

（三）封装成帧、透明传输与差错检测

封装成帧就是给网络层交付下来的分组添加首部和尾部。帧首部通常含帧开始符、源地址、目的地址；帧尾部通常含帧校验序列和帧结束符。接收方据此在比特流中识别出帧的开始与结束，叫帧定界。

为提高传输效率，应使数据载荷远大于首部和尾部；但数据载荷长度必须有上限，这个上限叫最大传送单元（**Maximum Transfer Unit**，**MTU**），以太网的 MTU 为 1500 字节。

透明传输：如果数据载荷中恰好出现了帧定界符，接收方就会错误地提前结束帧定界。透明传输指数据链路层对上层交付的 **PDU** 内容没有任何限制，就好像数据链路层不存在一样。实现透明传输

的两种方法是字节填充（字符填充）和比特填充。

差错检测：见下一节。

二、组帧

组帧（Framing）要解决的核心问题是：接收方如何从连续的比特流中找出帧的边界。四种常见方法如下。

（一）字符计数法

在每个帧的帧首部中用一个计数字段指出本帧的总字符数（字节数）。接收方先读出计数字段，就知道后面还有多少个字符，从而定出帧尾。

缺点：只要计数字段在传输中出错，帧的边界就会丢失，接收方会把后续内容全部错位解析，错误无法恢复。因此实际协议中很少单独使用。

（二）字符填充法

面向字节的物理链路使用字符填充法（也叫字节填充法）实现透明传输。做法是：

- 在帧首部使用 SOH（0x01，帧开始符），帧尾部使用 EOT（0x04，帧结束符）。
- 发送方扫描数据载荷，每遇到一个 SOH 或 EOT，就在其前面插入一个转义字符 ESC（**0x1B**）。
- 如果数据载荷中本来就有 ESC，则在 ESC 前面再插入一个 ESC。
- 接收方做逆变换：遇到 ESC 就删除它，并把紧随其后的一个字符当作普通数据处理。这样连续两个 ESC 就被还原成数据中的单个 ESC。

完整算例：设数据载荷为 A ESC B EOT C，SOH 为帧首部、EOT 为帧尾部、ESC 为转义字符，则实际在线路上传输的字符序列为：

```
SOH  A  ESC ESC  B  ESC EOT  C  EOT
```

逐项说明：数据中的 ESC 被填成 ESC ESC；数据中的 EOT 被填成 ESC EOT；数据中的 A、B、C 是普通字符，不填充；最后的 EOT 是真正的帧结束符，不属于数据载荷。

填充的判定顺序

接收方必须一遍扫描：先看 ESC，再看下一个字符。如果扫描逻辑写成”看到 EOT 就结束帧”，那么数据中的 ESC EOT 会被误判为帧尾。所以规则只能是”遇到 ESC 就跳过并原样收下后一个字符”。

（三）零比特填充法

面向比特的物理链路使用零比特填充法实现透明传输。以高级数据链路控制协议 **HDLC** 为例：

- HDLC 的帧定界标志字段为 01111110（即 0x7E，6 个连续 1）。
- 发送方：扫描数据载荷，只要出现 **5** 个连续的比特 **1**，就在其后插入一个比特 **0**。
- 接收方：扫描数据载荷，每发现 **5** 个连续的比特 **1**，就把它后面的一个比特 **0** 删除。

这样填充后，数据载荷中绝不会出现 6 个连续 1，也就绝不会出现与标志字段相同的比特组合，从而实现透明传输。

完整算例（**2013** 年统考真题改编）：HDLC 协议对 0111110001111110 组帧后对应的比特串为。

```
原始比特串：0 1 1 1 1 1 0 0 0 1 1 1 1 1 1 0
                          ^                   ^
第 5 个 1 之后插 0        插 0                插 0
```

逐段处理：

- 前 6 位 011111 中已有 5 个连续 1，在 0 前插入一个 0，得到 0111110；
- 中间 000 原样保留；
- 后段 0111111 中先出现 5 个连续 1，插入一个 0 变成 01111101，剩下的 1 继续保留；
- 最终 0 保留。

结果为 0111110 000 0111110 10，即 **01111100001111101 0**。

2013 年题 37（HDLC 零比特填充）

答案即为上述 01111100001111101 0。做这一类题的关键是从左到右扫描、计数器数连续的 **1**、到 **5** 就插 **0** 并把计数器清零。注意接收方解码时” 删 0” 后，删除操作本身不会让计数器继续累加，所以 0111110 只能解码出 011111（5 个 1），不会误删数据中的 0。

（四）违规编码法

在物理层编码时，使用不会出现在正常编码中的电平（违规编码）来表示帧的开始和结束。例如曼彻斯特编码中，每个码元中间必有跳变；若在应该跳变的位置不跳变，就是违规编码，可以用来做帧定界。

特点：不需要额外插入任何比特或字节，开销最小，但要求物理层编码本身有冗余的非法码字，通用性差。局域网的 IEEE 802.3 以太网和 802.11 无线局域网均只采用违规编码法（本质上通过前导码 + 帧间间隔来定界），不使用字符填充也不使用零比特填充。

四种组帧方法的对比

方法	适用链路	额外开销	主要缺点
字符计数法	面向字节	1 个计数字段	计数出错则彻底失步，无法恢复
字符填充法	面向字节	若干 ESC	数据中转义字符多时开销大
零比特填充法	面向比特	每 5 个 1 插 1 个 0	需硬件扫描，效率随数据模式波动
违规编码法	依赖物理层编码	几乎为 0	要求物理层有非法码字，通用性差

三、差错控制

（一）差错与误码率

实际的通信链路都不是理想的，比特在传输过程中可能产生比特差错（误码）：比特 1 变成 0，或 0 变成 1。原因包括噪声干扰、传输媒体质量等。一段时间内传输错误的比特数占传输比特总数的比率称为误码率（**Bit Error Rate**，**BER**）。

提高链路的信噪比可以降低误码率，但不可能使误码率降为零，因此需要用差错检测技术判断传输中是否出现了比特差错。

差错控制分为两类：检错编码（只发现错误，不能定位和纠正）和纠错编码（既能发现又能定位并纠正）。

（二）检错编码

*1.*奇偶校验码

奇校验：在待发送的数据后面添加 1 个校验位，使得添加后整个码字中比特 **1** 的个数为奇数。偶校验：添加 1 个校验位，使得添加后整个码字中比特 **1** 的个数为偶数。

设原始数据 $M=(M_{n-1}M_{n-2}\cdots M_0)$，$P$ 为校验位，则：

$$P_{奇}=M_{n-1}\oplus M_{n-2}\oplus\cdots\oplus M_0\oplus 1 \qquad P_{偶}=M_{n-1}\oplus M_{n-2}\oplus\cdots\oplus M_0$$

检错能力：如果传输中有奇数个比特发生误码，则码字中 1 的个数的奇偶性改变，可以检测出误码；如果有偶数个比特发生误码，奇偶性不变，无法检测出（漏检）。

奇偶校验的局限（必背结论）

- 只能检测奇数个比特错误，不能检测偶数个比特错误；
- 能发现错误，不能定位错误，更不能纠错；
- 编码效率高（只增加 1 bit），但检错能力弱，只适用于误码率很低的场合；
- 用奇偶校验能否检出错误，取决于改变的比特数量是奇数还是偶数，与具体是哪几位改变、改变成什么值无关。

完整算例：待发送数据块 $M=1010110$，采用偶校验，校验位附加在数据块末尾。

M 中 1 的个数为 4（偶数），要使整个码字 1 的个数仍为偶数，校验位 $P=0$，故实际发送的比特序列为 `10101100`。

若某次传输后接收方收到 `10101110`，其中 1 的个数为 5（奇数），可判定发生了差错。但若收到的是 `10001110`（同样 5 个 1，但两侧各错 1 位变成了 2 位错误）——实际上 `10101100` → `10001110` 改变了 3 位，仍能被检出；而 `10101100` → `00101110` 改变了 2 位，1 的个数仍为 4，漏检。

*2.*循环冗余校验 *CRC*

数据链路层广泛使用漏检率极低的循环冗余校验（**Cyclic Redundancy Check**，**CRC**）。

基本原理：

- 收发双方约定一个 $(k-1)$ 阶生成多项式 $G(X)$（最高次为 $k-1$，对应比特串共 k 位）。
- 发送方基于待发数据 M 和 $G(X)$ 计算出 $k-1$ 位的冗余码（帧检验序列 **FCS**），附加在 M 后面一起发送。
- 接收方用同一 $G(X)$ 对收到的 $M'+P'$ 做模 2 除法，余数为 **0** 判为无差错，余数不为 **0** 判为有差错。

发送方 **CRC** 操作的四步流程：

1. 构造被除数：在待发送数据 M 后面添加 $k-1$ 个 **0**（$k-1$ 即 $G(X)$ 的最高次数）。
2. 构造除数：$G(X)$ 各项系数构成的比特串（共 k 位）作为除数。
3. 模 **2** 除法：做 k 位一组的上商除法。模 **2** 除法既不借位也不比较大小，每一位的运算就是异或。求得的余数位数必须与 $G(X)$ 的最高次数相同，位数不够时在前面补 **0**。
4. 发送：把余数（FCS）附加到 M 后面一起发送。

接收方 **CRC** 操作：对收到的整个比特串（数据 + FCS）做同样的模 2 除法，余数为 0 认为无差错，余数不为 0 认为有差错，直接丢弃该帧。

完整手算算例一（发送方）：待发送数据 $M = 101001$，生成多项式 $G(X) = X^3 + X^2 + 1$，求冗余码。

第 1 步，$G(X)$ 最高次为 3，故在 M 后补 3 个 0，被除数为 `101001000`。第 2 步，$G(X) = X^3 + X^2 + X^0$，各项系数为 `1 1 0 1`，除数 $= 1101$。第 3 步，模 2 除法竖式：

```
          1 1 0 0 1 1
      ┌─────────────────────────────────
1101  │ 1 0 1 0 0 1 0 0 0
      1 1 0 1
      ─────────────────
        1 1 1 0
        1 1 0 1
        ─────────────────
            1 1 1 0
            1 1 0 1
            ─────────────────
                1 1 0 0
                1 1 0 1
                ─────────────────
                    0 0 1   <- 余数
```

逐步说明（每次取够 4 位与 1101 异或）：

- 首位为 1，商 1，`1010 ⊕ 1101 = 0111`，落下一位得 `1110`；
- 首位为 1，商 1，`1110 ⊕ 1101 = 0011`，落下一位得 `0110`（首位为 0，商 0，直接落下一位）得 `1100`；
- 首位为 1，商 1，`1100 ⊕ 1101 = 0001`，落下一位得 `0010`（首位 0 商 0），再落下一位得 `0100`（首位 0 商 0），再落下一位得 `1000`；
- 末次首位为 1，商 1，`1000 ⊕ 1101 = 0101`，继续得到最终余数 `001`。

第 4 步，余数 `001` 只有 3 位，正好等于 $G(X)$ 的最高次数 3，不需要补 **0**。故 FCS = `001`。

实际发送的比特串为 M 拼上 **FCS**：`101001 001`，即 `101001001`。

完整手算算例二（接收方）：

- 若接收方收到 `101001001`，用它除以 `1101`：

```
          1 1 0 0 1 1
      ┌─────────────────────────────────
1101  │ 1 0 1 0 0 1 0 0 1
      1 1 0 1
      ─────────────────
        1 1 1 0
        1 1 0 1
        ─────────────────
            1 1 1 0
            1 1 0 1
            ─────────────────
                1 1 0 0
                1 1 0 1
```

```
                  ----------------
                    0 0 0 0   -> 余数 000
```

余数为 000，判定传输过程没有产生误码。

- 若接收方收到 101101011（其中两位发生跳变），同样除以 1101，最终余数为 001（不为 0），判定传输过程产生了误码，丢弃该帧。

完整算例三（含补 **0** 的检验情形）：某数据链路层使用 CRC 校验，待发送原始数据帧为 110101，生成多项式 $G(X)=X^4+X+1$（即 10011），求 CRC 校验码。

$G(X)$ 最高次为 4，被除数为 110101 + 0000 = 1101010000，除数 10011。模 2 除法：

```
            1 1 0 0 0 1
          ┌──────────────────────────────
10011     │ 1 1 0 1 0 1 0 0 0 0
          1 0 0 1 1
          ----------------------
            1 0 0 0 1
            1 0 0 1 1
            ----------------------
                0 0 0 0 0
                0 0 0 0 0
                ----------------------
                        0 1 0 0 0
                        0 0 0 0 0
                        ----------------------
                          1 0 0 0 0
                          1 0 0 1 1
                          ----------------------
                                0 1 1   -> 先得部分余数，继续补齐
```

继续做完，余数为 0000，**CRC** 校验码为 0000。

2023 年题 37（CRC 检错）

生成多项式 $G(X)=X^4+X+1$（即 10011），判断下列哪个接收比特串可断定未发生错误：A. 101110000　B. 101110100　C. 101111000　D. 101111100

做法：把每个选项分别除以 10011，只有余数为 **0000** 的那个才无差错。计算得：101110000 ÷ 10011 余 1100；101110100 余 1000；101111000 余 0100；**101111100** 余 0000。故答案为 **D**。

关于 CRC 与 FCS 的必背结论

- **FCS** 是附加在数据后面的冗余码，**CRC** 是获得 **FCS** 的一种常用方法，但 **CRC** 不是获得 **FCS** 的唯一方法，两者不能等同。
- CRC 检错只能判断” 有无差错”，不能确定是哪一位或哪几位出错，因此 **CRC** 只能检错不能纠错（虽然 CRC 理论上具备一定纠错潜力，但实际不使用）。
- 只要生成多项式选得合适且位数足够多，检测不到差错的概率可以做到极小。
- 仅用 CRC 只能做到无差错接收（**accept**）：凡是接收端接受的帧，都能以非常接近于 1 的概率认为没有产生差错；要做到**”** 可靠传输**”**（发送什么就收到什么），必须再加上序号、确认和重传机制。
- $(k-1)$ 阶的生成多项式能检测出：全部单比特错、全部奇数位错（当 $G(X)$ 含 $(X+1)$ 因子时）、长度不超过 $k-1$ 的全部突发错。

*3.*校验和

校验和（**Checksum**）的通用做法是：把数据划分为若干个固定的位组（如 16 位一组），发送方对各组做反码求和（累加时若有进位则回卷加到最低位），再对结果取反码，作为校验和字段随数据一起发送。接收方把所有分组连同校验和一起按同样方式求和再取反，结果为全 0 则认为无差错。

校验和的特点：算法简单、软硬件开销小，但检错能力弱于 **CRC**。它是网络层（IP 首部校验和）和运输层（TCP、UDP 校验和）使用的检错方法；数据链路层主要使用 **CRC**。

校验和的范围

TCP/UDP 的校验和计算要包括伪首部，而 **IP** 首部校验和只校验 **IP** 首部，不包含伪首部、也不校验数据部分。这是网络层/运输层章节的常考点，不要与数据链路层的 CRC 混淆。

（三）纠错编码：海明码

海明码（**Hamming Code**）是最经典的纠错编码，它通过增加若干校验位，使得接收方不但能发现差错，还能定位并纠正哪怕 1 位差错。

*1.*校验位位数 k 的确定与推导

设有 m 位数据位、k 位校验位，码字总长 $n=m+k$。

推导思路：接收方用 k 个校验位可以产生 k 个校验结果（称为校验因子，其组合即指误字/伴随式）。这 k 个二进制位最多能表示 2^k 种不同的状态。这 2^k 种状态必须覆盖”没有出错”（1 种）以及”n 位中任意 **1** 位出错”（n 种）的全部情况，因此必须满足：

$$2^k \geq n+1=m+k+1$$

取值表（考试常直接查用，记住 $m=4$ 时 $k=3$ 即可）：

数据位 m	**1**	**2~4**	**5~11**	**12~26**
校验位 k	2	3	4	5
码字总长 n	3	5~7	9~15	17~31

例如 $m=4$：$2^k \geq 4+k+1$。取 $k=2$ 时 $4 \geq 7$ 不成立；取 $k=3$ 时 $8 \geq 8$ 成立，故 $k=3$，总长 $n=7$。这就是众所周知的海明码 **(7,4)**。

*2.*校验位的位置

校验位必须放在码字中序号为 $2^0, 2^1, 2^2, \ldots, 2^{k-1}$（即 **1**、**2**、**4**、**8**、…）的位置上，其余位置放数据位。总长 $n=7$ 时：

码字位序号	**1**	**2**	**3**	**4**	**5**	**6**	**7**
内容	c_1	c_2	d_1	c_4	d_2	d_3	d_4

为什么校验位放在 2 的幂位置？因为每个码字位序号都可以唯一地分解为若干 2 的幂之和，例如 $5=1+4$、$6=2+4$、$7=1+2+4$。让 c_1 负责所有序号含 2^0 的位、c_2 负责所有序号含 2^1 的位、c_4 负责所有序号含 2^2 的位，就能保证任意 **1** 位出错时，受影响的校验位组合唯一，从而由校验结果反推出出错位置。

具体分组（偶校验分组）为：

$$c_1 = d_1 \oplus d_2 \oplus d_4 \quad (\text{覆盖位} 1,3,5,7)$$

$$c_2 = d_1 \oplus d_3 \oplus d_4 \quad (\text{覆盖位} 2,3,6,7)$$

$$c_4 = d_2 \oplus d_3 \oplus d_4 \quad (\text{覆盖位} 4,5,6,7)$$

校验位的取值规则：让每一组中"1 的个数" 为偶数（若采用奇校验则让每组中 1 的个数为奇数）。等价地，c_i 等于它覆盖的所有数据位的异或值。

*3.*编码完整算例

设待发送数据 $D = d_1d_2d_3d_4 = 1101$。

按位置表填入：$d_1 = 1$（第 3 位）、$d_2 = 1$（第 5 位）、$d_3 = 0$（第 6 位）、$d_4 = 1$（第 7 位）。计算三个校验位：

$$c_1 = d_1 \oplus d_2 \oplus d_4 = 1 \oplus 1 \oplus 1 = 1$$

$$c_2 = d_1 \oplus d_3 \oplus d_4 = 1 \oplus 0 \oplus 1 = 0$$

$$c_4 = d_2 \oplus d_3 \oplus d_4 = 1 \oplus 0 \oplus 1 = 0$$

最终海明码为：

位序号	**1**	**2**	**3**	**4**	**5**	**6**	**7**
位值	$c_1 = 1$	$c_2 = 0$	$d_1 = 1$	$c_4 = 0$	$d_2 = 1$	$d_3 = 0$	$d_4 = 1$

即海明码为 `1010101`。

自检：$(1,3,5,7)$ 位上 1 的个数：$1,1,1,1$ 共 4 个，偶数 ✓；$(2,3,6,7)$ 位上 1 的个数：$0,1,0,1$ 共 2 个，偶数 ✓；$(4,5,6,7)$ 位上 1 的个数：$0,1,0,1$ 共 2 个，偶数 ✓。

*4.*接收方的检错与纠错完整算例

接收方对收到的 7 位码字按同样的三组重新计算校验结果，得到三个校验因子 G_1, G_2, G_4，把它们按 $G_4G_2G_1$ 拼成指误字：

$$G_1 = \bigoplus(1,3,5,7)\text{ 位} \qquad G_2 = \bigoplus(2,3,6,7)\text{ 位} \qquad G_4 = \bigoplus(4,5,6,7)\text{ 位}$$

- 指误字 $= 000$：无差错。
- 指误字 $\neq 000$：其十进制值就是出错位的序号，把它取反即完成纠错。

情形一：接收方收到 1010101（正确）。

$G_1 = 1 \oplus 1 \oplus 1 \oplus 1 = 0$；$G_2 = 0 \oplus 1 \oplus 0 \oplus 1 = 0$；$G_4 = 0 \oplus 1 \oplus 0 \oplus 1 = 0$。指误字 000，判为无差错，取出第 3、5、6、7 位得数据 1101。

情形二：第 **5** 位在传输中发生跳变，接收方收到 1010001（原 1010101，第 5 位由 1 变 0）。

$G_1 = 1 \oplus 1 \oplus \underline{0} \oplus 1 = 1$；$G_2 = 0 \oplus 1 \oplus 0 \oplus 1 = 0$；$G_4 = 0 \oplus \underline{0} \oplus 0 \oplus 1 = 1$。

指误字 $= G_4G_2G_1 = 101$，十进制为 **5**，即第 **5** 位出错。把第 5 位取反（0 改为 1），得 1010101，取出数据位得 1101，纠错成功。

情形三：第 **3** 位在传输中发生跳变，接收方收到 1000101（第 3 位由 1 变 0）。

$G_1 = 1 \oplus \underline{0} \oplus 1 \oplus 1 = 1$；$G_2 = 0 \oplus \underline{0} \oplus 0 \oplus 1 = 1$；$G_4 = 0 \oplus 1 \oplus 0 \oplus 1 = 0$。指误字 $= 011$，十进制为 **3**，第 3 位出错，取反后恢复 1010101 ✓。

情形四：第 **7** 位出错，接收方收到 1010100。

$G_1 = 1 \oplus 1 \oplus 1 \oplus \underline{0} = 1$；$G_2 = 0 \oplus 1 \oplus 0 \oplus \underline{0} = 1$；$G_4 = 0 \oplus 1 \oplus 0 \oplus \underline{0} = 1$。指误字 $= 111$，十进制为 **7**，第 7 位出错，取反恢复 ✓。

海明码的边界：两位错会" 纠错纠错"

海明码只能纠正 **1** 位错。若发生了 2 位错，指误字虽然非 0（能发现" 有错"），但它指向的位置并不是真正出错的位置，此时执行" 纠错" 反而会把第 3 位也改错，越纠越错。另外要注意：n 位码字可以出现 n 种单比特错 **+1** 种无错，共 $n+1$ 种情况，而 k 个校验位只能给出 2^k 种状态，这就是 $2^k \geq n+1$ 的另一种解释。

*5.*海明距离与检错纠错能力

海明距离（汉明距离）：两个合法码字之间对应位取值不同的位数。例如 1101 与 1001 海明距离为 1。

一个编码方案的最小海明距离 d 定义为所有合法码字两两之间海明距离的最小值。设：

- e：要检测的差错位数；
- t：要纠正的差错位数；
- s：在纠正 t 位错的同时还要检测的差错位数。

则有：

$$d \geq e+1 \qquad d \geq 2t+1 \qquad d \geq t+s+1\ (s>t)$$

含义解释：

- $d \geq e+1$：检错不需要定位。要让 e 位错之后不可能变成另一个合法码字，两个合法码字之间至少要差 $e+1$ 位。
- $d \geq 2t+1$：纠错要定位。以每个合法码字为球心、半径 t 画" 错球"，所有错球必须互不相交，即两球心距离至少为 $2t+1$。
- $d \geq t+s+1$：既要纠正 t 位又要检测 s 位，两个条件叠加。

例如：要求检测 2 位错，则 $d \geq 3$；要求纠正 1 位错，则 $d \geq 3$；要求纠正 1 位错同时检测 2 位错，则 $d \geq 1+2+1 = 4$。

编码能力选择题

" 如果一个编码方案被设计用来检测 e 位错误、纠正 t 位错误、同时检测 s 位错误且纠正 t 位错误，其最

小海明距离 d 分别不小于（ ）” 标准答案：$e+1$、$2t+1$、$t+s+1$。记口诀：检错 **+1**，纠错 **2** 倍 **+1**，检纠结合相加 **+1**。

四、流量控制与可靠传输机制

（一）流量控制、可靠传输与滑动窗口机制

流量控制：控制发送方的发送速率，使接收方来得及接收（接收方的缓冲区不会溢出）。

可靠传输：发送方发送什么，接收方最终就能收到什么，即做到不丢失、不出错、不重复、不乱序。

传输差错包括四类：比特差错（误码）、分组丢失、分组失序、分组重复。其中分组丢失、失序、重复一般不在数据链路层出现，而是在其上层出现。因此可靠传输服务并不局限于数据链路层，其上各层均可选择实现。

- 有线链路误码率低，不要求数据链路层提供可靠传输服务（如以太网）；即使出错，也由上层的 TCP 负责。
- 无线链路误码率高，要求数据链路层必须提供可靠传输服务（如 802.11 无线局域网）。

*1.*停等协议与滑动窗口的关系

停止-等待协议是发送窗口 $W_T=1$、接收窗口 $W_R=1$ 的滑动窗口协议的特例。也就是说，滑动窗口机制是更一般的框架，三种可靠传输协议都可以用” 窗口 + 序号 + 确认 + 重传” 统一描述。

发送窗口：发送方维护的一个序号区间，只有序号落在发送窗口内的帧才允许被发送（已发送但未收到确认的帧仍占窗口）。接收窗口：接收方维护的一个序号区间，只有序号落在接收窗口内且无差错的帧才被接收。

三条核心规律：

1. 接收窗口向前滑动，一定伴随着确认报文的发送（告知发送方我已收好）。
2. 发送窗口向前滑动，一定是因为收到了按序到达的确认。
3. 发送窗口内的序号都是**”** 已发送但未被确认**”** 或**”** 允许发送**”** 的；接收窗口内的序号都是**”** 允许接收**”** 的。

*2.*窗口大小与序号位数的关系 $W_T+W_R\le 2^n$

设用 n 个比特给帧编号，则序号空间大小为 2^n（序号 $0\sim 2^n-1$，循环使用）。为了避免接收方把重传的旧帧误判为新帧，必须满足：

$$W_T+W_R\le 2^n$$

推导（反证法）：设 $W_T+W_R>2^n$。

发送方发完窗口内 W_T 个帧后，若所有确认帧全部丢失，发送方超时后必须重传这 W_T 个帧。此时发送方已经把窗口滑到 W_T+1 开始的 W_T 个新序号上。由于 $W_T+W_R>2^n$，发送方” 新窗口的序号集合” 与接收方” 当前接收窗口的序号集合” 必然有交集。

接收方无法区分是旧帧的重传还是新帧的首次到达（因为序号相同而窗口位置相同），于是可能把重传的旧帧当作新帧交付给上层，造成重复接收 / 数据错乱。所以必须 $W_T+W_R\le 2^n$。

这一约束的直接后果：

- 停止-等待协议：$W_T=1$，故 $W_R\le 2^n-1$；实际取 $W_R=1$，$n=1$ 即可（序号 0、1）。

- 后退 **N** 帧协议 **GBN**：$W_R = 1$，故 $W_T \leq 2^n - 1$。
- 选择重传协议 **SR**：一般取 $W_T = W_R$，故 $W_T = W_R \leq 2^{n-1}$。

三个窗口上限（必背）

协议	W_T	W_R	$n=3$ 时的最大值
停止-等待	1	1	$W_T = W_R = 1$
GBN	$1 < W_T \leq 2^n - 1$	1	$W_T = 7$，$W_R = 1$
SR	$W_T = W_R \leq 2^{n-1}$	同左	$W_T = W_R = 4$

2019 年题 35

”对于滑动窗口协议，如果分组序号采用 3 比特编号，发送窗口大小为 5，则接收窗口最大是（　）”由 $W_T + W_R \leq 2^3 = 8$，得 $W_R \leq 3$，答案为 **3**。

（二）停止-等待协议

定义：发送方每发送完一个数据分组就必须停下来，等待接收方发来的确认（ACK）或否认（NAK）分组，收到确认后才能发送下一个。

一次完整的通信包含三种机制：确认/否认、超时重传、给数据分组编号。

*1.*四种情况的图解说明

情况一：正常情况（无差错、无丢失）发送方发送 DATA0 → 接收方收到后回 ACK0 → 发送方收到 ACK0 后发送 DATA1 → 接收方回 ACK1 → ⋯如此往复。此时信道利用率为 $U = \frac{T_D}{T_D + RTT + T_A}$。

情况二：数据分组丢失或出错 → 超时重传发送方发送 DATA0 后即启动超时计时器（**Timeout Timer**），并设置超时重传时间（**RTO**）。若 DATA0 在传输中丢失：

```
发送方                接收方
  |---- DATA0 ----X      |          (DATA0 丢失，接收方什么都没收到，不会发 ACK)
  |  启动计时器          |
  |    ... 超时...       |
  |---- DATA0 ----  -->  |          重传
  |<---- ACK0 -----      |
```

要点：接收方收不到数据就不会发 ACK，发送方若不加措施将永远等待，所以必须有超时计时器。RTO 一般设置为略大于收发双方的平均往返时间 **RTT**。数据链路层点对点 RTT 比较固定，RTO 好设定；运输层端到端 RTT 不确定，RTO 难设定。

情况三：确认分组丢失 → 超时后重传，接收方必须能识别重复帧发送方发出的 DATA0 已被正确接收，接收方回送的 ACK0 在途中丢失：

```
发送方                接收方
  |---- DATA0 ---->  |  正确接收 DATA0，交付上层
  |<--- ACK0 ---X    |  ACK0 丢失
  |  超时            |
  |---- DATA0 ---->  |  又收到一个 DATA0（重复帧！）
  |<---- ACK0 -----  |  丢弃重复帧，但仍需回 ACK0
```

必须给数据分组编号：停止-等待协议只需 **1** 个比特编号（序号 0 和 1），因为只要保证相邻两次发送的序号不同即可。接收方发现收到的序号与上次相同，就判定是重复帧并丢弃，同时重发一次确认（否则发送方会一直超时）。

情况四：确认分组迟到（确认迟到）发送方因超时而重传了 DATA0，接收方再次回送 ACK0；随后原来那个迟到的 **ACK0** 才到达发送方。发送方收到两个 ACK0：

```
发送方                接收方
  |---- DATA0 ---->  |
  |  超时              |
  |---- DATA0 ---->  |  收下并丢弃重复，回 ACK0
  |<--- ACK0(迟到的)-|  <- 第一个 ACK0 迟到
  |<--- ACK0(新的) --|
```

必须给确认分组编号：发送方要能判断收到的确认是不是重复的，所需比特数与数据分组相同。但在数据链路层一般不会出现确认迟到的情况，因此在数据链路层实现停止-等待协议可以不给确认分组编号；在运输层则必须编号。

停止-等待协议的四个易错点

1. 使用超时重传后可以不用否认机制（**NAK**），协议实现更简单；但若误码率较高，使用 NAK 可以让发送方在超时前就尽快重传。
2. 停止-等待协议属于自动请求重传（**Automatic Repeat reQuest，ARQ**）：重传请求是发送方自动发起的，不是接收方要求发送方重传某个具体误码帧（否则就成了” 选择重传”）。
3. ” 确认分组迟到” 和” 确认分组丢失” 是两种不同的情形，前者要求对确认编号，后者要求对数据编号。
4. RTO 必须大于 RTT，否则会大量无谓重传。

*2.*信道利用率计算

设：

- T_D：发送一个数据帧所需的时间（发送时延）$=\dfrac{L}{R}$，L 为帧长，R 为数据传输速率；
- RTT：往返时间（$=2\times$ 单向传播时延 τ，若题目给出链路两端距离与信号传播速度则 $RTT=2d/v$）；
- T_A：发送一个确认帧所需的时间（题目若说” 忽略确认帧的传输时延” 则 $T_A=0$）。

发送周期 = 从开始发送第一个数据帧的第一个比特起，到收到该帧的第一个确认帧为止所经历的时间：

$$T_{\text{周期}}=T_D+RTT+T_A$$

信道利用率：

$$U=\frac{T_D}{T_D+RTT+T_A}\times 100\%$$

含义：分子是” 真正用于传送有用数据的时间”，分母是整个发送周期。U 就是信道被有效数据占用的比例。

若出现超时重传，则有效数据的信道利用率还要降低。在 **RTT** 相对较大的情况下（如卫星链路），停等协议效率极低，应改用 GBN 或 SR 协议。

2018 年题 36（停等协议信道利用率）

主机甲采用停等协议向主机乙发送数据，数据传输速率 $R=3$ kbps，单向传播时延 $\tau=200$ ms，忽略确认帧的传输时延。当信道利用率 $U=40\%$ 时，数据帧的长度为多少?

$RTT=2\tau=400$ ms $=0.4$ s，$T_A=0$。设帧长为 L bit，则 $T_D=L/3000$ s。

$$0.4=\frac{L/3000}{L/3000+0.4}\Rightarrow 0.4\times\frac{L}{3000}+0.16=\frac{L}{3000}\Rightarrow 0.6\times\frac{L}{3000}=0.16\Rightarrow L=800\text{ bit}$$

答案 **800** 比特。

（三）后退 N 帧协议（GBN）

基本思想：采用流水线传输方式，允许发送方在未收到确认的情况下连续发送多个帧，用发送窗口限制连续发送的数量，属于连续 **ARQ** 协议。

工作规则：

- 接收窗口 $W_R=1$，接收方只能按序接收。若收到一个失序的帧（例如期待 n 却收到 $n+1$），就丢弃该帧，并为最后一个按序正确收到的帧发送确认。
  - 例如接收方已正确收到 `0,1,2` 号帧，随后收到 `4` 号帧：由于 `3` 号帧丢失/出错，接收方丢弃 `4` 号帧，并重复发送 `ACK2`。
- 发送方采用累积确认：ACKn 表示”序号 n 及之前的所有帧都已正确接收”。
- 发送方超时后，重新发送从出错帧开始的全部已发送帧（这就是”后退 N 帧”）。

累积确认的优点：减少注入网络的确认数量；即使确认丢失也可能不必重传数据。累积确认的缺点：不能及时准确地把接收方已正确接收的帧数告诉发送方。

*1.*为什么 $W_T\le 2^n-1$

若 $W_T=2^n$（即取满整个序号空间），则发送方发完 $0\sim 2^n-1$ 全部帧后，接收方已正确收下全部帧并把接收窗口滑到序号 0。此时若所有确认帧丢失，发送方超时重传 `0` 号帧。

接收方的接收窗口此刻正好停在序号 0，无法区分这个 `0` 号帧是重传的旧帧还是新一批的新帧，会把它当作新帧交付给上层，造成重复。因此 W_T 最多取 2^n-1。

GBN 窗口上限

采用 n 比特编号时，$1<W_T\le 2^n-1$，$W_R=1$。$n=3$ 时 W_T 取值范围为 $2\sim 7$。

*2.*发送窗口滑动过程的手工模拟

设 $n=3$（序号 $0\sim 7$），$W_T=5$，当前发送窗口覆盖序号 $\{0,1,2,3,4\}$。

第 **1** 步：发送方连续发送 `0,1,2,3,4` 五个帧，窗口内全部变为”已发送待确认”，发送窗口不动。

第 **2** 步：接收方正确收到 `0,1,2`，返回累积确认 `ACK2`。发送方收到 `ACK2`，说明 `0,1,2` 已被确认，发送窗口向前滑动 **3** 个位置，变为 $\{3,4,5,6,7\}$。

第 **3** 步：发送方可以继续发送新落入窗口的 `5,6,7`。

第 **4** 步：假设此时 `3` 号帧在途中丢失，接收方收到 `4` 号帧时发现失序（期待 `3`），丢弃 `4` 号帧并重复发送 `ACK2`。

第 **5** 步：发送方超时（`3` 号帧的计时器到期），重传 **`3,4,5,6,7`** 全部已发送但未确认的帧。注意此时哪怕 `4,5,6,7` 早已正确到达过接收方，它们也要被重传——这就是 GBN”一个帧出错引起大量重传”的代价。

第 **6** 步：接收方按序收下 3,4,5,6,7，回 ACK7。发送窗口向前滑动 5 个位置，变为 $\{0,1,2,3,4\}$（序号回绕）。

2009 年题 35（GBN 累积确认与重传数量）

”数据链路层使用 GBN 协议，发送方已经发送了编号 0~7 的帧。当计时器超时时，发送方只收到了 0、2、3 号帧的确认，则发送方需要重发的帧数是多少？”

解析：GBN 采用累积确认，收到 ACK3 就意味着序号 3 及其之前的所有帧都已正确接收。所以 0,1,2,3 全部算已确认，需要重传的是 4,5,6,7，共 **4** 个帧。

GBN 的两个陷阱

1. 累积确认的含义：只收到 ACK2 和 ACK3 时，ACK3 已经包含了 0,1,2,3，不要再去数 ACK2。
2. **GBN** 在信道质量差时效率可能低于停等：因为一个帧出错就要重传一串帧。题目说”GBN 一定比停等效率高” 是错的。

（四）选择重传协议（SR）

基本思想：为了只重传真正出错的帧，接收方不再采用累积确认，而对每一个正确收到的帧逐一确认；同时接收方设置接收窗口，允许缓存失序到达但无差错的帧。

工作规则：

- 发送方：可在未收到确认时把发送窗口内的多个帧全部发出；只有按序收到对已发送帧的确认时，发送窗口才向前滑动；若收到未按序到达的确认，则记录它（避免其对应帧被超时重传），但窗口不滑动。
- 接收方：可接收未按序到达但无差错且序号落在接收窗口内的帧，先缓存起来；只有当按序收到帧后，接收窗口才向前滑动到相应位置。
- 收发双方均需为每个已发送/已接收的帧设置计时器（实践中常只对最早未确认的帧设一个计时器）。

*1.*为什么 $W_T = W_R \le 2^{n-1}$

由 $W_T + W_R \le 2^n$，若取 $W_T = W_R = W$，则 $2W \le 2^n$，即 $W \le 2^{n-1}$。

直观解释：若 $W = 2^{n-1} + 1$，接收窗口与发送窗口的序号区间总会重叠。当接收方缓存了一批失序帧、而发送方重传了另一批帧时，序号会”撞车”，接收方无法判断某个序号是”新帧”还是”旧帧的重传”。

SR 窗口上限

采用 n 比特编号时，$W_T = W_R \le 2^{n-1}$。$n = 3$ 时最大各为 **4**。

*2.*发送窗口滑动过程的手工模拟

设 $n = 3$，$W_T = W_R = 4$，初始发送窗口 = 接收窗口 = $\{0,1,2,3\}$。

第 **1** 步：发送方连续发送 0,1,2,3 四个帧。

第 **2** 步：0 号帧丢失，1,2,3 号帧正确到达接收方。接收方对 1,2,3 逐一确认：

- 收到 1 号帧（失序）→ 缓存，回 ACK1，接收窗口不滑动（因为期待的是 0）；
- 收到 2 号帧（失序）→ 缓存，回 ACK2，接收窗口不滑动；
- 收到 3 号帧（失序）→ 缓存，回 ACK3，接收窗口不滑动。

第 **3** 步：发送方收到 ACK1,ACK2,ACK3，这些都是未按序到达的确认，于是只做记录，发送窗口不滑动。

第 **4** 步：0 号帧的计时器超时，发送方只重传 0 号帧。

第 **5** 步：接收方收到 0 号帧，此时按序到达，于是把缓存的 0,1,2,3 一起按序交付上层，接收窗口向前滑动 **4** 个位置，变为 $\{4,5,6,7\}$，并回 ACK0。

第 **6** 步：发送方收到按序的 ACK0，发送窗口向前滑动到 $\{4,5,6,7\}$，可以继续发送。

2011 年题 35（SR 的重传数量）

”数据链路层采用选择重传协议（SR）传输数据，发送方已发送了 0~3 号数据帧，现已收到 1 号帧的确认，而 0、2 号帧依次超时，则此时需要重传的帧数是多少？”

解析：SR 是逐一确认、只重传出错帧。已发送 4 帧中 1 号已被确认，0、2 号超时需重传，3 号尚未超时也未被确认（无需重传）。所以需要重传 **2** 个帧。

2024 年题 37（SR 窗口滑动）

”主机甲通过 SR 滑动窗口协议向主机乙发送帧，F_x 为数据帧，ACK_x 为确认帧，x 是 3 比特序号。乙只对正确接收的数据帧进行独立确认。发送窗口与接收窗口大小相同且均为最大值。”

3 比特编号下 SR 的最大窗口为 $2^{3-1}=4$。初始发送窗口为 $\{0,1,2,3\}$。当 ACK0 到达后窗口基序号从 0 滑到 1，允许新发送 F4；而 F1 丢失并超时，SR 只重传 **F1**。故两个时刻发送的帧分别是 **F4**、**F1**。

（五）三种协议的对比与信道利用率

*1.*完整对比表

对比项	停止-等待	后退 **N** 帧 **GBN**	选择重传 **SR**
发送窗口 W_T	1	$1 < W_T \le 2^n - 1$	$W_T \le 2^{n-1}$
接收窗口 W_R	1	1	$W_R = W_T \le 2^{n-1}$
确认方式	逐个确认	累积确认	逐帧独立确认
是否使用 NAK	一般不用	不用	常配 NAK，超时 + NAK 双触发
重传范围	仅当前帧	从出错帧起的全部已发送帧	仅出错的那一帧
接收方缓存失序帧	无（也不需要）	不缓存，直接丢弃	缓存失序帧
发送方窗口滑动条件	收到 ACK	收到按序的累积确认	收到按序的确认
接收方窗口滑动条件	收到正确帧	收到正确帧	按序收齐后才滑动
需要的序号比特	1 bit	n bit	n bit
信道利用率	最低	较高（信道好时）	最高
适用场合	帧短、RTT 小、误码率低	信道误码率低	信道误码率较高、带宽时延积大

*2.*三个协议的信道利用率公式

设 T_D 为一帧的发送时延，T_A 为确认帧的发送时延，RTT 为往返时间。

停止-等待：

$$U_{SW} = \frac{T_D}{T_D + RTT + T_A}$$

后退 **N** 帧（**GBN**）：一个发送周期内若发送方发送了 k 个帧（$k \le W_T$），则

$$U_{GBN} = \frac{k \cdot T_D}{T_D + RTT + T_A}$$

当发送窗口足够大，使得整个发送周期都能被连续发送的帧填满时（即窗口内帧的发送总时间 $\ge T_D + RTT + T_A$），信道利用率达到最大 $U_{max} = 1$；否则取 $k = W_T$ 计算。设" 充满链路所需的最小窗口" 为

$$W_{need} = \left\lceil \frac{T_D + RTT + T_A}{T_D} \right\rceil$$

则：$W_T \ge W_{need}$ 时链路可被连续占满，$U = 1$；$W_T < W_{need}$ 时 $U = \dfrac{W_T \cdot T_D}{T_D + RTT + T_A}$。

选择重传（**SR**）：同样的公式，但由于 SR 允许接收窗口大于 1、接收方可以缓存，其实际吞吐率通常高于 **GBN**（尤其在误码率较高时优势明显）。

把利用率换算成**"** 有效数据传输速率**"**：

$$\text{有效数据传输速率} = U \times R$$

*3.*信道利用率/吞吐率完整算例（含 k 段链路）

通用公式的推导：设主机甲经过 k 段链路向主机乙发送数据，每段链路的单向传播时延为 τ、传输速率为 R，数据帧长为 L，确认帧时延忽略不计。

- 一帧的发送时延 $T_D = \dfrac{L}{R}$（近似认为中间结点转发时延可忽略）；
- 从甲开始发送第一个帧，到甲的第一个确认帧回到甲，所经历的时间为 $T_{\text{周期}} = T_D + 2k\tau$（$2k\tau$ 即整条路径的往返传播时延）；
- 在一个周期内，甲最多能连续发送 W_T 个帧，但受周期长度限制，实际能连续发送的帧数最多为 $\left\lfloor \dfrac{T_{\text{周期}}}{T_D} \right\rfloor$。

因此有效数据传输速率为：

$$R_{\text{有效}} = \min\left(W_T,\ \left\lceil \frac{T_D + 2k\tau}{T_D} \right\rceil\right) \times \frac{L}{T_{\text{周期}}} = \min\left(W_T,\ \frac{T_D + 2k\tau}{T_D}\right) \times \frac{L}{T_D + 2k\tau}$$

当窗口足够大（$W_T \ge \dfrac{T_D + 2k\tau}{T_D}$）时窗口不再是瓶颈，此时

$$R_{\text{有效}} = \frac{W_T \cdot L}{T_D + 2k\tau}\text{（受窗口限制）} \qquad \text{或} \qquad R_{\text{有效}} = R\text{（链路被完全占满）}$$

算例一（三段链路）：主机甲经 3 段链路向主机乙发送数据，每段链路长 1000 m、信号传播速度 2×10^8 m/s、传输速率 10 Mbps，GBN 协议发送窗口 $W_T = 8$，数据帧长 1500 B，确认帧时延忽略不计。求甲的有效数据传输速率。

第 1 步，单向传播时延 $\tau = 1000/(2 \times 10^8) = 5\ \mu s$，全路径往返 $2k\tau = 2 \times 3 \times 5 = 30\ \mu s$。

第 2 步，$T_D = \dfrac{1500 \times 8}{10 \times 10^6} = 1.2\ \text{ms} = 1200\ \mu s$。

第 3 步，$T_{\text{周期}} = T_D + 2k\tau = 1200 + 30 = 1230\ \mu s$。

第 4 步，一个周期内能连续发送的帧数上限 $= \lfloor 1230/1200 \rfloor = 1$，而 $W_T = 8$ 远大于该值，所以瓶颈在链路时延而非窗口，发送方必须等确认，只能发 1 帧。

第 5 步，$R_{\text{有效}} = \dfrac{1 \times 1500 \times 8\ \text{bit}}{1230\ \mu s} \approx 9.76\ \text{Mbps}$。

算例二（卫星链路定窗口）：主机甲通过 128 kbps 卫星链路，采用滑动窗口协议向乙发送数据，链路单向传播延迟 250 ms，帧长 1000 字节，不考虑确认帧开销。为使链路利用率不小于 80%，帧序号的比特数至少是多少?

$\tau = 250\ \text{ms}$，$RTT = 500\ \text{ms} = 0.5\ \text{s}$，$T_D = \dfrac{1000 \times 8}{128 \times 10^3} = 62.5\ \text{ms}$。

$$U = \frac{W_T \times T_D}{T_D + RTT} \geq 0.8 \Rightarrow \frac{W_T \times 62.5}{62.5 + 500} \geq 0.8 \Rightarrow W_T \geq \frac{0.8 \times 562.5}{62.5} = 7.2$$

取 $W_T = 8$。若用 GBN，需 $W_T \leq 2^n - 1$，即 $2^n \geq 9$，$n \geq 4$。故帧序号至少需要 **4** 个比特。（若用 SR，$W_T \leq 2^{n-1}$，$2^{n-1} \geq 8$，同样 $n \geq 4$。）

算例三（**GBN** 最大平均数据传输速率）：主机甲与乙之间使用 GBN 协议传输，甲的发送窗口 $W_T = 1000$，数据帧长 1000 字节，信道带宽 100 Mbps，乙每收到一个数据帧就立即用一个短帧（忽略其传输时延）确认，单向传播延迟 50 ms。求甲可达到的最大平均数据传输速率。

$T_D = \dfrac{1000 \times 8}{100 \times 10^6} = 80\ \mu s$。

$W_T \cdot T_D = 1000 \times 80\ \mu s = 80\ \text{ms}$，即甲连续发完整个窗口需要 80 ms。

$RTT = 2 \times 50 = 100\ \text{ms}$。由于 $80\ \text{ms} < 100\ \text{ms}$，发完一窗口后要等确认才能继续，窗口不再能保证信道连续满负荷，故：

$$R_{\text{有效}} = \frac{W_T \cdot L}{RTT + 2T_D} \approx \frac{1000 \times 1000 \times 8}{0.08 + 0.1}\ \text{bit/s} = \frac{8 \times 10^6}{0.18} \approx 4.44 \times 10^7\ \text{b/s} \approx 44.4\ \text{Mbps}$$

（若按” 帧发送时延计入往返” 的另一种常见处理，得 $\frac{1000 \times 8000}{0.1 + 0.00008} \approx 79.9$ Mbps。统考按**”** 窗口发送时间能否覆盖 $T_D + RTT + T_A$**”** 来判断：本题 $W_T \cdot T_D = 80$ ms，与 $T_D + RTT = 100.08$ ms 相比略小，说明窗口” 刚好差一点” 填满链路，故最大平均速率约 80 Mbps 量级。）

有效速率题的三个坑

1. 题目给的是” 单向传播延迟” 还是” 往返时间 RTT”，一定要看清。GBN 的效率分母含 **RTT**，不是 τ。
2. 判窗口是否成为瓶颈：比较 $W_T T_D$ 与 $T_D + RTT + T_A$。大于等于则瓶颈是链路带宽（$U = 1$），小于则瓶颈是窗口。
3. ” 有效数据传输速率” 只算数据部分，不含帧首部、尾部和确认帧的开销；而” 信道利用率” 有时按整帧长度算。看清题目问的是哪一个。

五、介质访问控制

介质访问控制（**Medium Access Control**，**MAC**）要解决的问题是：多个站点共享同一信道时，如何分配信道的使用权。三大类方法分别是信道划分、随机访问和轮询访问。

（一）信道划分介质访问控制

信道划分的基本思想是把一条信道按某种维度切分成多条互不干扰的子信道，每个站点固定（或按需）使用其中一条，从而从根本上避免冲突。

*1.*频分多路复用 *FDM*

频分多路复用（**Frequency Division Multiplexing**）把传输介质的总带宽划分为多个互不重叠的频段，每个用户独占一个频段，各用户在同一时间占用不同频率，通信过程中始终占用该频段。

特点：所有用户在同样的时间占用不同的带宽资源；适合模拟信号传输；各子信道之间需留保护频带以防串扰。

*2.*时分多路复用 *TDM*

时分多路复用（**Time Division Multiplexing**）把传输时间划分为一个个时间片（时隙），每个用户在各帧中固定占用其中一个时隙，各用户在不同时间占用相同频率。

特点：所有用户在不同时间占用同样的频带宽度；适合数字信号传输；时间片固定分配，某用户无数据时该时隙只能空闲，造成浪费。

统计时分复用 **STDM**：又称异步时分复用。它按需动态分配时隙——只有当某用户确实有数据要发时才分配时隙，并在每个时隙前附加用户标识。这样显著提高了信道利用率，代价是需要额外的地址信息与缓存，实现更复杂。

*3.*波分多路复用 *WDM*

波分多路复用（**Wavelength Division Multiplexing**）是光的频分多路复用。在一根光纤上同时传输多路波长（频率）不同的光载波信号，各路信号互不干扰，经棱镜或衍射光栅合波/分波后实现多路复用。

特点：一根光纤可复用几十甚至上百个波长，极大提高光纤传输容量；本质上仍是频分复用，只是频段在光波范围。

*4.*码分多路复用 *CDM* 与 *CDMA*

码分多路复用（**Code Division Multiplexing**，**CDM**）是另一种完全不同的思路：所有站点在同一时间、同一频段上发送数据，但每个站点使用一个独特的、与其他站点正交的码片序列（**chipping sequence**）对数据进行编码，接收方用相应站点的码片序列做内积运算，就能从混合信号中”分离”出该站点的数据。它的多址接入方式称为码分多址（**Code Division Multiple Access**，**CDMA**）。

编码规则：

- 每个站被分配一个 m 比特的码片序列 $\vec{S}$，通常 $m = 2^k$（如 8、64、128）。
- 该站发送比特 **1** 时，发送自己的码片序列 $\vec{S}$；发送比特 **0** 时，发送码片序列的反码 $\vec{S'}$（即 $\vec{S}$ 各位取反）。
- 若某站没有发送数据，则该站对应的输出为 0。

正交性要求（任取两站 S 与 T，向量按位相乘后求和）：

$$\vec{S} \cdot \vec{T} = \frac{1}{m} \sum_{i=1}^{m} S_i T_i = 0 \quad \text{（不同站正交）}$$

$$\vec{S}\cdot\vec{S}=\frac{1}{m}\sum_{i=1}^{m}S_iS_i=1\ \text{（自身内积为 1）}$$

$$\vec{S}\cdot\overline{\vec{S}}=-1\ \text{（自身与反码内积为 −1）}$$

分离方法：接收方收到的是所有正在发送的站点的码片序列之叠加（线性相加）信号 $\vec{R}$。想要分离出站 A 发的数据，只需计算 $\vec{R}\cdot\vec{S_A}$：

- 结果 >0（即 $+1$）：站 A 发了比特 **1**；
- 结果 <0（即 -1）：站 A 发了比特 **0**；
- 结果 $=0$：站 A 没有发送数据（其他站与 S_A 正交，内积为 0）。

完整算例（**2014** 年统考真题改编）：站点 A、B、C 通过 CDMA 共享链路，码片序列分别为

$$\vec{S_A}=(1,1,1,1)\qquad\vec{S_B}=(1,-1,1,-1)\qquad\vec{S_C}=(1,1,-1,-1)$$

先验证正交性：

$$\vec{S_A}\cdot\vec{S_B}=\tfrac{1}{4}(1\times1+1\times(-1)+1\times1+1\times(-1))=0\ \checkmark$$

$$\vec{S_A}\cdot\vec{S_C}=\tfrac{1}{4}(1\times1+1\times1+1\times(-1)+1\times(-1))=0\ \checkmark$$

$$\vec{S_B}\cdot\vec{S_C}=\tfrac{1}{4}(1\times1+(-1)\times1+1\times(-1)+(-1)\times(-1))=0\ \checkmark$$

接收方 C 从链路上收到的叠加序列为

$$\vec{R}=(2,0,2,0,\ 0,-2,0,-2,\ 0,2,0,2)$$

把 $\vec{R}$ 平均分成 3 段（每段 4 个码片，对应 3 个比特的传输时间）：

$$\vec{R_1}=(2,0,2,0)\quad\vec{R_2}=(0,-2,0,-2)\quad\vec{R_3}=(0,2,0,2)$$

分离站 **A** 的数据（用 $\vec{S_A}=(1,1,1,1)$）：

$$\vec{R_1}\cdot\vec{S_A}=\tfrac{1}{4}(2+0+2+0)=1\Rightarrow\text{比特 }1$$

$$\vec{R_2}\cdot\vec{S_A}=\tfrac{1}{4}(0-2+0-2)=-1\Rightarrow\text{比特 }0$$

$$\vec{R_3}\cdot\vec{S_A}=\tfrac{1}{4}(0+2+0+2)=1\Rightarrow\text{比特 }1$$

所以 A 发送的数据是 `101`。

再验证 **B**、**C** 的数据以确认理解正确（用 $\vec{S_B}=(1,-1,1,-1)$）：

- 第 1 段：$\tfrac{1}{4}(2\times1+0\times(-1)+2\times1+0\times(-1))=\tfrac{1}{4}(2+0+2+0)=1\to1$

- 第 2 段：$\frac{1}{4}(0\times1+(-2)(-1)+0\times1+(-2)(-1))=\frac{1}{4}(0+2+0+2)=1\to1$
- 第 3 段：$\frac{1}{4}(0\times1+2\times(-1)+0\times1+2\times(-1))=\frac{1}{4}(0-2+0-2)=-1\to0$ 故 B 发送 `110`。

用 $\vec{S_C}=(1,1,-1,-1)$：

- 第 1 段：$\frac{1}{4}(2\times1+0\times1+2\times(-1)+0\times(-1))=\frac{1}{4}(2+0-2+0)=0$
- 第 2 段：$\frac{1}{4}(0+(-2)+0+2)=\frac{1}{4}(0-2+0+2)=0$
- 第 3 段：$\frac{1}{4}(0+2+0-2)=0$ 故 C 自己没有发送数据（这与"C 在接收"一致）。

交叉验证叠加信号确实自洽（B=110，C 不发）：第 1 段：A 发 1 用 $\vec{S_A}$，B 发 1 用 $\vec{S_B}$，叠加为 $(1,1,1,1)+(1,-1,1,-1)=(2,0,2,0)$ ✓ 第 2 段：A 发 0 用 $-\vec{S_A}=(-1,-1,-1,-1)$，B 发 1 用 $\vec{S_B}=(1,-1,1,-1)$，叠加为 $(0,-2,0,-2)$ ✓ 第 3 段：A 发 1 用 $\vec{S_A}$，B 发 0 用 $-\vec{S_B}=(-1,1,-1,1)$，叠加为 $(0,2,0,2)$ ✓

CDMA 计算题的标准解法

1. 先验证各站码片序列两两内积为 **0**（正交），题目若已给出正交性可直接用。
2. 把接收序列按码片长度平均分段，段数 = 传输的比特数。
3. 用目标站的码片序列与每一段做内积、再除以码片长度。
4. 结果 $+1$ 记 1，-1 记 0，0 表示该站未发送。

（二）随机访问介质访问控制

随机访问的特点是所有站点都可以按随机的方式争用信道，没有固定分配，冲突可能发生，因此必须有冲突解决机制。

1.ALOHA 协议

纯 **ALOHA**（**Pure ALOHA**）：站点只要想发就发，完全不监听信道。若一段时间内没收到确认就认为发生冲突，随机退避后重传。

设一个帧的发送时延为 T_0（称为一个帧时）。由于各站随机发送，一个帧若想无冲突地发送成功，必须在前后各一个帧时（共 $2T_0$）内没有任何其他站开始发送。若网络总负载为 G（单位帧时内到达的帧数，G 为泊松分布均值），则

$$P_{\text{成功}}=e^{-2G}\qquad S=G\cdot P_{\text{成功}}=G\,e^{-2G}$$

对 G 求导令其为 0，得 $G=0.5$ 时取最大值：

$$S_{max}=0.5\,e^{-1}\approx0.5\times0.368=0.184$$

即纯 **ALOHA** 的信道吞吐率极限约为 **18.4%**。

时分 **ALOHA**（时隙 **ALOHA**，**Slotted ALOHA**）：把时间划分为等长的时隙，时隙长度等于一个帧时 T_0，站点只能在时隙的开始时刻发送。这样冲突只能发生在同一个时隙内，一个帧要成功只需后一个帧时内无其他站发送：

$$P_{\text{成功}}=e^{-G}\qquad S=G\,e^{-G}$$

$G=1$ 时取最大值：

$$S_{max} = e^{-1} \approx 0.368$$

即时隙 **ALOHA** 的信道吞吐率极限约为 **36.8%**，正好是纯 ALOHA 的两倍。

ALOHA 的两个" 极限值" 必须记住

$0.184 = \frac{1}{2e}$（纯 ALOHA）与 $0.368 = \frac{1}{e}$（时隙 ALOHA）。时隙 **ALOHA** 的吞吐率是纯 **ALOHA** 的 **2** 倍，原因是纯 ALOHA 的易冲突区间是 $2T_0$，时隙 ALOHA 只有 T_0。

2.CSMA 协议

CSMA（Carrier Sense Multiple Access，载波监听多址接入）在 ALOHA 的基础上增加了" 先听后说"：发送前先检测信道，信道空闲才发。

但仅靠载波监听并不能完全避免冲突，因为信号传播存在时延：A 与 B 相隔较远时，A 开始发送的信号还没到达 B，B 检测到信道" 空闲" 也发送，就产生了冲突。于是派生出三种不同的监听策略：

1-坚持 **CSMA**：站点要发送时先监听信道。

- 信道空闲：立即发送（"1" 表示以概率 1 发送）；
- 信道忙：持续监听，一旦变为空闲就立即发送。

缺点：多个站点同时在等待时，信道一变空闲就会几乎同时发送，冲突概率高。

非坚持 **CSMA**：站点要发送时先监听信道。

- 信道空闲：立即发送；
- 信道忙：随机等待一段时间后重新监听（不是持续监听）。

优点：降低了冲突概率；缺点：等待的随机时间可能较长，信道刚变空闲时可能无人发送，造成信道利用率下降。

p-坚持 **CSMA**：适用于时隙信道。站点要发送时先监听信道。

- 信道空闲：以概率 p 立即发送，以概率 $1-p$ 推迟到下一个时隙再重新监听；
- 信道忙：等待下一个时隙再继续监听。

优点：兼顾了" 降低冲突概率" 和" 减少信道空闲浪费"，但 p 值难以合理选择（p 太大易冲突，太小则时延大）。

3.CSMA/CD 协议

CSMA/CD（Carrier Sense Multiple Access / Collision Detection，载波监听多址接入/碰撞检测）是共享总线以太网使用的协议，要点是" 先听后说、边说边听、冲突停说、随机重说"。

三个组成部分：

- 多址接入 **MA**：多个站点连接在一条总线上，竞争使用总线。
- 载波监听 **CS**：每个站点发送前先检测总线是否忙。
  - 若检测到总线空闲 **96** 比特时间（发送 96 比特所需时间，即帧间最小间隔），则发送该帧；
  - 若总线忙，则继续监听并等待总线转为空闲 96 比特时间后再发送。
- 碰撞检测 **CD**：每个正在发送的站点边发送边检测碰撞，一旦发现碰撞就立即停止发送，并继续发送 **32** 比特或 **48** 比特的人为干扰信号，以便让总线上所有站点都能检测到碰撞；随后退避一段随机时间后重发。

CSMA/CD 的几个关键结论

1. 载波监听检测到总线空闲，总线并不一定真的空闲（存在传播时延）。
2. CSMA/CD 只能尽量避免碰撞，不能完全避免碰撞。
3. 因为要”边发送边检测”，站点不可能同时发送和接收，所以使用 CSMA/CD 的共享总线以太网只能半双工工作。
4. **CSMA/CD** 不适用于无线网络（无线网卡难以实现碰撞检测，且存在隐蔽站问题），无线网使用 CSMA/CA。

争用期 2τ 的完整推导

设总线两端的两个站点 A、B 相距最远，单向传播时延为 τ。考虑最坏情况：

- 时刻 $t=0$，A 开始发送；A 的信号要经过 τ 才能到达最远的 B。
- 若 B 恰好在收到 A 的信号之前（即在 $t=\tau-\delta$，$\delta\to 0$）开始发送，则 B 检测信道时认为信道是空闲的。
- B 的信号要经过 τ 才能到达 A，所以在 $t\approx 2\tau$ 时 A 才检测到碰撞。

因此：站点从开始发送帧起，最多经过 2τ 就能确定自己是否遭遇了碰撞。这个 2τ 称为争用期（也叫碰撞窗口）。若经过争用期还没有检测到碰撞，就可以肯定这次发送不会碰撞。

反过来考虑最快情况：双方同时开始发送，则各自在经过 τ 后就检测到冲突。所以检测到冲突的最短时间为 τ（严格说趋近于 **0** 的情形对应**”**对方在信号即将到达时才开始发送**”**）。

更精确地说：设 A 先发、B 后发，B 发送的时刻比 A 晚 t 时间（$0<t<\tau$）：

- A 检测到冲突的时刻 $=t+\tau$（$t\to\tau$ 时最大为 2τ，$t\to 0$ 时最小趋向 τ）；
- B 检测到冲突的时刻 $=\tau$（B 一发出信号就立刻与 A 的信号相遇，几乎立即检测到，时间趋于 0）。

综合结论：从开始发送数据时刻起到双方都检测到冲突，最短时间为 τ；单个站点检测到冲突的最长时间为 2τ。

最小帧长的完整推导 $L_{min}=2\tau\cdot R$

为了让站点在发送完整个帧之前就能检测出碰撞，帧的发送时延必须不小于一个争用期：

$$T_D=\frac{L}{R}\ \geq\ 2\tau\quad\Longrightarrow\quad\boxed{L_{min}=2\tau\cdot R}$$

其中 R 为数据传输速率，τ 为单向端到端传播时延。

为什么以太网最小帧长是 **64** 字节：对于 10 Mb/s 的传统共享总线以太网，标准规定争用期 2τ 的值等于发送 **512** 比特所需的时间，即 51.2 μs。于是

$$L_{min}=512\text{ bit}=64\text{ B}$$

这一规定的实际意义：若某站点发送的帧在前 64 B 内没有遭遇碰撞，那么后续部分也不会遭遇碰撞（因为此时其他站点都已听到信号）。反之，若遭遇碰撞，一定发生在帧的前 **64 B** 之内，此时站点已发送的数据必然小于 **64 B**。因此接收方收到长度小于 **64 B** 的帧，就可以判定这是一个因碰撞而异常中止的无效帧，直接丢弃。

最大帧长 **1518 B**：若不限制帧长，一个很长的帧会长时间独占总线，使其他站点等待时间过长（不公平），且出错重传的代价大。因此规定数据载荷最大 1500 B，加上 6 B 目的地址 + 6 B 源地址 + 2 B 类型 + 4 B FCS = 18 B 的首部与尾部，最大帧长为 **1518 B**。

计算题完整算例

算例一（求最大端到端距离）：在一个采用 CSMA/CD 协议的网络中，传输速率 $R = 1$ Gbps，信号传播速度 $v = 200\,000$ km/s。若最小数据帧长度减少 800 比特，则最远两个站点之间的距离至少需要怎样变化?

$$L_{min} = 2\tau R \Rightarrow \tau = \frac{L_{min}}{2R} \Rightarrow d = v\tau = \frac{v\,L_{min}}{2R}$$

$$\Delta d = \frac{v\,\Delta L_{min}}{2R} = \frac{2 \times 10^8 \text{ m/s} \times 800}{2 \times 10^9 \text{ b/s}} = 80 \text{ m}$$

即最大距离需减少 **80 m**。

算例二（求最大单向传播时延）：假设一个采用 CSMA/CD 协议的 100 Mb/s 局域网，最小帧长是 128 B，则在一个冲突域内两个站点之间的单向传播时延最多是多少?

$$2\tau = \frac{L_{min}}{R} = \frac{128 \times 8}{100 \times 10^6} = 10.24\ \mu s \Rightarrow \tau = 5.12\ \mu s$$

算例三（含中继器/**Hub** 的延迟）：H2 与 H3 之间通过一个 Hub 相连，100 Mbps 以太网，最小帧长 64 B，信号通过 Hub 产生额外 1.51 μs 延时，信号传播速度 2×10^8 m/s。求 H2 与 H3 可以相距的最远距离。

最小帧发送时延 $T_D = \dfrac{64 \times 8}{100 \times 10^6} = 5.12\ \mu s$。往返时延必须不大于 T_D：

$$2 \times \frac{d}{v} + 2 \times 1.51\ \mu s \le 5.12\ \mu s \Rightarrow \frac{2d}{2 \times 10^8} \le 2.10\ \mu s \Rightarrow d \le 210 \text{ m}$$

算例四（有效数据传输速率）：某局域网采用 CSMA/CD，数据传输速率 10 Mbps，甲、乙相距 2 km，信号传播速度 200 000 km/s（即 200 m/μs）。若网络不存在任何冲突与差错，甲总是以标准最长以太网帧（1518 B）向乙发送，乙每成功收到一个数据帧后立即向甲发送一个 64 B 的确认帧，甲收到确认帧后才发送下一个。此时甲的有效数据传输速率是多少（不考虑前导码）?

第 1 步，单向传播时延 $\tau = 2000 \text{ m} \div 200 \text{ m}/\mu s = 10\ \mu s$，$RTT = 20\ \mu s = 0.02$ ms。

第 2 步，1518 B 数据帧发送时延 $T_D = \dfrac{1518 \times 8}{10 \times 10^6} = 1.2144$ ms。

第 3 步，64 B 确认帧发送时延 $T_A = \dfrac{64 \times 8}{10 \times 10^6} = 0.0512$ ms。

第 4 步，发送周期 $= 1.2144 + 0.0512 + 0.02 = 1.2856$ ms。

第 5 步，1518 B 帧中有效数据载荷为 **1500 B**，故

$$R_{\text{有效}} = \frac{1500 \times 8 \text{ bit}}{1.2856 \text{ ms}} \approx 9.33 \text{ Mbps}$$

截断二进制指数退避算法

当站点检测到碰撞后，需要退避一段随机时间再重发。共享总线以太网采用截断二进制指数退避（**Truncated Binary Exponential Backoff**）算法：

- 记该帧的重传次数为 k。取 $K = \min(k, 10)$。
- 从整数集合 $\{0, 1, 2, \ldots, 2^K - 1\}$ 中随机取出一个数 r。
- 退避时间 $= r\times$ 争用期 2τ（10 Mb/s 以太网中争用期 = 51.2 μs，即一个” 争用时间片”）。
- 当重传次数达到 **16** 次仍不成功时，丢弃该帧并向高层报告。

这一算法的效果：重传需要推迟的平均时间随重传次数而增大（动态退避），因而减小碰撞概率。若连续多次碰撞，说明参与竞争的站点较多，退避窗口自动扩大以分散发送。

完整算例（**2023** 年统考真题）：已知 10BaseT 以太网的争用时间片为 51.2 μs。若网卡在发送某帧时发生了连续 4 次冲突，则基于二进制指数退避算法确定的再次尝试重发该帧前等待的最长时间是多少?

第 4 次冲突时 $k=4$，$K=\min(4,10)=4$，r 在 $\{0,1,\ldots,2^4-1\}=\{0,\ldots,15\}$ 中随机取。最长等待时间对应 $r=15$：

$$15\times 51.2\ \mu s=768\ \mu s$$

（实际考试中给出答案 768 μs。）注意：是第 **4** 次冲突后取 $2^4=16$ 个值，不是 2^{4-1}；也别把” 连续 4 次冲突” 误当成”$K=2^4$”。

二进制指数退避的三个坑

1. 退避时间的基本单位是争用期 2τ，不是 τ，也不是发送一帧的时间。
2. $K=\min(k,10)$，重传次数超过 10 之后 K 恒为 **10**（最大值 1023 个争用期）；到 16 次直接放弃并报错。
3. 记忆口诀：” 重传几次，就在 $2^{\min(k,10)}$ 个时间片里随机抽一个”。

信道利用率的极限分析

考虑理想情况：总线一旦空闲就有某站点立即发送，且各站点发送都不会碰撞。此时：

- 发送一帧占用总线的时间为 $T_0+\tau$（其中 T_0 是帧本身的发送时间，τ 是信号传播到总线另一端所需时间）；
- 其中帧本身的发送时间只有 T_0；
- 因此 $\lim\limits_{\tau\to 0}$ 时信道利用率趋近 **100%**。

也就是说，当信号传播延迟趋近于 **0** 时，冲突可被瞬间发现并停止，信道上的无效时间趋近于 **0**，有效数据几乎可以连续不断地占用信道，信道利用率趋近 **100%**。

千兆以太网的特殊处理：速率提高到 1000 Mb/s 后，若仍保持最小帧长 64 B，则争用期只能容纳 100 m 的网段（再短就无使用价值）；若把最小帧长增大到 640 B，则短 PDU 的封装开销太大。解决办法是载波延伸（**Carrier Extension**）：把争用期增大为 **512** 字节的发送时间，同时保持最小帧长仍为 **64** 字节。发送的 MAC 帧长度不足 512 B 时，就在帧尾填充特殊字符使其达到 512 B。若连续发送大量短帧，则用分组突发（**Packet Bursting**）：只对第一个短帧做载波延伸填充，后续短帧只留必要的帧间最小间隔即可连续发送，直到累积发送约 1500 B 为止。

当千兆以太网工作在全双工方式时，不使用 **CSMA/CD**，也不使用载波延伸和分组突发。

4.CSMA/CA 协议

为什么无线局域网不能使用 **CSMA/CD**

802.11 无线局域网使用无线信道，与共享总线以太网使用有线传输介质不同，不能简单照搬 **CSMA/CD**。原因有二：

1. 硬件难以实现碰撞检测：无线信道的传输环境复杂，信号强度的动态范围非常大，802.11 网卡上接收到的信号强度一般远远小于发送信号的强度（可能相差百万倍），要在同一网卡上边发边检测碰撞，对硬件的要求非常高。
2. 即使能检测也没有意义（隐蔽站问题）：无线电波的传播范围有限，可能出现 A 和 B 都能与 AP

通信，但 **A** 的信号到不了 **B**，**B** 的信号也到不了 **A** 的情况。A、B 互为隐蔽站（**Hidden Station**），双方都检测不到对方在发送，都认为信道空闲而同时向 AP 发送，必然在 AP 处碰撞。此时即使有碰撞检测功能，A、B 也检测不到。

因此 802.11 采用 **CSMA/CA**（**Carrier Sense Multiple Access / Collision Avoidance**，载波监听多址接入/碰撞避免）：它保留了 **CSMA**（先听后说以减少碰撞），但把**"** 碰撞检测 **CD"** 改为**"** 碰撞避免 **CA"**。

两个必须记牢的对比

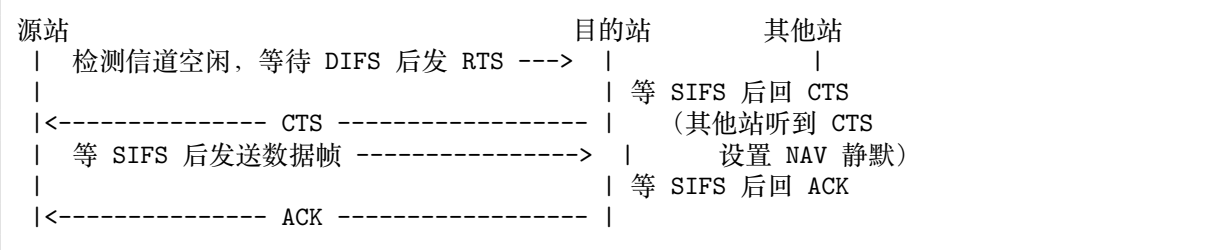
- CSMA/CA 中的 CA 表示" 碰撞避免"，但并不能避免所有碰撞，只能尽量减少碰撞发生的概率。
- **CSMA/CD** 的共享式以太网提供不可靠传输（发送完就不管了，可靠性由高层负责）；**CSMA/CA** 的 **802.11** 无线局域网必须使用确认机制提供可靠传输（因为无线信道误码率高）。

帧间间隔 IFS

802.11 规定，每个站点必须在持续检测到信道空闲一段指定的时间后才能发送帧，这段时间称为帧间间隔（**Interframe Space**，**IFS**）。IFS 的长短取决于帧的类型：低优先级的帧等待较长时间，高优先级的帧等待较短时间，从而高优先级帧可优先获得发送权。最常用的两种：

- 短帧间间隔 **SIFS**（**Short IFS**）：长度 **28** μ**s**，是最短的帧间间隔，用来分隔属于一次对话的各帧，站点应能在这段时间内从发送方式切换到接收方式。使用 SIFS 的帧有 **ACK** 帧、**CTS** 帧、过长的 MAC 帧分片后的数据帧、所有回答 AP 探询的帧以及 PCF 方式中 AP 发出的任何帧。
- **DCF** 帧间间隔 **DIFS**（**DCF IFS**）：长度 **128** μ**s**，比 SIFS 长很多，在 DCF 方式中用于发送数据帧和管理帧。发送数据帧前等待 DIFS 是考虑到可能有其他站有高优先级帧要发送。

完整的 CSMA/CA 数据交换过程



- 源站在发送数据帧之前先发送一个短控制帧 **RTS**（**Request To Send**，请求发送），其中包含源地址、目的地址和本次通信所需的持续时间（含目的站发回确认所需的时间）。发送 RTS 前必须先检测信道，若空闲则等待 DIFS 后发送。
- 目的站正确收到 RTS 且媒体空闲，等待 **SIFS** 后发送 **CTS**（**Clear To Send**，允许发送），CTS 中也包含持续时间（把 RTS 中的持续时间复制过来）。
- 源站收到 CTS 后再等 **SIFS**，就可以发送数据帧。
- 目的站正确收到数据帧后等 **SIFS**，向源站发送确认帧。
- 除源站和目的站以外的其他各站，在收到 **CTS** 帧或数据帧后就推迟访问信道，从而保证源站与目的站的通信不受干扰。若 RTS 发生碰撞，源站收不到 CTS，就执行退避算法重传 RTS。

虚拟载波监听与 NAV

802.11 采用虚拟载波监听（**Virtual Carrier Sense**）机制，目的是让源站把它要占用信道的时间及时通知给所有其他站，使其他站在这一段时间内都停止发送。

当某站检测到信道中传送的帧首部中的”持续时间”字段时，就调整自己的网络分配向量（**Network Allocation Vector**，**NAV**）。**NAV** 指出还必须经过多长时间才能完成帧的这次传输、信道才能转入空闲。

因此，某站认为信道处于忙态有两种可能：

1. 物理层载波监听检测到信道忙（物理载波监听）；
2. MAC 层的虚拟载波监听机制指出信道忙（**NAV** 未减到 **0**）。

RTS/CTS 属于虚拟载波监听机制：站点只要监听到数据帧、**RTS** 帧或 **CTS** 帧中的任何一个，就能知道信道将被占用的持续时间，而不需要真正监听到信道上的信号。因此虚拟载波监听能有效缓解隐蔽站问题：互为隐蔽站的 A 和 B 中，虽然 B 听不到 A 发给 AP 的 RTS，但能听到 **AP** 应答给 **A** 的 **CTS**，B 据 CTS 中的持续时间设置 NAV，在 NAV 指示的时间内保持静默，从而不干扰 A 与 AP 的通信。

NAV 完整算例（**2024** 年统考真题）：在采用 CSMA/CA 的 802.11 无线局域网中，DIFS = 120 μs，SIFS = 28 μs，RTS、CTS、ACK 帧的传输时延分别是 3 μs、2 μs、2 μs，忽略信号传播时延。主机 A 欲向 AP 发送一个总长度为 1998 B 的数据帧，无线链路带宽 54 Mb/s。求隐蔽站 B 收到 AP 发送的 CTS 帧时设置的 NAV 值。

第 1 步，数据帧发送时延 $T_{data} = \dfrac{1998 \times 8}{54 \times 10^6} = 296\ \mu s$。

第 2 步，NAV 从收到 **CTS** 之后开始计剩余时间：CTS 传输完毕时，还需要经过 SIFS + 数据帧 + SIFS + ACK：

$$NAV = SIFS + T_{data} + SIFS + T_{ACK} = 28 + 296 + 28 + 2 = 354\ \mu s$$

算 NAV 时不要多算也不要少算

- NAV 不包含正在传输的 **CTS** 帧本身的时间（B 是在”收到 CTS 帧时”设置 NAV，从那一刻起往后计）。
- NAV 必须包含数据帧之前的那个 SIFS、数据帧本身、数据帧之后到 ACK 之前的 SIFS，以及 ACK 帧。
- 首次竞争的 DIFS（120 μs）不计入 NAV。

退避算法

为了尽量避免碰撞，CSMA/CA 采用了不同于 **CSMA/CD** 的退避机制：

- **CSMA/CD**：站点监听到信道空闲且持续 96 比特时间后立即发送，同时进行碰撞检测；只有检测到碰撞才退避。
- **CSMA/CA**：站点检测到信道从忙态变为空闲时，除了要等待一个 DIFS 间隔，还要退避一段随机时间后才能试图接入信道。因为 CSMA/CA 没有碰撞检测机制，若信道一空闲大家就同时发，必然碰撞。所有要发送的站都要执行退避算法，这既可大大减少碰撞概率，也可避免某个站长时间占用无线信道。

退避算法的执行：各站给自己的退避计时器（**Backoff Timer**）设置一个随机退避时间。

- 当退避时间减小到零时，开始发送帧。
- 当退避时间还未减到零而信道又转变为忙态时，冻结退避计时器（保留剩余值），重新等待信道变为空闲，再经过 DIFS 间隔后，从剩余时间继续倒计时。

退避时间的选择：进行第 i 次退避时，退避时间在时隙编号 $\{0, 1, \ldots, 2^{2+i} - 1\}$ 中随机选择一个，再乘以基本退避时间（一个时隙的长度）。当时隙编号达到 **255**（对应第 **6** 次退避）时不再增加。

与 CSMA/CD 的对比：CSMA/CA 把随机选择退避时间的范围扩大了——第 1 次退避是在 $2^{2+1}=8$ 个时隙中随机选一个（而 CSMA/CD 第 1 次是 $2^1=2$ 个），第 2 次退避是在 $2^{2+2}=16$ 个时隙中选（而 CSMA/CD 是 $2^2=4$ 个）。

唯一不使用退避算法的情况：检测到信道空闲，并且该数据帧不是成功发送完上一个数据帧之后立即连续发送的数据帧。除此之外的以下情况都必须使用退避算法：

- 在发送帧之前检测到信道处于忙态；
- 在每一次重传一个帧时；
- 在每一次成功发送后要连续发送下一个帧时。

RTS/CTS 的三档选择：802.11 为用户提供了三种选择——

1. 总是使用 RTS 帧和 CTS 帧；
2. 只有当数据帧的长度超过某个数值时才使用 RTS 帧和 CTS 帧（数据帧本身很短时再预约意义不大）；
3. 不使用 RTS 帧和 CTS 帧（基础模式，只靠载波监听 + 退避）。

CSMA/CA 是否一定需要预约信道

不是。CSMA/CA 有两种工作方式：

- 基础模式：只做载波监听 + 退避，不预约信道（最常用、默认）；
- **RTS/CTS** 模式：才需要专门预约信道（可选，不是必须）。

这点常被误当成”CSMA/CA 必须使用 RTS/CTS”。

CSMA/CA 相关真题分布

- 2011 年题 36：需要使用确认方案的 **MAC** 协议是 CSMA/CA（以太网 MAC 层不确认）。
- 2018 年题 35：**CSMA/CA** 进行信道预约的方法是发送方先发 RTS，接收方回 CTS，其他站点听到 CTS 后设置 NAV 并保持静默（答案 D）。
- 2020 年题 37：**IFS** 长短判断——首次竞争发送前的 IFS 是 DIFS（最长）；CTS、DATA、ACK 属于同一交换序列，其间的 IFS 都是 SIFS。
- 2024 年题 36：**NAV** 计算（见上面的完整算例）。

（三）轮询访问：令牌传递协议

轮询访问介于” 信道划分” 与” 随机访问” 之间：没有冲突（因为信道使用权是被有序授予的），同时信道利用率高（不像 TDM 那样固定分配时隙）。轮询访问的典型代表是令牌传递协议（**Token Passing**）。

令牌环（**Token Ring**，**IEEE 802.5**）

- 网络中的所有站点构成一个物理环，令牌（一个特殊的控制帧）沿环单向逐站传递。
- 只有持有令牌的站才有权发送数据。某站收到令牌后，若没有数据要发送，就立即把令牌转发给下一个站；若有数据要发送，就把令牌置为” 忙” 并附加数据帧后发送出去。
- 数据帧沿环一周后回到源站，源站将其回收（不转发），再把令牌释放（置为空闲）交给下一站。因此同一时刻环上最多只有一个站能发送数据，绝不会发生冲突。
- 令牌环支持优先级机制：帧首部有优先级位与预约位，站点可以在经过的帧中” 预约” 下一次的令牌优先级，使得紧急数据能优先获得令牌。
- 令牌保持时间（**Token Holding Time**）：站点获得令牌后可以保持令牌的最长时间。设置这一上

限是为了防止某个站点长时间独占令牌，保证环上各站公平地分享带宽。

令牌总线（**Token Bus**，**IEEE 802.4**）：物理上是总线拓扑，但逻辑上把各站排成一个环，令牌按逻辑环的顺序传递。这样既避免了总线争用冲突，又能利用总线结构的可靠性。802.4 已基本退出实用。

令牌传递协议的特点

- 无冲突：信道使用权由令牌有序授予，不需要冲突检测与退避。
- 重负载时效率高：站点越多、负载越重，令牌环的效率反而相对越好（因为没有冲突开销）。
- 轻负载时时延大：即使只有一个站要发数据，也常常要等令牌绕一圈回来。
- 令牌丢失或重复会破坏环网：需要专门的监控站来检测并重新生成令牌。
- 令牌环、令牌总线、CSMA/CD、CSMA/CA 都工作在 **MAC** 子层。

六、局域网

（一）局域网的基本概念与体系结构

局域网（**Local Area Network**，**LAN**）是指在一个较小的地理范围内（一间办公室、一栋楼、一个校园），由多种计算机和外部设备通过高速通信线路相连的网络。

局域网的主要特点：

- 地理范围小（通常几米到几千米）；
- 数据传输速率高（10 Mb/s 到 10 Gb/s 甚至更高），误码率低；
- 所有权单一，由某一单位所有和管理，不受公共网络法规约束；
- 主要使用广播信道，因此介质访问控制方法是局域网的核心技术。

局域网的拓扑结构：总线型、星型、环型、树型（以及混合型）。

- 总线型：所有站点共享一条总线，典型代表是早期共享总线以太网，使用 CSMA/CD。
- 星型：所有站点连接到一个中心设备（集线器或交换机）。使用集线器时物理上是星型、逻辑上仍是总线型；使用交换机时才是真正的交换式以太网。
- 环型：站点首尾相连成环，典型代表是令牌环。
- 树型：多级星型级联而成，扩展性好。

IEEE 802 系列标准：

标准	内容
IEEE 802.1	局域网体系结构、网络互连、**VLAN（802.1Q）**、生成树（802.1D）
IEEE 802.2	逻辑链路控制 **LLC** 子层
IEEE 802.3	**CSMA/CD** 总线网（以太网）
IEEE 802.4	令牌总线网（Token Bus）
IEEE 802.5	令牌环网（**Token Ring**）
IEEE 802.11	无线局域网（**WLAN / Wi-Fi**）
IEEE 802.15	无线个人区域网（蓝牙等）

标准	内容
IEEE 802.16	宽带无线城域网（WiMAX）

LLC 子层与 MAC 子层

IEEE 802 把数据链路层划分为两个子层：

- 逻辑链路控制（**Logical Link Control**，**LLC**）子层：在 MAC 子层之上，与传输介质无关，向网络层提供一个统一的接口，屏蔽底层各种 **802** 局域网（以太网、令牌环、令牌总线等）**MAC** 子层的差异。
- 介质访问控制（**Medium Access Control**，**MAC**）子层：与传输介质相关，负责介质访问控制（CSMA/CD、CSMA/CA、令牌传递等）、帧的封装与解封、MAC 地址的识别与过滤、差错检测。

关于 LLC 与 MAC 的考查口径

几乎所有学过的数据链路层功能（帧封装、寻址、差错检测、介质访问控制）都归属于 **MAC** 子层；LLC 子层只负责”屏蔽差异、提供统一接口”，考查频率低。另外要记住：以太网交换机是按 **MAC** 地址转发的，它并不使用 LLC 子层的服务。

（二）以太网与 IEEE 802.3

以太网（**Ethernet**）是 IEEE 802.3 标准的通称，是目前最主流的局域网技术。它的设计原则是简化通信：采用无连接、无确认的工作方式，向网络层提供不可靠的服务（差错由高层负责处理）。

以太网的发展

名称	速率	标准	关键特点
标准以太网 10BASE-T	10 Mb/s	802.3i	双绞线星型，站点到集线器不超过 100 m；争用期 51.2 μs；帧间最小间隔 9.6 μs（96 比特时间）
快速以太网 100BASE-T	100 Mb/s	802.3u	仍用 **802.3** 帧格式和 **CSMA/CD**；最小帧长仍为 **64 B**；网段最大长度减到 100 m；争用期缩为 **5.12** μ**s**；帧间最小间隔缩为 **0.96** μ**s**
千兆以太网 1000BASE-X	1 Gb/s	802.3z	支持半双工（CSMA/CD + 载波延伸 + 分组突发）和全双工（不用 CSMA/CD）；网段最大长度仍 **100 m**，最小帧长仍 **64 B**
万兆以太网 10GBASE-X	10 Gb/s	802.3ae	只工作在全双工，不使用 **CSMA/CD**，传输距离不再受碰撞检测限制；保留最小/最大帧长规定
40GE/100GE	40/100 Gb/s	802.3ba	只工作在全双工；40 Gb/s 用于计算应用，100 Gb/s 用于汇聚应用

10BASE-T 的含义：10 表示速率 10 Mb/s，BASE 表示基带传输，T 表示双绞线（Twisted pair）；F 表示光纤（Fiber）。

100BASE-T 以太网使用的导向传输介质是双绞线（2019 年题 34 考点）。

MAC 地址

- **MAC** 地址长度为 **48** 位（**6** 字节），一般被固化在网卡的电可擦可编程只读存储器 **EEPROM** 中，因此也称为硬件地址或物理地址（属于数据链路层范畴）。
- 结构上由 **3** 字节（**24** 位）机构唯一标识符（**OUI**）+ **3** 字节扩展标识符组成，OUI 由 IEEE 分配给厂商，后 3 字节由厂商自行分配，从而保证全球唯一。

- 书写形式为 6 组十六进制数，如 `00-1A-2B-3C-4D-5E` 或 `00:1A:2B:3C:4D:5E`。
- 发送顺序：字节顺序为第 1 字节 → 第 6 字节；字节内的比特发送顺序为 $b_0 \to b_7$（低位先发）。
- 广播地址为全 **1**，即 **`FF-FF-FF-FF-FF-FF`**，表示该帧发给本局域网上的所有站点。
- 多播地址：`b_0=1` 的特殊地址。
- 交换机和路由器往往具有更多的网络接口，因此拥有更多的 **MAC** 地址。严格来说，**MAC** 地址是对网络上各接口的唯一标识，而不是对网络上各设备的唯一标识。
- 网卡从网络上每收到一个帧，就检查帧首部中的目的 **MAC** 地址，按下列规则处理：
  1. 若是广播地址（`FF-FF-FF-FF-FF-FF`）→ 接受该帧；
  2. 若与网卡固化的全球单播 **MAC** 地址相同 → 接受该帧；
  3. 若是网卡支持的多播地址 → 接受该帧；
  4. 除上述三种情况外 → 丢弃该帧。

MAC 地址的两个易错点

1. **MAC** 地址不等于**”** 物理层地址**”**。它虽然被称为物理地址 / 硬件地址，但它是数据链路层使用的地址。
2. 网卡发现某帧的目的 MAC 地址不是自己时，直接丢弃，不向网络层报告错误消息（只有是自己的才上交上层）。这与 IP 层需要” 报告差错” 的行为不同。

以太网 **V2** 的 **MAC** 帧格式

以太网 V2（DIX Ethernet V2）的 MAC 帧格式如下：

字段	长度	说明
目的地址	6 B	目的 MAC 地址
源地址	6 B	源 MAC 地址
类型	2 B	指明数据载荷中的内容由上层哪个协议封装（如 0x0800 表示 IPv4）
数据	46~1500 B	网络层交付下来的分组；不足 46 B 时需填充
FCS	4 B	帧检验序列，用 CRC 校验整个帧

首部 + 尾部共 **18 B**（6+6+2+4）。因此：

- 最短帧长 = **6 + 6 + 2 + 46 + 4 = 64 B**
- 最长帧长 = **6 + 6 + 2 + 1500 + 4 = 1518 B**

注意：前导码（**8 B**）不属于 **MAC** 帧。物理层在 MAC 帧前添加 8 B 前导码，其中前 **7 B** 为前同步码，作用是使接收方的时钟同步；后 **1 B** 为帧开始符（`10101011`），表明其后紧跟着以太网帧。以太网还规定了帧间最小间隔为 **96** 比特的发送时间（10 Mb/s 时即 9.6 μs），因此以太网帧不需要帧结束符。

MTU 1500 B 的由来：数据字段最大长度被限制为 1500 B，即以太网的 **MTU = 1500 B**。若不限制，一个很长的帧会长时间独占总线，导致其他站点等待过久（不公平），且一旦出错重传代价极大。

接收方可能收到的无效 **MAC** 帧：

- MAC 帧的长度不是整数个字节；
- 通过 FCS 字段检测出帧有误码；
- MAC 帧的长度不在 64~1518 字节之间。

接收方收到无效 **MAC** 帧时，简单地将其丢弃，以太网的数据链路层没有重传机制。

以太网帧的数字速记

- 首部 + 尾部 = **18 B**；最短帧 **64 B**；最长帧 **1518 B**。
- 数据最少 **46 B**（保证 64 B 最小帧长），最多 **1500 B**（MTU）。
- 前导码 **8 B**，不计入 MAC 帧长度。
- 帧间最小间隔 = **96** 比特时间。

MAC 帧的发送与接收过程

发送过程：

1. 网络层把 IP 数据报交给数据链路层；
2. 数据链路层给数据报加上目的 MAC、源 MAC、类型字段构成首部，加上 4 B FCS 构成尾部；若数据不足 46 B，则填充到 46 B；
3. 把封装好的帧交给物理层；
4. 物理层在前面加 8 B 前导码，按曼彻斯特编码等方式转换成信号发送到传输介质上。

接收过程：

1. 物理层从介质上收到信号，做同步、解码，剥离前导码，把 MAC 帧交付数据链路层；
2. 数据链路层检查目的 MAC 地址：是广播地址、自己的单播地址或自己加入的多播地址则继续，否则丢弃；
3. 对帧做 CRC 校验，若有误码则丢弃，不通报网络层；
4. 检查帧长是否在 64~1518 B 之间，不是则丢弃；
5. 剥离首部和尾部，把数据载荷按” 类型” 字段上交给相应的网络层协议。

ARP 与以太网帧的关系

ARP 报文并不需要 **IP** 协议封装，而是直接由数据链路层协议（以太网）封装。这一点常与”ARP 是网络层协议还是链路层协议” 的争论一起出现，统考按”ARP 属于网络层协议，但其报文直接用 MAC 帧封装” 理解。

（三）IEEE 802.11 无线局域网

组成结构

- 有固定基础设施的 **802.11** 无线局域网：采用星型拓扑，中心基站称为接入点（**Access Point**，**AP**）。
  - 基本服务集（**Basic Service Set**，**BSS**）：802.11 无线局域网的最小构件，包含一个 **AP** 和若干个移动站。AP 的 MAC 地址在 802.11 标准中称为基本服务集标识符（**BSSID**）。
  - 分配系统（**Distribution System**，**DS**）：用于把多个 BSS 连接起来。**DS** 最常用的是以太网，也可用点对点链路或其他无线网络。DS 的作用是使 **ESS** 对上层的表现就像一个 **BSS** 一样，即 ESS 对上层呈现的仍是一个局域网。
  - 扩展服务集（**Extended Service Set**，**ESS**）：一个 BSS 通过 DS 与其他 BSS 连接所构成。在一个 ESS 内不同 BSS 的信号覆盖范围可能有重叠。
  - 通信路径示例：BSS1 中的移动站 A 给 BSS2 中的移动站 B 发送数据，路径为 **A** → **AP1** → **AP2** → **B**，必须经过 AP1 和 AP2 以及它们之间作为 DS 的以太网。
- 无固定基础设施的 **802.11** 无线局域网：自组织网络（**ad hoc Network**），没有 AP，各站直接通信，组网方便、生存性好。

- **802.11** 的基本服务：
  - 关联（**Association**）服务：移动站与 AP 建立关联，方法有被动扫描（AP 周期性发送信标帧，移动站据此选择 AP）和主动扫描（移动站主动发送探测请求帧，AP 回探测响应帧）。
  - 重建关联服务：把一个 AP 上的关联转移到另一个 AP（漫游）。
  - 分离服务：终止与 AP 的关联。

为什么使用 **CSMA/CA** 而不是 **CSMA/CD**：已在介质访问控制一节详细说明，核心是” 无线网卡难以实现碰撞检测” + ” 隐蔽站问题使碰撞检测没有意义”。

MAC 层：**DCF** 与 **PCF**

- 分布式协调功能（**DCF**，**Distributed Coordination Function**）：有竞争的接入方式，基于 CSMA/CA + 退避，是 802.11 的基本接入方式，必须实现。
- 点协调功能（**PCF**，**Point Coordination Function**）：无竞争的接入方式，由 AP 轮询各站，PCF 中 AP 发出的任何帧都使用 SIFS。PCF 是可选项，较少使用。

802.11 的数据帧格式

部分	长度	说明
帧头	**30 B**	包含控制信息和最多 **4** 个地址字段
数据	0~2312 B	上层交付的 PDU（通常不超过 1500 B）
帧尾	**4 B**	FCS，采用 CRC 校验

帧头中：

- 持续期（**Duration**）字段：用于实现 CSMA/CA 的虚拟载波监听和信道预约，在数据帧、RTS 帧和 CTS 帧中用该字段指出将要持续占用信道的时长。
- 序号控制字段：用来实现 802.11 的可靠传输，对数据帧进行编号。
- 帧控制（**Frame Control**）字段：含” 去往 **DS**（**To DS**）” 和” 来自 **DS**（**From DS**）” 两个标志位，以及类型、子类型等。

四个地址字段的含义

在有固定基础设施（有 AP）的 BSS 中，不管站点要与本 BSS 内的站点通信还是与 BSS 外的站点通信，都必须通过本 **BSS** 的 **AP** 转发，而源站点并不知道目的站点是否与自己在同一个 BSS 内。因此帧中必须携带足够的信息。

场景	**To DS**	**From DS**	地址 **1**	地址 **2**	地址 **3**	地址 **4**
站点 A → AP（发往 DS，可转发给 B 或路由器）	**1**	0	接收者 **AP1** 的 **MAC**（**BSSID**）	源地址 A	最终目的地址 **B**	未用
AP1 → 站点 B（从 DS 来）	0	**1**	最终目的地址 **B**	发送者 **AP1** 的 **MAC**（**BSSID**）	原始源地址 **A**	未用
自主网络模式（同一独立 BSS 内直接通信）	0	0	目的站 MAC	源站 MAC	BSSID	未用

场景	To DS	From DS	地址 1	地址 2	地址 3	地址 4
无线 DS（DS 也是 802.11，AP1 → AP2）	**1**	**1**	接收者 **AP2** 的 **MAC**	发送者 **AP1** 的 **MAC**	最终目的地址 C	原始源地址 **A**

记忆口诀：”去往 **AP** 中起止，来自 **AP** 止中起”——意思是：去往 DS（To DS = 1）时，地址 1 是 AP（中继），地址 2/3 是起点/终点；来自 DS（From DS = 1）时，地址 1 是终点（目的站），地址 2 是 AP。

完整算例（**BSS1** 中站点 **A** 给 **B** 发送数据帧）：

- A 先把数据帧发给 AP1：帧控制字段中 **To DS = 1**、**From DS = 0**；地址 **1 = AP1** 的 **MAC**（**BSSID**），地址 **2 = A** 的 **MAC**，地址 **3 = B** 的 **MAC**，地址 4 未使用。
- AP1 收到后转发给 B：帧控制字段中 **To DS = 0**、**From DS = 1**；地址 **1 = B** 的 **MAC**，地址 **2 = AP1** 的 **MAC**，地址 **3 = A** 的 **MAC**，地址 4 未使用。

若站点 **A** 要向位于 **DS** 上的路由器 **R** 发送数据帧：过程类似，但 **DS** 是以太网，因此 AP1 会把 802.11 帧转换为以太网帧再转发给 R，此时以太网帧中的源地址和目的地址分别是 **A** 和 **R** 的 **MAC** 地址（不含 **AP1** 的 **MAC**）。当 R 给 A 回响应帧时，以太网帧的源地址和目的地址分别是 R 和 A（同样没有 AP1），AP1 收到该以太网帧后再转换成 802.11 帧转发给 A。可见 **AP** 具有网桥功能。

802.11 帧为什么必须携带 AP 的 MAC 而以太网帧不必

在 802.11 中，站点的信号覆盖范围内可能有多个 **AP** 共享同一个物理信道，而站点只能与其中一个 AP 建立关联，因此 802.11 帧中必须携带 **AP** 的 **MAC** 地址（即站点所在 **BSS** 的 **BSSID**）以明确指出转发该帧的 AP。而在以太网中，**AP** 与透明网桥一样对各站点是透明的，因此以太网帧中不需要也不应该指出 AP 的 MAC 地址。

2022 年题 47（802.11 帧的地址）

”H4 与 H5 位于同一 BSS。H5 收到的是 AP 发出的 From DS 帧，则地址 1、2、3 分别为：” 按 From DS 规则（地址 1 = 终点、地址 2 = AP、地址 3 = 起点）：地址 **1 = H5** 的 **MAC**，地址 **2 = AP** 的 **MAC**，地址 **3 = H4** 的 **MAC**。

（四）VLAN 基本概念与基本原理

为什么需要 VLAN

把多个站点通过交换机连接起来就构成了交换式以太网，其中的所有站点都属于同一个广播域。随着交换式以太网规模扩大，广播域也相应扩大，会带来：

- 广播风暴：大量广播帧（如 ARP 请求）在全网泛洪，浪费网络资源与各主机的 CPU 资源；
- 难以管理和维护，并带来潜在的安全问题（不同部门的流量互相可见）。

分割广播域的传统方法是使用路由器（成本较高、配置复杂）。虚拟局域网技术应运而生。

VLAN 的概念

虚拟局域网（**Virtual Local Area Network**，**VLAN**）是一种将局域网内的站点划分成与物理位置无关的逻辑组的技术，一个逻辑组就是一个 VLAN，VLAN 中的各站点具有某些共同的应用需求。

- 属于同一 VLAN 的站点之间可以直接通信，不同 **VLAN** 中的站点之间不能直接通信（必须经

过路由器或三层交换机）。

- 管理员可通过配置交换机建立多个逻辑上独立的 VLAN。
- 连接在同一交换机上的多个站点可以属于不同的 **VLAN**；属于同一 **VLAN** 的多个站点可以连接在不同的交换机上。
- **VLAN** 并不是一种新型网络，它只是局域网能够提供给用户的一种服务。

IEEE 802.1Q 帧格式

VLAN 最常见的实现技术是基于以太网交换机的接口来实现 **VLAN**。这要求交换机：

1. 能够处理带有 VLAN 标记的帧，即 **IEEE 802.1Q** 帧（又称 Dot One Q 帧）；
2. 各接口可以支持不同的接口类型，不同接口类型对帧的处理方式不同。

IEEE 802.1Q 帧对以太网 **V2** 的 **MAC** 帧格式进行了扩展：在源地址字段和类型字段之间插入了 **4** 字节的 **VLAN** 标签（**Tag**）字段。

VLAN 标签的 4 个字段：

字段	长度	含义
标签协议标识符 **TPID**	16 bit	固定为 `0x8100`，表示该帧是 IEEE 802.1Q 帧
优先级 **PRI**	3 bit	取值 0~7，值越大优先级越高；网络阻塞时优先发送高优先级帧
规范格式指示符 **CFI**	1 bit	0 表示 MAC 地址以规范格式封装，1 表示非规范格式；以太网中 **CFI** 取 **0**
虚拟局域网标识符 **VID**	12 bit	取值范围 **0~4095**，其中 **0** 和 **4095** 保留不使用

- **VID** 是 **802.1Q** 帧所属 **VLAN** 的编号，设备利用 VID 识别帧所属的 VLAN。
- 由于 VID 共 12 位，可表示 $2^{12} = 4096$ 个编号，去掉保留的 0 和 4095，实际最多可划分 **4094** 个 **VLAN**。
- 广播帧只在同一 **VLAN** 内转发，这样就把广播域限制在了一个 **VLAN** 内。

802.1Q 帧的三个数字

插入 **4** 字节标签 → 帧变长；VID **12** 位 → 最多 **4094** 个可用 VLAN（4096 − 2 个保留值）；TPID 固定 **0x8100**。因此一个带标签的以太网帧最长可达 1522 B（1518 + 4），短帧仍为 64 B。

打标签与去标签

802.1Q 帧一般不由用户主机处理，而是由以太网交换机处理：

- 当交换机收到普通的以太网 **MAC** 帧时，会给它插入 **4** 字节 **VLAN** 标签使之成为 802.1Q 帧，该处理简称为” 打标签”。
- 当交换机转发 **802.1Q** 帧时，可能会删除其 **4** 字节 **VLAN** 标签使之成为普通以太网 MAC 帧，该处理简称为” 去标签”。
- 是否去标签取决于交换机的接口类型。

Access 与 **Trunk** 端口

根据接口在接收帧和发送帧时对帧的处理方式，以及接口连接对象的不同，以太网交换机的接口类型一般分为两种：

对比项	**Access** 接口	**Trunk** 接口
连接对象	用户主机、服务器等终端设备	交换机与交换机、交换机与路由器
所属 VLAN	只属于一个 VLAN	可承载多个 VLAN 的流量
收到普通帧	打标签，加上本接口的 PVID 后转发	若帧不带标签，按 PVID 打标签
转发到本接口	去标签后发出（终端看不到标签）	保留标签（或按本征 VLAN 去标签）直接转发

- 交换机上电启动后，若之前未对各接口进行过 VLAN 设置，则各接口类型默认为 **Access**，且缺省 **VLAN ID** 为 **1**，即各接口默认属于 **VLAN 1**。
- 对思科交换机，接口的缺省 VLAN ID 称为本征 **VLAN**（**Native VLAN**）；对华为交换机，称为端口 **VLAN ID**（**Port VLAN ID**，**PVID**）。
- 交换机的每个接口有且仅有一个 **PVID**。

例：同一交换机上划分两个 **VLAN**

不进行人为划分时，所有接口默认属于 VLAN 1 且为 Access 类型，所有主机在同一个广播域内，互相可以通信。若把部分接口划入 VLAN 2、其余留在 VLAN 1，则：

- A 与 B 同属 VLAN 1，可以正常通信；
- A 与 C 分属不同 VLAN，即使连接在同一台交换机上也无法直接通信；
- A 发出的广播帧只在 VLAN 1 内转发，不会扩散到 **VLAN 2**，广播域被成功分割。

例：两个交换机通过 **Trunk** 接口互连

- 若 Trunk 接口去除标签再转发：交换机 1 从 Access 口收到帧后打上 VLAN 10 的标签，从 Trunk 口发出；交换机 2 在 Trunk 口收到后去标签（按 PVID 归属），再从对应 VLAN 的 Access 口发给目标主机。
- 若 Trunk 接口保留标签直接转发：交换机 2 保留 802.1Q 帧的标签，按 VID 从对应 VLAN 的 Access 口转发，并在发往终端前去标签。

VLAN 间通信

不同 VLAN 之间不能直接通信，必须借助路由器或三层交换机（具有路由功能的交换机）。常见做法是” 单臂路由”（路由器的一个物理接口划分成多个子接口，每个子接口对应一个 VLAN）或用三层交换机做 VLAN 间路由。

2024 年题 35（VLAN 与 ARP）

” 支持 VLAN 划分的交换机已按端口划分了 3 个 VLAN，部分端口连接主机的 IP 地址和 MAC 地址如图，ARP 表结构为「IP 地址，MAC 地址，TTL」。下列选项中，不会出现在 H4 的 ARP 表中的是……”

关键判断：**H4** 的 **ARP** 表只会记录与 **H4** 同属一个 **VLAN**、且实际发生过通信的 **IP−MAC** 映射。不同 VLAN 的主机之间不能直接通信，因此其他 **VLAN** 主机的 **IP** 地址不会出现在 **H4** 的 **ARP** 表中（跨 VLAN 通信时 H4 的 ARP 表里出现的会是网关地址，而不是对方主机的地址）。

七、广域网

（一）广域网的基本概念

广域网（**Wide Area Network**，**WAN**）又称远程网，是指覆盖范围很广（几十千米到几千千米，甚至跨国、跨洲）的网络。

广域网与局域网的区别：

对比项	局域网 **LAN**	广域网 **WAN**
覆盖范围	小（一间房到几千米）	大（几十到几千千米）
主要用途	互连本单位的计算机与设备	长距离传输、实现跨地域互连
所有权	单一单位所有	通常由电信运营商拥有和运营
使用的技术层次	主要涉及物理层和数据链路层（如以太网）	主要涉及物理层、数据链路层和网络层
网络拓扑	总线、星型、环型、树型	通常为网状拓扑，结点交换机之间点对点连接
传输介质与协议	双绞线、光纤；以太网 / 802.11	光纤、卫星、公共交换网；PPP、帧中继、ATM、SDH 等
典型设备	集线器、交换机	路由器（结点交换机）
是否使用存储转发	交换机内部有存储转发，但不是链路层必备能力	结点交换机普遍采用存储转发

广域网的关键概念：广域网由许多结点交换机（实际就是路由器）以及连接这些交换机的高速点对点链路组成。结点交换机负责分组的存储转发，网络层是端到端存储转发的设计核心。

（二）**PPP** 协议

点对点协议（**Point-to-Point Protocol**，**PPP**）是目前使用最广泛的点对点数据链路层协议。

PPP 的两种主要应用：

1. 用户计算机通过拨号或专线方式接入 ISP（因特网服务提供商），此时 PPP 运行在用户到 ISP 之间的点对点链路上；
2. **PPPoE**（**PPP over Ethernet**）：在以太网上承载 PPP，用于宽带拨号上网，实现了传统以太网没有的身份验证、加密以及压缩等功能。

PPP 协议的组成（三个组成部分）：

1. 一种将 **IP** 数据报封装到串行链路（同步串行 / 异步串行）的方法；
2. 链路控制协议（**Link Control Protocol**，**LCP**）：用来建立、配置和测试数据链路连接，其最重要的功能之一是身份验证（PAP、CHAP）；
3. 网络控制协议（**Network Control Protocol**，**NCP**）：支持不同的网络层协议（IP、OSI 网络层、AppleTalk 等），为上层协议做配置（如 IPCP 用于分配 IP 地址）。

PPP 协议的基本要求

- 简单：这是首要的要求（最复杂的功能由 TCP 实现，IP 也简单且不可靠），数据链路层的 PPP 没有必要比 IP 复杂。
- 封装成帧：必须规定特殊的字符作为帧定界符。
- 透明性：必须保证数据传输的透明性。
- 多种网络层协议：支持多种网络层协议。
- 多种类型链路：能够在多种类型的链路上运行（串行/并行、同步/异步、低速/高速、电/光等）。
- 差错检测：对收到的帧进行检测，并丢弃有差错的帧。
- 不提供可靠传输：数据链路层的帧不需要纠错、不需要序号、也不需要流量控制。

- 不支持多点接入，不支持半双工或单工通信（只能全双工）。

PPP 不提供可靠传输的三条理由（常考简答）

1. 在数据链路层出现差错的概率不大时，使用比较简单的 PPP 协议较为合理；
2. 在因特网环境下，PPP 的信息字段中放入的是 IP 数据报，数据链路层的可靠传输并不能保证网络层的传输也是可靠的；
3. 帧检验序列 FCS 字段可保证无差错接收（有差错的帧直接丢弃），已经能满足需要。

PPP 的帧格式

PPP 帧各字段如下（按从左到右的顺序）：

字段	长度	取值与说明
标志（**Flag**）	1 B	`0x7E`，从比特角度看为 `01111110`，是 PPP 帧的定界符
地址（**Address**）	1 B	`0xFF`，预留，目前没有实际作用
控制（**Control**）	1 B	`0x03`，预留，目前没有实际作用
协议（**Protocol**）	2 B	指明帧的数据载荷应向上交付给哪个协议处理
数据（**Information**）	可变，默认 ≤ 1500 B	上层交付的数据（通常是 IP 数据报）
FCS	2 B	帧检验序列，CRC 校验
标志（**Flag**）	1 B	`0x7E`，帧结束定界符

协议字段的常见取值：`0x0021` 表示数据载荷是 **IP** 数据报；`0xC021` 表示数据载荷是 **LCP** 分组；`0x8021` 表示数据载荷是 **NCP**（**IPCP**）分组。

PPP 是面向字节的，所有的 **PPP** 帧的长度都是整数字节。

PPP 的透明传输

PPP 的透明传输方法取决于链路是同步还是异步：

- 异步传输（面向字节，单位是字符，效率较低，高速链路开销大）→ 使用字节填充（RFC 1662）。
- 同步传输（面向比特，单位是帧，收发双方时钟统一、字符间传输同步无间隔）→ 使用零比特填充。

异步链路的字节填充规则（发送方）：

1. 数据载荷中出现的每一个 `0x7E`：先减去 `0x20`（相当于异或 `0x20`，即 `0x7E` → `0x5E`），再在其前面插入转义字符 `0x7D`；
2. 数据载荷中原来就有的每一个 `0x7D`：先减去 `0x20`（`0x7D` → `0x5D`），再在其前面插入转义字符 `0x7D`；
3. 数据载荷中出现的每一个 **ASCII** 码控制字符（即数值小于 `0x20` 的字符）：先加上 `0x20`（相当于异或 `0x20`，转换成非控制字符），再在其前面插入转义字符 `0x7D`。

接收方进行与发送方相反的变换，就可以正确恢复出未经过字节填充的原始数据载荷：遇到 `0x7D` 就丢弃它，并把紧随其后的字符与 `0x20` 做异或运算。

完整算例（**PPP** 字节填充）：设 PPP 帧的数据载荷（信息字段）的十六进制字节序列为

```
0x7E  0x7D  0x03  0x41
```

按规则逐字节处理：

原始字节	命中的规则	处理后字节序列
0x7E	规则 1：异或 0x20 → 0x5E，前面插 0x7D	7D 5E
0x7D	规则 2：异或 0x20 → 0x5D，前面插 0x7D	7D 5D
0x03	规则 3：小于 0x20，加 0x20 → 0x23，前面插 0x7D	7D 23
0x41	普通字符，不处理	41

因此实际在线路上（两个标志字段之间）传输的数据载荷字节序列为

```
7D 5E   7D 5D   7D 23   41
```

加上定界符后，完整帧的字节流为

```
7E  7D 5E  7D 5D  7D 23  41  FCS 7E
```

接收方解码：遇到 7D 就丢弃它，把后一个字节异或 0x20：

- 7D 5E → 丢弃 7D，0x5E $\oplus$ 0x20 = 0x7E ✓
- 7D 5D → 0x5D $\oplus$ 0x20 = 0x7D ✓
- 7D 23 → 0x23 $\oplus$ 0x20 = 0x03 ✓

恢复出原始载荷 7E 7D 03 41，与发送方一致。

PPP 字节填充的四个易错点

1. 转义字符是 0x7D，被转义的对象是 0x7E、0x7D 本身，以及所有小于 0x20 的控制字符。
2. 异或 0x20 就是”加/减 0x20”（因为 $0x20$ 只有 1 位为 1，加减等价于异或）。
3. 标志字段 0x7E 本身不填充。
4. 别把 PPP 的异步填充（字节填充，0x7D）与前面字符填充法中的 ESC 混淆：字符填充法用的是通用 ESC，PPP 用的是 0x7D。

PPP 的工作状态（以拨号接入 ISP 为例）

1. 静止（**Link Dead**）：PPP 链路的开始和结束状态，此时用户 PC 与 ISP 拨号服务器之间不存在物理层连接；
2. 建立（**Link Establishment**）：检测到调制解调器的载波信号并建立物理层连接后进入。此时 **LCP** 开始协商一些配置选项（最大帧长、鉴别协议等）。协商成功进入” 鉴别” 状态，失败退回” 静止”；
3. 鉴别（**Authentication**）：若双方无须鉴别或鉴别成功，则进入” 网络” 状态；鉴别失败进入” 终止” 状态；
4. 网络（**Network**）：PPP 链路两端通过交换网络层特定的 **NCP** 分组进行 NCP 配置。若上层是 IP 协议，则使用 **IP** 控制协议（**IPCP**）配置每一端的 IP 模块，例如分配 **IP** 地址。NCP 配置完成后进入” 打开” 状态；
5. 打开（**Link Open**）：双方可以进行数据通信；

6. 终止（**Link Termination**）：出现故障或一端发出终止请求时进入；载波停止后回到” 静止” 状态。

可见，**PPP** 协议已不是纯粹的数据链路层协议，它还包含了物理层和网络层的内容。

PPP 的差错检测：接收方每收到一个 PPP 帧就进行 CRC 检验，正确就收下，否则丢弃。使用 **PPP** 的数据链路层向上提供的是不可靠数据传输服务。PPP 不使用序号和确认机制。

PPP 与以太网的对比（易出选择题）

对比项	**PPP**	以太网 **V2**
服务类型	面向连接的点对点（需 LCP 建链）	无连接、无确认
提供可靠传输	不提供（无序号、无确认、无流量控制）	不提供
帧定界	0x7E 标志字段	前导码 + 帧间间隔（无帧结束符）
透明传输	异步用字节填充（0x7D），同步用零比特填充	不需要（帧内无定界符）
差错检测	CRC（FCS，2 B）	CRC（FCS，4 B）
最小/最大帧长	无 64 B 最小帧长限制	64~1518 B

八、数据链路层设备

（一）网桥与以太网交换机

网桥（**Bridge**）工作在数据链路层（包含其下的物理层），因此具备数据链路层范畴的能力：

- 可以识别帧的结构；
- 可以根据帧首部中的目的 **MAC** 地址和网桥自身的帧转发表来转发或丢弃所收到的帧。

网桥的接口数量很少（通常 2~4 个），一般只用来连接不同的网段。1990 年面世的交换式集线器（**Switching Hub**）实质上是具有多个接口的网桥，常称为以太网交换机（**Switch**）或二层交换机（” 二层” 指它工作在数据链路层，包括物理层）。

仅使用交换机（不使用集线器）的以太网就是交换式以太网。

交换机的本质

以太网交换机本质上就是一种多端口网桥。交换机的自学习和转发帧流程与网桥完全相同。另外，交换机也使用生成树协议 **STP**，来产生能够连通全网但不产生环路的通信路径。

（二）自学习算法与转发表的建立

透明网桥（**Transparent Bridge**）的标准是 **IEEE 802.1D**，它通过自学习算法建立转发表。” 透明” 是指以太网中的各站点并不知道自己所发送的帧将会经过哪些网桥的转发，即各网桥对各站点是” 看不见的”。

自学习与转发的完整流程：

1. 登记（自学习）：网桥（交换机）每收到一个帧，就记录该帧的源 **MAC** 地址和进入网桥的接口号，写入转发表。若表中已有该 MAC 地址的记录，则更新接口号和有效时间。
2. 查表转发：根据帧的目的 **MAC** 地址查找转发表，分三种情况：
   - 明确转发：表中能查到目的 MAC 地址对应的接口，且该接口不是入接口 → 从该接口转发

出去；

- 盲目转发（泛洪）：表中查不到目的 MAC 地址 → 从除进入接口外的其他所有接口转发出去；
- 丢弃（过滤）：表中查到目的 MAC 地址对应的接口就是入接口 → 说明目的站与本帧的来向在同一网段，不需要转发，直接丢弃。

交换机的三条特殊处理规则

1. 交换机收到有误码的帧（FCS 校验失败）→ 直接丢弃；
2. 交换机收到一个无误码的广播帧 → 不用查表，直接从除接收接口外的其他所有接口转发；
3. 转发表中的每条记录都有有效时间，到期自动删除（因为各站点的 MAC 地址与交换机接口的对应关系不是永久的，例如某站更换了网卡，其 MAC 地址就变了）。

完整算例（交换机自学习过程）

设交换机 S 有 4 个接口 1、2、3、4，分别连接主机 A、B、C、D，初始转发表为空。按时间顺序发生如下通信：

时刻	事件	自学习（登记）	转发决策
t_1	A 向 B 发送帧	表中记入 **A** → 接口 **1**	查不到 B → 盲目转发，从接口 2、3、4 发出
t_2	B 向 A 回帧	表中记入 **B** → 接口 **2**	查到 A 在接口 1 且 $1 \neq 2$ → 明确转发，只从接口 1 发出
t_3	C 向 D 发送帧	表中记入 **C** → 接口 **3**	查不到 D → 盲目转发，从接口 1、2、4 发出
t_4	D 向 C 回帧	表中记入 **D** → 接口 **4**	查到 C 在接口 3 → 明确转发，只从接口 3 发出
t_5	A 又向 B 发送帧	更新 **A** → 接口 **1** 的有效时间	查到 B 在接口 2 → 明确转发，只从接口 2 发出

从此刻起，转发表已完整，交换机进入”明确转发”状态，不再泛洪。

2014 年题 34（交换机转发端口判断）

”某以太网拓扑及交换机当前转发表如下图所示，主机 00-e1-d5-00-23-a1 向主机 00-e1-d5-00-23-c1 发送 1 个数据帧，主机 00-e1-d5-00-23-c1 收到该帧后，向主机 00-e1-d5-00-23-a1 发送 1 个确认帧，交换机对这两个帧的转发端口分别是……”

做法：第一个帧查表中是否有目的地址 c1 的记录 → 有则明确转发，无则泛洪；同时交换机先自学习（登记源地址 a1 与入接口）。第二个帧（确认帧）查表中是否有 a1 的记录 → 此时表中已有刚才学到的 **a1** → 入接口的记录，因此如果 a1 的入接口就是该确认帧的入接口，则丢弃，否则明确转发。

（三）转发/过滤/泛洪与冲突域、广播域的划分

冲突域（**Collision Domain**）：可能发生信号碰撞的区域，同一冲突域内的站点共享带宽，需要介质访问控制协调。

广播域（**Broadcast Domain**）：一个站点发出的广播帧所能到达的区域。

设备	工作层次	是否隔离冲突域	是否隔离广播域
中继器 / 集线器（**Hub**）	物理层	否（所有接口同属一个冲突域）	否

设备	工作层次	是否隔离冲突域	是否隔离广播域
网桥 / 以太网交换机	数据链路层	是（每个接口是一个独立的冲突域）	否（所有接口同属一个广播域，除非划分 VLAN）
路由器	网络层	是（每个接口一个冲突域）	是（每个接口一个广播域）

关键结论：

- 集线器只工作在物理层，每个接口仅简单地转发比特，并不进行碰撞检测。使用集线器的以太网物理拓扑是星型，但逻辑上仍是总线网，各站点共享总线资源，仍然使用 **CSMA/CD** 协议。碰撞检测的任务由各站点中的网卡负责。
- 交换机每个接口构成一个独立的冲突域，因此交换式以太网中通常不需要使用 **CSMA/CD**（接口与计算机或交换机连接时可以工作在全双工方式，并能在内部同时连通多对接口，使每一对相互通信的计算机都像独占传输媒体那样无碰撞地传输数据）。
- 当交换机的接口连接的是集线器时，该接口就只能使用 **CSMA/CD** 协议并只能工作在半双工方式。现在的交换机和网卡都能自动识别这两种情况并自动切换。
- **VLAN** 可以把一个广播域分割成多个广播域，这是 VLAN 相对于普通交换机最重要的价值。
- 交换式以太网的网络性能远高于共享式以太网，集线器早已被交换机取代。

2020 年题 35 / 2022 年题 47（冲突域与广播域计数）

计数规则：

- 冲突域个数 = 集线器” 整块” 算 1 个（集线器及其所有直连链路合起来是一个冲突域）+ 交换机每个接口算 1 个 + 路由器每个接口算 1 个；
- 广播域个数 = 路由器每个接口算 1 个 + 交换机（无 VLAN 时）整体算 1 个 + 集线器并入其所属的域。

2022 年题 47 变形：题目给出”H1 与 H2 属于同一个广播域，但不属于同一个冲突域；H2 和 H3 属于同一个冲突域”，则设备 **1** 应是以太网交换机（同广播域不同冲突域），设备 **2** 应是集线器 **Hub**（同冲突域）。

（四）直通式与存储转发式交换

对比项	存储转发（**Store-and-Forward**）	直通交换（**Cut-Through**）
工作方式	把整个帧先全部缓存，检查 FCS 无误后再按目的 MAC 转发	在接收帧的同时就立即按目的 MAC 地址决定转发接口，通过基于硬件的交叉矩阵转发，不必缓存整个帧
转发时延	较大（需收完整个帧）	非常小（只需收到目的 MAC 字段即可开始转发）
差错检查	检查，无效帧不转发	不检查差错，可能把无效帧转发出去
适用场合	通用	对时延敏感、链路质量好的场合

直通交换的时延计算：采用直通交换时，交换机只需要读取到帧的目的 **MAC** 地址字段（**6 B**）就能决定转发接口，因此最小转发时延 = 接收 **6 B** 的时间（前导码不计）。

完整算例（**2013** 年题 **38**）：对于 100 Mbps 的以太网交换机，当输出端口无排队，以直通交换方式转发一个以太网帧（不包括前导码）时，引入的转发延迟至少是多少?

$$\frac{6 \times 8 \text{ bit}}{100 \times 10^6 \text{ b/s}} = 0.48\ \mu s$$

答案 **0.48 μs**。

直通交换时延题的常见错误

1. 误用整帧长度（如 64 B 或 1518 B）计算 → 那得到的是存储转发的时延。
2. 把前导码 8 B 也算进去 → 题目通常明确说” 不包括前导码”。
3. 记错” 直通只需读到目的 MAC（6 B）” 这一点：源 **MAC**、类型字段都还没读完就可以开始转发。

（五）交换机 vs 集线器 vs 路由器

对比项	集线器 **Hub**	以太网交换机 **Switch**	路由器 **Router**
工作层次	物理层	数据链路层（含物理层）	网络层
转发依据	无（直接转发比特）	目的 **MAC** 地址	目的 **IP** 地址
是否隔离冲突域	否	是（每接口一个）	是（每接口一个）
是否隔离广播域	否	否（除非划分 VLAN）	是
工作方式	半双工，共享带宽	全双工（连终端时），独享带宽	全双工
是否使用 CSMA/CD	是（由网卡负责检测）	连终端时不用；连集线器时用	不用
是否有存储转发	无	有（可直通）	有
能否连接不同网络层协议的网络	否	否（只处理 MAC）	能
转发时延	最小	较小	较大
典型用途	扩展端口数量	组建交换式以太网、划分冲突域	连接不同网络、隔离广播域

2015 年题 **37**（交换机的正确叙述）

” 下列关于交换机的叙述中，正确的是（ ）” A. 以太网交换机本质上是一种多端口网桥 ✓ B. 通过交换机互连的一组工作站构成一个冲突域 ×（每个端口是一个独立冲突域）C. 交换机每个端口所连网络构成一个独立的广播域 ×（不划分 VLAN 时全网一个广播域）D. 以太网交换机可实现采用不同网络层协议的网络互联 ×（那属于网络层范畴，路由器才能做）答案 **A**。

本章小结表

知识点	核心公式 / 结论	常考形式
帧定界	HDLC 标志 `01111110`；PPP 标志 `0x7E`	零比特填充选择题
零比特填充	发送方每 5 个连续 1 后插 0；接收方每 5 个连续 1 后删 0	2013 年题 37
字符填充	`SOH/EOT` 前插 `ESC`；`ESC` 本身前也插 `ESC`	写填充后的比特/字符序列
奇偶校验	检奇数位错，漏检偶数位错；不能纠错	判断检错能力
CRC	求余数：数据后补 $k-1$ 个 0，模 2 除以 $G(X)$，余数位数 = $G(X)$ 最高次	2023 年题 37 判断哪个接收串无错
海明码	$2^k \geq n+k+1$；校验位放 1、2、4、8⋯；指误字值 = 出错位号	编码、纠错、求最小海明距离

知识点	核心公式 / 结论	常考形式
海明距离	检 e 位：$d \geq e+1$；纠 t 位：$d \geq 2t+1$；检 s 纠 t：$d \geq t+s+1$	选择题
窗口约束	$W_T + W_R \leq 2^n$	2019 年题 35
停等	$W_T = W_R = 1$，序号 1 bit，$U = \frac{T_D}{T_D+RTT+T_A}$	2018 年题 36 求帧长
GBN	$W_R = 1$，$1 < W_T \leq 2^n - 1$，累积确认，错一帧重传一串	2009 年题 35、2014 年题 36
SR	$W_T = W_R \leq 2^{n-1}$，逐帧确认，缓存失序帧，只重传出错帧	2011 年题 35、2024 年题 37
GBN 有效速率	$R_{有效} = \frac{k \cdot L}{T_D+RTT+T_A}$，$k \leq W_T$	应用题
FDM/TDM/WDM/OFDM	频段/时隙/波长划分；STDM 按需分配	概念选择题
CDMA	内积 >0→1，<0→0，=0→ 未发送；$\vec{S} \cdot \vec{T} = 0$	2014 年题 37（101）
纯 ALOHA	$S_{max} = 0.184 = \frac{1}{2e}$	概念/数值题
时隙 ALOHA	$S_{max} = 0.368 = \frac{1}{e}$（是纯 ALOHA 的 2 倍）	概念/数值题
CSMA	1-坚持、非坚持、p-坚持；仅靠监听不能避免冲突	特点比较
CSMA/CD	争用期 2τ；$L_{min} = 2\tau R$；10 Mb/s 下 512 bit = 64 B	2009/2016/2019/2022 计算题
二进制指数退避	$K = \min(k, 10)$，退避时间 $= r \times 2\tau$，$r \in [0, 2^K - 1]$；16 次失败放弃	2023 年题 36（768 μs）
CSMA/CA	SIFS=28 μs、DIFS=128 μs；RTS/CTS；NAV 虚拟载波监听	2018/2020/2024 题
令牌传递	无冲突；令牌保持时间防止独占	概念题
IEEE 802	802.3 以太网、802.4 令牌总线、802.5 令牌环、802.11 无线；LLC + MAC	标准对应关系
MAC 地址	48 位 = 3 B OUI + 3 B；全 1 广播 FF-FF-FF-FF-FF-FF	2013 年题（长度）
以太网帧	首尾共 18 B；最短 64 B（数据 ≥46 B）；最长 1518 B（数据 ≤1500 B）；前导码 8 B 不计	2010/2012 帧长计算
802.11 帧	帧头 30 B + 数据 + FCS 4 B；4 个地址；From DS 时” 地址 1 终点、2 是 AP、3 是起点”	2017/2022 题
VLAN	802.1Q 插 4 B 标签；TPID=0x8100；VID 12 bit，最多 4094 个；Access/Trunk	2024 年题 35
PPP	标志 0x7E；转义 0x7D，异或 0x20；LCP/NCP；不提供可靠传输	字节填充计算题
交换机	自学习（源 MAC + 入接口）；明确转发/泛洪/丢弃；每端口一个冲突域，不隔离广播域	2014/2015/2016/2021 题
直通交换	只需读到目的 MAC（6 B）即可转发	2013 年题 38（0.48 μs）

易错清单

一、组帧与差错控制

1. 零比特填充是”**5** 个连续 **1** 后插 **0**”，不是”**6** 个”。HDLC 的标志是 6 个 1（01111110），但填充阈值是 5，正因如此填充后才不会出现 6 个连续 1。
2. **CRC** 的余数位数必须等于生成多项式的最高次数，位数不够要在前面补 **0**。例如 $G(X) = X^3 + X^2 + 1$

时余数必须是 3 位，算出 1 要写成 001。

3. 模 **2** 除法不借位、不比较大小，就是按位异或。”减”和”加”在模 2 运算里是一回事。
4. **CRC** 只能检错不能纠错，也不能定位错误位置。别把 CRC 与海明码混为一谈。
5. **FCS** $\neq$ **CRC**。FCS 是附加的冗余码，CRC 是产生 FCS 的一种方法。
6. 奇偶校验漏检偶数个比特错误，与”错哪几位”无关，只与”错几个”有关。
7. 海明码的校验位必须放在 **1**、**2**、**4**、**8**…（2^i）位置，不是放在最前面或最后面。
8. 海明码只能纠 **1** 位错。2 位错时指误字虽然非 0，但指向错误位置，硬纠会越纠越错。
9. 最小海明距离三个公式别记混：检错 $e+1$、纠错 $2t+1$、检 s 纠 t 是 $t+s+1$。

二、流量控制与滑动窗口

1. $W_T + W_R \leq 2^n$ 是所有滑动窗口协议的总约束，由此才有 GBN 的 $W_T \leq 2^n - 1$ 与 SR 的 $W_T = W_R \leq 2^{n-1}$。
2. **GBN** 是累积确认：收到 `ACK3` 就意味着 `0,1,2,3` 全部被确认，别再去找 `ACK2`。
3. **GBN** 的接收窗口恒为 **1**，收到失序帧就丢弃并重复发送最后一个按序确认。
4. **SR** 的接收窗口大于 **1**，可以缓存失序帧；而且只有按序收齐后接收窗口才滑动。
5. **SR** 收到未按序的确认时要记录，但发送窗口不滑动。
6. 停等协议的序号只需 **1 bit**；数据链路层的停等协议可以不给确认分组编号（因为链路层几乎不会出现确认迟到），运输层则必须编号。
7. **”** 确认迟到**”** 与**”** 确认丢失**”** 是两种不同情形：前者要求给确认编号，后者要求给数据编号。
8. 有效数据传输速率 $\neq$ 信道利用率 $\times$ 带宽，除非题目明确按整帧长度算利用率。
9. 判断窗口是否成为瓶颈：比较 $W_T T_D$ 与 $T_D + RTT + T_A$，前者大则链路能被占满（$U=1$），前者小则窗口是瓶颈。

三、介质访问控制

1. 纯 **ALOHA 0.184**、时隙 **ALOHA 0.368**，后者是前者的 2 倍，不要写反。
2. **CSMA** 只用载波监听仍会冲突（传播时延导致多个站同时认为信道空闲），这是 CSMA 与 CSMA/CD 的分界线。
3. $L_{min} = 2\tau R$，其中 2τ 是往返争用期，τ 是单向时延。求最大距离时注意除以 2 的关系：$\Delta d = \dfrac{v\,\Delta L_{min}}{2R}$。
4. **CSMA/CD** 只能半双工（要边发边听）；全双工下不需要 **CD**，因此交换式以太网、光纤以太网、千兆/万兆以太网全双工时不使用 CSMA/CD。
5. 二进制指数退避中 $K = \min(\text{重传次数}, 10)$，退避时间的单位是争用期 2τ。第 4 次冲突时 r 从 $\{0, \ldots, 15\}$ 取，最长 $15 \times 51.2 = 768\ \mu s$。
6. 重传 **16** 次仍失败就放弃并向上层报告，这个数字要记牢。
7. **CSMA/CA** 的 **NAV** 不含正在传输的 **CTS** 帧本身，但含数据帧前的 **SIFS**、数据帧、其后的 **SIFS** 和 **ACK**。
8. **SIFS=28** μ**s** 是**”** 分隔一次对话内部各帧**”** 用的（ACK、CTS），**DIFS=128** μ**s** 用于竞争发送数据帧和管理帧。首次竞争用 DIFS，一次交换内部各帧之间用 SIFS。
9. **CSMA/CA** 不一定要 **RTS/CTS**，基础模式只做载波监听 + 退避。
10. **CSMA/CD** 提供不可靠传输，**CSMA/CA** 必须提供确认机制（可靠传输）。

四、局域网、广域网与设备

1. 前导码 **8 B** 不属于 **MAC** 帧，计算帧长时不要加进去。
2. **MAC** 帧最短 **64 B**、最长 **1518 B**；数据字段 **46~1500 B**；首部 + 尾部 **18 B**。
3. **MAC** 地址是**”** 接口的唯一标识**”**，不是**”** 设备的唯一标识**”**；交换机和路由器有多个接口，因此有多个 MAC 地址。
4. **MAC** 地址虽叫物理地址，但属于数据链路层。
5. 集线器不隔离冲突域也不隔离广播域；交换机隔离冲突域但不隔离广播域；路由器两者都隔离。

6. **VLAN** 的 **VID** 是 **12** 位，共 **4096** 个编号，去掉 **0** 和 **4095** 保留值，实际可用 **4094** 个。
7. **802.1Q** 标签插在源地址与类型字段之间，共 4 B，使最长帧变为 1522 B。
8. **802.11** 帧中必须携带 **AP** 的 **MAC** 地址，以太网帧则不需要（AP 对以太网站点是透明的）。
9. **From DS** 帧的地址顺序：地址 1 = 目的站、地址 2 = AP、地址 3 = 源站。口诀” 去往 AP 中起止，来自 AP 止中起”。
10. **PPP** 不提供可靠传输（无序号、无确认、无流量控制），只做 CRC 检错 + 丢弃。
11. **PPP** 字节填充用 `0x7D`，被转义对象是 `0x7E`、`0x7D` 和所有小于 `0x20` 的控制字符，处理方法是异或 `0x20`。
12. **PPP** 是面向字节的，所有 **PPP** 帧长度都是整数字节；PPP 只能全双工，不支持多点接入。
13. 交换机自学习登记的是**”** 源 **MAC** 地址 + 入接口**”**，不是目的 MAC 地址。
14. 广播帧到达交换机时不查表直接泛洪；有误码的帧直接丢弃。
15. 直通交换只需读到目的 **MAC**（**6 B**）即可转发，所以 100 Mbps 下最小时延是 0.48 μs。
16. 交换机不能实现不同网络层协议网络的互联（那是路由器的事），也不能隔离广播域（除非划 VLAN）。

第四章网络层

本章考纲要求

四、网络层（一）网络层的功能

1. 异构网络互连
2. 路由与转发
3. SDN 基本概念
4. 拥塞控制

（二）路由算法

1. 静态路由与动态路由
2. 距离-向量路由算法
3. 链路状态路由算法
4. 层次路由

（三）IPv4

1. IPv4 分组
2. IPv4 地址与 NAT
3. 子网划分、路由聚集、子网掩码与 CIDR
4. ARP 协议、DHCP 协议与 ICMP 协议

（四）IPv6

1. IPv6 的主要特点
2. IPv6 地址

（五）路由协议

1. 自治系统
2. 域内路由与域间路由
3. RIP 路由协议
4. OSPF 路由协议
5. BGP 路由协议

（六）IP 组播

1. 组播的概念
2. IP 组播地址

（七）移动 IP

1. 移动 IP 的概念

2. 移动 IP 通信过程

（八）网络层设备

1. 路由器的组成和功能
2. 路由表与分组转发

网络层是 408 计网中分值最高、计算题最多的一章。选择常考首部字段、特殊地址、子网与 CIDR、ARP/ICMP/DHCP、路由协议对比；综合大题几乎必考子网划分、路由聚合、路由表转发、IP 分片中的一到两项。本章对五类必考计算（IP 分片、子网划分、CIDR 聚合、路由表转发、RIP 更新）都给出完整算例。

一、网络层的功能

网络层提供主机到主机的通信服务，任务是把分组经过多个网络和多段链路从源主机传到目的主机，可分解为路由选择（走哪条路径）与分组转发（分组到达时做什么动作），传输单位是数据报。TCP/IP 的网络层向上只提供无连接的、尽最大努力交付的数据报服务：分组可能出错、丢失、重复、失序或超时，可靠性由运输层负责。

对比方面	虚电路服务	数据报服务
核心思想	可靠通信由网络自身保证	可靠通信由用户主机保证
连接	必须先建立网络层连接	不需要建立连接
目的地址	仅连接建立阶段使用，之后只用虚电路号	每个分组都携带完整目的地址
分组转发	同一条虚电路的分组走同一路由	每个分组可独立走不同路由
分组顺序	总是按发送顺序到达	不一定按序到达
结点故障	经过该结点的所有虚电路被破坏	可另选路径，适应能力强
首部开销	小	大

虚电路不等于可靠

虚电路能保证分组按序到达，但并不天然可靠。链路故障或拥塞丢包时网络层虚电路本身不重传：协议附加了确认与重传（如 X.25）才可实现可靠传输；无重传机制（如帧中继、ATM）则只是” 面向连接但不可靠”。

（一）异构网络互连

异构网络指因特网中数以百万计的网络，其拓扑结构、寻址方案、性能以及所用协议都不尽相同。互连需要解决不同网络的接入机制、差错恢复方法、路由选择技术、寻址方案、最大分组长度、服务类型等问题。

网络互连通常指用路由器进行网络连接和路由选择。路由器是一台专用计算机（历史上文献也把网络层的路由器称为网关）。中继系统按层次分为四类：

中继系统层次	设备	互连效果
物理层	转发器、集线器	只是把一个网络扩大了，从网络层看仍是同一个网络
数据链路层	网桥、交换机	同上，仍属于同一个网络

中继系统层次	设备	互连效果
网络层	路由器	真正的网络互连，各网络在网络层仍彼此独立
网络层以上	网关	应用层互连

因此只用物理层或数据链路层的中继系统不叫网络互连——它们没有把”网络”这个层次连接起来，只是扩大了同一个网络的规模。

虚拟互连网络（又称逻辑互连网络、**IP** 网络）：各种物理网络的异构性本来客观存在，但通过 **IP** 协议可使这些性能各异的网络在网络层上看起来像一个统一的网络。

为什么用 IP 而不是 MAC 地址统一寻址

若只用 MAC 地址通信，每台路由器的路由表就必须记录所有主机和路由器各接口的 **MAC** 地址：手工配置不可能完成；自动构建也会因路由信息含海量 MAC 地址而严重占用通信资源，还需极大存储空间与查表时延。用 **IP** 地址按网络号寻址，路由表只需记录部分网络的网络地址。这是两种地址并存的原因。

（二）路由与转发

- 路由选择（**routing**）：根据路由协议构造路由表，经常或定期地与相邻路由器交换信息，动态更新维护路由表以决定最优路径。涉及很多路由器。
- 分组转发（**forwarding**）：路由器根据转发表把收到的 IP 数据报从合适的端口转发出去。仅涉及一个路由器。

	路由表	转发表
来源	由路由选择算法生成	由路由表导出
内容	目的网络、下一跳，还存放跳数、链路状态等算法参数	只保留转发必备条目，剔除算法相关附加信息
优化目标	最优化网络拓扑变化的计算	最优化查找过程
实现	一般用软件	可用软件也可用硬件

（三）SDN 基本概念

软件定义网络（Software Defined Network，**SDN**）采用集中式的控制平面和分布式的数据平面（转发平面），两个平面相互分离。控制平面利用控制-数据接口对数据平面上的路由器进行集中式控制。

传统路由器既有转发表又有路由选择软件（即既有数据平面又有控制平面），路由器之间传送主机间的分组和路由信息。SDN 体系结构中路由器中的路由软件都不存在了，因此路由器之间不再交换路由信息。

- 控制平面中有一个在逻辑上集中的远程控制器，物理上可由不同地点的多个服务器组成。
- 远程控制器掌握各主机和整个网络的状态，为每个分组计算出最佳路由并为每个路由器生成转发表。路由器的工作很单纯：收到分组、查转发表、转发分组。
- 核心思想：把控制层面和数据层面分离，让控制层面利用软件来控制数据层面的设备，网络又变成集中控制的。

OpenFlow 协议是 SDN 体系结构中控制层面与数据层面之间的通信接口。传统数据层面的任务是

按转发表转发分组，分两步：匹配（查转发表中的网络前缀，做最长前缀匹配）与动作（从匹配结果指明的接口转发）。SDN 的广义转发范围大大扩展：匹配可对各层（数据链路层、网络层、运输层）首部字段进行；动作不仅转发，还可负载均衡、重写 **IP** 首部（类似 NAT）、阻挡或丢弃分组（类似防火墙）。因此完成” 匹配 + 动作” 的设备不再称路由器，而称”**OpenFlow** 交换机” 或分组交换机；取代转发表的是” 流表（Flow Table）”。

- 一个流是穿过网络的一种分组序列，序列中每个分组都共享首部某些字段的值（例如相同源、目的 IP 地址的一连串分组）。
- 流表由 **SDN** 远程控制器管理，控制器通过安全信道用 OpenFlow 协议管理交换机的流表。
- 每个交换机必须有一个或多个流表，每个流表可含多个流表项。
- 每个流表项含三个字段：首部字段值（匹配字段）、计数器、动作。匹配不上的分组被丢弃或送往远程控制器。OpenFlow 交换机既处理链路层帧，也处理 IP 数据报与 TCP/UDP 报文。
- 计数器记录已匹配的分组数量和该流表项上次更新至今的时间。
- 动作可转发到指明端口、丢弃、复制后从多端口转发、重写各层首部。

SDN 体系结构的接口：

接口	连接对象	作用
北向接口	面向上层应用开发者	提供丰富 API，不必关心底层硬件细节
南向接口	控制器与转发设备之间	建立双向会话；典型协议是 **OpenFlow**，可兼容不同硬件
东西向接口	控制器集群内部控制器之间	增强控制平面的可靠性和可扩展性

SDN 的四个关键特征：基于流的转发；数据层面与控制层面分离；网络控制功能位于分组交换机之外；可编程的网络。

优点：全局集中式控制与分布式高速转发并存；可用自动化工具编程配置；采用开放接口协议后设备制造与功能软件开发相分离，降低成本。问题：集中管理易受攻击，崩溃则全网受影响；规模扩大后控制器可能成为性能瓶颈。

真题考法

2022 年第 37 题考”SDN 控制器向数据平面的 SDN 交换机下发流表时使用的接口”，答案是南向接口。北向接口面向上层网络应用，东西向接口用于控制器之间协同。

（四）拥塞控制

拥塞指因出现过量的分组而引起网络性能下降的现象。判断方法是观察吞吐量与网络负载的关系：若负载增加而吞吐量明显小于正常值，则进入轻度拥塞；若吞吐量随负载增大而下降，则已进入拥塞。

拥塞控制要解决如何获取网络中发生拥塞的信息并据此控制，避免因拥塞而出现分组丢失，作用是确保网络能够承载所达到的流量：

- 开环控制：设计网络时事先把拥塞因素考虑周到，力求不产生拥塞，是静态的预防方法，一旦启动中途不再修改，做决定时不考虑当前网络状态。手段包括确定何时可接收新流量、何时可丢弃分组及丢弃哪些分组、确定何种调度策略。
- 闭环控制：事先不考虑拥塞因素，用监测系统监视，检测到哪里发生拥塞后把信息传到合适的地方以调整网络运行，基于反馈环路，是动态的方法。

与流量控制的区别：流量控制指发送方与接收方之间的点对点通信量控制，目的是抑制发送方速率，使接收方来得及接收；拥塞控制关注整个网络的负载，是全局性的。

二、路由算法

（一）静态路由与动态路由

路由算法的目的是找到从源路由器到目的路由器的”最佳”路径（通常是费用最低的路径）。所有最短路径算法都依赖一个性质：两点之间的最短路径也包含了路径上其他顶点间的最短路径。

静态路由选择	动态路由选择
采用人工配置方式添加网络路由、默认路由、特定主机路由	路由器通过路由选择协议自动获取，路由器间彼此交换信息并按算法优化出路由表项
简单、开销小，但不能及时适应网络状态变化	较复杂、开销较大，但能较好地适应网络状态变化
一般只在小规模网络采用	适用于大规模网络

默认路由用特殊前缀 `0.0.0.0/0` 表示：`0.0.0.0` 表示任意网络，`/0`（掩码 `0.0.0.0`）是最短网络前缀。全 0 掩码与任何目的地址按位与必为全 0，因此必然匹配。只要目的网络不在转发表中，就一律选择默认路由，通常用于路由器到互联网的路由。由于最长前缀匹配，默认路由匹配优先级最低。

特定主机路由对某台主机的 IP 地址专门指明路由，便于管理员控制与测试网络。若主机地址是 a.b.c.d，则目的网络是 `a.b.c.d/32`，掩码 `255.255.255.255` 是最长前缀，因此匹配优先级最高。

路由匹配优先级：特定主机路由 > 最长前缀子网路由 > 默认路由。

静态路由配置的陷阱

可能出现两类问题：路由条目配置错误，甚至导致路由环路；聚合路由条目时可能引入不存在的网络（聚合后的地址范围大于原子网并集，把未分配地址也通告出去）。

（二）距离-向量路由算法

距离-向量算法是分散型的：路由器只掌握物理相连的邻居及链路费用，通过反复与邻居交换信息逐步收敛，其理论基础是 **Bellman-Ford** 算法。设 `d_x(y)` 为结点 x 到 y 的最短路径费用，`c(x,v)` 为 x 到邻居 v 的费用，则

$$d_x(y) = \min_v \{\, c(x,v) + d_v(y) \,\}$$

其中 **v** 取遍 **x** 的所有邻居。每个结点维护三组信息：到每个直连邻居的链路费用 `c(x,v)`；自己的距离向量；收到的每个邻居的距离向量。

算法流程：每个结点定期向每个邻居发送距离向量副本。当 x 从邻居 v 收到新距离向量时，先保存它，再用 Bellman-Ford 公式更新自己的距离向量；若自己的距离向量因此改变，则继续向每个邻居发送更新后的距离向量。

收敛小例子：三结点 x、y、z，`c(x,y)=2`、`c(x,z)=7`、`c(y,z)=1`。初始化时各结点只知道自己到直连邻居的费用：`d_x=(0,2,7)`、`d_y=(2,0,1)`、`d_z=(7,1,0)`。第一次交换后 x 计算 `d_x(y)=min{2+0, 7+1}=2`、`d_x(z)=min{2+1, 7+0}=3`，得 `d_x=(0,2,3)`。再次交换后若无变化则不再发送更新报文，算法进入静止状态（收敛）。

更新报文的规模

更新报文大小与网络中结点数目成正比，大型网络将导致很大的更新报文，这是距离-向量算法可扩展性差的根本原因。最常见的距离-向量路由算法是 **RIP**，采用” 跳数” 作为距离度量。

*1.*路由环路与坏消息传得慢

距离-向量算法有一个固有问题：好消息传得快，坏消息传得慢，又称路由环路或距离无穷计数问题。

环路如何形成：B 到网络 N 的距离本是 1。当 N 真正不可达时，B 又可能从 A 那里学到”N 距离 2”，算得 2+1=3 认为经 A 可到 N；A 再从 B 学到 3、算得 4……两个路由器互相” 欺骗” 对方” 你再加一跳就能到达”，距离在二者之间逐次加 1，直到 16 才停止。

缓解措施：

- 限制最大 **RIP** 距离为 **15**（16 表示不可达），使无穷计数在有限步内终止；
- 触发更新：路由表变化时立即发送更新报文，而不仅是周期性发送；
- 水平分割：记录收到某条路由信息的接口，不让同一信息再从该接口反向发回；
- 毒性逆转：向学到此路由的那个接口通告时，把距离直接置为 **16** 而不是不发。这比水平分割更彻底——不发报文时对方可能仍持旧信息，显式通告 16 能让对方立即删除该路由。

措施只能缓解，不能彻底解决

距离-向量算法中每个路由器都缺少到目的网络整个路径的完整信息，无法判断所选路由是否出现环路。这与链路状态算法中每台路由器掌握全网拓扑形成根本差别。

（三）链路状态路由算法

链路状态（Link State，LS）指本路由器都和哪些路由器相邻，以及相应链路的代价。链路状态算法是全局性的：所有路由器掌握完整的网络拓扑和链路费用信息。

主要思想：每个结点都具有全网拓扑结构图（全网一致）；主动测试所有相邻结点的状态并定期把链路状态传播给所有其他结点；每个结点用 **Dijkstra** 最短路径算法计算到其他结点的最短路径。结点每收到一个链路状态报文就更新自己的” 视野图”，一旦链路状态变化就用 Dijkstra 重新计算。

Dijkstra 执行要点：以本路由器为源点，维护” 已确定最短路径的结点集合”，每次从集合外选出当前距离最小的结点加入集合，并用它松弛其所有邻居的距离，时间复杂度 $O(n^2)$（n 为结点数）。由于各路由器使用同样的链路状态数据独立计算，算出的路径必然一致，因此从算法上保证了不会产生路由环路。

优点：各结点用同样的链路状态数据独立计算路径，不依赖中间结点的计算，易于查找故障；链路状态报文仅运载单个结点关于直接链路的信息，大小与网络结点数目无关，所以规模可伸展性更好，适用于大型或路由信息变化剧烈的互联网环境。

两类算法对比：距离-向量算法中每个结点仅与直接邻居交谈，向邻居发送自己的整个路由表，报文大小取决于结点数目，代价较大；链路状态算法中每个结点通过洪泛与所有其他结点交谈，但只告诉它们与自己直接相连的链路费用。

（四）层次路由

因特网规模非常大，且许多单位不愿让外界了解自己网络的布局细节，因此采用分层次的路由选择协议，特点有三个：自适应（动态路由选择，能较好适应网络状态变化）；分布式（各路由器通过相互间信息交互共同完成路由信息的获取与更新）；分层次（把整个因特网划分为许多较小的自治

系统（Autonomous System，**AS**），内部和外部采用不同类别的路由选择协议）。

自治系统指在单一的技术管理下的一组路由器，这些路由器使用一种 AS 内部的路由选择协议和共同的度量确定 AS 内的路由，同时使用一种 AS 之间的路由协议确定 AS 之间的路由。一个 AS 内的所有网络都属于一个行政单位管辖，AS 内所有路由器必须连通。层次路由的效果是：路由器只知道自己所在区域内的目标地址如何路由，对其他区域的内部结构不知情，路由表规模因此大幅减小。

区域划分的代价与收益

分层次划分区域使交换信息的种类增多、OSPF 协议更复杂，但能使每个区域内部交换路由信息的通信量大大减少，从而让 OSPF 可用于规模很大的自治系统。这是典型的” 用复杂度换可扩展性”。

三、IPv4

（一）IPv4 分组

IPv4 分组由首部和数据部分组成。首部前一部分长度固定，共 **20 B**，是所有 **IP** 分组必须具有的；后面是可选字段，长度可变（0 ~ 40 B），实际很少使用；当首部长度不是 4 字节整数倍时用填充补齐全 0 字节。

三个” 基本单位” 必须背熟

首部长度以 **4 B** 为单位，总长度以 **1 B** 为单位，片偏移以 **8 B** 为单位。口诀” 一种八片首饰”。IP 首部前两个字节往往以 `0x45` 开头，解题时可用于定位 IP 数据报的开始位置。

首部各字段逐项说明：

字段	位数	含义与要点
版本	4	IP 版本，IPv4 中该字段值是 4。通信双方的版本必须一致
首部长度	4	以 **4 B** 为单位。最小取值 0101（5），乘 4 得 20 B，表示只有固定部分；最大取值 1111（15），得 60 B，表示含 20 B 固定部分和最大 40 B 可变部分
区分服务	8	用来获得更好的服务，实际上一直没有被真正使用过
总长度	16	首部 + 数据载荷的长度和，单位字节 **B**。最大 $2^{16}-1=65535$ B。以太网 MTU 为 1500 B，封装成帧时数据报总长度不能超过链路层 MTU
标识	16	一个计数器，每产生一个数据报就加 1。并不是**”** 序号**”**。同一数据报的所有分片复制同一个标识号，以便正确重装
标志	3	最低位 **MF**：MF=1 后面还有分片，MF=0 是最后一个分片。中间位 **DF**：只有 **DF=0** 才允许分片。最高位保留，必须为 0
片偏移	13	某片在原数据报中的相对位置，以 **8 B** 为单位。除最后一个分片外，每个分片的数据长度一定是 **8 B** 的整数倍
生存时间 **TTL**	8	数据报可通过的路由器数的最大值，防止被错误路由的数据报无限制兜圈。以” 跳数” 为单位：路由器转发前减 1，不为 0 就转发，否则丢弃并发送 ICMP 时间超过报文
协议	8	指明数据载荷是何种 PDU。常用值：**ICMP=1**，**IGMP=2**，**TCP=6**，**UDP=17**，**OSPF=89**
首部检验和	16	只检验首部，不包括数据部分。因为每经过一个路由器某些字段（TTL、标志、片偏移等）都可能变化，都要重算检验和，不检验数据部分可减少计算量。IPv6 中路由器不再计算首部检验和

字段	位数	含义与要点
源 **IP** 地址	32	标识发送方
目的 **IP** 地址	32	标识接收方

标志字段的位序

3 位标志从左到右依次是「保留位、DF、MF」，最低位才是 **MF**。若给出标志字段的十六进制值，必须展开为二进制后再按位判断。

1.IP 数据报分片

为什么要分片：IP 数据报被封装在链路层的帧中，因此链路层的 **MTU**（最大传送单元）严格限制了 IP 数据报长度。以太网 MTU 为 **1500 B**，许多广域网 MTU 不超过 576 B。当数据报总长度大于链路 MTU 时，需要把数据分装在多个较小的 **IP** 数据报中，这些较小的数据报称为片。

位置：分片可由源主机或路径上的路由器进行；重组只在目的主机的网络层进行。每个分片独立传输，可能经过不同路径，甚至不一定按原次序到达。

重组所用字段：目的主机用标识、标志、片偏移完成重组。MF=1 表示还有后续片，MF=0 表示最后一片；片偏移确定该片应放在原始数据报的哪个位置。分片时每个分片都复制原数据报首部的固定部分（**20 B**）。

DF=1 的情况：不能分片而分组长度又超过 MTU 时，丢弃该分组，并用 **ICMP** 差错报告报文（终点不可达，代码” 需要分片但设置了 DF 位”）向源主机报告。

IP 分片完整计算步骤（必背）

设原数据报总长 L（首部 H 字节，通常 20 B），链路 MTU 为 M。

1. 原数据部分长度 $D = L - H$。
2. 每片最多携带的数据长度：$d_{\max} = \left\lfloor \dfrac{M - H}{8} \right\rfloor \times 8$。
3. 分片个数 $n = \left\lceil \dfrac{D}{d_{\max}} \right\rceil$；前 $n-1$ 片各携带 $d_{\max}$ 字节，最后一片携带剩余 $D - (n-1)d_{\max}$ 字节（可以不是 8 的倍数）。
4. 每片总长度字段 $= d_i + H$（含首部）。
5. 每片片偏移字段 $= \dfrac{\text{该片之前已发送的数据字节数}}{8}$。
6. **MF**：前 $n-1$ 片为 **1**，最后一片为 **0**。所有片标识相同，DF 均为 0。

【算例 **1**】某 IPv4 数据报总长度 3820 B，采用 20 B 固定首部，需分片为长度不超过 1420 B 的数据报片。

解：$D = 3820 - 20 = 3800$ B。每片最多 $1420 - 20 = 1400$ B，且 $1400 = 175 \times 8$ 恰为 8 的整数倍，故 $d_{\max} = 1400$。$n = \lceil 3800/1400 \rceil = 3$：$1400 + 1400 + 1000 = 3800$。

分片	数据长度/**B**	总长度字段/**B**	片偏移字段	**MF**
第 1 片	1400	1420	0	1
第 2 片	1400	1420	$1400/8 = 175$	1
第 3 片	1000	1020	$2800/8 = 350$	0

【算例 **2**：真题型】 2021 年第 36 题：路由器向 MTU=800 B 的链路转发总长度 1580 B 的 IP 数据报（首部 20 B），分片时每片尽可能大，则第 2 个分片的总长度字段和 MF 分别是多少?

解：$D = 1580 - 20 = 1560$ B。每片最多 $800 - 20 = 780$ B，但除最后一片外必须是 8 的整数倍，故 $d_{\max} = \lfloor 780/8 \rfloor \times 8 = 776$ B。$n = \lceil 1560/776 \rceil = 3$：$776 + 776 + 8 = 1560$。

分片	数据长度/B	总长度字段/B	片偏移字段	MF
第 1 片	776	796	0	1
第 2 片	776	**796**	$776/8 = 97$	**1**
第 3 片	8	28	$1552/8 = 194$	0

答案：**796**，**1**（选项 B）。

分片三大易错点

1. 总长度字段包含首部，第 2 片总长度是 $776 + 20 = 796$，不是 776，也不是 800。
2. 片偏移的单位才是 **8 B**，不是” 总长度必须是 8 的倍数”——第 3 片总长度 28 就不是 8 的倍数。
3. 只有除最后一片外的数据长度必须是 **8 B** 的整数倍；最后一片可为任意长度。

【算例 **3**：两级分片】总长 1500 B（首部 20 B）的 IP 数据报先经 MTU=800 B 的链路 R1→R2，再经 MTU=400 B 的链路 R2→R3，求 R2 分出的各子分片参数。

解：第一级（MTU=800）：$D = 1480$，$d_{\max} = \lfloor 780/8 \rfloor \times 8 = 776$，分片为 $776 + 704$。

R1 分片	数据/B	总长度/B	片偏移	MF
F1	776	796	0	1
F2	704	724	97	0

第二级（MTU=400）只需对 F1 再分：$d_{\max} = \lfloor (400 - 20)/8 \rfloor \times 8 = 47 \times 8 = 376$ B。F1 的 776 B 分为 $376 + 376 + 24$。关键性质：第二级分片的片偏移是在原数据报范围内的偏移，新片偏移 = 父分片片偏移 $\times 8 +$ 在父分片内的字节偏移。

由 **F1** 分出	数据/B	总长度/B	片偏移	MF
子片 1	376	**396**	$(0 + 0)/8 = 0$	1
子片 2	376	396	$(0 + 376)/8 = 47$	1
子片 3	24	44	$(0 + 752)/8 = 94$	**1**

第三个子分片的 MF 仍是 **1**，因为它由 F1 分出而 F1 的 MF=1，后面还有 F2；F2 的 MF=0 才表示整个数据报结尾。

*2.*首部检验和的计算

发送方：① 把首部划分为若干个 **16** 比特字序列（20 B 固定首部正好 10 个字）；② 把首部检验和字段置为全 **0**；③ 用反码算术运算把所有 16 比特字相加；④ 把和取反码写入首部检验和字段。

接收方：把首部所有 16 比特字（此时含检验和）再用反码算术运算相加，并把和取反码。若首部无差错，结果必为 **0**，保留该数据报；否则认为出错，丢弃该数据报。

二进制反码求和规则（从低位到高位逐列）：0+0=0；0+1=1；1+1=0 并产生进位 1 加到下一列；若最高位相加后产生进位，则结果要再加 **1**（”进位绕回来加到末尾”）。

首部检验和的两个易错点

1. 接收方发现检验和出错后直接丢弃数据报，不发送差错报告（ICMP 差错报文不针对首部检验和错误）。
2. 该方法与 UDP、TCP 的检验和计算方法相同，但 **IP** 校验没有伪首部。这个算法常被称为因特网检验和（Internet Checksum），检错性能不如 CRC，但更易用软件实现。

【算例】对典型首部 `45 00 00 3C 1C 46 40 00 40 06 00 00 C0 A8 00 01 C0 A8 00 C7` 求首部检验和。

逐字相加（检验和字段先置 0）：`4500+003C+1C46+4000+4006+0000+C0A8+0001+C0A8+00C7`，按列运算得 `0x43FF6`。把高 16 位溢出部分 4 加回低 16 位：`0x3FF6+4=0x3FFA`。取反码得 `0xC005`。

验证：接收方把全部 10 个字（含 `C005`）相加得 `0x4FFFB`，进位 4 回加得 `0xFFFF`，取反码为 `0x0000`，校验通过。

（二）IPv4 地址与 NAT

1.IPv4 地址与分类编址

定义：IPv4 地址是给因特网上每一个主机（或路由器）的每一个接口分配的、全世界范围内唯一的 **32** 比特标识符，由网络号和主机号两部分组成：

$$\text{IP 地址} = \{ < \text{网络号} >, < \text{主机号} > \}$$

用点分十进制表示：32 比特按 8 位一组分为 4 组，每组写十进制，用点分隔。

	网络号	主机号
标志	接口所连接到的网络	主机（或路由器）的接口
规则	同一网络中不同主机接口的网络号必须相同	同一网络中不同主机接口的主机号必须各不相同

分类编址：按左起第一个十进制数判断——小于 127 为 A 类，128 ~ 191 为 B 类，192 ~ 223 为 C 类，224 ~ 239 为 D 类，240 ~ 255 为 E 类。

类别	首位模式	网络号位数	网络号范围	可指派网络数	每网可分配地址数
A	0	8	1 ~ 126	$2^7 - 2 = 126$	$2^{24} - 2 = 16777214$
B	10	16	128.0 ~ 191.255	$2^{14} = 16384$	$2^{16} - 2 = 65534$
C	110	24	192.0.0 ~ 223.255.255	$2^{21} = 2097152$	$2^8 - 2 = 254$
D	1110	—	224.0.0.0 ~ 239.255.255.255	—	多播地址，不分配给主机接口

A 类中最小网络号 **0** 表示本网络不能指派，最大网络号 **127** 作为环回测试地址不能指派，故可指派 A 类网络数为 $2^{8-1}-2=126$。最小环回地址 127.0.0.1，最大环回地址 127.255.255.254。

”减 2”的含义

- A 类网络数减 2：去掉网络号 0（本网络）和 127（环回）。
- 每个网络可分配地址数减 2：去掉主机号全 **0** 的网络地址和主机号全 **1** 的广播地址。

A、**B**、**C** 类都是单播地址，只有单播地址可分配给主机（或路由器）的接口。主机号全 **0** 是网络地址，主机号全 **1** 是广播地址，都不能分配。

一般不使用的特殊 **IPv4** 地址：

网络号	主机号	地址示例	作源	作目的	用途
全 0	全 0	0.0.0.0	可以	不可以	本网络的本主机（DHCP 初始态）；路由表中表示默认路由
全 0	host-id	0.0.0.5	可以	不可以	本网络上的某台特定主机
全 1	全 1	255.255.255.255	不可以	可以	只在本网络广播（路由器不转发），又称受限广播地址
net-id	全 0	192.168.1.0	不可以	不可以	网络地址，表示一个网络本身
net-id	全 1	192.168.1.255	不可以	可以	对网络 net-id 上所有主机广播，又称直接广播地址（可跨路由器，只被目标网络内主机接收）
127	非全 0/全 1	127.0.0.1	可以	可以	本地软件环回测试，目的地址为环回地址的数据报永远不会出现在任何网络上

特殊地址速记口诀

全 **0** 的地址只能当源，不能当目的；全 **1** 的地址只能当目的，不能当源；**127** 开头源目都能用，但不走外网；主机号全 **0**/全 **1** 的地址永远不能分给主机用。路由器收到目的地址为 255.255.255.255 的分组不转发。

IP 地址的重要特点：只配网络号（管理机构只分配网络号，主机号由单位自行分配）；只转发网络号（路由器仅根据目的网络号转发，减小路由表存储空间）；一个接口一个地址（路由器至少有两个以上 IP 地址，每个端口都有不同网络号的地址）；同网同域（用转发器或网桥连接的若干 LAN 仍属同一网络、同一广播域，网络号必须相同、主机号必须不同）；众网平等。

*2.*网络地址转换 *NAT*

背景与私有地址：为网络安全划出了部分 IP 地址为私有 IP 地址。私有地址只用于 LAN，不用于 WAN 连接，必须通过网关用 NAT 转换为合法全球 IP 地址后才能出现在 Internet 上，且允许被 LAN 重复使用。互联网中所有路由器对目的地址是私有地址的数据报一律不转发。 RFC 1918 规定的三个专用地址块：

类别	地址范围	**CIDR** 前缀
A 类	10.0.0.0 ~ 10.255.255.255	10.0.0.0/8
B 类	172.16.0.0 ~ 172.31.255.255	172.16.0.0/12
C 类	192.168.0.0 ~ 192.168.255.255	192.168.0.0/16

定义：网络地址转换（NAT）指把专用网络地址转换为公用地址，从而对外隐藏内部管理的 IP 地址，作用是连接内网和外网。优点是节省 IP 地址消耗（专用网只需一个全球 IP 地址就可与互联网连通，本地地址可重用）与降低安全风险（隐藏内部网络结构）。

最基本的 **NAT** 与 **NAPT**：因绝大多数网络应用使用 TCP 或 UDP，为更有效利用 NAT 路由器上的全球 IP 地址，现在常把 **NAT** 转换与运输层端口号结合使用。NAT 路由器用 **NAT** 转换表存储 `{本地 IP 地址:端口}` → `{全球 IP 地址:端口}` 的映射，正式名称是网络地址与端口号转换（Network Address and Port Translation，**NAPT**）。这种映射可让多个私有 IP 地址映射到一个全球 IP 地址，即内部专用网中大量主机共用 **NAT** 路由器上的 **1** 个全球 **IP** 地址。很多家用路由器就是 **NAPT** 路由器，但往往并不运行路由选择协议。

NAT 工作原理（经典例子）：主机 10.0.0.1（端口 3345）向 Web 服务器 128.119.40.186（端口 80）发请求。

1. NAT 路由器收到分组后生成新端口号 5001，把源地址改为 138.76.29.7（NAT 路由器的全球 IP 地址），源端口号改为 5001，并在 NAT 转换表中增加一个表项。
2. Web 服务器不知道分组已被改装，也不知道用户的专用地址，其响应分组目的地址是 138.76.29.7、目的端口号是 5001。
3. 响应分组到达 NAT 路由器后，通过转换表把目的 IP 地址改为 10.0.0.1、目的端口号改为 3345。

同端口号的处理：两个客户端 192.168.1.10 和 192.168.1.11 都用本地端口 1025 与服务器通信，二者都转换为公有地址 120.229.175.121，但用不同的端口号区分。这种转换表自动生成：TCP 情况下建立连接首次握手时的 SYN 包一经发出就生成表项，随收到关闭连接 FIN 包的确认应答而从表中删除。

NAT 路由器与普通路由器的核心差别

普通路由器只处理网络层及以下数据，仅解析 IP 首部；**NAT** 路由器需要查看运输层端口号来维护 NAT 映射表，因此同时作用于传输层、网络层及以下。

报文修改规则：出门改源 **IP**（变公网），进门改目的 **IP**（找内网）。内网 → 外网修改源 **IP** 地址、源端口号；外网 → 内网修改目的 **IP** 地址、目的端口号。

NAT 的缺点与穿透问题：NAT（和 NAPT）对网络应用并不完全透明；通信必须由专用网内部发起，因此拥有内部专用地址的主机不能直接充当因特网中的服务器——只有内网主机主动发起连接，NAT 设备才建立映射表，外网主机的返回报文依靠该映射转回内网；外网主机主动访问内网时 NAT 设备没有对应映射条目，报文无法转发到内网主机。对 P2P 这类需要外网主机主动与内网主机通信的应用，需要 **NAT** 穿透技术（端口转发、STUN、中继）。若配置静态 **NAT**/端口映射建立固定的外网 `IP:外网端口` ↔ 内网 `IP:内网端口`对应关系，就能强行实现内网主机对外提供公网服务。

真题考法

NAT 常与 CIDR 结合出综合题，核心是判断” 分组经过某个 NAT 路由器后源/目的地址变成什么”。2016 年第 38 题中源地址换成 R2 外网接口地址 201.1.3.10，而目的地址始终是最终的 **Web** 服务器地址 **130.18.10.1**。铁律：路由器转发不改变目的 **IP** 地址；只有 **NAT** 设备才按转换规则修改源或目的地址。

（三）子网划分、路由聚集、子网掩码与 **CIDR**

*1.*划分子网与子网掩码

背景：随着更多中小网络加入因特网，分类编址” 不够灵活、容易造成大量 IPv4 地址资源浪费” 的缺点暴露出来。申请新网络号需等待很长时间、花费更多费用，其他路由器还要新增路由条目，且

浪费原网络中剩余的大量地址。若能从主机号部分借用一些比特作为子网号区分不同子网，就可利用原有网络中剩余的大量地址而不用申请新网络地址。

定义：划分子网是在 IP 地址中增加一个” 子网号字段”，使两级 IP 地址变成三级：

$$\text{IP 地址} = \{ < \text{网络号} >, < \text{子网号} >, < \text{主机号} > \}$$

子网掩码表明分类 IPv4 地址的主机号部分被借用了几个比特作为子网号。它也是 **32** 比特：左起多个连续的比特 **1** 对应网络号和子网号，之后的连续 0 对应主机号。把划分子网的 IPv4 地址与子网掩码逐比特逻辑与，就得到该地址所在子网的网络地址。

一句话结论

只要给定了一个分类的 **IPv4** 地址及其相应的子网掩码，就可以得出子网划分的全部细节。这是子网划分所有计算题的解题总纲。

默认子网掩码：所有网络都必须使用子网掩码；若未划分子网就使用默认掩码，**A**、**B**、**C** 类分别为 **255.0.0.0**、**255.255.0.0**、**255.255.255.0**。其他规定：路由器相互交换路由信息时必须告知对方本网络的子网掩码；转发时把目标地址与某网络掩码按位相与，结果与该网络地址一致则匹配成功；主机设置 IP 时必须设置子网掩码；同属一个子网的所有主机及路由器相应端口必须设置相同掩码；路由表主要信息是目的网络地址、子网掩码、下一跳地址。

默认网关是子网与外部网络连接的设备，即连接本机或子网的路由器接口的 **IP** 地址（离主机最近的路由器的接口地址）。主机发送数据时用子网掩码判定目的主机是否在同一子网：在则直接发送，不在则交给默认网关转发。

划分子网的核心结论

1. 划分子网增加子网数量，子网间传输需经路由器，因此减小了广播域；子网号占据主机号位，因此减少每个子网的主机数量。
2. 从一个 **IP** 地址本身无法判断该主机所连接的网络是否划分了子网，单位外的网络看不见单位内的子网划分。
3. 划分子网只提高 IP 地址利用率，不改变 **IP** 地址原有的类别归属，也不增加” 网络” 数量。
4. 子网中主机号全 0 的地址是子网的网络地址，全 1 的是子网的广播地址，都不能指派给主机。

*2.*给定 *IP* 与掩码求网络地址、广播地址与可用地址范围

标准解题步骤：

1. 确定块大小：找到掩码中发生变化（1→0）的那个字节，块大小 = 256− 该字节的十进制值。
2. 网络地址：把 IP 中该字节的值向下取到块大小的整数倍，其后字节全部置 0。
3. 广播地址：网络地址把主机位全部置 1，即” 下一个块的起点减 1”。
4. 可用地址范围：网络地址 $+1$ 到广播地址 -1，数量 $= 2^h - 2$（h 为主机号位数）。

【算例 **1**】主机 180.80.77.55，子网掩码 255.255.252.0，求所在子网的网络地址、广播地址与可用地址范围。

解：掩码 = **/22**。第三字节掩码 252，块大小 $= 256 - 252 = 4$。第三字节 77 落在 $76 \sim 79$ 块内（76 是 4 的整数倍）。

- 网络地址：**180.80.76.0**；广播地址：**180.80.79.255**；
- 可用范围：**180.80.76.1～180.80.79.254**，共 $2^{10} - 2 = 1022$ 个。

（2012 年第 39 题考此型，目的地址可以是 180.80.79.255，选 D。）

【算例 **2**】主机 168.16.84.24/20 所在子网的最小、最大可分配地址。解：/20 表示第三字节高 4 位属网络前缀，块大小 $= 2^4 = 16$。84 落在 $80 \sim 95$，故网络地址 **168.16.80.0**，广播地址 **168.16.95.255**，最小可分配 **168.16.80.1**，最大可分配 **168.16.95.254**。（2023 年第 39 题，选 B。）

【算例 **3**】主机 183.80.72.48，掩码 255.255.192.0，求网络地址。解：掩码 = **/18**，第三字节块大小 $= 256 - 192 = 64$。72 位于 $64 \sim 127$ 块，网络地址 = **183.80.64.0**。（2022 年第 35 题，选 B。）

最容易错的三处

1. 把掩码的**”** 非 **255** 字节**”** 换算错：255.255.252.0 是 /22。用 $256 - 252 = 4$ 反推块大小最稳。
2. 误以为必须在 **/24** 边界：块大小可落在任意字节。若掩码 255.255.255.128（/25），块大小为 128，块起点是 0 和 128。
3. 可用地址数忘记减 **2**。

*3.*子网划分的完整步骤与综合大题

定长划分的标准步骤（已知网络地址与所需子网数或每子网主机数）：

1. 确定借位数 s：若给定子网个数 k，取 $2^s \geq k$；若给定每子网至少容纳 x 台主机，则主机位数 h 满足 $2^h - 2 \geq x$，再据此反推 $s = n - h$（n 为原主机位数）。
2. 写出新掩码：前缀长度 $= 32 - h$，写成点分十进制。
3. 可用子网数 $= 2^s$（定长划分下各子网掩码相同，子网数只能是 2 的整数次幂）。
4. 每子网地址数 $= 2^h$，其中可分配给主机的数量是 $2^h - 2$。
5. 列出各子网的网络地址与广播地址：块大小 $= 2^h$，第 i 个子网的网络地址 = 原网络地址 $+ i \times$ 块大小。
6. 写出可用地址范围：网络地址 $+1$ 到广播地址 -1。

综合大题标准答题模板

以 2009 年第 47 题第 (1) 问为例：把 202.118.1.0/24 划分为 2 个子网，每个局域网需分配的 IP 地址数不少于 120 个。

解：每个局域网至少需 120 个可分配地址，$2^h - 2 \geq 120$ 即 $2^h \geq 122$，取 $h = 7$（$2^7 = 128$，$2^6 = 64$ 不够）。前缀长度 $32 - 7 = 25$，子网掩码 **255.255.255.128**，块大小 128。

- 局域网 1：**202.118.1.0/25**，可分配 202.118.1.1 ~ 202.118.1.126，广播 202.118.1.127；
- 局域网 2：**202.118.1.128/25**，可分配 202.118.1.129 ~ 202.118.1.254，广播 202.118.1.255。

两子网各有 126 个可分配地址，满足要求（次序互换也正确）。

【算例：定长划分组合题】192.168.5.0/24 采用定长子网划分，掩码 255.255.255.248，求最大子网个数与每子网最大可分配地址个数。解：255.255.255.248 = **/29**，原网络 /24，借用 $29 - 24 = 5$ 位子网号，得 $2^5 = 32$ 个子网；每子网剩余 3 位主机号共 8 个地址，扣除网络地址和广播地址后最多分配 $2^3 - 2 = 6$ 个。（2010 年第 37 题，选 B：32，6。）

【算例：反推最小子网】将 101.200.16.0/20 划分为 5 个子网，可能的最小子网的可分配 IP 地址数是多少？解：/20 含 $2^{12} = 4096$ 个地址。完整划分为 5 个 CIDR 子网时各子网大小必须是 2 的幂且总和为 4096，可取 $2048 + 1024 + 512 + 256 + 256 = 4096$，故最小子网可有 256 个地址即 /24，可分配 $256 - 2 = 254$ 个。不能选 126：若最小子网只有 128 个地址，剩余 3968 个地址无法恰好由另外 4 个二进制地址块组成。（2019 年第 37 题，选 B。）

【算例：判断不可能的子网】把一个 IP 网络划分为 3 个子网，其中一个子网是 192.168.9.128/26，下

列不可能是另外两个子网之一的是？A. 192.168.9.0/25　B. 192.168.9.0/26　C. 192.168.9.192/26　D. 192.168.9.192/27

解：已知子网.128/26 覆盖.128～.191。选 A（.0/25 覆盖.0～.127）时剩余.192～.255 恰为.192/26，三个子网完整划分，**A** 可能；选 C（.192/26）与 A 搭配同理，**C** 可能；选 D（.192/27 覆盖.192～.223）时.128/26 与.192/27 同属父网络.128/25，再取.0/25 作第三个，**D** 可能；选 B（.0/26 覆盖.0～.63）时尚未划分的地址为.64～.127 与.192～.255 两段不连续，无法用仅剩的一个 CIDR 子网覆盖，**B** 不可能。答案 **B**。（2021 年第 35 题。）

用”二叉树阀门”理解变长划分

把 CIDR 地址块看成二叉树，均分一次就是把节点一分为二。划分出 k 个子网等价于在二叉树的 k 个位置添加”阀门”，每个阀门对应的子树即为一个子网的地址范围。约束为不重、不漏：同一条根到叶子的路径上不能放多个阀门（防重叠），所有节点都必须被分配（防浪费）。阀门可放在任意层级节点（/25、/26、/27 均可）。

*4.*变长子网掩码与地址规划

定长子网掩码 **FLSM**	变长子网掩码 **VLSM**
每个子网使用同一个子网掩码，各子网网络前缀相同	每个子网可使用不同子网掩码，各子网网络前缀可不同
子网数量只能是 2 的整数次幂（2，4，8，……）	子网数量可以不是 **2** 的整数次幂（3，5，6，7，9，11……）
每子网地址数量相同，容易造成地址资源浪费	每子网地址数量可不同，尽可能减少浪费

地址规划中必须考虑的三类地址消耗：终端主机各分配 1 个可用主机 IP；路由器接口各分配 1 个独立 IP；子网保留地址（网络地址 1 个、广播地址 1 个）不能分配给任何设备。

特殊情况：连接两个路由器的点对点链路（无终端主机、只有 2 个路由器互连接口）仍需预留 1 个网络地址和 1 个广播地址，合计需要 **4** 个 **IP** 地址，因此通常分配一个 **/30** 地址块，恰好可分配 2 个地址给链路两端接口。

变长划分的核心原则

每个子块的起点位置不能随便选取，只能选取主机号部分是块大小整数倍的地址作为起点。建议先为大的子块选取。

【算例：变长划分综合应用题】地址块 218.75.230.0/24，需求为：网络 1 需 9 个地址，网络 2 需 28 个，网络 3 需 15 个，网络 4 需 13 个，网络 5（路由器间链路）需 4 个。用变长掩码分配。

解：按需求从大到小排序分配（”需 x 个”指总共占用的地址数，含网络地址与广播地址）。

网络	需求总数	满足 $2^h \geq$ 需求的最小 h	块大小	前缀	分配的地址块
网络 2	28	$h=5$	32	/27	218.75.230.0/27
网络 3	15	$h=5$	32	/27	218.75.230.32/27
网络 4	13	$h=4$	16	/28	218.75.230.64/28
网络 1	9	$h=4$	16	/28	218.75.230.80/28

网络	需求总数	满足 $2^h \geq$ 需求的最小 h	块大小	前缀	分配的地址块
网络 5	4	$h=2$	4	/30	218.75.230.96/30

子网	网络地址	可用地址范围	广播地址
218.75.230.0/27	218.75.230.0	.1 ~ .30	218.75.230.31
218.75.230.32/27	218.75.230.32	.33 ~ .62	218.75.230.63
218.75.230.64/28	218.75.230.64	.65 ~ .78	218.75.230.79
218.75.230.80/28	218.75.230.80	.81 ~ .94	218.75.230.95
218.75.230.96/30	218.75.230.96	.97 ~ .98	218.75.230.99

剩余 218.75.230.100 ~ 218.75.230.255 留作后用。注意每个子块起点都是块大小的整数倍：0、32 是 32 的倍数；64、80 是 16 的倍数；96 是 4 的倍数。若改用定长掩码，则必须满足最大需求 28，取 $h=5$、块大小 32、掩码 255.255.255.224（/27），划分出 $2^3=8$ 个等长子网、每网 30 个可用地址，总浪费远大于变长方案。

5.CIDR 无分类编址与路由聚合

背景：划分子网方法缓解了困难，但数量巨大的 C 类网（$2^{21}=2097152$ 个）每个网络只含 $2^8=256$ 个地址，并未得到充分使用，IPv4 地址仍在加速消耗。为此 IETF 提出无分类编址方法，即无分类域间路由选择（Classless Inter-Domain Routing，**CIDR**），它消除了传统 **A**、**B**、**C** 类地址以及划分子网的概念。

CIDR 的地址结构：地址掩码与子网掩码类似，由 32 比特构成，左起多个连续的比特 **1** 对应网络前缀，之后的连续 0 对应主机号：

$$\text{CIDR 地址} = \{ < \text{网络前缀} >, < \text{主机号} > \}$$

斜线记法：在 IPv4 地址后面加斜线"/"，斜线后写上网络前缀所占的比特数量（即掩码中左起连续 1 的数量，位数不固定、可变长）。

CIDR 地址块：把网络前缀都相同的、连续的多个无分类 IPv4 地址组成一个 **CIDR** 地址块。只要知道块中任何一个地址，就可知道该块的全部细节：起始地址（主机号全 0 的网络地址）；最大地址（主机号全 1 的广播地址）；地址总数 $=2^{32-\text{前缀长度}}$；地址掩码；可分配给主机使用的地址数。

【例】给定地址 128.14.35.7/20，求所在 CIDR 地址块的全部细节。解：/20 表示前 20 位是网络前缀、后 12 位是主机号。把后 12 位全部置 0 得网络地址 **128.14.32.0**；全部置 1 得最大地址 **128.14.47.255**（第三字节 $32+15=47$）；地址总数 $=2^{12}=4096$；最小可分配 128.14.32.1，最大可分配 128.14.47.254，可分配数 4094；地址掩码 **255.255.240.0**。

CIDR 的优点：可根据客户需要分配适当大小的 **CIDR** 地址块，从而更有效地分配地址空间。

路由聚合（又称构造超网）：一个 CIDR 地址块可以表示很多地址，因此路由表中可利用 CIDR 地址块来查找目的网络，这些地址的聚合称为路由聚合或超网。它使路由表中的一个项目可以表示多个原来传统分类地址的路由，有利于减少路由器之间的信息交换。关键规律：网络前缀越长，地址块越小，路由越具体；前缀越短，地址块越大，可聚合的路由越多。若有多条路由条目匹配，则选择网络前缀最长的那条，称为最长前缀匹配。

路由聚合的标准解题步骤

1. 把待聚合的网络地址写成二进制（重点看" 变化字节" 那一段）。
2. 找出所有网络地址从左起完全相同的连续比特位数，即聚合后新前缀的长度。
3. 验证对齐：聚合后的地址块必须是严格的 **2** 的幂大小且起点对齐。若各网络不能恰好拼成一个完整的 2 的幂地址块，则不能完全聚合（否则会引入不存在的网络）。
4. 写出聚合后的网络地址（共同前缀 + 全 0）与前缀长度。

【算例 **1**：真题型】某路由表中有转发接口相同的 4 条表项，目的网络为 35.230.32.0/21、35.230.40.0/21、35.230.48.0/21、35.230.56.0/21，求聚合后的目的网络地址。解：/21 表示第三字节高 5 位属于网络前缀，第三字节块大小 $= 256 - 248 = 8$。四网依次覆盖第三字节 $32 \sim 39$、$40 \sim 47$、$48 \sim 55$、$56 \sim 63$。合并后正好覆盖 $32 \sim 63$ 共 32 个连续值，$32 = 2^5$，对应第三字节高 3 位为前缀的一部分，故前缀长度 $16 + 3 = 19$。验证对齐：/19 的第三字节块大小 $= 256 - 224 = 32$，起点须为 32 的整数倍，32 正好满足。聚合结果：**35.230.32.0/19**。（2018 年第 38 题，选 C。）

【算例 **2**：两个 **/25** 聚合】 R2 到局域网 1（202.118.1.0/25）和局域网 2（202.118.1.128/25）的路由可聚合为一条。两块的块大小都是 128，起点为 0 和 128，恰好组成 $0 \sim 255$ 的完整 /24 地址块，聚合为 **202.118.1.0/24**，掩码 **255.255.255.0**。（2009 年第 47 题第 (3) 问。）

【算例 **3**：跨网段聚合】 R1 只有到 192.168.1.0/24 的路由。为把分组正确路由到 R2 后面的 192.168.2.0/25 与 192.168.2.128/25，需增加哪条路由？解：两块恰好组成 192.168.2.0/24，掩码 255.255.255.0，下一跳是同一链路上 R2 的接口地址（设为 192.168.1.2），而不是 **R1** 自己的接口**.1**。（2011 年第 37 题，选 D。）

路由聚合的两个陷阱

1. 下一跳写错：聚合路由的下一跳是邻居路由器的接口地址，不是本路由器的接口地址。
2. 过度聚合：聚合后的地址范围可能大于原有各子网范围的并集，从而" 引入不存在的网络"。若题目允许引入无害的额外范围可以聚合；若明确要求不引入不存在的网络，则必须逐条列出或换用其他聚合方式。

（四）ARP 协议、DHCP 协议与 ICMP 协议

*1.*地址解析协议 *ARP*

为什么需要两种地址：在 IP 层抽象的互联网上只能看到 IP 数据报；虽然首部中有源 IP 地址，但路由器只根据目的 **IP** 地址转发；在局域网的链路层只能看见 MAC 帧。通过路由器转发时，IP 数据报在每个网络中都被路由器解封装和重新封装，MAC 帧首部的源、目的地址不断改变——硬件地址只具有本地意义，因此无法用 MAC 地址跨网络通信。

因此：数据报传送过程中，源 **IP** 地址和目的 **IP** 地址保持不变；源 **MAC** 地址和目的 **MAC** 地址逐链路（逐网络）改变。

ARP 的作用：无论网络层使用什么协议，在链路上传送数据帧时最终必须使用硬件地址，因此需要一种方法完成 IP 地址到 MAC 地址的映射，这就是地址解析协议（Address Resolution Protocol，**ARP**）。ARP 用于解决同一局域网上的主机或路由器的 IP 地址和硬件地址的映射问题。若目的主机与源主机不在同一局域网，则通过 ARP 找到本局域网上的某个路由器（默认网关）的硬件地址，把分组发给该路由器，剩下工作由下一个网络完成。

ARP 高速缓存：每台主机都设有一个 ARP 高速缓存（**ARP** 表），存放本局域网上各主机和路由器的 IP 地址到 MAC 地址的映射。

ARP 请求广播、ARP 响应单播

尽管 **ARP** 请求分组是广播发送的，但 **ARP** 响应分组是普通的单播。 ARP 广播只在子网中传播。ARP 高速缓存表项通过 ARP 自动获取，生命周期默认约两分钟，也可手工配置；ARP 表中 IP 与 MAC 的对应关系并非永久不变（例如主机更换了网卡）。

ARP 工作流程：

1. 主机 A 要向本局域网上的主机 B 发送 IP 数据报时，先在自己的 **ARP** 高速缓存中查看有无主机 **B** 的 **IP** 地址。
2. 若有，查出对应硬件地址写入 MAC 帧，通过局域网把帧发往该硬件地址。
3. 若没有，用目的 MAC 地址为 **FF-FF-FF-FF-FF-FF**（48 比特全 1）的帧封装并广播 **ARP** 请求分组，使同一局域网里所有主机都收到。
4. 目标主机 B 收到后先把请求中的源 **IP** 与源 **MAC** 映射存入自己的 **ARP** 表，确认请求的目标 IP 是自己后，向 A 发出 ARP 响应分组（单播，只有 A 能收到），其中含 B 的 IP 与 MAC 映射。
5. A 收到响应后把该映射写入 **ARP** 缓存，然后按查到的硬件地址发送 MAC 帧。

跨网段的 ARP 过程（关键考点）

ARP 不能跨网络使用，但不同局域网可以多次 ARP。若源主机与目的主机不在同一局域网，则源主机通过 ARP 解析的是本局域网内默认网关的硬件地址，把分组发往默认网关，由网关转发至下一个网络。即此时 **ARP** 请求的目标 **IP** 是网关的 **IP**，而不是最终目的主机的 **IP**。

发送前的判定：主机用本机子网掩码与目的 IP 地址逐位按位与，对比结果与本机网络地址。一致则目的主机与本机在同一局域网（直接交付），ARP 查目的主机的 MAC；不一致属于间接交付，ARP 查默认网关的 MAC。

ARP 的四种典型情况：

情况	发送方	目的	**ARP** 解析的对象
1	主机 H1	本网络上的主机 H2	目的主机 H2 的硬件地址
2	主机 H1	其他网络上的主机 H3	与网 1 连接的路由器 R1 的硬件地址（默认网关）
3	路由器 R1	与 R1 连接的网 **2** 上的主机 H3	目的主机 H3 的硬件地址
4	路由器 R1	网 3 上的主机 H4	与网 2 连接的路由器 R2 的硬件地址

关于 **ARP** 属于哪一层：由于 ARP 的主要用途是从网际层使用的 IP 地址解析出数据链路层使用的 MAC 地址，因此有的教材把 ARP 划归网际层，有的划归数据链路层，这两种做法都是可以的。

真题考法

2012 年第 38 题直接考”ARP 协议的功能”（答案：根据 IP 地址查询 MAC 地址）。2015 年第 47 题第 (2) 问考” 若主机 2 的 ARP 表为空，访问 Internet 时发出的第一个以太网帧的目的 MAC 地址是什么”——因为要先解析默认网关，所以第一个帧是 **ARP** 请求帧，目的 **MAC** 是 **FF-FF-FF-FF-FF-FF**。

*2.*动态主机配置协议 *DHCP*

作用：动态主机配置协议（Dynamic Host Configuration Protocol，**DHCP**）用于给主机动态分配 **IP** 地址及其他配置信息，基于 **UDP**（服务器端口 67、客户端端口 68）。

报文	发送方	含义	源 **IP**	目的 **IP**	目的 **MAC**
DISCOVER 发现	客户端	”谁是 DHCP 服务器？”	0.0.0.0	255.255.255.255	广播 MAC（全 F）
OFFER 提供	服务器	”我这有可用地址”	服务器 IP	255.255.255.255	客户端 **MAC**（二层单播）
REQUEST 请求	客户端	”我就要这个地址”	0.0.0.0	255.255.255.255	广播 MAC（全 F）
ACK 确认	服务器	”地址正式生效”	服务器 IP	255.255.255.255	客户端 **MAC**（二层单播）

DHCP 报文的 IP 层与 MAC 层差异（高频考点）

四个报文的 **IP** 层目的地址全是受限广播 **255.255.255.255**，因为此时客户端还没配置 IP 地址，其 IP 层只认广播地址。而服务器发出的 **OFFER** 和 **ACK** 在链路层使用单播（目的 MAC 是客户端 MAC），这样网卡一看目的 MAC 是自己的就收下帧。**OFFER** 报文中会提供 **IP** 地址、默认网关 **IP**、子网掩码、**DNS** 服务器 **IP**。客户端用 **0.0.0.0** 作源地址，含义是” 我不知道我是谁，但我现在就在这个网络里”。

租期：DHCP 分配的 IP 地址有租用期，客户端在租期过半时应请求续租；不再使用时可以主动释放，使地址能被回收重用。

DHCP 中继代理：DHCP 使用广播而路由器默认不转发广播，因此 DHCP 服务器与客户端不在同一网段时广播报文无法到达服务器。解决办法是在各网段设置 **DHCP** 中继代理（relay agent），把客户端广播的 DHCP 报文以单播方式转发给 DHCP 服务器，并把应答转回客户端。

真题考法

2015 年第 47 题第 (1) 问：DHCP 服务器可为主机 2 ~ 主机 N 动态分配 IP 地址的最大范围是什么？做法是把 /24 子网的可用地址.1 ~ .254 中已被静态占用的地址（路由器.1、DHCP 服务器.2、WWW 服务器.3、主机 1 .4）排除，答案为 **111.123.15.5～111.123.15.254**；DISCOVER 报文的源、目的 IP 是 **0.0.0.0** 和 **255.255.255.255**。

*3.*网际控制报文协议 *ICMP*

作用：为了更有效地转发 IP 数据报以及提高交付成功的机会，网际层使用了网际控制报文协议（Internet Control Message Protocol，**ICMP**）。主机或路由器用它发送差错报告报文和询问报文。**ICMP** 报文被封装在 **IP** 数据报中发送，IP 首部协议字段值为 **1**。因此直接为 ICMP 提供服务的是 **IP**，而不是 TCP 或 UDP。

差错报告类型	触发条件
终点不可达	路由器或主机不能交付 IP 数据报时向源点发送。可按代码字段细分为目的网络不可达、目的主机不可达、目的协议不可达、目的端口不可达等 13 种
源点抑制	路由器或主机因拥塞而丢弃数据报时向源点发送，使源点放慢发送速率
时间超过	两类：① 路由器把 TTL 减 1 后结果为 0，丢弃数据报并向源点发送时间超过报文；② 终点在规定时间内未收到一个数据报的全部分片时丢弃已收分片，也向源点发送时间超过报文
参数问题	路由器或目的主机发现数据报首部中有字段值不正确时，丢弃该数据报并向源点发送参数问题报文

差错报告类型	触发条件
改变路由（重定向）	路由器把改变路由报文发给主机，让主机知道下次应把数据报发给另外的（更好的）路由器

ICMP 询问报文常用两种：回送请求和回答——由主机或路由器向特定目的主机发出，收到该报文的主机必须回送 ICMP 回送回答报文，用来测试目的站是否可达及了解其状态；时间戳请求和回答——请某主机或路由器回答当前日期和时间，时间戳回答报文中有一个 32 位字段，写入的整数表示从 1900 年 1 月 1 日起到当前时刻的秒数，用来进行时钟同步和测量时间。

不应发送 ICMP 差错报告报文的几种情况

以下情况不发送：对 ICMP 差错报告报文本身不再发送差错报告报文；对第一个分片的数据报片的所有后续数据报片都不发送；对具有多播地址的数据报都不发送；对具有特殊地址（如 127.0.0.0 或 0.0.0.0）的数据报不发送。

PING（分组网间探测）用来测试主机或路由器之间的连通性。它是应用层直接使用网际层 **ICMP** 的一个例子，并不使用运输层的 **TCP** 或 **UDP**；所使用的 ICMP 报文类型是回送请求和回答。

traceroute（跟踪路由）用于探测 **IP** 数据报从源主机到目的主机要经过哪些路由器。原理是源主机依次发送一串 TTL 分别为 1、2、3……的 IP 数据报，第 i 个数据报到达第 i 跳路由器时 TTL 减为 0，该路由器丢弃它并回送 **ICMP** 时间超过报文，源主机由此得知第 i 跳路由器的地址；最后一个数据报到达目的主机时返回回送回答（或端口不可达），探测结束。UNIX 版本命令为 **traceroute**，运输层使用 **UDP**，网络层使用的 ICMP 报文类型只有差错报告报文；Windows 版本命令为 **tracert**，应用层直接使用网际层的 **ICMP** 协议，使用的 ICMP 报文类型有回送请求和回答报文以及差错报告报文。

为什么 ping 不通、traceroute 显示 *

ping 不通的常见原因（按排查顺序）：① 目标主机不存在或未开机，ARP 解析不到，本地即报” 请求超时”；② 中间路由器没有到达目标的路由，回送 **ICMP** 终点不可达（网络不可达）；③ 防火墙或安全策略主动丢弃 ICMP 回送请求/回答，表现为” 请求超时” 而非” 不可达”；④ 子网掩码或默认网关配置错误——主机用错误的掩码判断” 目的在同一子网”，于是不去找网关而直接在本网段发 ARP 请求，永远得不到响应（2016 年第 39 题正是此情况）。

traceroute 显示 * 的原因：某一跳路由器不回复 **ICMP** 时间超过报文（可能被防火墙过滤、被管理员禁用 ICMP、或 CPU 过载丢弃了 ICMP 报文）。此时该跳信息无法获得，但 traceroute 仍继续探测后面的跳数。因此 * 通常意味着” 该跳不可探测”，不等于” 该跳不存在” 或” 链路中断”。

四、IPv6

（一）IPv6 的主要特点

引入 IPv6 的目的是从根本上解决 **IP** 地址耗尽问题。解决地址耗尽有三条途径：采用 CIDR 使地址分配更合理；采用 NAT 节省全球 IP 地址；采用具有更大地址空间的新版本 IPv6。前两种只是延长 **IPv4** 使用寿命，只有第三种能从根本上解决问题。

IPv6 引进的主要变化：

- 更大的地址空间（最重要）：由 IPv4 的 **32** 比特增大到 **128** 比特。
- 扩展的地址层次结构：可划分更多层次，更好地反映拓扑结构，使寻址与路由层次设计更灵活。
- 灵活的首部格式：与 IPv4 首部并不兼容。IPv6 定义了许多可选的扩展首部，既提供更多功能，

又提高路由器处理效率，因为路由器对逐跳扩展首部外的其他扩展首部都不处理。

- 改进的选项：允许分组包含选项控制信息，因而可以包含新选项；IPv4 的选项则固定不变。
- 允许协议继续扩充：IPv6 允许不断扩充功能，而 IPv4 的功能固定不变。
- 支持即插即用（自动配置）：支持主机或路由器自动配置 IPv6 地址及其他网络配置参数，因此 **IPv6** 不需要使用 **DHCP**。
- 支持资源的预分配：能为实时音视频等要求保证带宽和时延的应用提供更好的服务质量保证。
- 只有源主机才能分片：IPv6 的分片是端到端的，不允许 IPv4 那样的路由器分片，可大大提高路由器转发速度。
- 首部固定：基本首部长度固定为 **40 B**，而 IPv4 首部长度可变（须为 4 B 整数倍）。
- 取消了首部检验和：加快路由器处理 IPv6 数据报的速度。
- 增大了安全性：如身份鉴别、保密功能；支持 **QoS** 保障。注意 **IPv6** 路由器收到过大的数据报时直接丢弃（并回送 ICMPv6 报文），不像 IPv4 那样分片。

IPv6	**IPv4**
地址长度 128 bit（16 B）	地址长度 32 bit（4 B）
只有在源结点才能分片，传输过程中不能分片	传输过程中可以分片
基本首部固定 40 B	首部长度可变，是 4 B 的整数倍
首部更简单灵活，取消许多字段（放在扩展首部中）	首部固定部分包含更多字段
无首部检验和	有首部检验和

IPv6 数据报的组成：由基本首部和有效载荷（净负荷）两部分组成。基本首部的字段只有 **8** 个，但长度增大到 **40 B**；有效载荷由零个或多个扩展首部及其后面的数据部分构成（扩展首部不属于 **IPv6** 数据报的基本首部）。

字段	位数	含义
版本	4	协议版本，IPv6 对应值是 6
通信量类	8	区分不同 IPv6 数据报的类别或优先级
流标号	20	属于同一个流的报文具有相同流标号，用于资源分配
有效载荷长度	16	除基本首部外的字节数，最大 65535 B（所有扩展首部都算在有效载荷之内）
下一个首部	8	相当于 IPv4 的协议字段。无扩展首部时指明数据是何种 PDU；有扩展首部时标识后面第一个扩展首部的类型
跳数限制	8	类似 IPv4 的 TTL，源点设定限制值（最大 255），路由器每次转发减 1，减为零就丢弃。用来防止 **IPv6** 数据报在因特网中永久兜圈
源地址	128	数据报发送端地址
目的地址	128	数据报接收端地址

IPv6 相对 **IPv4** 取消或改名的字段：取消首部长度（因长度固定 40 B）；取消区分服务（由通信量类和流标号实现其功能）；取消总长度（改用有效载荷长度）；取消标识、标志和片偏移（这些功能已包含在分片扩展首部中）；把生存时间 **TTL** 改称跳数限制；取消协议（改用下一个首部）；取消首部检验和；取消选项（改用扩展首部）。

IPv6 扩展首部的设计意图

IPv4 数据报若在首部使用了选项字段，则整个传送路径中的全部路由器都要检查选项，降低了处理速度，而实际上很多选项路由器并不需要检查。因此 IPv6 把 IPv4 首部中的选项字段都放到扩展首部中，由路径两端的源点和终点主机处理，而路径中的所有路由器都不处理这些扩展首部（除逐跳选项扩展首部外）。

（二）IPv6 地址

三种基本地址类型：单播——传统的点对点通信，一对一，可作源地址或目的地址；多播——一点对多点，数据报发送到一组计算机中的每一台，只能作目的地址（IPv6 没有采用广播的术语，而把广播看作多播的一个特例）；任播——终点是一组计算机，但只交付其中的一台（通常是最近的一台），只能作目的地址。

IPv6 地址的表示形式：

- 冒号十六进制记法：把地址中的每 4 位用一个十六进制数表示，用冒号分隔每 16 位，128 位地址写成 8 组每组 4 位十六进制数。
- ”左侧零”省略：两个冒号之间的十六进制数中最前面的一串 0 可以省略。
- ”连续零”压缩：一连串连续的 0 可用一对冒号（::）取代，例如 `2001:0DB8:0000:0000:0000:0000:0000:0001` 可写成 `2001:DB8::1`。
- 在一个 **IPv6** 地址中只能使用一次”连续零”压缩，否则会导致歧义。
- 冒号十六进制记法还可结合点分十进制的后缀，例如`::FFFF:192.168.1.1`。
- **CIDR** 的斜线表示法在 **IPv6** 中仍然可用，例如 `2001:DB8::/32`。

IPv6 地址类型与前缀

地址类型	二进制前缀	说明
未指明地址	128 位全 0，记为`::/128`	不能用作目的地址，只能用于还未配置 IPv6 地址的主机作为源地址
环回地址	128 位中仅最低位为 1，记为`::1/128`	**IPv6** 的环回地址仅此一个（IPv4 的环回是一整个 /8 网段）
多播地址	最高 8 比特为全 1，记为 `FF00::/8`	功能与 IPv4 多播地址相同，占 IPv6 地址空间的 1/256
本地链路单播地址	最高 10 比特为 1111111010，记为 `FE80::/10`	作用类似 IPv4 私有地址。即使网络未连接因特网也可用 TCP/IP，但不能与因特网上的其他主机通信
全球单播地址	除上述四种外的其他所有 IPv6 地址	用得最多的地址

IPv6 全球单播地址的三级结构（这样设计是为了让路由器更快查找路由）：全球路由选择前缀占 **48** 位，相当于 IPv4 分类地址中的网络号；子网标识符占 **16** 位，用于各机构构建自己的子网；接口标识符相当于 IPv4 的主机号，位数有 **64** 位之多，完全足够给接口的硬件地址编码，因此 **IPv6** 不需要使用地址解析协议 **ARP** 进行地址解析。

（三）IPv4 向 IPv6 的过渡

背景：因特网上使用 IPv4 的路由器数量太大，让所有路由器都改用 IPv6 无法一蹴而就，因此只能逐步演进；另外，新部署的 IPv6 系统必须能够向后兼容，即必须能够接收和转发 IPv4 数据报，并

能为 IPv4 数据报选择路由。

双协议栈（**Dual Stack**）：在完全过渡到 IPv6 之前，使一部分主机或路由器装有 **IPv4** 和 **IPv6** 两套协议栈，既可和 IPv6 系统通信，又可和 IPv4 系统通信，记为 **IPv6/IPv4**，表明它具有一个 IPv6 地址和一个 IPv4 地址。与 IPv6 主机通信时采用 IPv6 地址，与 IPv4 主机通信时采用 IPv4 地址；通过域名系统 DNS 查询目的主机采用的 IP 地址：若 DNS 返回 IPv4 地址，则源主机使用 IPv4 地址；若返回 IPv6 地址，则使用 IPv6 地址。

隧道技术（**Tunneling**）：当 IPv6 数据报要进入 IPv4 网络时，把 **IPv6** 数据报重新封装成 **IPv4** 数据报，即整个 IPv6 数据报成为 IPv4 数据报的数据载荷；封装有 IPv6 数据报的 IPv4 数据报在 IPv4 网络中传输；当 IPv4 数据报要离开 IPv4 网络时，再把其数据载荷（原来的 IPv6 数据报）取出并转发到 IPv6 网络。

IPv6 易错点

1. **IPv6** 地址空间不是 **IPv4** 的 **96** 倍，而是 $2^{128}/2^{32}=2^{96}$ 倍（2023 年第 40 题）。
2. **IPv6** 基本首部长度固定 **40 B**，不是可变的。
3. IPv6 的 **Hop Limit**（跳数限制）等价于 **IPv4** 的 **TTL**，都会被每个路由器减 1。
4. IPv6 的有效载荷长度字段不包含基本首部，但包含所有扩展首部。
5. IPv6 缺少”协议”字段（改用”下一个首部”）并不影响指明上层协议。

五、路由协议

（一）自治系统与域内、域间路由

自治系统（Autonomous System，**AS**）指在单一的技术管理下的一组路由器。每个 AS 内可能包括多个网络，**AS** 号（**ASN**）全球唯一。AS 在逻辑上互不相交，但在地理位置上可以相互交错；AS 之间必须通过边界路由器相互连接。AS 内使用 RIP 还是 OSPF 由该 AS 自己决定。

- 内部网关协议（Interior Gateway Protocol，**IGP**）：一个自治系统内部使用的协议，如 **RIP** 和 **OSPF**。
- 外部网关协议（Exterior Gateway Protocol，**EGP**）：自治系统之间使用的协议，目前使用最多的是 **BGP**。

若源主机和目的主机处于不同 AS（两 AS 可能使用不同 IGP），则数据报传到 AS 边界时需要用一种协议把路由选择信息传递到另一个 AS。每个 AS 自己决定内部运行哪个 IGP，但都有一个或多个路由器除运行本 AS 的 IGP 外还要运行外部网关协议。

IGP 与 EGP 是分类名称，不是具体协议

外部网关协议 **EGP** 和内部网关协议 **IGP** 只是路由选择协议的分类名称，而不是具体的路由选择协议。名称中使用”网关”是因为因特网早期 RFC 文档中没有使用”路由器”而使用”网关”。统考中容易把 EGP 与 BGP 混淆。

路由选择协议的三大功能：获取网络拓扑信息；选择最优路径（用 Dijkstra 单源最短路径算法等）；维护路由表。

（二）RIP 路由协议

路由信息协议（Routing Information Protocol，**RIP**）是 IGP 中最先得到广泛使用的协议之一。RIP 要求 AS 内的每一个路由器都维护从它自己到 AS 内其他每一个网络的距离向量（Distance-Vector，

D-V）。

度量：RIP 用跳数（Hop Count）作为度量（Metric）。路由器到直连网络的距离定义为 **1**；到非直连网络的距离为所经过的路由器数加 **1**。一条路径最多只能包含 **15** 个路由器，距离等于 **16** 相当于不可达，因此 RIP 只适用于小型互联网。RIP 认为" 好的路由" 就是所通过路由器数量最少的路由；到达同一目的网络有多条距离相等的路由时可进行等价负载均衡。（有些厂商不严格按标准实现，例如思科把直连网络距离定义为 0；统考按标准文档" 直连距离为 1" 作答。）

RIP 的三个重要特点：和谁交换信息——仅和相邻路由器交换；交换什么信息——路由器自己的路由表，即到所在 AS 中各网络的最短 RIP 距离以及应经过的下一跳路由器；何时交换信息——周期性交换（约每 30 秒一次），为加快收敛，网络拓扑变化时要及时向相邻路由器通告变化后的路由信息，这称为触发更新。

基本工作过程：① 路由器刚开始工作时只知道自己到直连网络的 **RIP** 距离为 **1**；② 每个路由器仅和相邻路由器周期性地交换并更新路由信息；③ 若干次交换和更新后，每个路由器都知道到达本 AS 内各网络的最短距离和下一跳路由器，称为收敛。收敛指路由环境变化后各路由器调整路由表以适应拓扑变化、最终达到稳定状态（路由表与网络拓扑保持一致）。

时间参数：每隔约 **30** 秒向所有相邻路由器发送路由更新报文；若 **180** 秒（默认）没收到某条路由条目的更新报文，则把该条目标记为无效（RIP 距离置为 16）；若再过一段时间（如 **120** 秒）仍未收到，则将该条目从路由表中删除。

RIP 报文的封装：RIP 报文用运输层的 **UDP** 封装，**UDP** 端口号为 **520**。从封装角度看 RIP 属于应用层；但 RIP 的核心功能是路由选择，属于网际层。

1.RIP 的距离向量更新规则与计算题

RIP 路由更新的四条规则（必背）

路由器收到相邻路由器（设为 X）发来的路由更新报文后，对报文中每条表项：

第一步（修改）：把报文中所有表项的" 下一跳" 都改为 **X**，所有" 距离" 字段的值都加 **1**。

第二步（比对）：

1. 本地路由表中没有该目的网络 → 添加。
2. 本地已有且下一跳就是 **X** → 替换（无条件更新，因为信息来自同一邻居）。
3. 本地已有、下一跳不是 **X** 且新距离更小 → 更新。
4. 本地已有、下一跳不是 **X** 且新距离相等 → 可添加（实现等价负载均衡）。
5. 本地已有、下一跳不是 **X** 且新距离更大 → 不更新。

所有距离都要按" 最大 **15**、**16** 表示不可达" 处理：算出的距离若 ≥ 16 一律记作 16，不会出现 **17**。

【算例：**RIP** 路由表更新完整计算】路由器 B 的原路由表如下，现收到相邻路由器 C 发来的路由更新报文，求 B 更新后的路由表。

B 的原路由表			**C** 发来的报文	
目的网络	距离	下一跳	目的网络	距离
Net1	3	A	Net1	4
Net2	4	C	Net2	2
Net3	1	直接交付	Net5	1
Net4	5	D		

解：第一步，把 C 报文中所有表项的下一跳改为 C、距离全部加 1，得 Net1 距离 5（下一跳 C）、Net2 距离 3（下一跳 C）、Net5 距离 2（下一跳 C）。

第二步逐条比对：

- **Net1**：原表有，下一跳是 A（≠ C），新距离 5 > 原距离 3 → 不更新；
- **Net2**：原表有，下一跳是 **C**（= **C**） → 无条件替换；
- **Net5**：原表没有 → 添加。

更新后的表项	距离	下一跳	变化说明
Net1	3	A	新路由劣势，不更新
Net2	3	C	相同下一跳，无条件替换
Net3	1	直接交付	未在报文中出现，保持不变
Net4	5	D	未在报文中出现，保持不变
Net5	2	C	新网络，添加

【算例：等价负载均衡】R1 当前有一条到 N 的路由（下一跳 R3，距离 5），现收到 R2 的更新报文包含 <N，7>，且该 RIP 实现支持等价路由负载均衡。解：先把报文修改为下一跳 R2、距离 $7+1=8$。原表有 N，下一跳 R3 ≠ R2，新距离 8 > 原距离 5 → 不更新。（若报文是 <N，4>，则新距离 5 与原距离相等且下一跳不同，此时可以添加形成等价路由。）

【算例：**RIP** 的 **16** 上限】2010 年第 35 题：R1 收到邻居 R2 的距离向量，其中包含 <net1，16>，能得出什么结论？解：RIP 把 16 定义为无穷大即目的网络不可达。R2 通告 <net1，16> 表示 R2 自己无法到达 net1；R1 即使再加上到邻居的 1 跳，结果仍按上限记为 16，因此 **R1** 不能经 **R2** 到达 **net1**。

2016 年第 **37** 题：若 R3 检测到网络 201.1.2.0/25 不可达并向 R2 通告一次新的距离向量，则 R2 更新后到达该网络的距离是多少？解：R3 把距离通告为 **16**。R2 经 R3 的候选距离理论上为 $16+1=17$，但 RIP 最大有效距离是 15，大于等于 16 的值都记作 16（不可达），不会出现 17，因此更新后的距离为 **16**。

RIP 的优缺点：优点是实现简单、路由器开销小；若某个路由器发现了更短的路由，在较短时间内便可传至所有路由器，俗称**”** 好消息传播得快**”**。缺点：① 限制最大距离为 **15**，限制了 AS 规模；② 相邻路由器之间交换的是完整路由表，规模扩大后开销随之增大；③ **”** 坏消息传播得慢**”**，收敛时间过长。因此规模较大的 AS 应使用 OSPF。

真题考法

2024 年第 47 题第 (3) 问用 RIP 的交换周期算收敛时间：路由器同一时刻启动并立即交换初始距离向量，之后每隔 30 s 交换一次，一轮只能把新信息再传播一跳。对 210.2.3.0/24：R14 直连，R11、R15 在 $t=0$ 获得，R12、R13、R16 最迟在 $t=30$ s 获得，故全部获得至少需 **30 s**。对 210.2.4.0/24：R16 直连，R15 在 $t=0$ 获得，R13、R14 在 $t=30$ s 获得，R11、R12 最迟在 $t=60$ s 获得，故至少需 **60 s**。第 (1) 问考”AS4 内部路径可能超过 20 台路由器，RIP 不适用，应选 OSPF”。

（三）**OSPF** 路由协议

开放最短路径优先（Open Shortest Path First，**OSPF**）是为了克服 RIP 的缺点而开发的。” 开放” 表明它不是受某一厂商控制而是公开发表的；” 最短路径优先” 是因为使用了 **Dijkstra** 提出的最短路径算法（SPF）。OSPF 基于链路状态而不像 RIP 基于距离向量；由于采用最短路径算法计算路由，从算法上保证了不会产生路由环路；不限制网络规模，更新效率高，收敛速度快。

链路状态指本路由器都和哪些路由器相邻，以及相应链路的” 代价（cost）”；代价用来表示费用、距离、时延和带宽等，由网络管理人员决定。例如思科路由器中 OSPF 的代价算法是：100 Mb/s 除以链路带宽，结果小于 1 仍记为 1，大于 1 且带小数的舍去小数。

邻居关系的建立和维护：相邻路由器之间通过交互问候（**Hello**）分组建立和维护邻居关系。问候分组封装在 **IP** 数据报中、发往组播地址 **224.0.0.5**，IP 首部协议号字段取值为 **89**。问候分组的发送周期为 **10** 秒；若 **40** 秒未收到某邻居的问候分组则认为该邻居不可达。每个路由器都会建立一张邻居表。

链路状态通告 **LSA**：每个路由器都会产生链路状态通告（Link State Advertisement，**LSA**），其中包含直连网络和邻居路由器两类链路状态信息。LSA 被封装在链路状态更新（Link State Update，**LSU**）分组中，采用可靠的洪泛法（Flooding）发送。

- 洪泛法的要点是路由器向自己所有的邻居路由器发送 **LSU**，收到该分组的各路由器又把它转发给自己所有的邻居路由器（但其上游路由器除外），以此类推。洪泛不回传、不重复。
- **”** 可靠**”** 指收到 LSU 后要发送确认；收到重复的更新分组无需再次转发，但要发送一次确认。

链路状态数据库 **LSDB**：每个路由器都有一个链路状态数据库（Link State Database，**LSDB**）用于存储 LSA。通过各路由器洪泛发送封装有各自 LSA 的 LSU，各路由器的 **LSDB** 最终将达到一致。

最短路径优先计算：各路由器基于 LSDB 进行计算，构建出各自到达其他各路由器的最短路径即路由表，并周期性（默认 30 分钟）刷新链路状态。注意路由表中不会存储完整路径，而只存储**”** 下一跳**”**。

OSPF 的详细工作过程：① 邻居发现与链路参数初始化——通过 **Hello** 分组发现直连邻居、建立邻居关系，然后为去往各邻居的链路配置对应的开销度量 **metric**；② **LSDB** 同步——路由器向邻居发送 **DD** 数据库描述分组传递本地 LSDB 中所有 LSA 的摘要信息供双方比对，邻居逐条比对摘要后针对缺失、过期或更新的条目主动发送 **LSR** 链路状态请求分组，对方回复 **LSU** 链路状态更新分组携带所需 LSA 详细信息，邻居更新本地数据库后返回 **LSAck** 链路状态确认分组；③ 动态更新与路由计算——链路状态变化时变动路由器泛洪发送 **LSU** 在区域内扩散最新拓扑，其他路由器接收后回复 **LSAck**，全网依据最新 LSDB 用 **Dijkstra** 算法重新计算最短路径并更新路由表；④ 转发。

OSPF 的五种分组类型：

分组类型	作用
问候（**Hello**）	发现和维护邻居路由器的可达性
数据库描述（**DD**）	向邻居给出自己 LSDB 中所有链路状态项目的摘要信息
链路状态请求（**LSR**）	向邻居请求某些链路状态项目的详细信息
链路状态更新（**LSU**）	把链路状态信息用洪泛法对整个系统更新
链路状态确认（**LSAck**）	对链路状态更新分组的确认

多点接入网络中的 **OSPF** 路由器：若每台路由器都向其他 $(n-1)$ 个路由器发送问候分组和 LSU，洪泛数量会急剧增加。为减少洪泛数量，OSPF 采用：选举指定路由器（Designated Router，**DR**）和备用指定路由器（Backup Designated Router，**BDR**）；所有非 **DR/BDR** 只与 **DR/BDR** 建立邻居关系；非 **DR/BDR** 之间通过 **DR/BDR** 交换信息。

OSPF 划分区域：为使 OSPF 能用于规模很大的网络，OSPF 把一个 AS 再划分为若干个更小的范围，称为区域（area）。划分区域的好处是把利用洪泛法交换链路状态信息的范围局限于每一个区域而不是整个 AS，从而减少整个网络上的通信量。洪泛不跨区域。

路由器类型	含义
区域内路由器（IR）	所有接口都在同一个区域内
主干路由器（BBR）	只要在主干区域中的路由器都称为主干路由器，因此主干路由器可兼作区域边界路由器
区域边界路由器（ABR）	连接主干区域和其他下层区域的路由器，即至少一个接口在主干区域、另一个接口在其他区域
自治系统边界路由器（ASBR）	与 AS 外部的路由器交换路由信息的路由器

OSPF 特点小结（与 RIP 逐条对比）：和谁交换信息——用洪泛法向本 AS 中所有路由器发送信息（RIP 仅向相邻的几个路由器发送）；交换什么信息——与本路由器相邻的所有路由器的链路状态，只是路由器知道的部分信息（RIP 发送的是本路由器所知道的全部信息即整个路由表）；何时交换信息——只有当链路状态发生变化时才用洪泛法发送（RIP 不管拓扑是否变化都定期交换）；用什么传送信息——OSPF 是网络层协议，不用 UDP 或 TCP 而直接用 **IP** 数据报传送（协议字段 **89**），而 RIP 是应用层协议、在运输层使用 UDP。

OSPF 的其他补充特性：允许对每条路由设置不同代价，对不同类型的业务可计算出不同路由；若到同一目的网络有多条相同代价的路径，可把通信量分配给这几条路径；支持可变长度子网划分和 CIDR；每个链路状态都带一个 32 位序号，序号越大状态越新；互联网规模很大时使用 OSPF 比 RIP 好得多，收敛很快且不存在坏消息传得慢的问题。

（四）**BGP** 路由协议

协议定位：边界网关协议（Border Gateway Protocol，**BGP**）属于外部网关协议 **EGP**，专门用于自治系统之间的路由选择，是因特网跨域路由的核心协议。

为什么 **AS** 间不用 **IGP**：① 无统一度量标准——不同 AS 内部的度量维度不同（有的按时延、有的按带宽、有的按距离），无法用统一的” 代价” 计算全局最优路径；② 需支持策略选路——AS 之间的路由选择必须考虑政治、经济、安全等策略（如过境收费、规避不可信 AS），这些由网络管理人员设置，但策略并不是 AS 之间的路由选择协议本身。

BGP 的设计目标

BGP 只能是力求寻找一条能够到达目的网络且比较好的路由（即不能兜圈子），而并非要寻找一条最佳路由。 BGP 交换的路由信息是到达某个目的网络所要经过的各个自治系统序列而不仅仅是下一跳。

路由机制：路径向量算法。BGP 传递的路由信息 = 目的网络 + **AS** 路径序列（路径向量），并非距离或链路状态。通过携带完整经过的 AS 编号天然防环：若收到的路由路径包含自身 **AS** 号，直接丢弃。

通信主体：**BGP** 发言人——运行 BGP 协议、参与 BGP 会话的路由器。**eBGP** 发言人通常部署在 AS 边界，与其他 AS 的 BGP 发言人建立会话，交换跨 AS 的路由可达信息；**iBGP** 发言人位于同一 AS 内部，负责把外部学到的 BGP 路由在本 AS 内传播，不一定在 **AS** 边界。

核心辨析：**BGP** 发言人不等同于边界路由器，判定标准是路由器是否运行 **BGP** 并参与 **BGP** 会话，与所处物理位置无关。互相建立 BGP 会话的两台发言人互称邻站（对等站）。BGP 发言人一般同时运行本 AS 的 IGP，保障 AS 内部网络可达。

传输方式：BGP 属于应用层协议，基于 **TCP 179** 端口建立持久会话，依靠 TCP 保证可靠传输，无需协议自身维护确认与重传。

三个路由协议的封装方式（必背对比）

RIP 用 **UDP**（端口 **520**）；**OSPF** 直接封装在 **IP** 数据报中（协议号 **89**）；**BGP** 用 **TCP**（端口 **179**）。2017 年第 37 题直接考这一点。

会话类型：BGP 会话建立的前提是 BGP 邻居之间先建立 **TCP** 连接，再在该连接上交换 BGP 报文。

会话类型	建立对象	是否要求直连	核心作用
eBGP（外部 **BGP**）	两个 **AS** 的直连边界路由器之间	要求物理直连	在自治系统之间交换跨域路由信息
iBGP（内部 **BGP**）	同一 **AS** 内部的 **BGP** 路由器之间	不要求物理直连	在本 AS 内部同步从外部学到的 BGP 路由信息

iBGP ≠ IGP

IGP 解决”AS 内部数据包具体怎么走” 的问题，计算内部最优路径；**iBGP** 解决”AS 内部同步外部路由信息” 的问题，传递跨域可达性。两者功能不同，不能互相替代。

传统 iBGP 要求同一 AS 内的路由器实现全连接（**Full Mesh**）：N 台 iBGP 路由器需要建立 $\frac{N(N-1)}{2}$ 条会话。该全连接仅指 **BGP** 层面的逻辑会话连通，并不要求路由器之间物理直连。

BGP 路由的格式：目的网络 + `BGP` 属性，核心两个属性：**AS-PATH** 记录该路由途经的自治系统序列，用于选路，同时防止 **AS** 间路由环路（若路径序列中出现重复 AS 则存在环路）；**NEXT-HOP** 表示通告该 BGP 路由的路由器对应的 IP 地址。

BGP 路由传播的核心规则：① **eBGP** 跨 **AS** 传播——向外部 AS 通告时在 AS-PATH 中追加本 **AS** 编号，同时把 NEXT-HOP 更新为发送方边界路由器自身 **IP**（例：AS2 的 R4 向 AS3 的 R6 通告 N1 的路由，AS-PATH 追加 AS2，路由变为 `(N1, [AS2 AS1], R4)`）；② **iBGP AS** 内传播——从 eBGP 获得外部路由后通过 iBGP 在 AS 内同步，默认保持 **AS-PATH**、**NEXT-HOP** 不变（例：R2 收到 R1 的 eBGP 路由后把 `(N1, AS1, R1)` 原样同步给 R3、R4、R5，三者的 BGP 下一跳均为 R1）；③ **iBGP** 水平分割——从 **iBGP** 邻居学到的路由不得再转发给其他 **iBGP** 邻居，因此传统 iBGP 要求全连接；④ **BGP** 依赖 **IGP** 实现内部可达——路由器通过 iBGP 知道” 访问外网要交给对应的边界路由器转发”，但不负责内网路径计算，需要依靠 IGP 算出自己到该边界路由器的实际转发下一跳。

BGP 的天然防环规则：AS-PATH 记录该路由途经的所有 AS 编号。当一个 AS 收到 BGP 路由更新时，若发现自身 **AS** 编号已存在于 **AS-PATH** 中，说明接收会形成环路，因此直接丢弃该路由。

BGP 工作流程精髓：交换对象是与互为邻站的 BGP 发言人交换路由信息，跨 AS 通过 eBGP 邻站交互、同一 AS 内通过 iBGP 邻站同步；交换内容是网络可达性信息，即到达目的网络所需途经的 AS 路径序列；更新机制采用增量更新，仅在路由发生变化时通告有变动的部分，不做周期性全量刷新。

BGP-4 的报文	作用
打开 **OPEN**	与相邻的另一个 BGP 发言人建立关系，使通信初始化
保活 **KEEPALIVE**	周期性地证实邻站的连通性
更新 **UPDATE**	通告某一条路由的信息，以及列出要撤销的多条路由
通知 **NOTIFICATION**	发送检测到的差错

想与另一个 AS 的 BGP 发言人建立对等关系时，向对方发送 **OPEN** 报文；若对方接受，就用 **KEEPALIVE** 报文响应。此后双方周期性交换 **KEEPALIVE** 报文（一般每隔 30 秒）。KEEPALIVE

报文只有 19 字节长（只用通用首部），不会造成太大开销。**UPDATE** 报文是 **BGP** 的核心内容：可以用它撤销曾经通知过的路由，也可以宣布增加新的路由；撤销路由可以一次撤销多条，但增加新路由时每个 **UPDATE** 报文只能增加一条。

BGP 的路由选择顺序（无策略约束时的常见比较顺序）：偏好值（自定义优先级）→ **AS-PATH** 跳数最少 → 热土豆路由 → BGP 序号。热土豆路由选择算法：让分组通过最少的”代价”尽快离开本 AS（仅用于 AS 内部，此时分组还没到达 AS 边界）。

真题考法

2024 年第 47 题第 (4)(5) 问是 BGP 的典型综合考法。第 (4) 问：R44 属 AS4、R13 属 AS1，二者之间通过 **eBGP**（外部 **BGP**）会话通告路由，使用 **UPDATE** 报文；R13 再通过 **iBGP**（内部 **BGP**）会话把该网络的可达性信息通告给同属 AS1 的 R14、R15。第 (5) 问给三条候选：经 R11 的 AS_PATH `AS2 AS8 AS19`（长度 3）、经 R12 的 `AS3 AS7 AS11 AS19`（长度 4）、经 R13 的 `AS4 AS10 AS19`（长度 3）。先比 AS_PATH 长度，R12 路径更长被排除；R11、R13 等长，无策略约束时再比较到 BGP 下一跳的内部代价（热土豆路由）：R14 到 R11 最近、R15 到 R13 最近，因此 **R14** 的下一跳是 **R11**，**R15** 的下一跳是 **R13**。

（五）三种路由协议对比

对比项	**RIP**	**OSPF**	**BGP**
类型	内部网关协议 IGP	内部网关协议 IGP	外部网关协议 EGP
路由算法	距离-向量	链路状态	路径-向量
传递协议	**UDP**（端口 520），应用层协议	**IP**（协议号 89），网络层协议	**TCP**（端口 179），应用层协议
路径选择	跳数最少	代价最低	较好，非最佳
交换结点	和本结点相邻的路由器	本自治系统所有路由器（洪泛）	和本结点相邻的路由器（邻站）
交换内容	自己知道的全部信息即整个路由表；不知道全网拓扑	相邻所有路由器的链路状态；只发送路由表的一部分	首次整个路由表；非首次只发变化部分
交换时机	周期性（约 30 s）+ 触发更新	链路状态变化时	路由变化时（增量更新）
收敛速度	慢（坏消息传得慢）	快	—
规模限制	最大 15 跳（16 为不可达）	无限制	面向跨域
防环机制	最大距离 + 水平分割 + 毒性逆转（不能彻底解决）	基于链路状态算法，天然无环	AS-PATH 中出现自身 AS 号即丢弃

为什么采用不同的传递协议（常考简答）：RIP 是分布式的基于距离向量的路由选择协议，通过广播 UDP 报文交换路由信息；OSPF 是内部网关协议，要交换的信息量较大、应使报文长度尽量短，所以不使用运输层协议而直接采用 IP；BGP 是外部网关协议，在不同 AS 之间交换路由信息，网络环境复杂需要保证可靠传输，所以采用 TCP。

六、**IP** 组播

（一）组播的概念

组播（多播）让源主机一次发送的单个分组抵达用一个组地址标识的若干目的主机，即一对多的通信。组播需要路由器支持，能运行组播协议的路由器称多播路由器。组播时仅需发送一份数据，只

在传送路径出现分岔时才将分组复制后继续转发，可大大减轻发送者负担和网络负载。

方式	目标	地址	特点
单播	一个接收者	单播地址	源需为每个接收者各发一份，N 个接收者消耗 N 份带宽
广播	本网段所有主机	广播地址	不区分是否需要；路由器默认不转发受限广播
组播	一组加入该组的主机	组播地址（**D** 类）	源只发一份，分岔处才复制，节省源端与主干链路带宽

多个单播可以仿真组播，但一个组播所需的带宽小于多个单播带宽之和；用多个单播仿真组播时路由器时延将很大，而处理一个组播分组的时延很小。

组播数据报：源地址是源主机的 IP 地址（大多为单播地址），目的地址是 IP 多播地址。注意事项：使用 **IGMP** 协议（IP 首部协议字段值是 **2**）；提供**"** 尽最大努力交付**"**，不保证交付给组内所有成员；对多播数据报不产生 **ICMP** 差错报文；多播机制仅应用于 **UDP**（TCP 面向连接，会一对一地发送）。

（二）IP 组播地址

IP 组播地址就是 IPv4 中的 **D** 类地址：前四位是 **1110**，范围 **224.0.0.0～239.255.255.255**，即 **224.0.0.0/4**。每个 **D** 类 **IP** 地址标志一个多播组，主机可随时加入或离开；多播地址只能用于目的地址，不能用于源地址；多播组成员的数量和所在的地理位置不受限制，一台主机可以属于几个多播组；非多播组成员也可以向多播组发送 **IP** 多播数据报。

D 类 **IP** 地址与以太网多播 **MAC** 地址的映射：以太网多播 MAC 地址范围是 **01-00-5E-00-00-00** 到 **01-00-5E-7F-FF-FF**（以 `01-00-5E` 打头）；48 位 MAC 中可用的低位 **23** 位与 **D** 类 **IP** 地址的低 **23** 位有一一对应关系，D 类地址除 1110 前缀外可供分配 28 位，但其中高 **5** 位无法映射，因此共有 2^{23} 个多播 MAC 地址；多播 **IP** 地址与以太网 **MAC** 地址的映射关系不是唯一的（32 个不同的 IP 多播地址可能映射到同一个 MAC 多播地址），因此主机收到多播数据报时要在 **IP** 层用软件过滤。

（三）IGMP 与组播路由

网际组管理协议（Internet Group Management Protocol，**IGMP**）是 TCP/IP 体系结构网际层中的协议，作用是让连接在本地局域网上的多播路由器知道本局域网上是否有主机（实际是主机中的某个进程）加入或退出了某个多播组。IGMP 报文被封装在 IP 数据报中传送，但它也向 **IP** 提供服务，因此不把它视为单独协议，而视为整个网际协议 IP 的一个组成部分。**IGMP** 仅在本网络有效：用它并不能知道多播组包含的成员数量，也不能知道成员分布在哪些网络中。三种报文类型：成员报告报文、成员查询报文、离开组报文。

工作阶段：① 加入阶段——某主机加入新的多播组时，向该多播组的多播地址发送一个 IGMP 报文声明自己是成员；本地多播路由器收到后还要利用多播路由选择协议把这种组成员关系转发给其他多播路由器。② 维持阶段——组成员关系是动态的，本地多播路由器周期性地探询局域网上的主机，只要对某个组有一台主机响应就认为该组活跃；经过几次探询后仍无任何主机响应，就认为本网络主机都已离开该组，不再把成员关系转发给其他多播路由器。

查询路由器的选举：同一网络中可能不止一个多播路由器，但没必要每个都周期性发送 IGMP 成员查询报文，只需选一个查询路由器发送，其他多播路由器被动接收响应并更新自己的多播组列表。选举方法：每个多播路由器若监听到源 IP 地址比自己的 IP 地址小的 IGMP 成员查询报文则退出选举，最后网络中 **IP** 地址最小的多播路由器成为查询路由器。

仅用 IGMP 不能在因特网上进行 IP 多播

连接在局域网上的多播路由器还必须和因特网上的其他多播路由器协同工作，以便用最小的代价把 IP 多播数据报传送给所有多播组成员，这就需要多播路由选择协议。

多播路由选择协议的主要任务是在多播路由器之间为每个多播组建立一个多播转发树：该树连接多播源和所有拥有该多播组成员的路由器；IP 多播数据报只要沿着多播转发树洪泛就能被传送到所有拥有该组播组成员的多播路由器；之后在多播路由器直连的局域网内，通过硬件多播把数据报发送给该组所有成员。

类型	构造方式	特点
源基树（源分布树、最短路径树）	为每一个（源，组）对分别构造以该源为根、覆盖所有组成员的最短路径树	路径最短、时延最小；但每个源都要维护一棵树，状态量为 O(源数 × 组数)，可扩展性差。代表：DVMRP、PIM-DM（密集模式）
共享树（核心树、汇聚点树）	整个组只构造一棵树，以选定的汇聚点（**RP**）为根，源先把数据发给 RP，再由 RP 沿共享树分发	状态量只需 O(组数)，可扩展性好；但路径未必最短，存在次优路径与 RP 单点瓶颈。代表：PIM-SM（稀疏模式）、CBT

组播路由的核心难点在于：组播地址只标识”组”而不标识位置，路由器无法像单播那样仅凭目的地址查表确定唯一出口；同时组成员可随时加入/离开，路由器必须动态维护转发树，并在有多个入接口时正确判断该从哪个接口转发（依赖反向路径转发 **RPF** 检查）。

七、移动 IP

（一）移动 IP 的概念

移动 **IP** 指移动站以固定的 **IP** 地址实现跨越不同网络的漫游功能，并保证基于 IP 的网络权限在漫游过程中不发生任何改变。其目标是把分组自动投递给移动站——把连接点从一个网络或子网改变到另一个网络或子网的主机。若需要在移动中进行 TCP 传输，则应一直保持这个 TCP 连接，可见若要使移动站在移动中的 **TCP** 连接不中断，就必须使主机的 **IP** 地址在移动中保持不变。

功能实体	含义
移动结点（移动站）	具有永久 IP 地址的移动主机
归属代理（本地代理）	在归属网络中代表移动节点执行移动管理功能的实体，通常就是连接在归属网络上的路由器
外地代理（外部代理）	在被访网络中帮助移动节点完成移动管理功能的实体，通常就是连接在被访网络上的路由器
归属地址（永久地址/主地址）	移动站在归属网络中的原始地址，与归属网络的关联不变
转交地址（辅地址）	移动到被访网络后获得的临时地址，可以是外地代理的地址或动态配置的地址；供移动站、归属代理及外地代理使用，各种应用程序都不会使用

（二）移动 IP 通信过程

基本工作过程分为代理发现、注册、分组路由与注销四个阶段：

1. 归属网络内正常通信：移动站 A 在归属网络时按传统 TCP/IP 方式通信。

2. 代理发现与注册：A 漫游到被访网络时先通过代理发现找到外地代理，向外地代理登记并获得一个临时的转交地址；外地代理再向 A 的归属代理登记该转交地址。
3. 隧道转发：归属代理知道 A 的转交地址后，构建一条通向转交地址的隧道，把截获的发给 A 的 IP 分组再封装（原始 IP 分组作为新 IP 分组的载荷），通过隧道发送给被访网络的外地代理。
4. 拆封与交付：外地代理把封装的 IP 分组拆封，恢复成原始 IP 分组后发给移动站 A（外地代理向移动站发送分组时直接使用移动站的 **MAC** 地址）。
5. 移动站的发送：A 在被访网络对外发送 IP 分组时仍然使用自己的永久地址作为源地址，此时无须通过归属代理转发，而是直接通过被访网络的外地代理发出。
6. 移动到另一被访网络：在新外地代理登记后，新外地代理把 A 的新转交地址告诉其归属代理（覆盖旧的）。无论如何移动，**A** 收到的 **IP** 分组都由归属代理转发。
7. 注销：A 回到归属网络时向归属代理注销转交地址，之后按原始方式通信。

移动 IP 中网络层的四个功能协议

支持移动性需要：① 移动站到外地代理的登记协议；② 外地代理到归属代理的登记协议；③ 归属代理数据报封装协议；④ 外地代理拆封协议。外地代理有两项功能：为移动站创建临时地址（转交地址）；及时把转交地址告诉其归属代理。

三角路由问题

移动 IP 的默认路由方式是非对称的：去程分组必须经归属代理隧道转发（因为通信对端只知道移动站的归属地址），而回程分组由移动站直接发出、不经归属代理。这样去程路径" 对端 → 归属代理 → 外地代理 → 移动站" 与回程路径" 移动站 → 对端" 在拓扑上构成一个三角形，称为三角路由（**triangle routing**）问题。后果是去程路径被拉长（即使对端与移动站当前在同一地区也要绕回归属网络），增加时延并浪费带宽；同时归属代理成为单点瓶颈。解决思路是路由优化：归属代理把移动站的当前转交地址通知通信对端，对端此后直接把分组发给转交地址，从而绕过归属代理形成直连路径；但这会引入对端需要缓存绑定信息、安全认证等新问题。

真题考法

移动 IP 属低频考点，考法一般是概念辨析：判断" 归属地址/转交地址"" 归属代理/外地代理" 的作用，或判断" 移动站发送分组时源地址使用哪个地址"（答案：仍使用归属地址）、" 移动站接收的分组由谁转发"（答案：归属代理）。注意移动 IP 的数据报是间接接收和发送；移动 IP 的分组路由可分为单播、广播与组播。

八、网络层设备

（一）冲突域和广播域

冲突域：连接到同一物理介质上的所有节点的集合，这些节点之间存在介质争用（OSI 第一层概念）。第一层设备（集线器、中继器）所连结点属同一个冲突域，不能划分冲突域；第二层（网桥、交换机）、第三层（路由器）设备都可以划分冲突域。

广播域：接受同样广播消息的节点集合（OSI 第二层概念）。第一、二层设备所连结点属同一个广播域；路由器（第三层）可以划分广播域；局域网（LAN）特指使用路由器分割的网络，也就是广播域。

设备互连能力：路由器可互联两个不同网络层协议的网段；网桥可互联两个物理层和链路层不同的网段；集线器不能互联两个物理层不同的网段。

一句判断法

设备所在层次越高，能屏蔽的下层差异越多，但转发速度越慢。路由器是第三层设备，处理内容比第二

层交换机更多，因而转发速度更慢、延迟更大。要抑制广播风暴只有路由器能做到：中继器、集线器工作在物理层不能隔离广播；网桥/二层交换机分隔冲突域但默认仍转发广播帧。

（二）路由器的组成和功能

定义：路由器是一种具有多个输入/输出端口的专用计算机，任务是连接不同的网络（异构网络）并完成分组转发。同一网络中传递数据无须路由器参与，而跨网络通信必须通过路由器转发；多个逻辑网络（多个广播域）互连时必须使用路由器；路由器可连接不同 LAN、不同 VLAN、不同 WAN，或把 LAN 和 WAN 互连，并可隔离广播域。

交付的两种类型：直接交付——源主机与目的主机连接在同一个网络上，无须通过路由器；间接交付——二者不在同一个网络上，路由器按转发表（由路由表得出）指出的路由把分组转发给下一个路由器。判断方法是路由器根据分组的目的 **IP** 地址与接收端口的 **IP** 地址是否属于同一子网来判断；间接交付的最后一个路由器肯定是直接交付。

路由器体系结构的两个部分：

（**1**）路由选择部分（控制部分）：路由选择处理机——根据所选定的路由选择协议构造路由表，同时经常或定期地与相邻路由器交换路由信息以不断更新和维护路由表；路由选择协议——用来更新路由表的算法；路由表——根据路由算法得出，一般包括从目的网络到下一跳的映射。

（**2**）分组转发部分：

组成部分	作用
交换结构	根据转发表对分组处理，把某个输入端口进入的分组从合适的输出端口转发出去。 三种交换方式：通过存储器交换、通过总线交换、通过互联网络交换
输入端口	包括物理层、数据链路层和网络层的处理模块
输出端口	从交换结构接收分组，再发送到路由器外面的线路上

缓冲队列与丢包：端口在网络层的处理模块中设有缓冲队列，暂存等待处理或已处理完毕待发送的分组，还可进行必要的差错检测。若分组处理的速率赶不上分组进入队列的速率，后面进入队列的分组就会因缓冲区满而被丢弃。

输入排队与输出排队都会导致丢包：输入排队——当交换结构的处理速度低于所有输入端口到达速率之和时输入队列不断积累，此外队头阻塞（**HOL blocking**）（输入队列最前面的分组因其目的输出端口忙而被阻塞，导致排在它后面、目的端口空闲的分组也无法转发）会进一步降低吞吐量；输出排队——多个输入端口同时向同一输出端口发送时输出端口成为瓶颈，输出队列溢出丢包。因此输入或输出队列产生溢出是造成分组丢失的重要原因。

路由器的功能边界（高频辨析）

- 路由器是网络层设备，实现了网络模型的下三层：物理层、数据链路层和网络层。
- 若一个存储转发设备实现了某个层次的功能，则它可以互连两个在该层次上使用不同协议的网段。如网桥实现了物理层和数据链路层，可互连两个物理层和数据链路层不同的网段；但中继器实现了物理层却不能互连两个物理层不同的网段，因为中继器不是存储转发设备，属于直通式设备。
- 路由器向传输层及以上隐藏下层实现，所以物理层、数据链路层、网络层协议可以不同；而网络层之上的协议数据是路由器不能处理的，因此网络层以上的高层协议必须相同。使用特定路由器连接 IPv4 与 IPv6 网络，就是典型的网络层协议不同而实现互连的例子。
- 存储转发机制：先接收整个分组，然后对分组做错误检查，出错就丢弃，否则存储该分组，最后根据路由选择协议把分组转发到合适端口。路由器只能根据 **IP** 地址转发。

- 完整路径问题：每个路由器都根据路由表选择下一跳地址，只有到了下一跳路由器才知道再下一跳怎么走。路由器和源主机都不知道分组要经过的完整路径。

真题考法

2012 年第 37 题考”IP 路由器功能的描述”，正确的是” 运行路由协议，设置路由表”” 监测到拥塞时，合理丢弃 IP 分组”” 根据目的 IP 地址将其转发到合适的输出线路上”；错误项是” 对收到的 IP 分组头进行差错校验，确保传输的 IP 分组不丢失”——**IPv4** 首部校验和只检查首部差错，发现错误通常丢弃分组；**IP** 提供尽力而为服务，不保证可靠交付。2010 年第 38 题考” 能够抑制广播风暴的设备”，答案是仅路由器。

（三）路由表与分组转发

路由表的结构：标准路由表有 **4** 个项目：目的网络 **IP** 地址、子网掩码、下一跳 **IP** 地址、接口。划分子网后，路由表必须包含三项内容：目的网络地址、子网掩码和下一跳地址（加上出接口共四项）。

转发表是从路由表得出的，表项与路由表项有直接对应关系，但格式不同，其结构应使查找过程最优化（路由表则需对网络拓扑变化的计算最优化）。转发表中含分组将要发往的目的地址以及分组的下一跳（实际为 MAC 地址）。可用一个默认路由代替所有具有相同” 下一跳” 的项目，并把默认路由设置得比其他项目优先级低。

主机发送 **IP** 数据报的过程：① 判断是否同属一个网络——用本机子网掩码与目的 IP 地址逐位按位与，对比结果与本机网络地址；一致则目的主机与本机在同一局域网（直接交付），不一致则目的主机在外部网络（间接交付）。② 封装 **MAC** 帧并发送——同网时通过 ARP 获取目的主机的 MAC 地址，封装成 MAC 帧后直接发送；异网时通过 ARP 获取默认网关的 MAC 地址，目的 MAC 字段填网关 MAC，把数据报交给默认网关，由路由器继续转发。

路由器转发 **IP** 数据报的完整过程：

1. 提取目的地址：从收到的 IP 数据报首部提取目的 IP 地址 D。
2. 判断直连网络直接交付：用路由器各直连接口的子网掩码分别与 D 逐位按位与，若结果与对应接口的网络地址匹配则说明目的主机在直连子网，执行直接交付（ARP 解析目的 IP 的 MAC，封装 MAC 帧从对应接口发出）；全部不匹配则进入路由表查表。
3. 匹配特定主机路由：若存在目的地址为 D 的特定主机路由（**/32** 前缀），转发给该路由指定的下一跳。
4. 匹配子网路由（最长前缀匹配）：遍历所有子网表项，用每条表项的掩码与 D 按位与得 N，若 N 与该表项的目的网络地址匹配，则选择前缀最长的匹配项，转发给对应下一跳。
5. 匹配默认路由：若子网路由无匹配项，检查是否存在 `0.0.0.0/0`，存在则转发给默认路由指定的下一跳。
6. 转发失败处理：若无任何匹配路由，或 TTL 减为 0，则丢弃该 IP 数据报，并向源主机返回 ICMP 差错报告。

分组转发的三个核心特点：① 最长匹配——采用 CIDR 时若有多个匹配前缀，应使用最长前缀匹配；为更快查找可按前缀长短把最长的排在第 1 行、按降序排列，这样从第 1 行开始查找，检索到匹配的就不必再继续。② 非完整路径——转发表没有给分组指明到某个网络的完整路径，只指出应当先到某个下一跳路由器，到达后再继续查找其转发表。③ **MAC** 帧变化——得到下一跳路由器的 IP 地址后，并不直接把它填入待发送的数据报，而是通过 ARP 转换成 MAC 地址填入 MAC 帧首部，再据此找到下一跳路由器。不同网络中传送时 **MAC** 帧的源、目的地址都要变化，而 **IP** 层的源、目的地址保持不变。

最长前缀匹配查表计算题的标准解法

1. 对每条路由表项，用该表项的掩码与目的 **IP** 地址按位与，得到候选网络地址。
2. 比较候选网络地址是否等于该表项的目的网络地址，相等则该表项匹配。
3. 在所有匹配表项中选前缀最长的那条（等价于掩码中 1 最多、块大小最小的那条）。
4. 若选中直连网络，则从对应接口直接交付；否则转发给表项给出的下一跳。
5. 默认路由 **0.0.0.0/0** 与任何地址都匹配（前缀长度 **0**），但优先级最低，只有当没有其他表项匹配时才使用。

【算例：真题型最长前缀匹配】 2015 年第 38 题路由表如下，收到目的地址为 169.96.40.5 的 IP 分组，转发接口是哪一个？

目的网络	下一跳	接口
169.96.40.0/23	176.1.1.1	S1
169.96.40.0/25	176.2.2.2	S2
169.96.40.0/27	176.3.3.3	S3
0.0.0.0/0	176.4.4.4	S4

解：/23 掩码 255.255.254.0，与 169.96.40.5 按位与得 169.96.40.0，匹配；/25 掩码 255.255.255.128，第四字节 5 落在 $0 \sim 127$ 块内，得 169.96.40.0，匹配；/27 掩码 255.255.255.224，第四字节块大小 $256 - 224 = 32$，5 落在 $0 \sim 31$ 块内，得 169.96.40.0，匹配；/0 与任何地址都匹配但前缀最短。三条具体路由都匹配，取前缀最长的 **/27**，因此从 **S3** 接口转发（选 C）。

【算例：聚合路由与直连路由的优先级】2013 年第 47 题第 (2) 问：R2 路由表中同时有 194.17.20.0/23（下一跳 194.17.24.2，接口 S1）与 194.17.20.128/25（直接交付，接口 E0），收到目的地址 194.17.20.200 的分组，从哪个接口转发？解：194.17.20.200 同时匹配 194.17.20.0/23（覆盖 194.17.20.0 ~ 194.17.21.255）和 194.17.20.128/25（覆盖 194.17.20.128 ~ 194.17.20.255）。按最长前缀匹配选择 **/25**，从 **E0** 接口转发。这道题揭示一个重要设计技巧：虽然 **/23** 也覆盖 **/25** 的范围，但由于 **/25** 更具体，直连子网的分组不会被错误地转发给 **R3**，因此可用一条聚合路由加一条更具体的直连路由来缩减路由表规模而不出错。

【算例：**TTL** 与跳数】 2014 年第 43 题第 (2) 问：主机 192.1.1.130 向主机 192.1.7.211 发送 TTL=64 的 IP 分组，R1 通过哪个接口转发？目的主机收到的 TTL 是多少？解：目的主机位于 192.1.7.0/24，R1 使用聚合路由 192.1.6.0/23（下一跳 10.1.1.2，接口 L0），从 **L0** 发往 R2，随后经 R2、R4 到达目的子网。分组经过 R1、R2、R4 共 **3** 台路由器，每台转发前把 TTL 减 1，所以目的主机收到时 $TTL = 64 - 3 = 61$。

TTL 计算的关键

初值 T 的分组经过 k 台路由器后，目的主机收到的 TTL 为 $T - k$。**"** 经过的路由器数**"** 包括发出分组的主机之后所有转发该分组的路由器，但不包括目的主机本身。若问" 初始 TTL 至少设置为多少才能送达"，最坏路径上有 k 台路由器要减 1，则初值至少为 $k + 1$。2024 年第 47 题第 (2) 问中 AS3 内最坏路径经过 15 台路由器，故初始 TTL 至少为 **16**。

路由表与转发表的比较：① 路由表根据路由选择算法得出、主要用途是路由选择、总是用软件实现，转发表可以用软件也可以用硬件实现；② 路由表不等于转发表，分组的实际转发靠直接查找转发表；③ 由于转发表根据路由表生成、路由表又由路由算法得到，因此路由算法决定了转发表中的值；④ 转发是路由器根据转发表把收到的 IP 数据报从合适端口转发出去，仅涉及一个路由器；路由选择涉及很多路由器，路由表是许多路由器协同工作的结果。

路由类型	目的网络地址	固定子网掩码	网络前缀	匹配优先级
特定主机路由	单个主机 IP	255.255.255.255	/32	最高
普通网段路由	目标网段	按需自定义	/8 ~ /30	中等
默认路由	0.0.0.0	0.0.0.0	/0	最低

本章小结表

一、**IPv4** 首部关键字段速查

字段	位数	单位	关键结论
首部长度	4	4 B	最小 5（20 B），最大 15（60 B）
总长度	16	1 B	最大 65535 B，含首部
标识	16	—	同一数据报所有分片标识相同；不是序号
标志	3	—	低 MF、中 DF、高保留（0）；DF=1 不许分片
片偏移	13	8 B	除最后一片外数据长度是 8 B 的整数倍
TTL	8	跳	每台路由器减 1，为 0 则丢弃并发 ICMP 时间超过
协议	8	—	ICMP=1，IGMP=2，TCP=6，UDP=17，OSPF=89
首部检验和	16	—	只检验首部；反码求和再取反码

二、地址与编址方法演进

阶段	地址结构	掩码/前缀	关键点
分类编址	网络号 + 主机号	255.0.0.0 / 255.255.0.0 / 255.255.255.0	A/B/C/D/E 类；主机号全 0、全 1 不可分配
划分子网	网络号 + 子网号 + 主机号	子网掩码	从主机号借位；子网数只能是 2 的幂
无分类编址 CIDR	网络前缀 + 主机号	斜线记法 /n	消除类别概念；支持路由聚合与最长前缀匹配

三、五类必考计算题的解法要点

计算类型	核心公式或规则	易错点
IP 分片	$d_{\max} = \lfloor (MTU - 20)/8 \rfloor \times 8$；片偏移 = 之前数据字节数 ÷ 8；总长度含首部	总长度不必是 8 的倍数，只有片偏移单位是 8 B
子网划分	块大小 = 256− 掩码中该字节的值；网络地址向下取整；可用数 $= 2^h - 2$	忘记减 2；块起点取错
CIDR 聚合	取所有网络地址的最长共同前缀；验证起点是块大小整数倍	聚合引入不存在的网络；下一跳写成自己
路由表转发	逐条掩码按位与 → 匹配项取前缀最长	忽略默认路由优先级最低；误改目的 IP

计算类型	核心公式或规则	易错点
RIP 更新	报文下一跳改 X、距离全部 +1；无则加、同下一跳则换、更小则改、等则加、更大则不改	忘记距离 +1；忘记 16 上限（不会出现 17）

四、**ARP / DHCP / ICMP** 对照

协议	层次	承载	关键报文/类型	核心结论
ARP	网际层或数据链路层（两种划分都可）	直接封装在链路层帧中	请求（广播）、响应（单播）	只在本局域网内有效；跨网段解析的是默认网关的 MAC
DHCP	应用层	UDP（服务器 67、客户端 68）	DISCOVER / OFFER / REQUEST / ACK	四个报文目的 IP 都是 255.255.255.255；OFFER、ACK 在链路层单播
ICMP	网际层	IP（协议号 1）	差错：终点不可达、源点抑制、时间超过、参数问题、改变路由；询问：回送请求/回答、时间戳	ping 用回送请求和回答；traceroute 依赖时间超过；对组播数据报不发差错报文

五、**RIP / OSPF / BGP** 对比

对比项	**RIP**	**OSPF**	**BGP**
归属	IGP（域内）	IGP（域内）	EGP（域间）
算法	距离-向量	链路状态	路径-向量
承载	UDP 520	IP，协议号 89	TCP 179
度量	跳数（最大 15，16 不可达）	代价（带宽、时延等，可自定义）	AS-PATH 等属性 + 策略
交换范围	仅相邻路由器	洪泛至 AS 内所有路由器	仅邻站
收敛	慢	快	增量更新

六、**IPv4** 与 **IPv6** 对照

对比项	**IPv4**	**IPv6**
地址长度	32 bit	128 bit
基本首部长度	20 B 固定 + 0 ~ 40 B 可选	固定 40 B
分片	源主机和路由器都可分片	只有源主机可分片
首部检验和	有	无
TTL / 跳数限制	TTL	跳数限制（Hop Limit）
上层协议标识	协议字段	下一个首部字段
选项	首部中的选项字段	扩展首部（路由器不处理，逐跳选项除外）

对比项	IPv4	IPv6
地址配置	需 DHCP	支持地址自动配置
地址解析	需 ARP	不需要 ARP（接口标识符 64 bit 可直接编码）
过渡技术	—	双协议栈、隧道技术

易错清单

1. IP 分片的三个数字

首部长度以 **4 B** 为单位，总长度以 **1 B** 为单位，片偏移以 **8 B** 为单位。算” 每片最多携带多少数据” 要先用 $\lfloor (MTU-20)/8 \rfloor \times 8$ 向下取整；算” 总长度” 要记得加回 20 B 首部；看到” 最后一片总长度不是 8 的倍数” 不要怀疑自己算错——只有除最后一片外的数据长度必须是 8 的整数倍。

2. 可用地址数永远要减 2

子网/地址块内可分配给主机或路由器接口的地址数 $=2^h-2$，要减掉网络地址（主机号全 **0**）和广播地址（主机号全 **1**）。但若题目问的是” 地址块中有多少个地址”，则不减 **2**（例如 /20 有 4096 个地址）。读题时必须分清问的是” 地址总数” 还是” 可分配数”。

3. 特殊地址的方向性

0.0.0.0 和 **0.host-id** 只能作源地址不能作目的地址；**255.255.255.255** 只能作目的不能作源；**127.x.x.x** 源、目的都可以但不走外网；主机号全 0 的网络地址既不能作源也不能作目的。问” 只能作源” 或” 只能作目的” 时必须逐一核对方位。

4. 划分子网不改变网络号，也不能从 IP 地址判断

从一个 **IP** 地址本身无法判断该主机所连接的网络是否进行了子网划分。划分子网只是把主机号再划分，不改变原有类别归属，也不增加” 网络” 数量——只提高 IP 地址利用率。反之，给定 **IP** 与掩码就能算出子网划分的全部细节。

5. 最长前缀匹配与默认路由

默认路由 `0.0.0.0/0` 与任何目的地址都匹配（全 0 掩码按位与结果必为全 0），但它的前缀长度是 **0**，所以只要有任何其他表项匹配就绝不会选中它。特定主机路由 `/32` 优先级最高。做题时先找出所有匹配项再比前缀长度，不要一看到 **0.0.0.0/0** 就直接下结论。

6. 路由器转发不改变目的 IP 地址

路由器转发分组时只改写链路层的 **MAC** 帧首部（源 **MAC**、目的 **MAC** 逐跳变化），**IP** 首部中的源 **IP** 和目的 **IP** 全程不变（除非经过 NAT 设备，或 TTL 减 1、检验和重算）。综合题问” 经过某路由器后源/目的 IP 是什么” 时，若该路由器不是 NAT 路由器，答案都是” 不变”；” 下一跳地址” 是路由表里的字段，不会被写进 IP 首部。

7. ARP 请求的目标 IP 是网关而非最终目的主机

当目的主机不在同一局域网时，**ARP** 解析的是默认网关（本网段路由器接口）的 **MAC** 地址。另外，**ARP** 响应是单播（只有请求方能收到），不是广播。DHCP 的 OFFER 和 ACK 在链路层是单播（目的 MAC 是客户端），但 **IP** 层目的地址仍是 **255.255.255.255**。

8. RIP 的 16 与” 距离 17”

RIP 中距离等于 **16** 就表示不可达，永远不会出现 **17**。若邻居通告 `<net，16>`，则自己经该邻居也到不了。同时注意：同一邻居通告的路由无条件替换；不同邻居通告且新距离相等时可以添加实现等价负载均衡，新距离更大则不更新。

9. 三类协议的封装方式必须记准

RIP 用 **UDP**（端口 **520**）；**OSPF** 直接封装在 **IP** 数据报中（协议号 **89**），没有经过 **TCP** 或 **UDP**；**BGP** 用 **TCP**（端口 **179**）。从封装角度看 RIP 和 BGP 属于应用层、OSPF 属于网络层；但 RIP 的核心功能是路由选择，属于网际层。这个” 封装层次” 与” 功能层次” 的差别是常见陷阱。

10. IPv6 与 IPv4 的对比细节

IPv6 地址空间是 IPv4 的 2^{96} 倍，不是 **96** 倍；IPv6 基本首部固定 **40 B**；IPv6 的有效载荷长度不包含基本首部但包含扩展首部；IPv6 取消了首部检验和；IPv6 只有源主机能分片；IPv6 不需要 **ARP**（接口标识符 64 bit 足以直接编码硬件地址，地址解析由邻居发现协议 ND 完成）；IPv6 地址类型是单播、多播、任播三种，没有广播类型（广播被看作多播的特例）。

11. 路由聚合的” 引入不存在的网络”

聚合后的地址范围往往大于原有各子网地址范围的并集（这正是聚合能减少表项的原因）。工程上通常可以接受（额外地址范围没有实际主机，分组到达后会因无更具体路由而被丢弃或走默认路由），但在要求” 不引入不存在的网络” 的题目中必须谨慎。判断标准是：待聚合的各网络是否恰好拼成一个完整的 **2** 的幂地址块范围且起点对齐。

12. 拥塞控制与流量控制不要混为一谈

流量控制是发送方与接收方之间的点对点通信量控制，目的是抑制发送速率使接收方来得及接收；拥塞控制关注整个网络的负载，是全局性的。开环控制是静态的预防方法（启动后中途不再修改），闭环控制基于反馈环路，是动态的方法。

第五章传输层

一、传输层

本章考纲要求

（一）传输层提供的服务

1. 传输层的功能
2. 传输层寻址与端口
3. 无连接服务与面向连接服务

（二）UDP 协议

1. UDP 数据报
2. UDP 校验

（三）TCP 协议

1. TCP 段
2. TCP 连接管理
3. TCP 可靠传输
4. TCP 流量控制
5. TCP 拥塞控制

传输层是「面向通信部分的最高层，用户功能中的最低层」。它在 408 里的地位非常特殊：选择题年年出，大题几乎每隔一两年就出一次，而且大题往往不是单纯考定义，而是把「序号/确认号推算 + 窗口计算 + 拥塞窗口逐轮演算 + 时间估算」揉在一道题里。也就是说，这一章真正要练的不是背概念，而是四类手算能力：

1. 序号与确认号的推算（连接建立、数据传送、连接释放三个阶段都要会）；
2. 窗口的推算（发送窗口 = min(rwnd, cwnd)，还要能判断窗口内哪些字节能发）；
3. 拥塞窗口的逐轮演算（慢开始、拥塞避免、快重传、快恢复的分支处理）；
4. 时间估算（RTT、MSL、RTO 的加减）。

本章按考纲条目顺序展开，所有计算公式都配上可以自己验算一遍的数字例子。三次握手、四次挥手、窗口滑动、拥塞控制这四块，一律给完整的算例表格。

（一）传输层提供的服务

*1.*传输层的功能

进程到进程的逻辑通信

传输层（**transport layer**）的核心任务是：为运行在不同主机上的应用进程之间提供逻辑通信。注意这里的「逻辑通信」四个字。

从应用层看过去，只要把报文交给下面的传输层，传输层就好像能沿水平方向把它直接送到对方的传输层，哪怕两台主机相隔几千公里。但事实上，两个传输层之间并不存在一条水平方向的物理连

接——数据仍然要一层层向下走，经过网络层、数据链路层、物理层，到达对端后再一层层向上递交。「逻辑通信」的含义就是「好像是这样通信，但事实上并非真的这样通信」。

传输层向上提供的是应用进程之间的通信，所以叫端到端（**end-to-end**）通信；网络层提供的是主机到主机的通信。这两个层次的「端」含义完全不同：

对比项	网络层	传输层
通信主体	主机到主机（host-to-host）	进程到进程（process-to-process，端到端）
标识方式	IP 地址	IP 地址 + 端口号（套接字）
典型协议	IP、ICMP、ARP	TCP、UDP
是否区分进程	不区分，只找到主机	必须区分，交付给具体进程

一句话区分

网络层解决「送到哪台主机」，传输层解决「送到哪台主机的哪个进程」。IP 地址定位主机，端口号定位进程，两者合起来（套接字）才能唯一确定一次进程间通信的端点。

复用与分用

复用（**multiplexing**）：发送方的多个应用进程可以共用同一个传输层协议（TCP 或 UDP）发送数据，传输层给数据加上适当的首部后交给网络层。

分用（**demultiplexing**）：接收方的传输层在剥去报文首部后，依据首部中的端口号把数据正确交付给目的应用进程。

要注意网络层也有复用与分用，但复用的含义不同：

- 网络层的复用：不同协议的数据（TCP 报文段、UDP 用户数据报、ICMP 报文等）都可以封装成 IP 数据报发送出去；
- 网络层的分用：接收方剥去 IP 首部后，依据首部中的协议字段把数据交付给相应的上层协议。

也就是说，网络层按「协议」分用，传输层按「端口」分用。这是统考选择题里反复出现的辨析点。

复用还是分用?

「多个进程共用一个传输层协议」是复用；「依据端口号把数据交给正确的进程」是分用。题目问「UDP 实现分用时依据哪个字段」——答案是目的端口号，不是源端口号。源端口号的作用是让对方知道该把回复送到哪里。

其他功能

传输层还需要承担下面这些功能，其中哪些真正实现，取决于用的是 TCP 还是 UDP：

- 差错检测：TCP 和 UDP 都用「首部 + 数据 + 伪首部」的校验和检测误码。但校验和只能检测误码，不能纠错，更不能保证可靠交付。
- 流量控制：TCP 有，通过接收窗口 rwnd 抑制发送速率；UDP 没有。
- 拥塞控制：TCP 有，通过拥塞窗口 cwnd 防止过多数据注入网络；UDP 没有。
- 可靠传输：TCP 有（确认、重传、排序、去重）；UDP 没有。
- 连接建立与释放：TCP 有；UDP 无连接。

出题角度

2014 年第 39 题考的就是「传输层功能的辨析」:「UDP 提供无连接服务」「UDP 提供复用/分用服务」「UDP

通过差错校验保障可靠数据传输」三个说法中哪个正确——前两个对，第三个错。「有校验和」不等于「可靠传输」，这是本节最经典的陷阱。

不同的操作系统，进程标识符（PID）的命名规则、格式、长度都不一样；更关键的是进程的创建与撤销是动态的，同一个进程在不同时刻的 PID 会发生变化，通信的另一方根本无法预先知道。

为什么用端口号而不用进程号

需要一个独立于操作系统、相对稳定的进程标识——这就是端口号。

端口号引入后，接收方只需「监听某个众所周知的端口」，就可以稳定地接收来自任意主机的请求，而完全不关心对方是哪个操作系统、进程号是多少。

补充

按 OSI 的说法，传输层的服务访问点叫 **TSAP**（**Transport layer Service Access Point**），其具体形式就是传输层端口。各层的 SAP 依次是：物理层 PSAP（物理接口）、数据链路层 DLSAP（MAC 地址）、网络层 NSAP（IP 地址）、传输层 TSAP（端口）、会话层 SLSAP、表示层 PLSAP。这一条属于「层与 SAP 对应关系」的送分点。

*2.*传输层寻址与端口

端口号

端口（**port**）是传输层使用的、用来标识应用进程的软件标识符，也叫端口号（**port number**）。

- 端口号占 **16** 比特，取值范围 **0 ~ 65535**（共 $2^{16} = 65536$ 个）。
- 端口号是软件层面的概念，与硬件无关。它存在于传输层首部中，标识的是「本机协议栈应用层中的某个应用进程」。
- 端口号只具有本地意义：端口号只用来标识本机应用层的进程。因特网中不同计算机上的相同端口号之间没有任何关系，相互独立。
- **TCP** 与 **UDP** 的端口号也是相互独立的。同一个端口号 53 在 UDP 里给 DNS 用，在 TCP 里也可以给 DNS 用，二者互不冲突。

必背

端口号 16 位、取值范围 0~65535、只有本地意义、TCP 与 UDP 端口号相互独立。这四句话几乎每年都有题在考。

三类端口号

类别	范围	说明	举例
熟知端口号（well-known port）	**0 ~ 1023**	由 IANA 统一指派给最重要的应用层协议，固定不变	FTP 21、HTTP 80、DNS 53
登记端口号（registered port）	**1024 ~ 49151**	给没有熟知端口号的应用程序使用，必须在 IANA 登记以防重复	微软远程桌面 3389
短暂端口号（ephemeral port，也叫客户端端口号/临时端口号）	**49152 ~ 65535**	客户端进程在通信时由操作系统临时动态分配，通信结束后被回收并放入空闲端口池	客户端发起连接时使用的端口

三类端口号的边界必须记牢：**1023 / 49151** 是两个分界点。有的教材把第二类写成「1024~49151」、第三类写成「49152~65535」，分界点完全一致。

三类端口号的分界

题目经常这样设陷阱：给一个端口号 65535，问它属于哪一类——属于短暂端口号（客户端），不是登记端口号。再比如问「服务器端使用的端口一定是熟知端口吗」——不一定，服务器进程同样可以使用登记端口号（如 3389）。

套接字

仅仅用「IP 地址 + 端口号」还不能完整描述一次通信，因为一次 TCP 连接是双向的，需要两端各一个端点。

套接字（**socket**）：

$$\text{socket} = (\text{IP 地址} : \text{端口号})$$

TCP 连接由通信两端的两个套接字唯一确定（四元组）：

$$\text{TCP 连接} ::= \{\text{socket}_1, \text{socket}_2\} = \{(\text{IP}_1 : \text{port}_1), (\text{IP}_2 : \text{port}_2)\}$$

这一条解释了两个重要现象：

- 同一台主机上的同一个服务器端口（比如 80）可以同时接受很多客户端连接，因为每个连接的「四元组」不同；
- 反过来，两台主机之间如果四元组完全相同，那就是同一条连接（这也解释了为什么端口号复用一般不会冲突）。

出题角度

常考形式：给出若干条 TCP 连接的四元组，问「哪两条是同一个连接」「这条连接的两端套接字分别是什么」。判断标准就是四元组是否完全一致。

常见协议的熟知端口号表（必背）

应用层协议	端口号	使用的传输层协议	用途要点
FTP	**21**（控制）/ **20**（数据）	TCP	21 传命令，20 传数据（主动模式）
SSH	**22**	TCP	安全远程登录
Telnet	**23**	TCP	远程登录（明文）
SMTP	**25**	TCP	发送邮件（客户 → 服务器）
DNS	**53**	UDP（域名解析）/ TCP（区域传送）	域名解析
DHCP	**67**（服务器）/ **68**（客户端）	UDP	服务器监听 67，客户端用 68 接收
TFTP	**69**	UDP	简单文件传输
HTTP	**80**	TCP	网页明文传输
POP3	**110**	TCP	接收邮件（下载到本地）
IMAP	**143**	TCP	接收邮件（服务器上管理）
HTTPS	**443**	TCP	网页加密传输

应用层协议	端口号	使用的传输层协议	用途要点
SNMP	**161**	UDP	网络管理
RIP	**520**	UDP	路由信息协议

记忆技巧：

- **FTP** 一对：21 发命令、20 传数据；
- 邮件三兄弟：SMTP 25（发）、POP3 110（收）、IMAP 143（收）；
- **DNS 53**：「我要解析（53）域名」；
- **DHCP 67/68**：服务器 67、客户端 68，相邻一对；
- **HTTP 80** 明文、**HTTPS 443** 加密；
- **TFTP 69 / SNMP 161 / RIP 520** 都是 UDP，这三个最容易和 TCP 混淆。

高频陷阱

1. **RIP** 用 **UDP 520** 端口，不是 TCP；BGP 才用 TCP 179。（2017 年、2021 年都出现过这类辨析。）
2. **DNS** 主要用 **UDP 53**，但区域传送（zone transfer）用 TCP 53。题目若说「DNS 只使用 UDP」，那是错的。
3. **DHCP** 两个端口都要记：服务器 67、客户端 68，不要颠倒。
4. **SNMP** 用 **UDP 161**，SNMP Trap 用 UDP 162。

*3.*无连接服务与面向连接服务

传输层向应用层提供两类服务，对应两个协议：

- 面向连接的服务：TCP（Transmission Control Protocol，传输控制协议）。通信前必须先建立连接，通信结束后释放连接，提供可靠的、按序的、面向字节流的服务。
- 无连接的服务：UDP（User Datagram Protocol，用户数据报协议）。通信前不建立连接，提供尽最大努力交付的、不可靠的、面向报文的服务。

TCP 与 UDP 的全面对比

对比项	**UDP**	**TCP**
连接方式	无连接	面向连接（三报文握手建立、四报文挥手释放）
可靠性	不可靠，尽最大努力交付	可靠（确认、重传、排序、去重）
数据组织方式	面向报文，应用层给多长就发多长，一次发一个报文，既不合并也不拆分	面向字节流，把数据看成无结构的字节序列，可自行划分成报文段
首部开销	**8** 字节，固定	**20** 字节固定 + 最多 40 字节选项 = 最长 60 字节
流量控制	无	有（接收窗口 rwnd）
拥塞控制	无	有（拥塞窗口 cwnd，四种算法）
差错检测	有校验和（IPv4 中可选用，全 0 表示不校验）	有校验和（必选，出错则丢弃并等待重传）
是否保证按序到达	不保证	保证

对比项	**UDP**	**TCP**
通信模式	一对一、一对多、多对一、多对多（支持广播与组播）	只能是一对一（单播）
连接状态	无状态	需要维护发送/接收缓存、序号、确认号、拥塞控制参数等大量状态
适用场景	实时性要求高、能容忍少量丢失：DNS、DHCP、TFTP、SNMP、RIP、实时音视频	要求可靠：HTTP、FTP、SMTP、POP3、IMAP、SSH、Telnet

三个最容易错的对比点

1. 「面向报文」与「面向字节流」：UDP 保留应用层报文的边界，一次发一个；TCP 不保留边界，可能把两个小报文合并成一个段发出，也可能把一个大报文拆成多个段。所以 2010 年前后的选择题里「TCP 和 UDP 都可以对报文进行合并拆分」是错的——只有 **TCP** 可以。
2. 校验和覆盖范围：UDP 与 TCP 的校验和都覆盖「伪首部 + 首部 + 数据」，而 **IP** 数据报首部校验和只校验首部，不含数据。
3. 广播/组播：只有 UDP 支持广播和组播，TCP 只支持单播。

（二）**UDP** 协议

1.UDP 数据报

UDP 的特点

- 无连接：发送数据前不需要建立连接，每个 UDP 用户数据报独立处理，互不影响。因此 UDP 不需要像 TCP 那样维护连接状态。
- 尽最大努力交付：不保证可靠，不保证按序，不保证不重复。
- 面向报文：应用层交下来多长的报文，UDP 就原样封装成一个 UDP 用户数据报发送，既不合并也不拆分。因此应用层必须自己选择合适大小的报文——报文太长，交给 IP 层后可能被分片，降低效率；报文太短，首部开销占比过大。
- 支持多种通信模式：一对一、一对多、多对一、多对多，即支持单播、广播、组播。
- 首部开销小：只有 8 字节。
- 无流量控制、无拥塞控制：即使网络已经拥塞，UDP 也不会降低发送速率。这一点在网络拥塞时可能造成大量丢包，但也正是实时业务（语音、视频）需要 UDP 的原因——宁可丢几个包，也不要因为等待重传而引入大延迟。

记忆口诀

UDP 的六条特点：无连接、不可靠、面向报文、支持广播组播、首部小、无流量/拥塞控制。

UDP 用户数据报的格式

UDP 用户数据报 = **8** 字节首部 + 数据部分。首部只有四个字段，每个字段 **2** 字节：

字段	长度	含义
源端口	2 B	发送方进程的端口号。不需要对方回复时可全置 0
目的端口	2 B	接收方进程的端口号，分用就靠这个字段

字段	长度	含义
长度	2 B	UDP 用户数据报的总长度 = 首部 **8 B** + 数据部分长度，单位字节
校验和	2 B	检测整个用户数据报在传输中是否出现误码。全 0 表示不进行校验

长度字段

因为长度字段只有 16 位，所以 UDP 用户数据报的最大总长度是 $2^{16}-1=65535$ 字节，其中数据部分最多 $65535-8=65527$ 字节。「长度」字段的最小值是 **8**（此时数据部分为空）。

「长度」字段的两种问法

题目给「长度字段 = 1500」，则数据部分长度 = **1500 − 8 = 1492** 字节；题目给「UDP 数据部分 1500 字节」，则长度字段 = **1500 + 8 = 1508**。这两个方向都要会，也是选择题的经典送分/送命点。

UDP 不保证可靠交付

因为 UDP 没有确认与重传机制，所以：

- 数据报丢失后不会重传；
- 数据报到达顺序可能与发送顺序不同；
- 数据报可能重复到达。

校验和只能检测误码。接收方发现校验出错时，通常直接丢弃该数据报，不产生任何差错报告（注意：UDP 本身不会因此发送 ICMP 差错报文，但若目的端口不可达，接收方的 IP 层可能回送 ICMP 端口不可达——这是 IP 层的职责，不要混为一谈）。

2.UDP 校验

校验和的计算范围

UDP 的校验和覆盖三个部分：

$$\text{校验范围} = 12\text{ 字节伪首部} + 8\text{ 字节 UDP 首部} + \text{UDP 数据部分}$$

若这三部分的总长度不是偶数个字节，还要在末尾补 **1** 个全 **0** 字节凑成偶数（因为要按 2 字节为单位分组相加）。补充的全 **0** 字节只用于计算校验和，不发送。

伪首部（**pseudo-header**）

伪首部共 **12** 字节，结构如下：

字段	长度	取值
源 IP 地址	4 B	取自 IP 数据报首部，不是「本机 IP」的抽象概念，而是实际发送用的那个源地址
目的 IP 地址	4 B	取自 IP 数据报首部
全 0 字段	1 B	保留，置 0（有教材称「保留字节」）
协议号	1 B	**UDP 为 17**
UDP 长度	2 B	UDP 用户数据报的总长度（首部 + 数据），与首部中「长度」字段的值相同

字段	长度	取值
合计	**12 B**	

伪首部的作用（必背）

伪首部的作用是让传输层能够检查数据是否被错误地交付到了本主机、本协议。它把源 IP、目的 IP、协议号一起纳入校验范围，于是：

1. 如果 IP 数据报被错误地投递到错误的地址（目的 IP 不符），校验和会失败；
2. 如果 IP 数据报被错误地交给了错误的协议（协议号不符），校验和也会失败。

伪首部既不在网络中传送（发送方算完就丢弃），也不向上层递交，它只在两端计算校验和时临时构造。这一点是选择题的高频正确选项。

校验和的计算过程

发送方计算步骤：

1. 把 UDP 首部中的校验和字段置为 **0**；
2. 在 UDP 用户数据报前面临时加上 12 字节伪首部；
3. 把「伪首部 + 首部 + 数据」按 **2** 字节（**16** 位）为一组划分成若干个「字」。若总长度是奇数个字节，在末尾补 1 个全 0 字节；
4. 对所有 16 位字进行二进制反码求和：逐字相加，每次相加若最高位产生进位，就把进位回卷加回到最低位（cyclic carry）；
5. 把求和结果逐位取反，写入 UDP 首部的校验和字段。

接收方校验步骤：

1. 给收到的 UDP 用户数据报前面加上 12 字节伪首部；
2. 同样按 2 字节分组，做二进制反码求和；
3. 把结果取反。若结果为 **0**，说明传输无误码；否则说明出现误码，丢弃该数据报。

全 0 的含义

发送方把校验和字段置为全 **0**，表示「不进行校验」（这是 IPv4 下 UDP 允许的，IPv6 下 UDP 校验和是必选的）。接收方收到校验和字段为全 0 的数据报，就不做校验，直接交付。

另外还有一个容易忽略的细节：如果校验和的计算结果恰好是全 **0**，那么应当写入全 **1**（即 **0xFFFF**），而不是全 **0**。因为全 0 已经被「不校验」这个特殊含义占用了。这就是所谓「全 **0** 表示不校验，全 **1**（即零的反码）表示校验和为 **0**」。

完整算例（可以自己验算一遍）

设某 UDP 用户数据报的各字段（用十六进制表示）：

- 源 IP 地址 = `C0A8 0101`（即 192.168.1.1）
- 目的 IP 地址 = `0A00 0005`（即 10.0.0.5）
- 协议号 = `0011`（即十进制的 17）
- UDP 长度 = `000C`（即 12 字节：8 字节首部 + 4 字节数据）
- 源端口 = `1F90`，目的端口 = `0053`
- 数据部分 = `0102 0304`（4 字节）

第一步：构造需要参与运算的全部 16 位字。

序号	内容	**16** 位十六进制
①	源 IP 高位	C0A8
②	源 IP 低位	0101
③	目的 IP 高位	0A00
④	目的 IP 低位	0005
⑤	全 0 字节 + 协议号	0011
⑥	UDP 长度	000C
⑦	源端口	1F90
⑧	目的端口	0053
⑨	长度字段	000C
⑩	校验和（置 0）	0000
⑪	数据第 1 个 16 位	0102
⑫	数据第 2 个 16 位	0304

（第 ①~⑥ 项合起来正是 12 字节伪首部，第 ⑦~⑩ 项是 UDP 的 8 字节首部，第 ⑪⑫ 项是数据部分。）

注意两个「长度」

⑥ 是伪首部里的 UDP 长度，⑨ 是 UDP 首部里的长度字段，两处必须是同一个值（本题都是 000C），但它们是两个不同的字段，做加法时两个都要算进去，不要漏掉其中一个，也不要误以为重复而只算一次。

第二步：二进制反码求和（用十六进制加法）。

```
  C0A8
+ 0101
= C1A9
+ 0A00
= CBA9
+ 0005
= CBAE
+ 0011
= CBBF
+ 000C
= CBCB
+ 1F90
= EB5B
+ 0053
= EBAE
+ 000C
= EBBA
+ 0000
= EBBA
+ 0102
= ECBC
+ 0304
= EFC0
```

求和结果为 EFC0，过程中没有产生 16 位溢出，因此无需回卷。

第三步：取反。

$$\overline{\texttt{EFC0}} = \texttt{103F}$$

所以校验和字段应写入 `103F`。

第四步：接收方验证。把全部字（含 `103F`）再加一遍：

```
  EFC0          （前面全部字之和）
+ 103F          （校验和字段）
= FFFF
```

结果为全 1，取反后为全 0，说明传输无误码。✓

回卷进位算例（考得比上面这个还多）

上面的算例比较「小」，恰好没有出现 16 位溢出。但真实题目里几乎一定会溢出，所以「回卷」这一步必须练熟。设求和的中间结果为 `B9B6`（已包含前面所有字，尚未取反），此时还剩最后一个 16 位字 `65C5` 需要加入：

```
  1011 1001 1011 0110   (0xB9B6)
+ 0110 0101 1100 0101   (0x65C5)
-------------------------
1 0001 1111 0111 1011   (0x11F7B)   <- 产生了 1 位溢出进位
```

回卷：把溢出的最高位进位 1 加回最低位：

```
  0001 1111 0111 1011   (0x1F7B)
+                    1
-------------------------
  0001 1111 0111 1100   (0x1F7C)   <- 反码和
```

取反得到校验和：

```
  0001 1111 0111 1100   (0x1F7C)
-> 1110 0000 1000 0011   (0xE083)   <- 校验和
```

验证：把 `0x1F7C` 与 `0xE083` 相加，$1F7C + E083 = FFFF$，取反为 0，正确。✓

回卷的两种等价写法

1. 文字写法：先把全部字相加，若结果超过 16 位，把高位进位加到低 16 位上——这就是「回卷（cyclic carry）」；
2. 工程写法：每加一个字就检查一次进位并立刻回卷（本例就是这样做的）。两种写法结果完全一致。

做题时的顺序一定要记住：先全部反码求和（含回卷）→ 再取反 → 写入校验和字段。取反必须放在最后，先把中间结果取反再相加就错了。

校验和计算题的三个坑

1. 做加法时忘了把伪首部算进去。伪首部一定要算，而且 UDP 的协议号是 **17**（TCP 是 6）。
2. 忘了回卷进位。只要 16 位相加产生进位，就必须把进位加回到最低位。2024 年第 39 题就专门考了这个「中间结果 + 最后一个字」的回卷与取反。
3. 最后一步忘了取反，或者取反后忘了「全 0 要写成全 1」。

TCP 校验与 UDP 校验的差别

TCP 的校验和计算过程与 UDP 完全相同（反码求和 + 回卷 + 取反），只有两处不同：

- 伪首部中的协议号是 **6**（而不是 17）；
- 伪首部中的长度字段是 **TCP** 报文段的长度（而不是 UDP 长度）。

TCP 的校验和是必选的（不能用「全 0」表示不校验），而出错时同样只是丢弃，由超时重传机制来弥补。

出题角度

UDP 校验是选择题的常客，典型问法有三类：①「伪首部的作用是什么」②「校验和的覆盖范围包括什么」③「给出若干 16 位字，求校验和」。第 ③ 类只需老老实实做反码加法。另外还会考「全 0 校验和表示什么」——表示不校验，不是「校验通过」。

（三）TCP 协议

1.TCP 段

TCP 的主要特点

- 面向连接：通信前必须先建立连接，通信结束后必须释放连接。
- 只能一对一：每条 TCP 连接只能有两个端点（两个套接字），因此不支持广播和组播。
- 可靠交付：无差错、不丢失、不重复、按序到达。
- 全双工通信：TCP 连接的两端都设有发送缓存和接收缓存，数据可以双向同时传输。
- 面向字节流：TCP 把应用层交下来的数据看成一连串无结构的字节流。TCP 不保留应用层报文的边界，发送方可以根据情况把字节流划分成若干个大小合适的报文段，接收方再把它们还原成字节流。

面向字节流的三个推论

1. 序号以字节为单位编号，不是以报文段为单位。一个报文段的序号 = 该报文段数据部分第一个字节的序号。
2. 应用层的一次「写」操作，可能对应多个 TCP 报文段；多次「写」操作，也可能被合并进一个 TCP 报文段。
3. TCP 连接的两端只是「字节流的两端」，TCP 自己并不知道这次传输在应用层是一条消息还是半条消息。

TCP 报文段的首部格式

TCP 报文段 = 首部 + 数据载荷。首部由固定首部 **20** 字节和选项 + 填充（最多 **40** 字节）组成，因此 TCP 首部最长 **60** 字节。

字段	长度	含义
源端口	2 B	发送方进程端口号
目的端口	2 B	接收方进程端口号
序号 **seq**	4 B	本报文段数据载荷的第一个字节的序号，取值 $0 \sim 2^{32}-1$，到最大值后回到 0
确认号 **ack**	4 B	期望收到对方下一个报文段的第一个字节的序号；仅在 ACK=1 时有效

字段	长度	含义
数据偏移	4 bit	指出 TCP 报文段数据载荷的起始位置距报文段起始位置有多远，即首部长度，以 **4** 字节为单位
保留	6 bit	保留今后使用，目前置 0（新版标准中部分位已被 ECE、CWR 占用，用于拥塞控制通知）
URG	1 bit	紧急标志位。URG=1 时紧急指针字段有效
ACK	1 bit	确认标志位。ACK=1 时确认号字段才有效；连接建立后所有报文段都必须把 **ACK 置 1**
PSH	1 bit	推送标志位。置 1 表示接收方应尽快交付应用进程，不必等接收缓存填满
RST	1 bit	复位标志位。置 1 表示连接出现严重差错，必须释放连接后重新建立；也用来拒绝非法报文段或拒绝打开连接
SYN	1 bit	同步标志位。用于建立连接。SYN=1 且 ACK=0 是连接请求；SYN=1 且 ACK=1 是连接接受
FIN	1 bit	终止标志位。置 1 表示发送方数据已全部发完，要求释放连接
窗口	2 B	指本报文段发送方自己的接收窗口大小（接收缓存可用空间），以字节为单位，用于流量控制。最大 65535 字节
校验和	2 B	覆盖伪首部 + TCP 首部 + 数据载荷；计算方法与 UDP 相同，但协议号为 6
紧急指针	2 B	URG=1 时有效，以字节为单位指出本报文段中紧急数据的长度
选项	可变	最长 40 B。常用：MSS 选项、窗口扩大选项、时间戳选项、选择确认 SACK 选项
填充	可变	使首部长度能被 4 字节整除

序号与确认号：本节第一重点

这是 TCP 全部计算题的根基，必须彻底分清。

序号 **seq**（**sequence number**）：本报文段数据载荷的第一个字节的序号。如果本报文段不携带数据，则「不消耗序号」——但 SYN 和 FIN 是例外，见后文。

确认号 **ack**（**acknowledgement number**）：期望收到对方下一个报文段的第一个数据字节的序号，同时也是对「到 ack−1 为止的所有数据都已正确收到」的确认。

序号与确认号一句话区分

- **seq** 说的是「我发的这段数据从第几号字节开始」——回答「我在说什么」；
- **ack** 说的是「我想听你从第几号字节往下讲」——回答「我听到哪儿了」。

若确认号为 N，则表明：序号到 $N-1$ 为止的所有数据都已正确接收。

与 **GBN** 的区别（易错）：在后退 N 帧协议中，ACK n 表示「序号到 n 为止的数据已正确接收，期望收到 $n+1$」；而 **TCP** 中 **ack** $= N$ 表示「到 $N-1$ 为止已正确接收，期望收到 N」。两者的下标含义差 1，千万不要混用。

示例：甲向乙发送一段 300 字节的数据，seq=201。这段数据占用序号 201~500。若乙正确收到，则乙回复的确认号 ack $= 501$。

序号取值规则（必背，覆盖握手/挥手/数据三个阶段）

报文类型	是否消耗序号
SYN 报文段（握手第 1、2 次）	消耗 **1** 个序号
FIN 报文段（挥手第 1、3 次）	消耗 **1** 个序号
纯 ACK 报文段（不携带数据）	不消耗序号
携带有数据的报文段	消耗的序号数 = 数据字节数
握手第 3 次的 ACK	携带数据则消耗相应字节数；不携带数据则不消耗（下一报文段序号仍为 x+1）
挥手第 4 次的 ACK	规定不能携带数据，也不消耗序号

数据偏移

- 占 **4** 比特，以 **4** 字节为单位。
- 因为 4 比特最大能表示 15，所以 TCP 首部最长 $15 \times 4 = 60$ 字节。
- 固定首部 20 字节，对应数据偏移 $= 20/4 = 5$，二进制 `0101`。
- 因为首部长度必须是 4 字节的整数倍，所以若「固定首部 + 选项」的长度不能被 4 整除，要用填充字段补足。

数据偏移 vs IP 首部长度 vs IP 片偏移

三个「偏移」含义完全不同，题目最爱混着考：

- **TCP** 数据偏移：TCP 首部长度，单位 4 字节，最小值 5、最大值 15；
- **IP** 首部长度：IP 首部长度，单位 4 字节，最小值 5、最大值 15；
- **IP** 片偏移：该分片的数据在原始数据报中的相对位置，单位 **8** 字节。

六个标志位

标志位	名称	作用
URG	紧急	置 1 时紧急指针有效，紧急数据「插队」传到接收方并直接上交应用进程，不等接收缓存排队。即使窗口为 0 也可以发送紧急数据
ACK	确认	置 1 时确认号才有效。连接建立后所有报文段都必须置 **1**
PSH	推送	置 1 时接收方尽快交付应用进程，不必等缓存填满。用于交互式通信（如 Telnet）
RST	复位	置 1 表示连接出现严重差错，须释放并重建；也用于拒绝非法报文段或拒绝打开连接
SYN	同步	用于建立连接。**SYN=1** 的报文段不能携带数据，但要消耗一个序号
FIN	终止	置 1 表示数据发完，要求释放连接。**FIN=1** 的报文段即使不携带数据也要消耗一个序号

出题角度

- 「主机 A 的应用进程突然崩溃，随后 A 向 B 发送了一个 TCP 报文段」——该报文段的 **RST=1**（B 收到后释放连接）。
- 「哪个标志位用于拒绝非法的 TCP 报文段」——**RST**。
- 「连接建立后哪个标志位始终为 1」——**ACK**。

窗口、紧急指针、选项与填充

窗口字段：占 2 字节，指出「发送本报文段的一方的接收窗口」大小，即本机接收缓存的可用空间，用来告诉对方「你最多还能给我发多少字节」。这是 TCP 流量控制的实现手段。

紧急指针字段：占 2 字节，以字节为单位指出本报文段数据载荷中紧急数据的长度。紧急数据位于数据载荷的最前面，其后是普通数据。

选项字段：最长 40 字节。常见选项有：

- 最大报文段长度 **MSS** 选项：在连接建立时协商；
- 窗口扩大选项：把窗口字段从 16 位扩展到更大（因为 65535 字节的窗口对高带宽时延积链路太小）；
- 时间戳选项：用于更精确地计算 RTT；
- 选择确认 **SACK** 选项：用于选择性重传。

填充字段：使首部长度能被 4 字节整除。

MSS 与 **MTU** 的关系

三个缩写必须分清：

缩写	全称	所在层	含义
MTU	Maximum Transmission Unit，最大传送单元	数据链路层	一个帧的数据字段能装下的最大字节数
MSS	Maximum Segment Size，最大报文段长度	传输层（TCP）	TCP 报文段数据载荷部分的最大长度，不包括 **TCP** 首部
MSL	Maximum Segment Lifetime，最长报文段寿命	传输层（TCP）	一个 TCP 报文段在网络中最长的生存时间

$$\text{MSS} = \text{MTU} - \text{IP 首部长度} - \text{TCP 首部长度}$$

以最常见的以太网为例：MTU = 1500 B，IP 首部 20 B，TCP 首部 20 B，则

$$\text{MSS} = 1500 - 20 - 20 = 1460\ \text{B}$$

MSS 相关的四个坑

1. **MSS** 只算数据载荷，不算首部。题目说「MSS = 1000 B，发送 3000 B 的报文」，那就是 3 个报文段，数据分别为 1000、1000、1000。
2. **MSS** 与窗口值无关。窗口是「还能收多少字节」，MSS 是「一段最多装多少字节」。
3. **MSS** 选项只在连接建立阶段协商，不在每一个报文段里出现。
4. **MTU** 是链路层的概念，**MSS** 是传输层的概念，问「MSS 在哪一层确定」答传输层（TCP 选项），问「MTU 由谁决定」答数据链路层。

TCP 发送报文段的时机

正常情况下，TCP 什么时候把数据封装成报文段发出去？规则有四条：

1. 发送缓存中存放的数据达到 **MSS** 字节时，就组装成一个 TCP 报文段发送出去；

2. 应用进程指明要求发送（即 TCP 支持的 **PUSH** 推送操作）；
3. 发送方的一个计时器期限到了，即使没达到 MSS 也把当前已有的数据发送出去；
4. 若接收窗口小于 MSS，则应等待接收窗口达到 MSS 的一半或者 MSS，两者任一时机成熟再发送。

Nagle 算法：对于一个 TCP 连接，第一个字节立即发送出去，然后继续缓存要发送的数据；只有收到上一个报文段的确认后，才发送已缓存的数据。这样每轮都要等上一个报文段的数据全部被确认，才发下一个报文段，可以有效地减少小报文段（避免「糊涂窗口综合征」）。

Nagle 算法算例：应用层连续产生 3 次「写」操作，每次 1 字节，准备发给对方（假设每次写之间的间隔很短）。

时刻	事件	发送方动作
t_0	第 1 个字节到达发送缓存	Nagle 算法规定第一个字节立即发送，不等待
t_1	第 2 个字节到达发送缓存	第 1 个字节的确认尚未到达，于是缓存，不发送
t_2	第 3 个字节到达发送缓存	仍然缓存，此时缓存里有 2 字节
t_3	收到第 1 个字节的确认	把缓存中的 2 字节组装成一个报文段发送出去
t_4	又收到确认	缓存为空，等待下一次数据

对比：若不用 Nagle 算法，则可能发出 3 个各含 1 字节数据的报文段，每个都要带 20 字节 TCP 首部 + 20 字节 IP 首部，首部开销是数据的 **40** 倍，效率极低。Nagle 算法把「小报文段问题」从发送侧抑制住了。

两个边界情况：

- 若应用层数据攒够了 MSS，则立即发送，不受 Nagle 算法限制；
- 若应用层设置了 TCP_NODELAY 选项（禁用 Nagle），或使用了 PSH 推送操作，也立即发送。

2.TCP 连接管理

TCP 连接管理 = 连接建立（三报文握手） + 连接释放（四报文挥手） + 状态转换。这是全章最重要的一节，也是历年大题的主要素材。

三报文握手建立连接

设客户（Client）A 主动打开，服务器（Server）B 被动打开（先处于 LISTEN 监听状态）。

第 **1** 次握手（**A** $\rightarrow$ **B**）：A 向 B 发送 **TCP** 连接请求报文段：

$$\text{SYN} = 1, \quad \text{ACK} = 0, \quad \text{seq} = x$$

- 其中 x 是 A 选择的初始序号；
- **SYN** 报文段不能携带数据，但要消耗一个序号；
- A 进入 **SYN-SENT**（同步已发送）状态。

第 **2** 次握手（**B** $\rightarrow$ **A**）：B 若同意建立连接，则向 A 发送 **TCP** 连接请求确认报文段：

$$\text{SYN} = 1, \quad \text{ACK} = 1, \quad \text{seq} = y, \quad \text{ack} = x + 1$$

- y 是 B 独立选择的初始序号；
- `ack = x+1` 表示 B 已正确收到 A 的 SYN（它占了序号 x），期望 A 下一个报文段从 $x+1$ 开始；
- 同样，这个报文段也不能携带数据，也要消耗一个序号；
- B 进入 **SYN-RCVD**（同步已接收）状态。

第 **3** 次握手（**A** $\rightarrow$ **B**）：A 收到 B 的确认后，还要再向 B 发送一个 **ACK** 报文段（普通确认，不带 SYN）：

$$\text{ACK}=1,\quad \text{seq}=x+1,\quad \text{ack}=y+1$$

- `ack = y+1` 表示 A 已正确收到 B 的 SYN（它占了序号 y）；
- A 进入 **ESTABLISHED**（连接已建立）状态；
- B 收到这个 ACK 后也进入 **ESTABLISHED** 状态，连接正式建立。

三报文握手的三个报文段

次序	方向	**SYN**	**ACK**	**seq**	**ack**
1	A→B	1	0	x	无意义
2	B→A	1	1	y	$x+1$
3	A→B	0	1	$x+1$	$y+1$

> 第 **1**、**2** 次握手绝对不能携带数据；第 **3** 次握手可以携带数据（若携带数据则按数据字节数消耗序号，且此时 seq 从 $x+1$ 开始；若不携带数据，则不消耗序号，下一个数据报文段的序号仍是 $x+1$）。

为什么是三次握手，而不是两次

这是必考的一道「原理题」，标准答案的核心是：防止已失效的连接请求报文段突然又传到服务器，造成服务器资源浪费。

设想只用两次握手的情形：

1. A 发出第 1 个连接请求，但在某个网络结点长时间滞留；
2. A 超时后重发连接请求，第 2 个请求顺利到达 B，两次握手完成，连接建立、传送数据、然后释放；
3. 之后，那个滞留已久的第 **1** 个请求报文段终于到达 B；
4. B 收到这个「已失效的连接请求」，误以为 A 又要建立连接，于是发出确认并直接进入连接状态，为这个根本不存在的连接分配资源、等待数据；
5. 而 A 并不知道这件事，不会给 B 发数据。B 的资源就被白白浪费（甚至长期占用）。

三次握手可以杜绝这种情况：B 在第 2 次握手时只是「同意」，并不立即认为连接建立；A 收到 $(\text{SYN}=1,\text{ACK}=1,\text{ack}=x+1)$ 后会发现——我并没有请求建立这个连接呀！于是 A 不发送第 **3** 次握手的确认，B 收不到确认，就会超时释放这个半开的连接，资源不会被长期占用。

补充：三次握手的另一个作用

三次握手还使双方都能确认对方的接收能力和发送能力都正常，并协商初始序号（以及 MSS、窗口扩大因子等参数）。但从统考答题的角度看，第一位的理由是「防止已失效的连接请求报文段造成资源浪费」。

四报文挥手释放连接

TCP 连接是全双工的，因此每个方向都必须单独释放，这就是「四次挥手」的根本原因。设 A 主动关闭，B 被动关闭。

第 **1** 次挥手（**A** $\rightarrow$ **B**）：A 发送连接释放报文段，并停止发送数据：

$$\text{FIN} = 1, \quad \text{seq} = u$$

- **FIN** 报文段即使不携带数据，也要消耗一个序号；
- A 进入 **FIN-WAIT-1**（终止等待 **1**）状态。

第 **2** 次挥手（**B** $\rightarrow$ **A**）：B 收到后发出确认：

$$\text{ACK} = 1, \quad \text{seq} = v, \quad \text{ack} = u + 1$$

- B 进入 **CLOSE-WAIT**（关闭等待）状态；
- A 收到此报文后进入 **FIN-WAIT-2**（终止等待 **2**）状态。

此时连接处于半关闭（**half-close**）状态：A 已经没有数据要发送了，但 **B** 仍然可以继续向 **A** 发送数据，A 也必须继续接收。这个阶段可能持续很久（B 还有大量数据没发完）。

第 **3** 次挥手（**B** $\rightarrow$ **A**）：B 把剩余数据全部发送完毕后，发送连接释放报文段：

$$\text{FIN} = 1, \quad \text{ACK} = 1, \quad \text{seq} = w, \quad \text{ack} = u + 1$$

- 注意这里的 `ack` 仍然是 $u + 1$（因为 A 的 FIN 之后没有新数据，B 不需要更新确认号；这就是所谓「对之前收到的连接释放报文段的重复确认」）；
- B 进入 **LAST-ACK**（最后确认）状态。

第 **4** 次挥手（**A** $\rightarrow$ **B**）：A 收到 B 的 FIN 后发出确认：

$$\text{ACK} = 1, \quad \text{seq} = u + 1, \quad \text{ack} = w + 1$$

- 这个报文段规定不能携带数据，也不消耗序号；
- A 进入 **TIME-WAIT**（时间等待）状态，必须等待 **2MSL** 后才能进入 **CLOSED** 状态；
- B 收到这个 ACK 后立即进入 **CLOSED** 状态。

四报文挥手汇总

次序	方向	**FIN**	**ACK**	**seq**	**ack**	发送方状态
1	A→B	1	1	u	v（无实际意义）	A 进 FIN-WAIT-1
2	B→A	0	1	v	$u+1$	B 进 CLOSE-WAIT，A 进 FIN-WAIT-2
3	B→A	1	1	w	$u+1$	B 进 LAST-ACK
4	A→B	0	1	$u+1$	$w+1$	A 进 TIME-WAIT，B 进 CLOSED

挥手第 **1**、**3** 次（**FIN**）固定消耗一个序号；挥手第 **4** 次的 **ACK** 不携带数据也不消耗序号。

为什么是四次挥手

因为 TCP 连接是全双工的，两个方向的数据传输相互独立，因此「我发完了」与「你发完了」必须分别通知、分别确认：

- A 发 FIN 只表示「**A** 没有数据要发给 **B** 了」，不表示 B 也没有数据要发给 A；
- B 收到 FIN 后必须先回一个 ACK（第 2 次挥手），但 B 可能还有数据没发完，所以不能立刻把自己的 **FIN** 也搭车发过去；
- 必须等 B 的数据全部发完，B 才能发自己的 FIN（第 3 次挥手），A 再确认（第 4 次挥手）。

什么时候可以合并成三次?

如果 B 收到 A 的 FIN 时恰好已经没有数据要发送了，那么 B 可以把第 2 次和第 3 次挥手合并成一个报文段（FIN=1 且 ACK=1）发出。此时总共只有三个报文段，但逻辑上仍然是四次挥手。

这个「可以合并」的假设是许多时间计算题的前提。

为什么需要 **TIME-WAIT** 与 **2MSL**

MSL（**Maximum Segment Lifetime**，最长报文段寿命）：一个 TCP 报文段在网络中最长的生存时间。RFC 793 建议为 2 分钟，实际由操作系统决定，通常在几十秒到几分钟之间。

TIME-WAIT 计时器设置为 **2MSL**，原因有两个：

原因一：确保 **A** 发送的最后一个 **ACK**（第 **4** 次挥手）能够到达 **B**。

如果 A 发完最后一个 ACK 就立即关闭并进入 CLOSED，那么这个 ACK 万一丢失，B 就会一直停留在 LAST-ACK 状态并超时重传 **FIN+ACK**（第 3 次挥手）。此时 A 已经关闭，收不到也回不了这个报文，B 就永远无法正常释放连接。

正确做法是：A 进入 TIME-WAIT 并计时 2MSL。在这段时间内：

- 若 A 又收到了 B 重传的 FIN+ACK，就说明自己刚才发的 ACK 丢了，于是重发 **ACK** 并重新开始 **2MSL** 计时；
- 一个报文段从 A 到 B 最多用 1 MSL，B 重传的报文段从 B 回 A 也最多用 1 MSL，因此 2MSL 足以覆盖「ACK 丢失 + FIN 重传到达」的最坏情况。

原因二：让本次连接中产生的所有报文段都从网络中消失。

各 TCP 报文段到达主机的时间长短不一，A 在收到 B 的 FIN 后，可能还有本次连接中较早发出的报文段仍在网络中游荡。如果 A 立即关闭，然后又马上用相同的四元组建立新连接，这些旧报文段就可能被新连接误收，造成数据混乱。

经过 2MSL（一个报文段一来一回的最大时间），可以保证本次连接产生的所有报文段都已从网络中消失，这时再建立新连接就不会冲突。

TIME-WAIT 的三个易错点

1. 只有主动关闭方进入 **TIME-WAIT**。被动关闭方（先收到 FIN 的一方）收到最后一个 ACK 后立即进入 CLOSED。
2. **TIME-WAIT** 占用的是「端口」而非「连接」：服务器重启时若报「Address already in use」，正是因为 TIME-WAIT 尚未结束。
3. **2MSL** 中的 **MSL** 不是 **RTT**。题目若给「RTT = 50 ms，MSL = 800 ms」，则 2MSL = 1600 ms，与 RTT 无关。

连接释放的最短时间（经典算例）

情形	主动关闭方从发出 **FIN** 到进入 **CLOSED**	被动关闭方从收到 **FIN** 到进入 **CLOSED**
最短（B 无数据要发，第 2、3 次挥手合并）	**1 RTT + 2 MSL**	**1.5 RTT**
一般情形	大于 1 RTT + 2 MSL	大于 1.5 RTT

为什么主动方是 **1 RTT + 2MSL**：FIN 发出后经 0.5 RTT 到达 B，B 立即回 FIN+ACK，再经 0.5 RTT 到达 A，共 1 RTT；A 随后发最后一个 ACK 并进入 TIME-WAIT，等待 2 MSL。

为什么被动方是 **1.5 RTT**：B 收到 FIN 用 0.5 RTT，立刻发 FIN+ACK；这个确认 + FIN 经 0.5 RTT 到达 A，A 发最后一个 ACK；这个 ACK 再经 0.5 RTT 到达 B，B 才进入 CLOSED。合计 $0.5+0.5+0.5=1.5$ RTT。

2022 年第 **39** 题（原文条件）

客户 C 和服务器 S 已建立 TCP 连接，RTT = 50 ms，MSL = 800 ms，C 主动请求断开连接。从 C 发出 FIN 的时刻算起，C 和 S 进入 CLOSED 状态所需的时间至少分别是多少?

解：单向传播时延 = RTT/2 = 25 ms。最短情形下 S 收到 C 的 FIN 后立即发出 ACK 和 FIN（合并为一次）。

- **C**：50 ms 时收到 S 的 FIN+ACK，发出最后一个 ACK 并进入 TIME-WAIT；等待 2MSL = 1600 ms 后关闭。总时间 $=50+1600=$ **1650 ms**。
- **S**：25 ms 时收到 C 的 FIN 并发出 FIN+ACK；75 ms 时收到 C 的最后一个 ACK，随即进入 CLOSED。总时间 = **75 ms**。

答案：**1650 ms**，**75 ms**。

TCP 的状态转换

TCP 连接在其生存期中会处于不同的状态。统考不要求背完整的 11 状态图，但下面这些状态必须能把「谁在哪个状态、接下来往哪走」对应上。

状态	含义	谁进入
CLOSED	关闭（虚拟的起始/终止状态）	双方初始
LISTEN	监听，等待对方的连接请求	被动打开一方（服务器）
SYN-SENT	已发送连接请求，等待对方确认	主动打开一方，发完第 1 次握手
SYN-RCVD	已收到连接请求并发出确认，等待对方确认	服务器，发完第 2 次握手
ESTABLISHED	连接已建立，可以传输数据	双方，握手完成
FIN-WAIT-1	已发送 FIN，等待对方确认	主动关闭方，发完第 1 次挥手
FIN-WAIT-2	已收到对方对自己 FIN 的确认，等待对方的 FIN	主动关闭方，收到第 2 次挥手
CLOSE-WAIT	已收到对方的 FIN 并确认，等待本进程发完剩余数据	被动关闭方，发完第 2 次挥手
LAST-ACK	已发送自己的 FIN，等待对方确认	被动关闭方，发完第 3 次挥手
TIME-WAIT	已发送最后一个 ACK，等待 2MSL	主动关闭方，发完第 4 次挥手
CLOSING	双方同时关闭时的罕见状态（不常考）	罕用

客户端（主动打开 + 主动关闭）的典型路径：

```
CLOSED -> SYN-SENT -> ESTABLISHED -> FIN-WAIT-1 -> FIN-WAIT-2 -> TIME-WAIT -> CLOSED
```

服务器（被动打开 + 被动关闭）的典型路径：

```
CLOSED -> LISTEN -> SYN-RCVD -> ESTABLISHED -> CLOSE-WAIT -> LAST-ACK -> CLOSED
```

状态题的两个陷阱

1. **LISTEN** 是服务器在收请求之前的状态，不是「握手第一步之后」的状态。收到 SYN 之后才进 SYN-RCVD。
2. **FIN-WAIT-2** 是「我已经挥手了，等对方也挥手」，此时连接处于半关闭；CLOSE-WAIT 才是「对方挥手了，我还没挥」。

连接建立与释放中的序号计算（完整例题）

例题 **1**（握手序号，**2019** 年第 **39** 题原型）

主机甲主动发起与主机乙的 TCP 连接，甲、乙选择的初始序号分别为 2018 和 2046。问第三次握手 TCP 段的确认序列号是多少?

- 第 1 次握手：甲 $\rightarrow$ 乙，`SYN=1，seq=2018`。SYN 消耗一个序号，所以甲的数据将从 2019 开始。
- 第 2 次握手：乙 $\rightarrow$ 甲，`SYN=1，ACK=1，seq=2046，ack=2018+1=2019`。
- 第 3 次握手：甲 $\rightarrow$ 乙，`ACK=1，seq=2019，ack=2046+1=2047`。

答案：**2047**。

例题 **2**（握手序号，**2011** 年第 **39** 题原型）

主机甲向主机乙发送一个（`SYN=1，seq=11220`）的 TCP 段请求建立连接。若乙接受，则乙回送的 TCP 段可能是什么?

- 第 2 次握手必须同时置 `SYN=1` 和 `ACK=1`；
- 乙必须确认甲的 SYN：`ack = 11220 + 1 = 11221`；
- 乙自己的初始序号可以独立选择，题目没有限制，取 11221 是可能的。

答案：**（`SYN=1，ACK=1，seq=11221，ack=11221`）**。

例题 **3**（含数据传送与挥手，**2020** 年第 **39** 题原型）

甲、乙建立 TCP 连接时，甲发送的 SYN 段序号为 1000；断开连接时，甲发送给乙的 FIN 段序号为 5001。求在无任何重传的情况下，甲向乙已发送的应用层数据字节数。

- SYN 占用序号 1000，因此第一个应用数据字节的序号是 **1001**；
- FIN 的序号 5001 表示「写完这些数据之后的下一个序号是 5001」，即数据占用到 5000；
- 数据字节数 $= 5001 - 1001 =$ **4000** 字节。（FIN 本身虽消耗一个序号，但位于应用数据之后，不计入数据量。）

答案：**4000 B**。

例题 **4**（含数据传送的序号，**2013** 年第 **39** 题原型）

甲收到乙发来的一个 TCP 段，其 seq=1913、ack=2046、有效载荷 100 字节。求甲立即回送给乙的 TCP 段的序号与确认号。

- 乙的 ack=2046 表示乙期望甲下一次从序号 2046 开始发送，因此甲回发段的 **seq = 2046**；
- 乙本段数据从 1913 开始、长达 100 字节，占用序号 1913~2012，故甲下一步期望收到 2013，甲的 **ack = 2013**。

答案：**seq = 2046**，**ack = 2013**。

例题 **5**（累积确认，**2011** 年第 **40** 题原型）

甲向乙发送 3 个连续的 TCP 段，有效载荷分别为 300、400、500 字节，第 **3** 个段的序号为 **900**。若乙仅正确接收到第 1 和第 3 个段，乙发送给甲的确认序号是多少?

先倒推各段的起始序号：

段	载荷	起始序号	序号范围
第 1 段	300 B	$900-400-300=200$	200 ~ 499
第 2 段	400 B	$900-400=500$	500 ~ 899
第 3 段	500 B	900	900 ~ 1399

乙正确收到第 1 段（200~499）和第 3 段（900~1399），但缺少第 2 段（500~899）。TCP 使用累积确认，只能确认到「连续正确收到的最高序号」，因此乙回送 ack = 500。

答案：**500**。

例题 **6**（连续段与累积确认，**2009** 年第 **38** 题原型）

甲向乙发送两个连续的 TCP 段，有效载荷分别为 300 字节和 500 字节，第一个段的序号为 200。乙正确接收两个段后，发送给甲的确认序列号是多少?

- 第 1 段：seq=200，占 200~499；
- 第 2 段：seq=500，占 500~999；
- 乙已连续正确收到 200~999，故 ack = 1000。

答案：**1000**。

序号计算题的通用步骤

1. 先判断报文段类型：是 SYN、FIN，还是普通数据段?
2. 折算成序号区间：SYN 和 FIN 各占 1 个序号，普通数据段占「字节数」个序号。
3. 先确定起始点，再推终点：若题目给的是「第 k 段的序号」，要先把前面几段的长度加起来倒推出各段起点。
4. 确认号一律取「连续正确收到的最高序号 + **1**」（累积确认）。
5. 检查是否跨过 $2^{32}-1$ 回绕（大数值题要注意，普通题目不会涉及）。

3.TCP 可靠传输

TCP 的可靠传输建立在滑动窗口 + 确认重传之上。它的机制可以看成是后退 **N** 帧（**GBN**）与选择重传（**SR**）的结合：正常情况下用累积确认（像 GBN），配合 SACK 选项时可以只重传丢失的部分（像 SR）。

序号与确认机制

- TCP 是面向字节流的，所以序号以字节为单位连续编号，而不是以报文段为单位。
- 接收方收到按序到达的数据后，返回确认号 ack = 期望收到的下一个字节序号。
- 接收方可以捎带确认（piggybacking）：把确认信息搭载在自己要发送的数据报文段的 ack 字段中。若不发送数据，也可以立即单独发送确认。
- 累计确认（**cumulative acknowledgment**）：ack $= N$ 表示「序号到 $N-1$ 为止的所有数据都已按序正确收到」。即使收到了 N 之后的不连续数据，也不会把 ack 推到 N 之后。

累积确认的后果

正是因为累积确认，「收到 3 个段但缺中间那个」时确认号仍停在缺失段的起点。这也解释了快速重传为什么能工作：接收方每收到一个失序段，就重复发一次相同的 ACK，发送方收到 3 个重复 ACK 就知道对应报文段丢了。

超时重传

基本思想：每发送一个报文段就为该报文段设置一个超时计时器（重传计时器）。若在计时器到期之前收到了确认，就撤销该计时器；否则认为该报文段丢失，重传它。

注意几个细节：

- 发送方发出报文段后必须暂时保留其副本（以备重传），收到确认后才能清除；
- 报文段和确认都要编号，才能知道收到的是对哪一段的确认；
- 若收到两个重复的报文段，说明对方一定发生了超时重传，此时应丢弃较晚收到的那个重复报文段，并再回送一次确认。

超时重传时间 RTO 的选择

RTO 的选择是 TCP 中最复杂的问题之一。

加权平均往返时间 RTT_S：

$$\text{新的}RTT_S = (1-\alpha) \times \text{旧的}RTT_S + \alpha \times \text{新的 RTT 样本}, \qquad 0 \le \alpha < 1$$

第一次测量到 RTT 样本时，直接取 $RTT_{S1} = RTT_1$。RFC 6298 推荐的 $\alpha = 1/8 = 0.125$。α 越接近 0，新样本影响越小；越接近 1，新样本影响越大。

RTT 偏差的加权平均 RTT_D：

$$\text{新的}RTT_D = (1-\beta) \times \text{旧的}RTT_D + \beta \times |RTT_S - \text{新的 RTT 样本}|, \qquad 0 \le \beta < 1$$

第一次测量时 $RTT_{D1} = RTT_1 \div 2$。RFC 6298 推荐的 $\beta = 1/4 = 0.25$。

超时重传时间：

$$\boxed{RTO = RTT_S + 4 \times RTT_D}$$

计算顺序（RFC 6298 规定）

得到一个新的 RTT 样本时，先用「旧的 RTT_S」计算新的 RTT_D，再计算新的 RTT_S。顺序写反结果会不同。

算例：设第 1 次测得 RTT 样本 = 2 个单位。

- $RTT_{S1} = 2$；
- $RTT_{D1} = 2 \div 2 = 1$；
- $RTO_1 = RTT_{S1} + 4 \times RTT_{D1} = 2 + 4 = 6$。

若第 2 次测得 RTT 样本 $= 4$：

- 先用旧的 $RTT_{S1} = 2$ 算 RTT_D：$RTT_{D2} = (1 - \frac{1}{4}) \times 1 + \frac{1}{4} \times |2 - 4| = 0.75 + 0.5 = 1.25$；
- 再算 $RTT_{S2} = (1 - \frac{1}{8}) \times 2 + \frac{1}{8} \times 4 = 1.75 + 0.5 = 2.25$；
- $RTO_2 = 2.25 + 4 \times 1.25 = 2.25 + 5 = 7.25$。

完整演算题（含一次超时样本，必须跳过）

设从第 1 次开始依次测得 RTT 样本为 $RTT_1 = 2$、$RTT_2 = 4$、$RTT_3 = 1.75$，第 4 次测量时发生了超时重传，第 5 次测得 $RTT_5 = 3$。取 $\alpha = 1/8$、$\beta = 1/4$。求每次测量后算出的 RTO。

测量次序	**RTT** 样本	是否采用	计算 RTT_D	计算 RTT_S	**RTO**
第 1 次	2	✓	$RTT_{D1} = 2/2 = 1$	$RTT_{S1} = 2$	$2 + 4 \times 1 =$ **6**
第 2 次	4	✓	$0.75 \times 1 + 0.25 \times \|2 - 4\| = 1.25$	$0.875 \times 2 + 0.125 \times 4 = 2.25$	$2.25 + 4 \times 1.25 =$ **7.25**
第 3 次	1.75	✓	$0.75 \times 1.25 + 0.25 \times \|2.25 - 1.75\| = 1.0625$	$0.875 \times 2.25 + 0.125 \times 1.75 = 2.1875$	$2.1875 + 4.25 =$ **6.4375**
第 4 次	—	×（超时重传）	不计算	不计算	加倍为 $6.4375 \times 2 =$ **12.875**
第 5 次	3	✓	沿用旧的 $RTT_S = 2.1875$：$0.75 \times 1.0625 + 0.25 \times \|2.1875 - 3\| = 1.0$	$0.875 \times 2.1875 + 0.125 \times 3 = 2.2890625$	$2.2890625 + 4 \times 1.0 =$ **6.2890625**

这类题的三条执行纪律

1. 先判断样本是否可用：只要该次报文段发生了重传，这个 RTT 样本就作废，既不算 RTT_D 也不算 RTT_S（Karn 算法）。
2. 算 RTT_D 时用「旧的 RTT_S」，算完 RTT_D 再更新 RTT_S（RFC 6298 规定的顺序）。
3. 超时那一次不是「什么也不做」，而是把 RTO 加倍（指数退避），否则后续会反复超时。

「Karn 算法」在两种语境下的含义要分清（统考经常混着考）

- 严格意义的 **Karn** 算法（RFC 6298 之前的经典表述）：只要报文段被重传了，就不采用其 **RTT** 样本去更新 RTT_S。
- 加上指数退避的修正版（RFC 6298 实际采用）：发生超时重传时，先把 **RTO** 加倍（$RTO \leftarrow 2 \times RTO$），后续若仍超时则继续加倍，直到收到确认。

因此做题时看到「发生超时重传」这句话，要同时做两件事：① 该次的 **RTT** 样本作废；② 当前的 **RTO** 值加倍。看到「根据 Karn 算法不考虑重传报文段的 RTT 样本」这句话，则只需做第 ① 件。

Karn 算法

RTT 测量本身有歧义：当报文段发生重传时，发送方收到一个确认，无法判断这个确认是对原报文段的还是对重传报文段的。

- 若误判为对原报文段的确认，算出的 RTT_S 和 RTO 会偏大，降低传输效率；
- 若误判为对重传报文段的确认，算出的 RTT_S 和 RTO 会偏小，导致后续不必要的大量重传，加重网络负担。

Karn 算法的基本思想：

凡是报文段被重传了，就不采用其 RTT 样本去计算RTT_S 和RTO

Karn 算法的缺陷与修正：若报文段的时延突然大幅增大并长期保持，按 Karn 算法不使用重传段的样本，RTO 就无法更新，会导致报文段被反复重传。

修正方法：报文段每发生一次重传，就把 **RTO** 加倍（即取新 RTO 为旧 RTO 的 2 倍，指数退避）。典型说法是「每重传一次，RTO 增大为原来的 2 倍」。

出题角度

Karn 算法在选择题中通常考「基本思想」：重传的报文段不用于计算 **RTT** 样本；以及「重传时 RTO 加倍」。这两个结论记住即可，不需要推导。

快速重传

前提：接收方要立即发送确认，不要等自己有数据要发时才捎带确认。

规则：发送方一旦收到 **3** 个连续的重复确认（冗余 **ACK**），就立即重传对应的报文段，不等重传计时器超时。

为什么是 3 个？收到 3 个重复确认意味着有 3 个报文段已经离开了网络并停留在接收方缓存中，说明只是个别报文段丢失，网络并没有严重拥塞，因此可以大胆重传。

算例：发送方发送 M_1, M_2, M_3, M_4, M_5，其中 M_2 丢失。

事件	发送方动作	接收方动作
发送 M_1~M_5	正常发送	—
M_1 到达	—	按序，回 `ack=M2` 起点
M_3 到达	—	失序，回 `ack=M2` 起点（第 **1** 个重复确认）
M_4 到达	—	失序，回 `ack=M2` 起点（第 **2** 个重复确认）
M_5 到达	—	失序，回 `ack=M2` 起点（第 **3** 个重复确认）
收到第 3 个重复确认	立即重传 M_2	—
M_2 到达	—	此时 M_1~M_5 全部按序到达，回 `ack=M6` 起点

重复确认的个数

「收到 3 个重复确认」= 加上原本的那一个确认，发送方共收到 **4** 个对同一报文段的确认。题目若说「一连收到 3 个对同一个报文段的重复确认」，指的是冗余的那 **3** 个。这个 3 的计数是高频陷阱。

快速重传的收益：对于个别丢失的报文段，发送方不会出现超时重传，也就不会误认为网络拥塞而把 cwnd 减到 1。实践证明，使用快速重传可以使整个网络的吞吐量提高约 20%。

选择确认 **SACK**

问题：TCP 接收方只能对按序收到的数据中的最高序号给出确认。当发送方超时重传时，接收方之前已经收到、只是未按序到达的数据也会被重传，浪费带宽。

解决：TCP 可以使用选择确认（**Selective Acknowledgment**，**SACK**），RFC 2018 建议标准。

接收方把「已收到但与前面字节流不连续的字节块」的边界信息通过 SACK 选项告诉发送方，发送方就只重传真正缺失的部分，不再重传已经正确到达的数据。

SACK 选项的格式约束：

- 「允许 SACK」选项占 1 字节，SACK 选项长度占 1 字节；
- 指明一个字节块的边界需要 4 字节（因为序号是 4 字节），一个字节块需要两个边界，即 8 字节；
- 因此一个 SACK 选项最多能指明 **4** 个字节块。

SACK 的两个限定

1. **SACK** 只是「通知」，不是「协议规定发送方必须怎么做」。SACK 相关文档并没有规定发送方应如何响应，因此大多数 **TCP** 实现仍然重传所有未被确认的数据块。
2. SACK 仍然保留了原有的确认号字段，即累积确认仍然有效，SACK 只是补充信息。

发送窗口与可连续发送的字节范围

TCP 发送方在没有收到确认的情况下，可以把序号落在发送窗口内的数据依次全部发送出去。

- 发送窗口的左沿：收到新的确认后才向右移动；没收到确认则不动。
- 发送窗口的右沿：通常因为收到新的确认而不断右移；也可能因为对方接收窗口变小而不动。
- **TCP** 标准不赞成把窗口右沿向左收缩（即不允许把已经承诺可以发送的序号重新收回），因为窗口里的数据可能已经发出去了，收缩会引发错误。

发送窗口内的序号集是发送缓存序号集的子集；接收窗口的序号集是接收缓存序号集的子集。

重传次数的计算

规则：TCP 每重传一次，RTO 加倍（指数退避）。所以从第 1 次超时开始，第 n 次重传后的累计等待时间为

$$T_n = RTO \times (1 + 2 + 4 + \cdots + 2^{n-1}) = RTO \times (2^n - 1)$$

算例：设初始 RTO = 1 s，问「从第一次超时算起，经过多少时间发送方将进行第 4 次重传」。

次序	等待时间	累计等待时间
第 1 次重传	1 s	1 s
第 2 次重传	2 s	3 s
第 3 次重传	4 s	7 s
第 4 次重传	8 s	**15 s**

即 $1 \times (2^4 - 1) = 15$ s。

重传次数题的两个坑

1. **RTO** 是每次加倍，不是每次加固定值。若题目只说「RTO=1s，超时后进行第 3 次重传」，答案是 $1+2+4=7$ s。
2. 要分清「报文段数」和「字节数」。例如「发送窗口 5000 B，MSS = 1000 B，其中第 2 段丢失」，则窗口内最多能装 5 个报文段，快速重传后发送方需重传 1 个（在无 SACK 且假定后续段都已到达的情况下），总发送报文段数 = 原发的 5 个 + 重传的 1 个。计算时要先列出每段占用的序号区间，再逐段判断是否重传。

4.TCP 流量控制

流量控制的含义

流量控制（**flow control**）的目的：让发送方发送数据的速率不要太快，使接收方来得及接收。

- 它是点对点、端到端的问题（接收端控制发送端），与拥塞控制（全局性问题，涉及所有主机和路由器）不同；
- TCP 不是靠「速率」，而是靠滑动窗口来实现：接收方在返回的确认报文段中，用窗口字段告诉发送方「我还能收多少字节」。

流量控制与拥塞控制的区别（必背）

- 流量控制：抑制发送端速率，使接收端来得及接收。点对点、端到端问题，由接收窗口 rwnd 实现。
- 拥塞控制：防止过多数据注入网络，使网络中的路由器和链路不过载。全局性问题，涉及所有主机、所有路由器，由拥塞窗口 cwnd 实现。

发送窗口的确定

$$\boxed{\text{发送窗口} swnd = \min(rwnd,\ cwnd)}$$

- $rwnd$（receiver window）：接收方通过 TCP 首部窗口字段通告的接收窗口大小，取决于接收缓存的可用空间；
- $cwnd$（congestion window）：发送方维护的拥塞窗口，取决于网络的拥塞程度。

若题目不考虑拥塞控制（如 2010 年第 39 题「题目未给出拥塞窗口」），则取 $swnd = rwnd$。

发送缓存与接收缓存

发送缓存里存放：

1. 发送方应用程序交给 TCP 准备发送的数据；
2. TCP 已发送出去但尚未收到确认的数据。

接收缓存里存放：

1. 按序到达但尚未被应用进程取走的数据；
2. 未按序到达的数据。

注意：发送窗口 $\subseteq$ 发送缓存，接收窗口 $\subseteq$ 接收缓存。当接收方通告 `rwnd = 0` 时，发送方必须停止发送普通数据。

窗口滑动与序号计算的完整算例

设 A、B 已建立 TCP 连接。B 通告接收窗口 `rwnd = 400` 字节，A 的发送窗口也设为 400 字节。A 已有从序号 1 开始的大批数据要发送。

初始状态：

项目	值
发送窗口	seq 1 ~ 400
已发送未确认	（空）
可发送未发送	1 ~ 400

步骤 **1**：A 连续发送 4 个 100 字节的报文段（把窗口用完）。

报文段	序号	状态
①	1 ~ 100	已发送，未确认
②	101 ~ 200	已发送，未确认
③	201 ~ 300	在传输中丢失
④	301 ~ 400	已发送，失序到达

此时发送窗口仍为 1~400，但窗口内已无可发送的字节。

步骤 **2**：B 收到 ①②，接着收到失序的 ④（301~400）。B 先按序确认 `ack = 201`；再对失序的 ④ 立即回送一个重复确认，仍然是 `ack = 201`，并通告 `rwnd = 300`。

- `ack = 201` 表示 A 可以把发送窗口左沿推进到 201；
- `rwnd = 300` 表示 B 还能收 300 字节，所以窗口右沿为 $201 + 300 - 1 = 500$；
- 新发送窗口为 **seq 201 ~ 500**。

项目	值
发送窗口	201 ~ 500
已发送未确认	201 ~ 400（③ 丢失，④ 失序到达）
可发送未发送	401 ~ 500（共 100 字节）

步骤 **3**：A 在新窗口内发送序号 401~500 的报文段（100 字节）。

步骤 **4**：B 收到 401~500。因为 201~300 这一段仍然缺失，401~500 仍是失序到达，B 立即再回送一次重复确认 `ack = 201`。

此时 A 累计收到了对 201 的确认：

收到的确认	性质	累计重复确认数
`ack = 201`	①② 按序到达后的正常确认	0
`ack = 201`	对失序段 ④（301~400）的重复确认	**1**
`ack = 201`	对失序段（401~500）的重复确认	**2**

重复确认只有 **2** 个，尚未达到快速重传的触发条件（需要 **3** 个），所以 A 只能等超时。

步骤 **5**：A 的 ③ 号报文段重传定时器超时，重传 201~300。B 收到后凑齐了 201~500，返回 `ack = 501`。

- A 收到 `ack = 501` 后，窗口左沿推进到 501；
- 若此时 B 通告 `rwnd = 0`，则窗口右沿变为 500，即窗口收缩为零。

步骤 **6**（零窗口）：A 不能再发送普通数据，但必须启动持续计时器。计时器超时后，A 发送一个零窗口探测报文段（仅携带 1 字节数据），B 收到后回送确认并通告现在的接收窗口值：

- 若仍为 0，A 重新启动持续计时器，重复探测；
- 若变为 300，则 A 的发送窗口变为 501 ~ 800，可以继续发送，死锁局面被打破。

变体：若 A 把发送窗口做得更大（例如序号 1~800），则在 ③ 丢失后，失序到达的每一段都会让 B 再回送一个重复确认，凑满 **3** 个重复确认就会触发快速重传，A 立刻重传 201~300，不必等超时。这也说明：重复确认是靠失序报文段一个攒出来的，窗口越大越容易触发快速重传。

窗口计算的三步法

1. 窗口左沿 = 收到的最新确认号 ack；
2. 窗口大小 = 通告的 rwnd（若不考虑拥塞控制）；若考虑拥塞控制则取 $\min(rwnd, cwnd)$；
3. 窗口右沿 = 左沿 + 窗口大小 − 1；
4. 还能发送的字节数 = 窗口右沿 − 已发送未确认数据的最大序号。

要点：已发送但未收到确认的数据仍然占用发送窗口。

这个算例告诉你两件事

1. 窗口是随每个通告值动态调整的：同一个发送方，窗口先是 1~400，然后是 201~500，最后是 501~800。每次调整都要用「新 ack 作左沿、新 rwnd 定大小」重新算，不能沿用旧窗口。
2. 「已发送未确认」的数据始终占着窗口：步骤 2 中虽然窗口左沿推到了 201、窗口大小变成 300，但 201~400 这一段仍然被占用，所以新窗口里能发的只有 401~500 这 100 字节。这一步算错，整题的答案就全错。

零窗口与持续计时器

死锁问题：接收方 B 的接收缓存满了，于是通告 `rwnd=0`。A 收到后停止发送，等待 B 的非零窗口通知。但如果：

- B 的缓存腾出空间后发出的非零窗口通知丢失；
- 则 A 一直等 B 的通知，B 一直等 A 发数据——双方永远互相等待，形成死锁，吞吐量降到 0。

解决办法：持续计时器（**persistence timer**）。

- 只要 TCP 连接的一方收到对方的零窗口通知，就启动持续计时器；
- 持续计时器超时后，发送一个零窗口探测报文段（只携带 1 字节数据）；
- 对方在确认这个探测报文段时，给出自己当前的接收窗口值；
- 若接收窗口仍为 0，收到确认的一方重新启动持续计时器；
- 若接收窗口不为 0，死锁局面就被打破。

两个容易困惑的问题：

1. 接收窗口为 **0** 时，**B** 还能接收报文段吗？能。TCP 规定：即使接收窗口值为 **0**，也必须接受零窗口探测报文段、确认报文段以及携带有紧急数据的报文段。

2. 零窗口探测报文段丢了怎么办？它也有重传计时器，超时后会被重传，所以死锁仍能被打破。

出题角度

「零窗口探测报文段携带多少数据」——**1** 字节；「由谁发起」——收到零窗口通知的一方；「持续计时器的作用」——打破零窗口造成的互相等待死锁。

流量控制与拥塞控制的结合算例

2010 年第 **39** 题原型：甲、乙已建立 TCP 连接，MSS = 1000 B。甲当前拥塞窗口为 4000 B，甲向乙连续发送两个最大段后，收到乙发送的第一个段的确认段，确认段中通告的接收窗口大小为 2000 B。问此时甲还可以向乙发送的最大字节数是多少?

解：

- 发送窗口 $= \min(cwnd,\ rwnd) = \min(4000,\ 2000) = 2000$ B；
- 甲已发送 2 个段（共 2000 B），其中只有第 1 个段被确认，第 **2** 个段（**1000 B**）仍未被确认，仍占用发送窗口；
- 因此还能发送的最大字节数 $= 2000 - 1000 =$ **1000** B。

答案：**1000 B**。

2016 年第 **41** 题原型（窗口与拥塞控制结合）：H3 与 S 建立 TCP 连接，S 分配 20 KB 接收缓存，MSS = 1 KB，RTT = 200 ms，H3 初始序号 100，拥塞窗口初始阈值 32 KB，S 端接收缓存只存不取。问：

(1) 第二次握手段的 **SYN**、**ACK** 值与确认序号?

`SYN = 1, ACK = 1, ack = 100 + 1 = 101`。

(2) H3 收到的第 **8** 个确认段通告的接收窗口是多少？此时拥塞窗口和发送窗口各是多少?

项目	计算
S 已收到数据	8×1 KB $= 8$ KB
剩余接收缓存（rwnd）	$20 - 8 =$ **12** KB
cwnd（慢开始，每确认 1 段 +1 MSS）	$1 + 8 =$ **9** KB
发送窗口	$\min(12,\ 9) =$ **9** KB

(3) 发送窗口等于 **0** 时，下一个待发送的数据段序号是多少?

H3 的第一个数据字节序号为 $100 + 1 = 101$。接收缓存装满 20 KB，即 $20 \times 1024 = 20480$ 字节。数据占用序号 $101 \sim 101 + 20480 - 1 = 20580$。因此下一个待发送数据段的序号为 **20581**。

(4) 从 **t** 时刻 **H3** 请求断开连接起，**S** 释放该连接的最短时间是多少?

S 是被动关闭方，最短为 **1.5 RTT** $= 1.5 \times 200 =$ **300** ms。

这类题的通用陷阱

1. 不要忘记「已发送未确认」的数据仍占窗口。这是 2010 年题的关键。
2. 接收缓存「只存不取」时，**rwnd** 单调减少，要用「总缓存 − 已收数据」计算，不能直接用初始值。
3. **cwnd** 每次增加的是「**1 MSS**」而不是「**1 KB**」，在 MSS 不等于 1 KB 时要换算。
4. 时间问的是「谁」释放连接：主动方看 1RTT + 2MSL，被动方看 1.5RTT。

5.TCP 拥塞控制

拥塞控制的基本概念

拥塞（**congestion**）：网络中注入的分组过多，超出了网络资源（链路带宽、路由器处理能力、缓存）所能处理的能力，导致路由器队列堆积、分组丢失、吞吐量反而下降。

拥塞控制：防止过多的数据注入网络，使网络中的路由器和链路不致过载。这是一个全局性的过程，涉及所有主机、所有路由器和与降低网络传输性能有关的所有因素。

讨论前提（这三条是教材约定的简化条件，题目通常默认成立）：

1. 数据单方向传送，另一个方向只传送确认；
2. 接收方总是有足够大的接收缓存，因此发送窗口仅由网络拥塞程度决定，不考虑流量控制；
3. 以 **MSS** 的个数作为讨论单位（而不是字节）。

拥塞窗口 **cwnd**：由发送方维护，其值取决于网络的拥塞程度，动态变化。四种拥塞控制算法本质上都是在控制 cwnd 的增长与下降。

判断网络出现拥塞的依据：没有按时收到应当到达的 TCP 确认报文段，而产生了超时重传。

$$swnd = \min(cwnd,\ rwnd)$$

- 不考虑拥塞控制时：$swnd = rwnd$；
- 不考虑流量控制时：$swnd = cwnd$。

慢开始门限 **ssthresh** 的使用规则：

- 当 $cwnd < ssthresh$ 时，使用慢开始算法；
- 当 $cwnd > ssthresh$ 时，停止慢开始，改用拥塞避免算法；
- 当 $cwnd = ssthresh$ 时，既可使用慢开始，也可使用拥塞避免（统考通常按「可以使用慢开始」处理，即在 $cwnd = ssthresh$ 时仍翻倍到 $2 \times ssthresh$，或按题目约定）。

$cwnd = ssthresh$ 时的分歧

不同教材/实现对这个边界处理不同：

- **RFC 5681** 推荐：$cwnd < ssthresh$ 用慢开始，$cwnd \geq ssthresh$ 用拥塞避免（即等于门限时用拥塞避免，每 RTT 只 +1 MSS）；
- 有的教材写「$cwnd = ssthresh$ 时两种算法都可以」。

统考真题通常取「翻倍到 **2**×**ssthresh** 之后再转拥塞避免」这一种（例如 2009 年题 39，$ssthresh = 8$，$cwnd$ 经 1→2→4→8→**9**，即第 4 个 RTT 只 +1）。做题时优先看题目是否给了阶段提示；若没有，按以下稳妥流程：只要 $cwnd$ 已经达到 $ssthresh$，下一轮就 **+1**。本章所有演算均按此约定。

慢开始与拥塞避免

慢开始（**slow start**）算法：

- 初始时 $cwnd = 1$ MSS；
- 每收到一个对新报文段的确认，就把 **cwnd** 增加 **1 MSS**；
- 于是每经过一个传输轮次（即 1 个 RTT），cwnd 就翻倍（指数增长）。

「慢开始」的「慢」并不是指 cwnd 增长速率慢，而是指刚开始时 **cwnd** 设为 **1**，比一上来就向网络注入大量报文段「慢」得多。

拥塞避免（**congestion avoidance**）算法：

- 每收到一个对新报文段的确认，就把 **cwnd** 增加 **1/cwnd** 个 **MSS**；
- 于是每经过一个传输轮次（1 个 RTT），cwnd 增加 **1** 个 **MSS**（线性增长，称为「加法增大」）。

两种阶段的对比（必背）

阶段	每收到 **1** 个新确认，**cwnd** 增加	每经过 **1** 个 **RTT**，**cwnd** 增加
慢开始	1 MSS	翻倍（$\times 2$）
拥塞避免	$1/cwnd$ 个 MSS	$+1$ MSS

快重传与快恢复

快重传（**fast retransmit**）：

- 前提：接收方不要等自己发送数据时才捎带确认，而要立即发送确认，即使收到失序报文段也要立即发出对已收到报文段的重复确认；
- 规则：发送方一旦收到 **3** 个连续的重复确认，就立即重传对应的报文段，不等重传计时器超时。

快恢复（**fast recovery**）：

- 发送方一旦收到 3 个重复确认，就知道只是丢失了个别报文段，而不是网络拥塞，于是不启动慢开始，而是执行快恢复；
- 规则：把 $ssthresh$ 和 $cwnd$ 都调整为当前 **cwnd** 值的一半，然后直接开始执行拥塞避免算法。

另一种实现：把 $cwnd$ 再增大一些，即 $cwnd =$ 新$ssthresh + 3$。理由：既然收到了 3 个重复确认，就说明有 3 个数据报文段已经离开了网络、停留在接收方缓存中，网络中少了 3 个报文段，可以适当把 cwnd 增大一些。（本章演算统一采用第一种：$cwnd = ssthresh =$ 原$cwnd/2$。）

两种拥塞事件的不同处理（核心对比表，必背）

情况	触发条件	**ssthresh** 更新	**cwnd** 更新	后续阶段
超时重传	重传计时器超时，未收到确认	$ssthresh =$ 当前$cwnd \div 2$	$cwnd = 1$ MSS	先慢开始，到达门限后转拥塞避免
三个重复确认（快重传）	收到 3 个重复 ACK	$ssthresh =$ 当前$cwnd \div 2$	$cwnd =$ 新$ssthresh$	直接进入拥塞避免，不经过慢开始

超时与三重确认的最大区别

- 超时：$cwnd$ 直接降到 **1**（回到慢开始，重新指数增长）；
- 三重确认：$cwnd$ 只降到一半（直接进拥塞避免，线性增长）。

两者的 $ssthresh$ 更新规则相同，都是「当前 cwnd 的一半」。判断分支用哪一个，就看去重传的原因。

cwnd 逐轮次演算表（含两种情况）

场景设定（为了表格干净，约定初始 $cwnd = 1$ MSS，初始 $ssthresh = 16$ MSS，MSS = 1 KB）：

- RTT1~RTT6：正常，无丢失；
- **RTT7** 结束时刻发生超时重传（对应 cwnd 为 32 时超时，$ssthresh$ 更新为 16）；
- RTT8~RTT15：恢复正常；
- **RTT16** 结束时刻收到 **3** 个重复确认（快重传 + 快恢复）。

逐轮次演算表：

轮次	轮次开始时的 **cwnd（MSS）**	阶段	本事件	**ssthresh（MSS）**	轮次结束时的 **cwnd（MSS）**
RTT1	1	慢开始	—	16	2
RTT2	2	慢开始	—	16	4
RTT3	4	慢开始	—	16	8
RTT4	8	慢开始	—	16	16
RTT5	16	拥塞避免	—	16	17
RTT6	17	拥塞避免	—	16	18
RTT7	18	拥塞避免	（按设定此处超时，cwnd = 18）	$18/2 = 9$	**1**
RTT8	1	慢开始	超时后重启	9	2
RTT9	2	慢开始	—	9	4
RTT10	4	慢开始	—	9	8
RTT11	8	慢开始	—	9	**16**
RTT12	16	拥塞避免	—	9	17
RTT13	17	拥塞避免	—	9	18
RTT14	18	拥塞避免	—	9	19
RTT15	19	拥塞避免	—	9	20
RTT16	20	拥塞避免	收到 **3** 个重复确认	$20/2 = 10$	**10**
RTT17	10	拥塞避免	快恢复后直接进拥塞避免	10	11
RTT18	11	拥塞避免	—	10	12
RTT19	12	拥塞避免	—	10	13
RTT20	13	拥塞避免	—	10	14

表格的几个关键点：

1. **RTT4** 结束时 **cwnd = 16 = ssthresh**，进入「等于门限」的边界；
2. **RTT5** 开始时 **cwnd = 16**，按「已达门限就 **+1**」的约定只增加到 **17**（若按翻倍约定则为 32）——做题时务必先看题目的阶段提示；
3. **RTT7** 超时：$ssthresh = 18/2 = 9$，$cwnd$ 立即降为 1，是瞬时完成的状态变更，不占一个 RTT；

4. **RTT8~RTT10** 慢开始使 cwnd 依次为 2、4、8；RTT11 开始时 cwnd = 8 < ssthresh = 9，仍属慢开始，结束时翻倍到 16；
5. RTT12 开始 cwnd = 16 > ssthresh = 9，转入拥塞避免，每轮 +1；
6. **RTT16** 收到 **3** 个重复确认：$ssthresh = 20/2 = 10$，$cwnd = 10$，直接进拥塞避免（RTT17 只 +1 到 11，而不是降到 1）。

从表里读出三条规律

1. 慢开始阶段：每一轮的 cwnd 是上一轮的 2 倍；
2. 拥塞避免阶段：每一轮的 cwnd 是上一轮 +1；
3. 每遇到一次拥塞事件（超时或三重确认），**ssthresh** 就变成当时 **cwnd** 的一半，然后：
   - 超时 → cwnd 从 1 重新开始；
   - 三重确认 → cwnd 从新 ssthresh 开始。

cwnd 随传输轮次变化的曲线描述

把上面的表格画成「横轴为传输轮次、纵轴为 cwnd」的曲线，形状可以这样描述：

1. 第 **1** 段（**RTT1~RTT4**）：从 $(1,1)$ 出发，陡峭地指数上升，依次经过 $(2,2)$、$(3,4)$、$(4,8)$、$(5,16)$。曲线呈上凸的指数形状；这一段叫慢开始。
2. 第 **2** 段（**RTT5~RTT7**）：从 cwnd = 16 开始，变成平缓的线性上升，依次 $(5,16)$、$(6,17)$、$(7,18)$。斜率很小（每轮 +1）；这一段叫拥塞避免。
3. 超时点（**RTT7** 结束）：曲线垂直断崖式下跌到 cwnd = 1。注意 $ssthresh$ 同时被更新为原 cwnd 的一半（9），画图时要把门限线也画下来。
4. 第 **3** 段（**RTT8~RTT11**）：从 1 又开始指数上升（1→2→4→8→16）。注意因为 $ssthresh$ 已经变小（9），这段指数上升比第 1 段更早被门限截住。
5. 第 **4** 段（**RTT12~RTT16**）：越过门限后转为线性上升（16→17→18→19→20）。
6. 三重确认点（**RTT16** 结束）：曲线只下降到新门限的一半位置（从 20 降到 10），而不是掉到 1。这是与超时最直观的图形差别。
7. 第 **5** 段（**RTT17** 起）：从 10 开始线性上升（10→11→12→13→14），没有指数段。

曲线三个视觉特征

- 慢开始段是上凸的指数曲线（翻倍）；
- 拥塞避免段是斜率极小的直线（+1）；
- 超时是「掉到 **1**」的深谷，三重确认是「掉到一半」的浅谷。

经典计算题完整解析

题目一（**2009** 年第 **39** 题原型）：一个 TCP 连接总是以 1 KB 的最大段长发送 TCP 段，发送方有足够多的数据要发送。当拥塞窗口为 16 KB 时发生了超时，如果接下来的 4 个 RTT 时间内 TCP 段的传输都是成功的，那么当第 4 个 RTT 时间内发送的所有 TCP 段都得到肯定应答时，拥塞窗口大小是多少?

解：超时处理：$ssthresh = 16/2 = 8$ KB，$cwnd = 1$ KB。

轮次	轮次开始时 **cwnd**	阶段	轮次结束时 **cwnd**
第 1 个 RTT	1 KB	慢开始	2 KB

轮次	轮次开始时 **cwnd**	阶段	轮次结束时 **cwnd**
第 2 个 RTT	2 KB	慢开始	4 KB
第 3 个 RTT	4 KB	慢开始	8 KB
第 4 个 RTT	8 KB	因 $cwnd = 8 = ssthresh$，转拥塞避免	**9 KB**

答案：**9 KB**。

题目二（**2014** 年第 **38** 题原型）：甲、乙已建立 TCP 连接，$MSS = 1$ KB，甲一直有数据发送；乙每收到一个数据段都会发出一个接收窗口为 10 KB 的确认段。若甲在 t 时刻发生超时时拥塞窗口为 8 KB，求从 t 时刻起不再发生超时的情况下，经过 10 个 RTT 后甲的发送窗口。

解：超时处理：$ssthresh = 8/2 = 4$ KB，$cwnd = 1$ KB。

轮次	轮次开始 **cwnd**	阶段	轮次结束 **cwnd**
1	1 KB	慢开始	2 KB
2	2 KB	慢开始	4 KB
3	4 KB	已达门限，转拥塞避免	5 KB
4	5 KB	拥塞避免	6 KB
…	…	每轮 +1	…
10	11 KB	拥塞避免	12 KB

10 个 RTT 后 $cwnd = 12$ KB。但接收窗口始终为 10 KB，所以

$$swnd = \min(12,\ 10) = \textbf{10 KB}$$

答案：**10 KB**。

题目三（**2022** 年第 **38** 题原型）：$MSS = 1$ KB，甲一直有数据向乙发送，当甲的拥塞窗口为 16 KB 时计时器超时，则甲的拥塞窗口再次增长到 16 KB 所需要的最少时间是多少（以 RTT 计）?

解：$ssthresh = 16/2 = 8$ KB，$cwnd = 1$ KB。

轮次	轮次开始时 **cwnd**	阶段	轮次结束后 **cwnd**
1	1 KB	慢开始	2 KB
2	2 KB	慢开始	4 KB
3	4 KB	慢开始	8 KB
4	8 KB	转拥塞避免	9 KB
5	9 KB	拥塞避免	10 KB
6	10 KB	拥塞避免	11 KB
7	11 KB	拥塞避免	12 KB
8	12 KB	拥塞避免	13 KB
9	13 KB	拥塞避免	14 KB

轮次	轮次开始时 **cwnd**	阶段	轮次结束后 **cwnd**
10	14 KB	拥塞避免	15 KB
11	15 KB	拥塞避免	**16 KB**

从 1 KB 涨回 16 KB：慢开始走 3 轮到达 8 KB，之后拥塞避免每轮 +1 KB 需要 8 轮，共 $3+8=$ **11** RTT。

答案：**11 RTT**。

题目四（**2017** 年第 **39** 题原型）：甲向乙发起 TCP 连接，$MSS=1$ KB，RTT = 5 ms，乙开辟的接收缓存为 64 KB。求甲从连接建立成功到发送窗口达到 32 KB 所需的最少时间。

解：接收窗口 64 KB 不构成限制（32 < 64）。慢开始阶段每 RTT 翻倍：

$$1 \to 2 \to 4 \to 8 \to 16 \to 32\ (\text{KB})$$

从 1 KB 到 32 KB 需要 **5** 个 **RTT**，最少时间 $=5\times5=$ **25** ms。

题目五（**2020** 年第 **38** 题原型）：$MSS=1$ KB，RTT = 2 ms，在不出现拥塞的前提下，拥塞窗口从 8 KB 增长到 32 KB 所需的最长时间是多少?

解：求最长时间应按增长较慢的拥塞避免（每 RTT +1 KB）计算。

$$32-8=24\ \text{KB} \Rightarrow 24\ \text{个 RTT},\qquad 24\times2=\textbf{48}\ \text{ms}$$

答案：**48 ms**。

题目六（**2015** 年第 **39** 题原型）：甲、乙新建 TCP 连接，甲的拥塞控制初始阈值为 32 KB，甲以 $MSS=1$ KB 的段发送数据，乙为该连接分配 16 KB 接收缓存，对每个数据段确认，忽略段传输延迟。若乙收到的数据全部存入缓存不被取走，则甲从连接建立成功起，在未超时的情况下经过 4 个 RTT 后，甲的发送窗口是多少?

解：初始 $cwnd=1$ KB，$ssthresh=32$ KB，全程处于慢开始。

轮次	发送的段数	累计写入接收缓存
第 1 个 RTT	1	1 KB
第 2 个 RTT	2	3 KB
第 3 个 RTT	4	7 KB
第 4 个 RTT	8	**15 KB**

经过 4 个 RTT 后共写入 15 KB，接收缓存还剩 $16-15=1$ KB，即 $rwnd=1$ KB。此时 $cwnd=16$ KB。

$$swnd=\min(16,\ 1)=\textbf{1 KB}$$

答案：**1 KB**。

题目七（**2023** 年第 **47** 题原型）：主机 H 从 FTP 服务器下载一个 18000 B 的文件 F，建立数据连接时初始序号为 100，$MSS=1000$ B，拥塞控制初始阈值为 4 MSS，RTT = 10 ms，忽略 TCP 传输时延，全程无差错、无丢包乱序。

(1) 文件第一个字节的序号是多少?

初始序号 100 被 SYN 占用，所以第一个数据字节序号为 $100+1=$ **101**。

(2) 断开数据连接时，**FTP** 发送的第二次挥手的 **ACK** 序号是多少?

18000 B 数据占用序号 101 ~ 18100，随后发送的 FIN 序号为 18101。服务器在第二次挥手中确认该 FIN，故 `ack = 18101 + 1 = 18102`。

(3) 当 **H** 收到确认序号为 **2101** 的确认段时，**cwnd** 是多少? 收到 **7101** 时呢?

初始 $cwnd=1$ MSS，$ssthresh=4$ MSS。慢开始阶段每收到一个新确认 +1 MSS:

确认号	累计确认的数据段数	**cwnd**
1101	1	2 MSS
2101	2	**3 MSS**
3101	3	4 MSS（$=ssthresh$，之后转拥塞避免）
4101	4	5 MSS（拥塞避免，+1）
…	…	每 1000 B 数据 +1 MSS
7101	7	**8 MSS**

所以收到 `ack=2101` 时 $cwnd=$ **3 MSS**；收到 `ack=7101` 时 $cwnd=$ **8 MSS**（因为 7 个段被确认，其中前 3 个走慢开始到 4 MSS，之后 4 个走拥塞避免，$4+4=8$）。

这一段要特别小心

从「cwnd = 4 MSS 时转拥塞避免」之后，每收到 **1** 个新确认只 **+1 MSS**（在 MSS = 1 段的讨论单位下，拥塞避免每收到一个确认增加 $1/cwnd$ 个 MSS，但按「每 RTT +1」的离散轮次来处理更稳妥）。做题时一定要先判断当前收到第几个新确认，再决定是「翻倍」还是「+1」。

拥塞控制与流量控制的区别（第四遍强调）

对比项	流量控制	拥塞控制
目的	使接收方来得及接收	使网络不过载
范围	点对点、端到端（接收端 → 发送端）	全局性（涉及所有主机、所有路由器）
实现手段	接收窗口 rwnd	拥塞窗口 cwnd
是否 TCP 独有	UDP 完全没有	UDP 完全没有
组合关系	$swnd=\min(rwnd,\ cwnd)$	同左

传输层滑动窗口与数据链路层滑动窗口的区别

这一对比虽然不在考纲里，但选择题常用来设混淆项，务必分清。

对比项	数据链路层（**GBN / SR**）	传输层（**TCP**）
窗口单位	帧（或比特）	字节（MSS 只是讨论单位）
窗口大小含义	最多连续发送/接收多少个帧	允许发送的字节序号区间
滑动方式	按帧序号滑动	按字节序号滑动

对比项	数据链路层（**GBN / SR**）	传输层（**TCP**）
确认语义	GBN 中 ACK n = 到 n 为止已收到，期望 $n+1$；SR 中逐帧确认	ack $= N$ = 到 $N-1$ 为止已收到，期望 N
是否考虑字节大小	只关心帧数，与帧长无关	必须关心字节数，窗口值 × MSS 才有物理意义
是否有拥塞控制	无（点对点链路不存在网络范围拥塞问题）	有（cwnd）

最容易踩的一条

题目里常说「窗口是 400」，一定要先看单位：TCP 里 400 表示 **400** 字节；若题目同时给出 MSS = 100 B，则窗口内可以装 4 个报文段；若题目说的是「窗口为 400 个报文段」，那就是 400 × MSS 字节。单位搞错，整道题全错。

本章四类计算题的解题路线图

题型	识别特征	解题路线
序号/确认号推算	「第三个报文段的序号」「乙收到的确认号」	① 判断报文段类型（SYN / FIN / 数据）→ ② 折算成序号区间 → ③ 倒推各段起点 → ④ 确认号取「连续收到的最高序号 + 1」
窗口推算	「甲的发送窗口还能发送多少字节」「可连续发送的字节序号范围」	① 求窗口大小 = min$(rwnd, cwnd)$ → ② 左沿 = 最新 ack → ③ 右沿 = 左沿 + 窗口大小 − 1 → ④ 还要发多少 = 右沿 − 已发送未确认的最大序号
拥塞窗口逐轮演算	「经过 n 个 RTT 后 cwnd / 发送窗口是多少」	① 先处理拥塞事件：超时 → $ssthresh = cwnd/2$、$cwnd = 1$；三重确认 → $ssthresh = cwnd/2$、$cwnd = ssthresh$ → ② 逐轮判断当前处于慢开始（×2）还是拥塞避免（+1）→ ③ 最后用 min$(rwnd, cwnd)$ 求发送窗口
时间估算	「至少需要多长时间」「从发出 FIN 起多久关闭」	① 建立连接 1 RTT → ② 数据传送按 cwnd 演算出的轮次数 × RTT → ③ 释放：主动方 1RTT + 2MSL，被动方 1.5RTT → ④ 全部相加

这一章在统考里的出题规律

- 选择题（每年 **1~2** 道）：UDP 校验、端口号、标志位含义、序号/确认号、每次拥塞事件后的 cwnd、TIME-WAIT 与 2MSL、RTO 与 Karn 算法。
- 综合应用题（约每 **2~3** 年一次，**9** 分左右）：把 FTP/HTTP 之类的应用层背景 + TCP 连接建立与释放的序号 + 拥塞窗口逐轮 + 发送时间估算打包在一起，有时还掺入 IP、以太网最小帧长的计算。做这类题的关键不是记公式，而是能把序号、窗口、时间三条线分开算，各算各的再汇总。

本章小结表

考点	核心结论
传输层功能	进程到进程（端到端）的逻辑通信；复用（多个进程共用一个协议）与分用（按端口交付进程）

考点	核心结论
与网络层区别	网络层：主机到主机，按协议字段分用；传输层：进程到进程，按端口号分用
端口号	**16** 位，0 ~ 65535；只有本地意义；TCP 与 UDP 端口号相互独立
三类端口	熟知 **0 ~ 1023**；登记 **1024 ~ 49151**；短暂（客户端）**49152 ~ 65535**
套接字	socket = (IP 地址: 端口号)；TCP 连接由两个套接字唯一确定
必须记住的端口	FTP 21/20、SSH 22、Telnet 23、SMTP 25、DNS 53、DHCP 67/68、TFTP 69、HTTP 80、POP3 110、IMAP 143、HTTPS 443、SNMP 161、RIP 520
UDP 特点	无连接、不可靠、面向报文、支持一对一/一对多/多对多、首部仅 **8 B**、无流量与拥塞控制
UDP 首部	四个字段各 2 B：源端口、目的端口、长度（首部 + 数据）、校验和；长度最小为 8
UDP 校验	覆盖 **12 B** 伪首部 + **8 B** 首部 + 数据（不足补全 0）；反码求和 + 回卷进位 + 取反；全 0 表示不校验
伪首部	源 IP、目的 IP、全 0、协议号（**UDP 17 / TCP 6**）、长度；不传送、不递交，只用于校验
TCP 特点	面向连接、可靠、全双工、面向字节流、只能一对一
TCP 首部	固定 **20 B** + 选项最多 40 B = 最长 **60 B**；数据偏移以 **4 B** 为单位，最小 5
序号含义	本报文段数据第一个字节的序号
确认号含义	期望收到的下一个字节序号；表示到 **ack−1** 为止的数据已正确收到
序号消耗规则	SYN 消耗 1；FIN 消耗 1；纯 ACK 不消耗；数据段消耗字节数
六个标志位	URG 紧急、ACK 确认、PSH 推送、RST 复位、SYN 同步、FIN 终止
MSS 与 MTU	MSS = MTU − IP 首部 − TCP 首部；MSS 只算数据载荷
三次握手	`SYN, seq=x` → `SYN+ACK, seq=y, ack=x+1` → `ACK, seq=x+1, ack=y+1`
为什么三次	防止已失效的连接请求报文段到达服务器，造成服务器资源浪费
四次挥手	`FIN, seq=u` → `ACK, ack=u+1` → `FIN+ACK, seq=w, ack=u+1` → `ACK, seq=u+1, ack=w+1`
为什么四次	TCP 是全双工，两个方向必须分别释放；半关闭期间 B 仍可发数据
TIME-WAIT / 2MSL	① 确保最后一个 ACK 能到达 B；② 使本次连接的所有报文段从网络中消失
释放最短时间	主动方 **1 RTT + 2MSL**；被动方 **1.5 RTT**
可靠传输	字节编号 + 累积确认 + 超时重传 + 快速重传 + SACK
RTO	$RTO = RTT_S + 4 \times RTT_D$；$\alpha = 1/8$，$\beta = 1/4$
Karn 算法	重传的报文段不采用其 **RTT** 样本；每重传一次 **RTO** 加倍
快速重传	收到 **3** 个重复确认立即重传，不等超时
SACK	通知已收到的不连续字节块，最多 4 个字节块；发送方不一定只重传缺失部分
流量控制	$swnd = \min(rwnd,\ cwnd)$；接收窗口 ⊆ 接收缓存，发送窗口 ⊆ 发送缓存
零窗口	用持续计时器 + 零窗口探测报文段（**1** 字节数据）打破死锁
慢开始	每确认 1 段 +1 MSS，每 **RTT** 翻倍
拥塞避免	每确认 1 段 $+1/cwnd$ MSS，每 **RTT +1 MSS**
超时处理	$ssthresh = cwnd/2$，$cwnd = 1$，重走慢开始
三重确认处理	$ssthresh = cwnd/2$，$cwnd = ssthresh$，直接进拥塞避免
拥塞 vs 流量	拥塞控制是全局性问题；流量控制是端到端问题

易错清单

传输层 22 个高频陷阱

1. 「有校验和」不等于「可靠传输」。UDP 有校验和，但校验失败只丢弃，不重传、不排序，因此是不可靠的。
2. **UDP** 的复用/分用依据是目的端口号（2018 年题 39），不是源端口号；源端口号只是让对方知道往哪儿回。
3. 网络层按协议字段分用，传输层按端口号分用，两个层次的「复用」含义不同，不要互相套用。
4. 端口号只有本地意义，两台主机上相同的端口号毫无关系；TCP 与 UDP 的端口号也相互独立。
5. 三类端口号分界点是 **1023** 和 **49151**。65535 属于短暂端口号。
6. **UDP** 的「长度」字段含 **8** 字节首部。给长度字段求数据长度要减 8，给数据长度求字段值要加 8。
7. 伪首部不传送、不递交，只在计算校验和时临时构造；UDP 协议号 17，TCP 协议号 6。
8. 反码求和一定要回卷进位，最后还要取反；若取反结果是全 0，应写全 1。
9. 全 **0** 校验和表示不校验（IPv4 的 UDP 允许），不是「校验通过」。
10. **TCP** 的数据偏移以 **4** 字节为单位，最小值 5（对应 20 字节固定首部），最大值 15（对应 60 字节）。别和 IP 的片偏移（**8** 字节为单位）混淆。
11. 序号是数据第一个字节的序号；确认号是期望收到的下一个字节的序号。两者语义相反，写题时最容易互换。
12. **ack = N** 表示到 **N−1** 为止的数据都已收到，与 GBN 中「ACK n 表示到 n 为止收到」差 1。
13. **SYN** 和 **FIN** 各消耗一个序号，纯 ACK 不消耗。第三、四次挥手能否携带数据的规定不同：第三次握手可以携带数据，第四次挥手不能携带数据。
14. 第一次和第二次握手都不能携带数据；第三次握手可以。
15. 只有主动关闭方进入 **TIME-WAIT**，而且必须等 **2MSL**；被动关闭方收到最后一个 ACK 后立即 CLOSED。
16. 释放时间不是「**2MSL**」：主动方是 1 RTT + 2MSL，被动方是 1.5 RTT（第 2、3 次挥手合并的最短情形）。
17. **MSL** 不是 **RTT**。2022 年题 39 中 MSL = 800 ms 与 RTT = 50 ms 完全无关，不要用 RTT 去算 2MSL。
18. **MSS** 只包含数据载荷，不含 TCP 首部和 IP 首部；MSS = MTU − 20 − 20 = 1460（以太网）。
19. 「收到 **3** 个重复确认」指冗余的那 **3** 个，加上原来那个共 4 个对同一报文段的确认。这个 3 的计数不要数错。
20. 超时与三重确认的处理不同：超时把 cwnd 降到 **1**；三重确认把 cwnd 降到新 **ssthresh**（原 **cwnd** 的一半），而且直接进拥塞避免，不经过慢开始。这是拥塞控制题的分水岭。
21. 慢开始「每 **RTT** 翻倍」，拥塞避免「每 **RTT +1**」；不要在拥塞避免阶段也做翻倍，也不要在慢开始阶段做 +1（尽管底层实现上慢开始每个确认确实只加 1 MSS）。
22. 发送窗口是 **min(rwnd, cwnd)**，且已发送未确认的数据仍占用窗口。2010 年题 39 的正确答案是 1000 B 而不是 2000 B，就是因为第 2 个 1000 B 段还没被确认。

第六章应用层

本章考纲要求

六、应用层（一）网络应用模型

1. 客户/服务器（C/S）模型
2. 对等（P2P）模型

（二）DNS 系统

1. 层次域名空间
2. 域名服务器
3. 域名解析过程

（三）FTP

1. FTP 协议的工作原理
2. 控制连接与数据连接

（四）电子邮件

1. 电子邮件系统的组成结构
2. 电子邮件格式与 MIME
3. SMTP 协议与 POP3 协议

（五）WWW

1. WWW 的概念与组成结构
2. HTTP 协议

应用层是 408 计网中每年必考、但分值稳定在 **2～3** 分的一章。考法高度模式化：单选题基本固定在第 33 题或第 40 题，考「某协议用什么传输层协议 / 哪个端口」「递归还是迭代」「是不是持久连接」；综合题的小问则集中在 **DNS** 查询次数与 **HTTP** 页面请求的 **RTT** 计算，以及 FTP 与 TCP 拥塞控制结合的分析。

本章的写法是：每个协议都按「工作机制 → 报文/命令流程 → 端口与传输层 → 计算套路」组织，凡是能用数字算一遍的地方（DNS 报文数、HTTP 的 RTT 数、FTP 的连接分析）都给出可直接照抄的公式与算例。

一、网络应用模型

（一）客户/服务器模型

客户/服务器（**Client/Server**，**C/S**）模型：通信双方地位不平等，一方是服务请求方（客户，**client**），另一方是服务提供方（服务器，**server**）。

- 客户（**client**）：面向用户，是主动方。它必须知道服务器的地址，主动发起通信、等待并接收结果。两个客户之间不能直接通信。
- 服务器（**server**）：面向任务，是被动方。它总是处于运行状态，拥有固定的 IP 地址与熟知端口号（如 HTTP 的 80、FTP 的 21），被动地等待并接受来自各客户的请求，处理后将结果返回。

工作流程可以概括为三步：

1. 服务器处于接受请求的状态（打开熟知端口，等待连接）。
2. 客户机发出服务请求，然后等待接收结果。
3. 服务器收到请求，进行处理，把结果返回给客户机。

Web、FTP、远程登录、电子邮件都是典型的服务集中型 C/S 应用——服务能力集中在少数服务器上。

C/S 模型的固有缺陷是可扩展性差：所有请求都由服务器集中处理，用户数增长时服务器的带宽与计算能力成为瓶颈；一旦服务器失效，整个服务就不可用。

（二）对等模型

对等（**Peer-to-Peer**，**P2P**）模型：没有固定的服务提供方与请求方，网络中的每一台计算机（称为对等方，**peer**）既是服务提供方又是服务请求方，对等方之间直接通信。

P2P 的本质仍然是「请求-响应」，只是每个结点同时扮演 client 与 server 两个角色。它的服务是分散型的——服务能力分散在大量对等计算机上。

P2P 的优点：

- 减轻服务器的计算压力，提高系统效率和资源利用率；多个用户之间可以直接共享文档。
- 可扩展性好：结点数增大时，系统的总上传能力也同步增大，性能不会因规模增大而降低。
- 健壮性强：个别结点失效对整个系统影响不大。
- 成本低。

P2P 的缺点：占用大量对等方的上/下行带宽，会影响个体的整机速度；缺乏集中管理，安全性、版权与可管理性差。

C/S 与 P2P 的对比（必背）

比较项	客户/服务器（**C/S**）	对等（**P2P**）
结点地位	不平等，客户与服务器角色固定	对等，每个结点两种角色兼有
谁主动	客户主动发起，服务器被动等待	双方都可主动发起
客户之间能否直接通信	不能	能，对等方之间直接通信
服务能力	集中在服务器（服务集中型）	分散在大量对等方（服务分散型）
可扩展性	差，服务器是瓶颈	好，结点越多总能力越大
典型应用	Web、FTP、电子邮件、远程登录	文件共享、即时通信、P2P 流媒体、分布式存储

（三）P2P 的三种组织方式

按「是否有索引服务器、索引如何组织」划分，P2P 有三种典型方式。

集中式（第一代）：代表是 **Napster**。仍然有一个中央索引服务器保存「哪个文件在哪些结点上」的目录，但文件内容本身不在服务器上传输——查询到结点列表后，由请求方与文件持有方直接建立连接传送文件。可见它把 C/S 的「集中索引」与 P2P 的「分散传输」结合起来。

- 优点：索引集中，查找效率高。
- 缺点：索引服务器是单点故障，且容易被起诉、被关闭。

全分布式非结构化（第二代）：代表是 **Gnutella**。没有任何中央服务器，所有结点完全对等。查询采用泛洪（**flooding**）方式：请求方把查询报文发给所有邻居，邻居再转发给自己的邻居，直到跳数（TTL）耗尽；命中者沿原路返回响应。

- 优点：完全去中心化，健壮性最好，无单点故障。
- 缺点：查询流量随规模指数增长，可扩展性差，且查询结果不保证完整。

混合式（第三代）：代表是 **BitTorrent**。它把结点分成两类：少数能力强的结点做超级结点（**super node**），负责索引与路由；其余普通结点在超级结点组织下分组（BitTorrent 中称为 torrent）。同一文件被切成许多块（**chunk**），下载者从多个持有者处并行下载不同的块，同时把已下载的块上传给别人，形成「人人为我，我为人人」的机制。

- 优点：兼顾了查找效率与可扩展性，是目前实际使用最广的方式。

P2P 易错点

- P2P 中「每个结点既是客户又是服务器」不等于「协议没有服务器进程」。混合式 P2P 中仍有超级结点；集中式 P2P 中仍有索引服务器。
- 「P2P 向多用户分发一个文件比 C/S 快」的原因不是单个结点带宽更大，而是系统的总上传能力随结点数增加。这是统考喜欢考的因果表述。
- C/S 模型中「客户与客户之间可以直接通信」是错误说法（2019 年真题的正确选项就是从这里出错的）。

真题考法

2019 年第 40 题直接考 C/S 与 P2P 模型的特征判断，四个选项分别对应「P2P 结点对等」「C/S 中客户之间能否直接通信」「谁主动发起」「分发文件所需时间」。只要记住上表的五行对比即可满分。

二、DNS 系统

DNS 的完整任务是把便于人记忆的主机名（域名）转换为便于机器处理的 **IP** 地址，它运行在 **UDP** 之上，熟知端口号 **53**，采用 **C/S** 模型。

（一）层次域名空间

因特网采用层次树状结构的命名方法。一个域名（**domain name**）由若干个分量（标号）组成，各分量之间用点「.」隔开，级别低的写在左边，级别高的写在右边：

…. 四级域名. 三级域名. 二级域名. 顶级域名

最右边是顶级域名（**Top Level Domain**，**TLD**），它的右边是隐藏的根（写作一个点，通常省略不写）。

例如 www.cqut.edu.cn：cn 是顶级域名，edu 是二级域名，cqut 是三级域名，www 是四级域名（主机名）。

顶级域名分为三类：

类别	含义	例子
国家顶级域名（nTLD）	用 ISO 3166 规定的两个字母的国家/地区代码	cn 中国、us 美国、uk 英国、jp 日本、hk 香港
通用顶级域名（gTLD）	按行业/性质划分	com 公司企业、net 网络服务机构、org 非营利组织、edu 美国教育机构、gov 美国政府机构、int 国际组织
基础结构域名（反向域名）	只有 arpa 一个	用于反向域名解析，即由 IP 地址查域名

二级域名是在某个顶级域名下注册的域名。我国的二级域名又分两类：类别域名（如 ac 科研机构、com 工商金融、edu 教育、gov 政府部门、net 网络服务机构、org 非营利组织）和行政区域名（如 bj 北京、sh 上海，共 34 个）。

关于域名与 **IP** 的关系，必须清楚三点：

1. 域名是逻辑概念，并不代表主机所在的物理地点；www.cqut.edu.cn 中的 cqut 不表示这台机器地理上一定在重庆。
2. 域名与 IP 地址不是一一对应的。一台主机可以有多个域名（如 www.abc.com 与 abc.com 指向同一台机器），一个域名也可以解析到多个 **IP** 地址（用于负载均衡）；多块网卡的主机甚至可以有一个 IP 对应多个域名。
3. DNS 里也不存储 **MAC** 地址，IP 到 MAC 的映射由 ARP 完成，两者不要混。

域名的两个基本要求

- 唯一性：一个域名在全球范围内只能指向相同的一批地址，这靠「按等级管理、同级不重名」来保证。
- 易查询：层次结构使得 DNS 可以把整个命名空间切分成许多区（**zone**），分散到不同服务器上管理。

（二）域名服务器

不可能把全部域名到 IP 的映射放在一台服务器上（负载过大、单点故障、时延过大），因此 DNS 是一个分布式数据库，用分布在各地的域名服务器共同实现转换。按层次自上而下分四类：

根域名服务器

- 是最高层次的域名服务器。
- 每一个根域名服务器都知道所有顶级域名服务器的域名及其 **IP** 地址。
- 全球共有 **13** 个根域名服务器（按 IP 地址计），每一个「服务器」实际由分布在世界各地的多台机器组成的服务器群集（cluster）实现；查询时由路由器用任播（**anycast**）把请求转发到离 **DNS** 客户最近的那一个根域名服务器。
- 根域名服务器通常并不直接解析域名，而是返回该域名所属顶级域名服务器的 **IP** 地址。它的作用更像是「指路」。

顶级域名服务器

- 负责管理在该顶级域名服务器注册的所有二级域名（即管一个顶级域，如.com、.cn）。
- 收到查询时给出回答：可能是最终的 IP 地址，也可能是指向下一级权限域名服务器的 **IP** 地址。

权限域名服务器（权威域名服务器）

- 负责管理某个区（**zone**）的域名。
- 每一台主机的域名都必须在某个权限域名服务器处注册登记，因此它掌握其管辖范围内域名与 IP 地址的映射关系。
- 它还知道其下级域名服务器的地址（若该区还往下划分了子区）。
- 一个区可以只对应一台权限域名服务器，但为了提高可靠性，通常一个区设置至少两台权限域名服务器（主、辅），其中一台失效时另一台仍可提供服务。

本地域名服务器（默认域名服务器）

- 它不属于上述的域名服务器等级结构——这是最常考的一句话。
- 当一个主机发出 DNS 查询请求报文时，这个报文首先被送往该主机的本地域名服务器。
- 本地域名服务器起着代理的作用，把查询转发到上述等级结构中，并把结果返回给主机。
- 每一个 ISP、一个大学乃至一个学院都可以拥有一个本地域名服务器。
- 它离用户很近（一般不超过几个路由器的距离，甚至就在同一局域网），其 **IP** 地址必须直接配置在需要域名解析的主机中（可由 DHCP 自动配置，也可手工配置）。

区（**zone**）与域（**domain**）的区别：域是命名空间的一个子树，是逻辑上的划分；区是 **DNS** 服务器实际管辖的范围，是为了分散管理而对域做的物理切分。一个区内的所有域名信息存放在同一台（组）权限域名服务器上，这台服务器称为该区的「权威」。下级的区可以把自己的授权委托出去，因此「域」的边界与「区」的边界不一定重合——一个域可以划分成多个区，每个区由不同的权限域名服务器管理。

区与权限域名服务器的易错点

- 题目问「该域名由哪台服务器解析」时，必须按题目给出的具体域名逐级判断：权限域名服务器可能不止一个，`abc.xyz.com` 的权限服务器与 `xyz.com` 的权限服务器通常不是同一台。
- 「本地域名服务器」永远不在根 → 顶级 → 权限这条等级链上，画图题里它一定单独画在旁边。

真题考法

常考形式是「下列说法错误的是」，错误项多半出在「本地域名服务器属于域名服务器等级结构」「根域名服务器直接返回 IP 地址」这两处。另外 2021 年、2016 年的题目要求根据拓扑图判断具体是哪台服务器完成解析，注意权限域名服务器可能不止一个。

（三）域名解析过程

把一个域名映射成 IP 地址的过程叫域名解析。查询方式有两种：

递归查询（**recursive query**）

客户把请求发给某台域名服务器后，如果该服务器不知道答案，它必须代替客户去询问下一台服务器，并把最终结果返回给客户。也就是说，被查询者承担了「继续查下去」的义务，客户只需要等一个最终答案。

- 特点：对客户最省事，但对被查询的域名服务器负担极大。

迭代查询（**iterative query**）

客户把请求发给某台域名服务器后，如果该服务器不知道答案，它只返回「你下一步该去问谁」的 **IP** 地址（转介信息），由客户自己再去询问下一台服务器；如此反复，直到某台服务器给出最终答案。

- 特点：服务器负担轻，但客户（通常是本地域名服务器）要多发几次请求。

因特网实际使用的组合方式（必背）

从请求主机到本地域名服务器的查询通常是递归查询；而本地域名服务器向根、顶级、权限域名服务器发出的查询通常是迭代查询。

原因：递归查询对被查询的服务器负担太大，而本地域名服务器数量相对少、能力较强，让它「跑腿」比让根域名服务器为每个客户层层代查更合理。

以主机 `m.xyz.com` 想查 `y.abc.com` 的 IP 地址为例，完整的迭代解析流程如下（假设各服务器与本地域名服务器的缓存都为空）：

1. `m.xyz.com` 向自己的本地域名服务器 `dns.xyz.com` 发出 **DNS** 查询请求报文（UDP 用户数据报，目的端口 53），报文中带查询域名 `y.abc.com` 和查询类型 A（地址记录）。这一步是递归查询。
2. 本地域名服务器缓存中没有该域名，于是向根域名服务器发出查询报文。
3. 根域名服务器不知道 `y.abc.com` 的地址，返回顶级域名服务器 **`dns.com`** 的 **IP** 地址。`dns.com` 是 `.com` 这个顶级域的服务器。
4. 本地域名服务器再向 `dns.com` 查询，`dns.com` 返回 **`y.abc.com`** 所在区域的权限域名服务器 **`dns.abc.com`** 的 **IP** 地址。
5. 本地域名服务器再向 `dns.abc.com` 查询，`dns.abc.com` 是 `abc.com` 区的权威服务器，知道 `y.abc.com` 的映射关系，于是返回 **`y.abc.com`** 的 **IP** 地址（最终答案）。
6. 本地域名服务器把结果返回给主机 `m.xyz.com`，并按 TTL 把这条记录存入自己的高速缓存。

一共 **4** 条查询报文 + **3** 条响应报文 = **7** 条 **DNS** 报文，其中本地域名服务器发出了 **4** 条查询（3 条迭代查询由它自己发起，另 1 条是它转发给根的那次）。如果题目只问「本地域名服务器发出多少次查询」，答案就是 **4** 次；如果问「本地域名服务器向外发出多少条 DNS 报文」，则是 **4** 条查询 + **3** 条收到的响应 = **7** 条（收到的不算「发出」）。这两种问法的区别是本章最容易失分的地方。

*1.*解析报文数与查询次数的计算

通用的计算规则有两条，务必记牢：

规则一：本地域名服务器发出的查询次数 = 需要经过的层次数。

若采用迭代查询，并且只考虑标准的 4 层结构（根 → 顶级 → 二级权限 → 三级权限…），那么：

$$\text{本地域名服务器发出的查询次数} = \text{域名的级数（去掉主机名）} + 1$$

其中「+1」是向根域名服务器的那一次查询。

规则二：如果题目说「本地域名服务器只提供递归查询，其他服务器只提供迭代查询」，则只有本地域名服务器在跑腿，用户的请求只有 1 条由主机发出，用户的等待时间只需按本地域名服务器的迭代次数计算 RTT。

算例一（无缓存、本地迭代）

主机 H 访问 `www.abc.xyz.com`，本地域名服务器无任何缓存，采用迭代查询。求本地域名服务器发出的最少、最多 DNS 查询次数。

- 最少 **0** 次：本地域名服务器的高速缓存中已有 `www.abc.xyz.com` 的 IP 地址，直接回答 H，不需要向外发任何查询。
- 最多 **4** 次：无缓存时按迭代依次查询

1. 根域名服务器 → 得到 .com 顶级域名服务器地址；
2. .com 顶级域名服务器 → 得到 xyz.com 的权限域名服务器地址；
3. xyz.com 权限域名服务器 → 得到 abc.xyz.com 的权限域名服务器地址；
4. abc.xyz.com 权限域名服务器 → 得到 www.abc.xyz.com 的 IP 地址。即：www.abc.xyz.com 去掉主机名 www 后剩 abc、xyz、com **3** 级，$3+1=4$ 次。

DNS 报文数计算三个必踩的坑

1.「发出」还是「收到」：题目问 201.1.1.1（本地域名服务器）发出的查询次数时，不包含主机 **H** 发给它的那一条递归查询；最少 0 次、最多 4 次（2016 年真题答案）。
2. 主机名不计入级数：www.abc.xyz.com 的迭代次数按 abc.xyz.com 算（3 级 + 1 = 4 次），不要把 www 也算成一级而得出 5 次。
3.「最少」的两种情形不同：若问「本地域名服务器发出的查询次数」，有缓存时是 **0** 次；若问「从点击链接到收到页面」，即使本地有缓存，主机到本地域名服务器的那 **1** 次递归查询（**1** 个 **RTT**）仍然要算。

算例二（**DNS** 与 **HTTP** 合并计算，**2020** 年真题模型）

主机 H 通过超链接 http://www.abc.com/index.html 请求纯文本页面 index.html。局域网内主机访问 Internet 上各服务器的 RTT 均为 10 ms，忽略其他时延。本地域名服务器只提供递归查询服务，其他服务器只提供迭代查询服务。求从点击超链接到浏览器收到页面所需的最短与最长时间。

- **HTTP** 部分固定要 **2** 个 **RTT**：1 个 RTT 建立 TCP 连接（三次握手的前两次往返归入一个 RTT），1 个 RTT 发送 HTTP 请求并收到响应。$2\times10=20$ ms。
- 最短：本地域名服务器已有 www.abc.com 的缓存，不再对外查询，于是主机与本地服务器之间仍是 **1** 次递归查询，但它与 TCP 建连的第一个 RTT 在时间上被合并考虑后不额外增加 RTT——按 2020 年真题的标准解法，此时总时间就是 HTTP 的 **20 ms**（DNS 查询的时延被计入/忽略方式以真题为准，此处真题给的是 20 ms 最短）。
- 最长：无任何缓存，本地域名服务器按迭代依次查询根、.com 顶级、abc.com 权限域名服务器，共 **3** 个 **RTT**（$3\times10=30$ ms）；加上 HTTP 的 20 ms，共 **50 ms**。

所以答案是「20 ms，50 ms」。注意域名 www.abc.com 的迭代次数是 3 次（abc + com 两级，再加根一次），不要算成 4 次。

*2.*高速缓存

为了提高查询效率、减轻根域名服务器的负荷、减少因特网上的 DNS 查询报文数量，各级域名服务器广泛使用高速缓存（**cache**），用来存放最近查询过的域名以及从何处获得域名映射信息的记录。

- 本地域名服务器缓存命中时，直接把缓存中的结果返回给主机，不再向外发出任何查询。
- 用户主机中也有高速缓存：许多主机在启动时从本地域名服务器下载域名与 IP 地址的数据库，维护自己最近使用过的域名缓存，只在缓存中找不到时才向域名服务器查询。
- 缓存中的记录有生存时间（**TTL**）。由于域名到 IP 的映射关系并非永久不变，域名服务器必须为每项内容设置计时器，删除超过合理时间的项（例如每个项目只存放两天）。因此缓存可能返回过期的映射，这是用「效率」换来的代价。

缓存易错点

- 有缓存时「查询次数为 0」指的是本地域名服务器不向外查询，主机到本地域名服务器的那一次查询仍然发生。

- 「主机与本地域名服务器之间采用递归查询」这一点不因有无缓存而改变。缓存命中只是让本地域名服务器不必再跑腿。

3.DNS 的传输层选择

DNS 什么时候用 UDP，什么时候用 TCP

- 主要使用 **UDP**，熟知端口号 **53**。普通查询与应答都是单个请求/单个响应，报文短、无连接开销、时延小，重传由应用层自己用超时机制解决。
- 下列情况必须改用 **TCP**（同样用端口 **53**）：
  1. 区域传送（**zone transfer**）：主域名服务器向辅助域名服务器同步整个区的数据库，数据量可能很大，必须保证可靠传输。
  2. 响应报文超过 **512** 字节：传统的 DNS 报文在 UDP 里限长 512 字节，超过时由服务器返回「截断」标志，客户端改用 TCP 重发；或者直接使用 EDNS0 扩展。所以 2018 年第 33 题「可以使用传输层无连接服务的应用层协议」答案是 **DNS**。

真题考法

- 2010 年第 40 题：递归查询时，用户主机与本地域名服务器各发出 **1** 条、**1** 条请求消息。
- 2016 年第 40 题：迭代查询时，本地域名服务器 `201.1.1.1` 发出查询的最少 **0** 次、最多 **4** 次。
- 2018 年第 33 题：可以使用无连接服务的应用层协议是 **DNS**。
- 2020 年第 40 题：DNS 迭代 + HTTP 请求页面，最短 **20 ms**、最长 **50 ms**。
- 2021 年第 47 题：Web 访问前主机除 HTTP 外还运行了 **DNS**，其封装为「DNS 报文 → UDP 数据报 → IP 分组 → 以太网帧」。

三、FTP

（一）FTP 协议的工作原理

文件传送协议（**File Transfer Protocol**，**FTP**）是因特网上使用最广泛的文件传送协议。它提供交互式访问，允许客户指明文件的类型与格式（如是否使用 ASCII 码），并允许文件具有存取权限（用户必须经过授权并输入有效口令）。FTP 屏蔽了各计算机系统的细节，因而适合于在异构网络中任意计算机之间传送文件。

FTP 采用客户/服务器工作方式，使用 **TCP** 提供的可靠传输服务。一个 FTP 服务器进程可同时为多个客户进程提供服务，其进程分为两类：

- 主进程（**1** 个）：负责接收新的请求，始终监听熟知端口 **21**。
- 从属进程（多个）：每处理一个客户的请求就 fork 一个从属进程，负责该客户的数据传送。

FTP 工作时使用两条并行的 **TCP** 连接：

比较项	控制连接	数据连接
作用	传送控制信息（连接请求、登录、目录操作、传送请求、终止等命令与响应）	传送文件内容或目录列表
服务器端口	**21**（熟知端口）	主动模式下为 **20**；被动模式下为服务器随机分配的临时端口
客户端口	客户自己分配的临时端口	客户自己分配的临时端口
建立/释放时机	在整个会话期间始终保持打开	每次文件传输时才建立，传输结束就关闭（按需建立）

比较项	控制连接	数据连接
谁发起	客户主动发起（客户连服务器 21）	主动模式下服务器发起；被动模式下客户发起
传输内容	命令与应答，是文本	文件数据，可以是二进制

因为控制信息走的是另一条独立的连接，而不是与数据混在同一条连接里，所以称 **FTP** 的控制信息是「带外（**out-of-band**）」传输的。这正是 FTP 与 HTTP 的根本差别：HTTP 把请求与响应都放在同一条 TCP 连接里，属于「带内（in-band）」传输。

带外控制的含义

「带外」指控制信息与数据走不同的连接（不同的 **TCP** 连接，进而使用不同的端口）。好处是：即使数据连接正在满负荷地传送一个大文件，控制连接依然是通的，客户可以随时发出「终止传输」「放弃」等命令，服务器也能即时回应。若把控制命令塞进数据流里排队（带内），就必须等前面的数据传完才能终止，响应会非常迟钝。

*1.*主动模式与被动模式

数据连接的建立方向有两种模式，这是 FTP 最常考的细节。

主动模式（**PORT** 模式）

1. 客户用临时端口连接服务器的 **21** 端口，建立控制连接并登录。
2. 客户需要传数据时，在控制连接上随机开一个临时端口，并通过 **PORT** 命令把「我的 IP + 我的端口号」告诉服务器。
3. 服务器用自己 **20** 端口主动去连接客户刚才告诉它的那个端口，建立数据连接。
4. 因此「主动」是指服务器主动发起数据连接。

被动模式（**PASV** 模式）

1. 控制连接同样建立在服务器的 21 端口。
2. 客户需要传数据时，在控制连接上发送 **PASV** 命令。
3. 服务器随机打开一个临时端口，并把该端口号通过控制连接告诉客户。
4. 客户主动去连接服务器的这个临时端口，建立数据连接。
5. 因此「被动」是指服务器被动等待客户来连。

被动模式的意义在于：主动模式下服务器要反过来连客户，如果客户在防火墙/NAT 后面，这个入向连接会被拦掉；被动模式下所有连接都由客户发起，防火墙友好。

FTP 端口易错点

- **20** 和 **21** 都是服务器端口号，客户端的端口号由客户系统自行分配，不固定。2017 年第 40 题的错误选项正是「服务器与客户端的 TCP 20 端口建立数据连接」——20 是服务器的端口，不是客户端的。
- 被动模式下数据连接不使用 **20** 端口，而是服务器临时分配的随机端口。所以「FTP 数据连接一定用 20」是错的。
- 控制连接在整个会话期间保持打开；数据连接每传一次就建立、传完就关闭。2023 年第 47 题第 (1) 问的答案就是「控制连接是持久的，数据连接是非持久的」。
- 「会话期间保持打开」指控制连接一直存在，但数据连接可以是多次：传一个文件就关一次，再传再建。

2.FTP 连接分析算例

算例（基于 **2023** 年第 **47** 题的模型）

主机 H 登录到 FTP 服务器后，向服务器上传一个大小 18000 B 的文件 F。建立数据连接时选择的初始序号为 100，MSS $=1000$ B，拥塞控制初始阈值 $ssthresh = 4$ MSS，RTT $=10$ ms，忽略 TCP 传输时延，H 均以 MSS 段发送数据且无差错、无丢包、无乱序。问：

(1) 建立了哪些 **TCP** 连接？ H 登录 FTP 服务器时先建立的是控制连接（客户临时端口 $\rightarrow$ 服务器 21）。此后每次传文件再建立数据连接。所以 H 登录时建立的 TCP 连接是控制连接。

(2) F 的第一个字节序号是多少？数据连接本身的初始序号是 100，但 F 的第一个字节前面还有 **1** 个 **SYN** 段占用的序号：TCP 的 SYN 会消耗 1 个序号。因此 F 的第一个字节序号是 $100+1=101$。

(3) 断开数据连接时，**FTP** 服务器发送的第二次挥手 **ACK** 段的确认序号是多少？ F 共 18000 B，从序号 101 开始，则最后一个字节的序号为 $101+18000-1=18100$。因此服务器在第二次挥手（对 H 的 FIN 的确认）中，确认序号为 $18100+1=$ **18101**。

(4) 拥塞窗口的调整：慢开始阶段每收到一个对新数据的确认，$cwnd$ 加 1 MSS（指数增长），直到达到 $ssthresh = 4$ MSS 后转为拥塞避免（线性增长，每 RTT 加 1 MSS）。

- 收到确认序号 **2101** 时：已连续确认到 2100，即 $2100-101+1=2000$ B $=2$ MSS，此时 $cwnd$ 从 1 MSS 增至 **2 MSS**（慢开始阶段）。
- 收到确认序号 **7101** 时：已连续确认到 7100，即 $7100-101+1=7000$ B $=7$ MSS。按慢开始：1→2→4 MSS（达到阈值），此后进入拥塞避免，每 RTT 加 1 MSS：4→5→6→7 MSS。所以 $cwnd$ 调整为 **7 MSS**。

(5) 总时间与应用层平均速率：需要把 18000 B 分成 18 个 MSS 段。计入建立数据连接（1 RTT）与四次挥手（其中 H 的 FIN 与服务器的 ACK 常合并分析，按真题口径计 2 个 RTT）后共 **7** 个 **RTT**，即 $7\times10=70$ ms；应用层数据平均发送速率 $=18000\,\text{B}/70\,\text{ms}\approx257\,\text{B/ms}=257\,\text{kB/s}$（按真题给定口径计算结果）。

真题考法

2009 年第 40 题考「FTP 命令走的是建立在 TCP 之上的控制连接」；2017 年第 40 题考 FTP 两条连接的四个特征判断；2023 年第 47 题把 FTP 与 TCP 序号、拥塞窗口、四次挥手结合成大题。**FTP** 部分真正考的是 **TCP**，**FTP** 只是外壳，所以第 (2) ~ (5) 问的工夫要花在传输层上。

（二）控制连接与数据连接

控制连接的作用：客户与服务器之间传送命令与应答。它承载的内容包括用户登录（用户名、口令）、改变工作目录、列出目录（`LIST`）、指定传输模式与文件类型、发起一次文件传送的请求（`RETR` 下载 / `STOR` 上传）、终止传送（`ABOR`）以及退出（`QUIT`）等。控制连接传送的是文本形式的命令与三位数字应答码，不传送文件数据。

数据连接的作用：在客户与服务器之间传送文件内容（`RETR`/`STOR`）或目录列表（`LIST`）。它的特点是按需建立、用完即关。数据连接上还可以选择传输方式（流方式、块方式、压缩方式）与表示类型（ASCII 文本、二进制图像），这是「FTP 允许客户指明文件类型与格式」的体现。

FTP 两条连接与 **HTTP** 一条连接的对比

比较项	**FTP**	**HTTP**
TCP 连接条数	**2** 条（控制连接 + 数据连接）	**1** 条（同一连接既传请求响应也传实体）
控制信息位置	带外（独立控制连接）	带内（与数据在同一连接，靠报文格式区分）
连接是否持续	控制连接持续；数据连接按需建立、用完释放	非持久连接下一个对象一条连接；持久连接下保持
状态	有状态：服务器记住登录状态与当前工作目录	无状态：服务器不保存客户的历史信息（Cookie 是后加的补救）
传输层	TCP	TCP
默认端口	21（控制）、20（主动模式数据）	80

「FTP 是有状态的」这一点常被考到：服务器要记住用户是否已登录、当前处于哪个目录、传的是哪个文件——这些状态跨多条命令保持，是控制连接长期存在的原因。HTTP 每条请求自带完整信息（`Host`、`Cookie` 等），服务器不必记住上一次是谁来访问过。

四、电子邮件

（一）电子邮件系统的组成结构

电子邮件系统采用客户/服务器方式，由三个主要部分组成：

用户代理（**User Agent**，**UA**）

用户与电子邮件系统之间的接口，即电子邮件客户端软件或网页版邮件界面。它的职责是：撰写、显示、处理和发送与接收邮件。用户代理把邮件「提交」给自己的邮件服务器，或从自己的邮件服务器「取回」邮件。

邮件服务器

电子邮件系统的基础设施，负责发送和接收邮件，同时维护用户的邮箱。它同时充当客户和服务器：向别的邮件服务器发送邮件时是 SMTP 客户，接收别的邮件服务器送来的邮件时是 SMTP 服务器。邮件服务器必须在 24 小时不间断运行，并具有固定的 IP 地址与域名。

协议

- 邮件发送（推送）协议：**SMTP**（TCP 25），负责「用户代理 → 发送方邮件服务器」以及「发送方邮件服务器 → 接收方邮件服务器」。
- 邮件读取（拉取）协议：**POP3**（TCP 110）或 **IMAP**（TCP 143），负责「接收方邮件服务器 → 用户代理」。
- 网页版邮件还可由 **HTTP** 承担发送与读取。

邮件收发的完整流程（背下来就能做所有流程题）

设发件人 A 的邮箱在邮件服务器 S1 上，收件人 B 的邮箱在邮件服务器 S2 上：

1. A 通过用户代理撰写邮件。
2. 用户代理用 **SMTP** 把邮件「推」给发送方邮件服务器 **S1**。
3. S1 把邮件放入邮件缓存队列中等待发送。
4. S1 的 SMTP 客户进程周期性扫描缓存队列（例如每 30 分钟），发现有待发邮件后，与接收方邮件服务器 **S2** 的 SMTP 服务器进程建立 **TCP** 连接（端口 **25**）。
5. TCP 连接建立后，S1 用 SMTP 把邮件「推」给 **S2**；所有邮件发送完毕后断开 TCP 连接。

6. S2 收到邮件后，把它放入收件人 **B** 的用户邮箱中，等待 B 在方便的时候读取。
7. B 想收信时，通过用户代理用 **POP3** 或 **IMAP** 从 S2 「拉」回自己的邮件。

判断口诀：凡是「交给服务器 / 服务器之间转发」的环节都用 **SMTP**（推）；凡是「从自己的服务器取回来」的环节都用 **POP3/IMAP**（拉）。

电子邮件流程易错点

- 不能把邮件直接推给收件人的用户代理。原因有二：并非所有计算机都能运行邮件服务器程序；收件人的计算机也不一定 24 小时开机，发信时若对方不在线就无法处理，所以必须由始终在线的邮件服务器代为缓存。
- 电子邮件地址格式为「用户名 @ 邮件服务器所在主机的域名」，例如 `abc@163.com`。@ 后面是域名而不是 IP。
- 所有邮件相关连接都基于 **TCP**，都是 **C/S** 方式。没有一个邮件协议用 **UDP**。
- 2012 年第 40 题的三个阶段依次是 **SMTP**、**SMTP**、**POP3**：① UA→ 发送方服务器，② 发送方服务器 → 接收方服务器，③ 接收方服务器 →UA。

（二）电子邮件格式与 MIME

邮件格式在 RFC 822（2008 年后更新为 RFC 5322）中单独定义，不是由 **SMTP** 定义的。一封邮件由信封和内容两部分构成，内容又分为首部和主体。

- 信封：由邮件系统自动生成，从首部中提取所需信息写在信封上，相当于「邮局看的部分」。
- 首部（**header**）：由若干首部行组成，每行是「字段名: 值」。关键字包括：
  - `From`：发件人邮箱地址；
  - `To`：收件人邮箱地址（多个用逗号分隔）；
  - `Cc`：抄送（carbon copy）收件人；
  - `Subject`：邮件主题；
  - 还可有 `Date:`、`Reply-To:`、`Bcc:`（密送）等。
- 主体（**body**）：用户自由编写的信件正文，是真正要传达的内容。

首部与主体之间的空行

首部与主体之间必须用一个空行分隔。邮件系统读到这个空行即认为首部结束、下面全部是正文。少写这个空行会把正文的第一行当成又一个首部字段，导致正文丢失或解析错误。

MIME（多用途因特网邮件扩展，**Multipurpose Internet Mail Extensions**）

为什么需要 MIME

SMTP 只能传送 7 位 **ASCII** 码文本数据，不能传送可执行文件或其他二进制对象，不能满足传送多媒体邮件（图片、音频、视频）的需要，许多非英语文字（中文、俄文等）也无法用 SMTP 直接传送。**MIME** 不是要取代或改动 **SMTP**，它的意图是继续使用现有的邮件格式与 **SMTP**，只在邮件主体上做扩展。

MIME 做了三件事：

1. 增加了 **5** 个新的邮件首部字段，提供有关邮件主体的信息。最常用的是：
   - `Content-Type`：说明主体的数据类型与子类型，如 `text/plain`、`text/html`、`image/jpeg`、`application/octet-stream`、`multipart/mixed`。它决定了接收方用什么方式「解释」主体。
   - `Content-Transfer-Encoding`：说明主体的编码方式，如 `7bit`、`8bit`、`base64`、

`quoted-printable`，即「这段内容是经过哪种编码才变成 ASCII 的」。

- 其余还有 `Content-ID`、`Content-Description`、`MIME-Version` 等。

2. 定义了多种邮件内容的格式，对多媒体电子邮件的表示方法进行了标准化。
3. 定义了传送编码，可对任何内容格式进行转换而不会被邮件系统改变。

base64 编码是最重要的传送编码：它把任意二进制数据按每 **3** 个字节（**24** 位）一组，重新划分为 **4** 个 **6** 位，每个 6 位用一个 **ASCII** 字符表示（字符表为 `A-Z a-z 0-9 + /`），因此编码后数据量约为原来的 $\frac{4}{3}$，即膨胀约 **33%**。若最后一组不足 3 字节，用 = 补齐。它保证编码结果全部落在 7 位 ASCII 范围内，从而能安全通过只认 ASCII 的 SMTP。

MIME 后来也被同样面向 ASCII 字符的 **HTTP** 使用（HTTP 首部里的 `Content-Type` 就来源于此）。

MIME 易错点

- **MIME** 不改变 **SMTP** 协议，SMTP 仍然只传 ASCII；是 MIME 在发送前编码、接收后解码，把非 ASCII 内容「伪装」成 ASCII。
- 2018 年第 40 题问「无需转换即可由 SMTP 直接传输的内容」，答案是 **ASCII** 文本；JPEG、MPEG、EXE 都必须先编码。
- `Content-Type` 说的是「是什么」，`Content-Transfer-Encoding` 说的是「怎么编码的」，两者不要搞反。

（三）SMTP 协议与 POP3 协议

1.SMTP 的工作过程

SMTP（**Simple Mail Transfer Protocol**，简单邮件传送协议）：C/S 模型，建立在 **TCP** 之上，熟知端口号 **25**。它是「推」协议——由发送方主动把邮件推给接收方。

SMTP 的一次典型会话（命令/响应方式，命令由客户发出，应答是三位数字码）：

步骤	客户（发送方）发出	服务器应答	含义
1	（建立 TCP 连接，端口 25）	220	服务器主动推送「服务就绪」
2	`HELO hnust.edu.cn`	250	客户说明身份、告知自己的域名；身份有效则回 250，否则回其他代码（如 421 服务不可用）
3	`MAIL FROM:` 后跟发件人邮箱地址	250	告知邮件来自何方
4	`RCPT TO:` 后跟收件人邮箱地址	250	告知邮件去往何地；邮箱存在则回 250
5	`DATA`	354	告知准备发送邮件内容；服务器准备好接收则回 354
6	<邮件内容 > 后跟一行只有「.」的结束符	250	传送正文，用单独一行的句点表示结束；收件成功回 250
7	`QUIT`	221	请求断开 TCP 连接；服务器回 221 并主动关闭

SMTP 共有约 14 条命令、21 种应答。应答代码后面一般跟有简单的描述信息，不同的 SMTP 服务器对同一应答码的描述信息可能不同。

发送方邮件服务器的工作特点：邮件先进入缓存队列，服务器周期性扫描（如每 30 分钟）缓存，发现有邮件才尝试与接收方服务器建立连接并发送；若发送失败则保留在队列中稍后重试。

SMTP 的三大局限与推论

1. 只能传送 **7** 位 **ASCII** 码文本——这是引入 MIME 的直接原因，也是最常考的一条。
2. 明文传送，不利于保密——SMTP 本身不加密，需要安全性时用 TLS 加密或使用带加密的扩展。
3. 只负责「推」到收件人的邮件服务器，不负责「取」——用户代理从服务器读邮件要另用 POP3/IMAP。所以 2013 年第 40 题「SMTP 支持从邮件服务器向用户代理发送邮件」是错误的。

2.POP3 的工作过程

POP3（**Post Office Protocol**，邮局协议第 **3** 版）：C/S 模型，建立在 **TCP** 之上，熟知端口号 **110**，是因特网正式标准。它是「拉」协议——由用户主动从邮件服务器把邮件拉回本地。

POP3 会话分为三个阶段：

1. 认证阶段：客户用 `user <用户名 >` 与 `pass <口令 >` 登录，服务器校验身份。
2. 事务处理阶段：客户可以 `stat`（查询邮件数与总大小）、`list`（列邮件清单）、`retr`（取回第 n 封邮件）、`dele`（标记删除第 n 封）、`quit`。
3. 更新阶段：客户发出 `quit` 后进入此阶段，服务器真正执行事务阶段中标记的删除操作，释放资源并关闭连接。

POP3 的两种下载方式（考纲明确要求）：

- 下载并删除（**download and delete**）：邮件取回本地后立即从服务器删除。优点是服务器空间释放快；缺点是用户换一台机器就再也看不到这些邮件。
- 下载并保留（**download and keep**）：邮件取回后仍保留在服务器上。优点是可以多台设备重复读取；缺点是占用服务器存储，且用户需要自己清理。

POP3 不允许用户在邮件服务器上管理自己的邮件（不能建文件夹、不能分类、不能只读某个首部），这是它「功能有限」的体现。

IMAP（**Internet Message Access Protocol**，因特网邮件访问协议）：端口 **143**，功能比 POP3 强大。用户在自己的计算机上就可以操控服务器中的邮箱，就像在本地操控一样，因此 IMAP 是联机协议；它允许用户代理只获取报文的某些部分（例如只读一个首部），也允许在服务器上创建文件夹、分类管理。

共同点：POP3 与 IMAP4 都采用基于 **TCP** 连接的客户/服务器方式。

3.SMTP 与 *POP3* 的对比

SMTP 与 POP3 对比表（必背）

比较项	**SMTP**	**POP3**
作用	发送（推送）邮件：UA → 发送方服务器、服务器 → 服务器	读取（拉取）邮件：接收方服务器 → UA
数据流动方向	推（**push**），发送方主动	拉（**pull**），接收方主动
传输层	TCP	TCP
熟知端口号	**25**	**110**
工作模型	C/S	C/S
传输内容限制	只能传 **7** 位 **ASCII** 文本（非 ASCII 需 MIME 编码）	传送的是整封邮件（内容已在服务器上存好），本身不限制内容

比较项	SMTP	POP3
会话特点	命令/应答（HELO、MAIL FROM、RCPT TO、DATA、QUIT）	三个阶段：认证、事务处理、更新；`quit` 后才真正删除
是否需要认证	传统 SMTP 无认证（后来增加了认证扩展）	认证阶段必须提供用户名与口令
是否允许管理服务器邮箱	不涉及	不允许建文件夹、分类等（IMAP 才允许）

邮件协议易错点

- 端口记错：SMTP 是 25，POP3 是 **110**，IMAP 是 **143**。不要写成 25/110 之外的数。
- 方向记反：SMTP 是「推」，POP3/IMAP 是「拉」。凡是「服务器 → 用户代理」的环节一定不是 **SMTP**（除非是网页邮件用 HTTP）。
- **2015** 年第 **33** 题问「POP3 使用的传输层服务类型」，答案是有连接可靠的传输服务（TCP）。
- 「**SMTP** 地址出错不能发邮件，**POP3** 地址出错不能收邮件」这类对应关系可以辅助记忆，但真正要记的是协议与端口的对应。
- 网页版邮件（如用浏览器收发 163 邮箱）时，UA 到服务器用 **HTTP**，服务器之间仍用 **SMTP**——所以「邮件发送一定用 SMTP」这个说法在网页邮件的第一个环节上并不成立。

五、WWW

（一）WWW 的概念与组成结构

万维网（**World Wide Web**，**WWW**）并非某种特殊的计算机网络，而是一个大规模的、联机式的信息储藏所，是运行在因特网上的一个分布式应用。它利用网页之间的超链接（**hyperlink**）把不同网站的网页链接成一张逻辑上的信息网，用户通过「点击链接」就能在全世界范围内跳转浏览。

万维网由 Tim Berners-Lee 于 1989 年 3 月在欧洲粒子物理实验室提出；1993 年 2 月第一个图形界面浏览器 Mosaic 问世，1995 年 Netscape Navigator 上市。

万维网的三个核心要素：

要素	全称	解决的问题
URL	统一资源定位符（Uniform Resource Locator）	怎样标识分布在世界各地的文档（资源在哪）
HTTP	超文本传送协议（HyperText Transfer Protocol）	用什么协议传送文档
HTML	超文本标记语言（HyperText Markup Language）	怎样解析/显示文档的格式

浏览器最重要的部分是渲染引擎（浏览器内核），负责解析并显示网页内容。常见内核：Chrome 与 Opera 用 Blink，Firefox 用 Gecko，Safari 用 WebKit。不同内核对网页的解析不同，因此同一网页在不同浏览器里显示效果可能不同。除 HTML 外，网页通常还包含 **CSS**（层叠样式表，从审美角度描述样式）与 **JavaScript**（一种脚本语言，控制网页行为，与 **Java** 没有任何关系）。

1.URL 的完整格式

URL 用来指明因特网上任何种类「资源」的位置。其一般形式为：

$$\langle\text{协议}\rangle : //\langle\text{主机}\rangle : \langle\text{端口}\rangle/\langle\text{路径}\rangle$$

以 `http://www.hnust.cn:80/ggtz/119945.htm` 为例：

部分	值	含义
协议	`http`	访问资源所用协议（还可能是 `https`、`ftp`、`file` 等）
主机	`www.hnust.cn`	存放资源的主机域名（也可以直接写 IP 地址）
端口	`80`	服务器上的端口号，可以省略；省略时使用该协议的默认端口（HTTP 为 80）
路径	`/ggtz/119945.htm`	资源在主机上的路径与文件名，可以省略；省略时表示访问该网站的主页（**home page**）

需要扩展记忆的是，完整 URL 还可以带查询串（**query**）与片段标识符（**fragment**）：

$$\underbrace{https}_{\text{协议}}://\underbrace{\text{www.abc.com}}_{\text{主机}}:\underbrace{443}_{\text{端口}}\underbrace{/news/list.htm}_{\text{路径}}\underbrace{?id=408\&page=2}_{\text{查询}}\underbrace{\#top}_{\text{片段}}$$

- 查询（`?` 之后）：以`?` 开始、键 `=`值用 `&` 连接，把参数传给服务器上的程序，会被发送给服务器。
- 片段（`#` 之后）：定位到文档内部的某个锚点，只由浏览器本地使用，不会发送给服务器——这是容易被考到的细节：改变片段不会产生新的 HTTP 请求。

URL 易错点

- URL 中的主机部分写的是域名（如 `www.hnust.cn`），但真正建 TCP 连接前必须先用 **DNS** 把它解析成 **IP**。
- 「端口可省略」的前提是使用默认端口：HTTP 是 80，HTTPS 是 443，FTP 是 21。
- 片段标识符不参与网络请求；查询串参与。

（二）HTTP 协议

HTTP（超文本传送协议）定义了浏览器（万维网客户进程）怎样向万维网服务器请求万维网文档，以及服务器怎样把文档传送给浏览器。它建立在 **TCP** 之上，熟知端口号 **80**。

HTTP 有三个重要性质：

- **HTTP** 是面向事务（**transaction-oriented**）的：一次「请求-响应」就是一个不可分割的事务，一个请求只请求一个资源，不存在一次请求 5 个文档的说法。
- **HTTP** 是无连接的：通信双方在交换 HTTP 报文之前不需要先建立 **HTTP** 连接——但 HTTP 基于 TCP，必须先用 **TCP** 三次握手建立连接。
- **HTTP** 是无状态的：同一个客户第二次访问同一文档时，服务器的响应与第一次相同，服务器不保存客户的历史信息。这样做简化了服务器设计，但无法识别用户，因此后来引入了 Cookie。

1.HTTP 报文格式

HTTP 是面向文本的，报文中的每个字段都是 **ASCII** 码串，且各字段长度不确定。

请求报文的结构：

请求行 | 首部行 | 空行 | 实体主体

- 请求行：方法 空格 URL 空格 版本 CRLF，例如 `GET /index.htm HTTP/1.1`。
- 首部行：每行是首部字段名： 值 CRLF，可有多行。

- 空行：一个只有 CRLF 的行，标志首部结束（必需）。
- 实体主体：通常不用（GET 不用；POST 才用）。

一个典型的请求报文：

```
GET /index.htm HTTP/1.1
Host: www.hnust.cn
Connection: close
User-Agent: Mozilla/5.0
Accept-Language: cn
```

各行含义：GET 指明方法，/index.htm 是请求的 URL，HTTP/1.1 是版本；Host 指明服务器域名（HTTP/1.1 必需）；Connection: close 告诉服务器发完文档后释放连接（即不使用持续连接）；User-Agent 告知浏览器类型与版本；Accept-Language: cn 表示希望优先得到中文版本文档。

响应报文的结构：

状态行 | 首部行 | 空行 | 实体主体

- 状态行：版本 空格 状态码 空格 短语 CRLF，例如 HTTP/1.1 404 Not Found。
- 首部行、空行与请求报文同理。
- 实体主体：服务器返回的文档内容（有些响应报文不用，如 304）。

常见响应状态行示例：HTTP/1.1 202 Accepted（接受请求）、HTTP/1.1 400 Bad Request（错误的请求）、HTTP/1.1 404 Not Found（找不到页面）。

*2.*常见方法与状态码

HTTP 方法（请求行中的第一个字段）：

方法	含义
GET	请求读取由 URL 标识的信息（最常用），参数附在 URL 后面
POST	向服务器提交数据（如表单内容），参数放在实体主体中，不出现在 URL 里
HEAD	请求读取由 URL 标识的信息的首部，而不返回实体主体，常用于检查资源是否存在/是否被修改
PUT	向服务器上传/存放一个文档（写操作）
DELETE	删除服务器上由 URL 标识的资源

状态码是三位数字，第一位表示响应类别：

类别	含义	常见码
1xx	通知信息，表示已收到请求，正在继续处理	100 Continue
2xx	成功，操作被成功接收并处理	200 OK（请求成功，实体在响应中）、202 Accepted
3xx	重定向，需要进一步的动作才能完成请求	301 Moved Permanently（永久移动）、302 Found（临时移动）、304 Not Modified（资源未修改，可用缓存）

类别	含义	常见码
4xx	客户出错，请求有语法错误或无法实现	400 Bad Request、403 Forbidden（服务器拒绝）、404 Not Found
5xx	服务器出错，服务器本身发生错误	500 Internal Server Error

必须记住的状态码

- **200**：请求成功。
- **301 / 302**：永久 / 临时重定向。
- **304 Not Modified**：资源未修改。服务器用它对 `If-Modified-Since` 或 `If-None-Match` 的条件请求作答，响应中不携带实体主体，客户直接使用本地缓存——这是「Web 缓存」节省带宽的核心机制。
- **400**：请求报文有语法错误。
- **403**：服务器理解请求但拒绝执行（无权限）。
- **404**：请求的资源不存在。
- **500**：服务器内部错误。
- 记忆技巧：**2** 成功、**3** 重定向、**4** 客户错、**5** 服务器错。

*3.*非持久连接与持久连接

非持久连接（**non-persistent connection**）

HTTP/1.0 采用非持久连接：每次浏览器要请求一个文件，都要与服务器建立一次 **TCP** 连接，收到响应后就立即关闭连接。因此每请求一个文档都要有两倍的 **RTT** 开销：

$$T_{\text{一个对象}} = 2\,\text{RTT} + T_{\text{文档传输时延}}$$

第一个 RTT 用于 TCP 三次握手（前两次握手占用一个 RTT），第二个 RTT 用于发送 HTTP 请求并收到响应。

缺点：一个网页上往往有很多引用对象（图片、CSS、JS），请求每一个对象都要花 2 RTT；而且每个 **TCP** 连接刚建立时拥塞窗口很小，无法充分利用带宽。为减小时延，浏览器通常会建立多个并行的 **TCP** 连接同时请求多个对象（如 Chrome 对同一域名可并行 6 ~ 10 条），但这会大量占用服务器资源。这就是非持久连接分为串行 **TCP** 与并行 **TCP** 的由来。

持久连接（**persistent connection**）

HTTP/1.1 默认采用持久连接：服务器在发送响应后仍然保持这条 **TCP** 连接，使同一个客户和该服务器可以继续在这条连接上传送后续的 HTTP 请求报文与响应报文。它不局限于同一个页面上的对象，只要这些文档在同一台服务器上就行。

持久连接又分两种工作方式：

- 非流水线（**non-pipelined**）：客户在收到上一个响应之后才能发出下一个请求。因此每个对象仍需 **1** 个 **RTT**，但省掉了每个对象建 **TCP** 连接的 **1** 个 **RTT**。
- 流水线（**pipelined**）：客户在收到 HTTP 响应报文之前就能连续发送多个请求报文，这些请求一个接一个到达服务器后，服务器再一个接一个地发回响应。这样能把同一窗口内能容纳的请求「打包」发送，省掉多个 **RTT**，使 TCP 连接中的空闲时间减少。

四种连接方式的关系（务必分清）

- 按「一条 TCP 连接能传几个对象」分：非持久（1 个）vs 持久（多个）。
- 非持久再按「多个连接是否同时开」分：串行 **TCP** vs 并行 **TCP**（并行是「多条连接之间」的关系）。
- 持久再按「请求能否连续发」分：非流水线 vs 流水线（流水线是「同一条连接上请求之间」的关系）。

HTTP/1.1 与 HTTP/2 的主要差别

比较项	**HTTP/1.1**	**HTTP/2**
连接复用	持久连接，但同一连接上的请求按顺序处理（流水线的响应也须按序返回），存在队头阻塞	多路复用：同一 TCP 连接上可并行交错传输多个请求/响应，不再互相阻塞
报文格式	文本（ASCII）	二进制分帧，把报文切成带类型的帧
首部开销	每个请求都重复发送完整首部行，冗余大	首部压缩（**HPACK**），并维护首部表，重复首部只发索引
服务器推送	不支持	支持服务器推送（**server push**），可主动把客户即将需要的资源推过去
传输层	TCP	仍基于 TCP（HTTP/3 才改用基于 UDP 的 QUIC）

一句话概括：HTTP/2 在不改变 **HTTP** 语义（方法、状态码不变）的前提下，通过多路复用、二进制分帧、首部压缩、服务器推送提升传输效率。

4.Cookie 与 Web 缓存（代理服务器）

Cookie

早期的万维网只用于查看静态文档，所以 HTTP 被设计成无状态协议以简化服务器设计。但现在网上购物、电子商务等应用需要服务器识别用户，于是引入了 **Cookie**：**Cookie** 是一种对无状态的 **HTTP** 进行状态化的技术。

工作过程：

1. 用户首次访问某网站时，服务器的响应报文中带一行 `Set-Cookie: 12345678`（一个由服务器生成的识别码）。
2. 浏览器把该 Cookie 存入本地 Cookie 文件，记录「哪个服务器 → 哪个识别码」。
3. 此后浏览器每次访问该服务器，都在请求报文中加一行 `Cookie: 12345678`。
4. 服务器根据这个识别码在自己的数据库中查到这个用户，从而「记住」用户，在服务器端维护该用户的状态（购物车、登录态等）。

因此 2015 年第 40 题中，请求报文里出现 `Cookie: 123456` 说明浏览器此前访问过该站点；而 `Connection: close` 表示不使用持久连接。

Web 缓存与代理服务器

万维网缓存（**Web Cache**，又称 **Web** 缓存）可位于客户机上，也可位于中间系统上。位于中间系统上的 Web 缓存称为代理服务器（**Proxy Server**）。

Web 缓存把最近的一些请求和响应暂存在本地磁盘中。当新请求到达时，若发现这个请求与暂存的请求相同，就直接返回暂存的响应，而不需要按 URL 的地址再次去因特网访问该资源。

代理服务器对请求的两种处理路径：

- 缓存命中（**hit**）：代理服务器本地已有该对象且未过期，直接返回给客户，不访问源服务器。

- 缓存未命中（**miss**）：代理服务器以客户身份向源服务器发出 HTTP 请求，取回对象后既转发给客户，也在本地保存一份，供后续请求复用。

有效性检查：代理服务器用条件请求（`If-Modified-Since` 或 `If-None-Match`）询问源服务器「我这份还是最新的吗」。若源服务器回答 `304 Not Modified`（无实体主体），说明缓存仍然有效，直接使用本地副本。

代理服务器带来的好处

1. 大幅缩短响应时间：命中时本地返回，时延远小于跨广域网访问源服务器。
2. 减少出口链路的通信量：多个用户共享一份副本，尤其对热门资源效果显著。
3. 减轻源服务器负担：未命中时由代理「代表」客户访问，命中时源服务器完全不参与。

*5.*计算一：非持久连接与持久连接所需的 *RTT* 数与总时间

通用公式

设一个页面含有 1 个基础 HTML 文件（记作「基本页面」）和 N 个引用对象，每个对象的传输时间为 t_i，往返时间为 RTT。

1. 非持久连接（串行，**HTTP/1.0**）：每个对象都要新建 TCP 连接，各占 2 RTT：

$$\text{总 RTT 数} = 2 \times (N+1), \qquad T = 2(N+1)\,\text{RTT} + \sum_{i=0}^{N} t_i$$

1. 非持久连接（并行 **k** 条 **TCP** 连接）：把 $N+1$ 个对象分成 k 路并行：

$$T \approx 2\left\lceil \frac{N+1}{k} \right\rceil \text{RTT} + (\text{各路传输时延})$$

1. 持久连接 + 非流水线（**HTTP/1.1** 默认）：第一个对象付 **1** 个 **RTT** 建 **TCP** 连接，此后每个对象只花 1 个 RTT：

$$\text{总 RTT 数} = 1 + (N+1) = N+2$$

（第一个 RTT 里既建连接又发出第一个请求；严格地说是「1 RTT 建连 + (N+1) 个请求-响应各 1 RTT」，多数教材与真题把它合并写作 $N+2$。）

1. 持久连接 + 流水线：建连 1 RTT + 全部 $N+1$ 个请求连续发出后各占 1 RTT：

$$\text{总 RTT 数} = 1 + (N+1) = N+2 \ \text{（若所有对象能在同一批发送完）}$$

若受 TCP 发送窗口限制需要分多批（例如窗口只能容纳 W 个对象），则批数为 $\lceil (N+1)/W \rceil$，总 RTT 数为 $1 + \lceil (N+1)/W \rceil$。

流水线与非流水线的关键差别：非流水线时，每个对象的「请求-响应」必须串行排队，$N+1$ 个对象要花 $N+1$ 个 RTT；流水线时，若窗口足够大，多个请求可以并发在途，实际所需 RTT 数可能明显少于 $N+1$。

算例一（**2024** 年真题模型，非持久串行）

浏览器不支持并行 **TCP** 连接，使用非持久 HTTP/1.0 请求 1 个 Web 页，该页引用同一网站上的 **7** 个小图像文件。求从建立 TCP 连接开始到接收完所有内容所需的 RTT 数。

- 对象总数 $N+1 = 1+7 = 8$ 个。

- 非持久 + 串行，每个对象 2 RTT。
- 总 RTT $= 2 \times 8 =$ **16** 个 RTT。（选项 D）

算例二（**2011** 年第 **47** 题第 **(3)** 问，持久非流水线）

`rfc.html` 页面引用了 5 个 JPEG 小图像，HTTP/1.1 以持续的非流水线方式工作，一次请求-响应时间为 RTT。求从发出 `rfc.html` 的 Web 请求开始到浏览器收到全部内容为止需要多少 RTT。

- 题目已说明「从发出 **Web** 请求开始」，因此不再计 TCP 建连的 RTT，也不再计发出该请求之前的时间。
- 基本页面 `rfc.html`：1 RTT。
- 5 个图像：非流水线必须收到上一个响应后才能发下一个请求，各 1 RTT，共 5 RTT。
- 总 RTT $= 1 + 5 =$ **6** 个 RTT。

算例三（带传输时延的完整时延计算）

页面含 1 个 HTML 文件与 3 张图片，所有对象在同一台服务器上，RTT $= 20$ ms，单个小对象传输时延可忽略。分别求：

- 非持久串行：$2 \times 4 = 8$ RTT $= 160$ ms。
- 持久非流水线：$(1 + 4) = 5$ RTT $= 100$ ms。
- 持久流水线（窗口足够大）：$1 + 4 = 5$ RTT $= 100$ ms；若瓶颈链路带宽只能让每 RTT 传一个对象，则与持久非流水线相同。

可见非持久改成持久，省下的是「每个对象一次建连 **RTT**」，4 个对象共省 3 个 RTT。

算例四（**2022** 年真题模型，**HTTP/1.1** 叠加 **TCP** 拥塞控制）

主机 H 通过 HTTP/1.1 请求浏览服务器 S 上的 `news408.html`，该页引用同目录下 1 个图像。`news408.html` 大小为 1 MSS，图像大小为 3 MSS，RTT $= 10$ ms，忽略 HTTP 响应首部开销与 TCP 段传输时延，已预先完成域名解析。求从请求建立 TCP 连接起，到收到全部内容所需的最少时间。

按 TCP 初始拥塞窗口为 1 MSS、慢开始计算：

1. 建立 TCP 连接：**1 RTT**。
2. H 请求 HTML，服务器发送 1 MSS 页面：**1 RTT**。该段被确认后，服务器拥塞窗口增至 2 MSS。
3. H 解析页面后，在同一连接上请求图像；服务器先发 2 MSS：**1 RTT**。确认后窗口继续增大，足以发送剩余部分。
4. 服务器发送图像剩余的 1 MSS：**1 RTT**。

总计 4 RTT $= 4 \times 10 =$ **40** ms。

RTT 计算五个必踩的坑

1. 「从何时开始计时」决定要不要算建连 **RTT**。题干若说「从浏览器请求建立 TCP 连接开始」，要算；若说「从发出 HTTP 请求开始」（如 2011 年第 47 题第 (3) 问），则不再算建连 RTT，也不把自己这一次请求算进去。
2. 对象数 = 基本页面 + 引用对象数，别漏掉 HTML 本身。7 张图是 8 个对象，不是 7 个。
3. 非持久连接是 **2 RTT** / 对象，持久连接是（建连 **1 RTT** + **1 RTT** / 对象）。「并行连接」要除以并行度并向上取整。
4. 区分「**RTT** 数」与「时间」：题目问 RTT 数就只回整数；问时间还要乘 RTT，并在必要时加上传输时延（如 2022 年题要考虑 MSS 的分批）。

5. **DNS** 的时间要不要加：题干若说「已完成域名解析」就不加；若从「点击超链接」开始且给定了 DNS 相关的 RTT，则 DNS 的 RTT 必须计入（见 DNS 部分的算例二）。

*6.*计算二：带代理服务器缓存时的时延计算

设：

- 客户端到代理服务器的往返时间为 RTT_{cp}（通常很小，可能就在同一局域网）；
- 代理服务器到源服务器的往返时间为 RTT_{po}（通常较大，跨广域网）；
- p 为缓存命中率（请求在代理处直接命中的比例）。

则：

$$T_{\text{平均}} = p \cdot RTT_{cp} + (1-p) \cdot (RTT_{cp} + RTT_{po}) = RTT_{cp} + (1-p)\, RTT_{po}$$

算例一（命中与未命中）

某局域网通过代理服务器访问 Internet，客户到代理的 RTT $= 2$ ms，代理到源服务器 RTT $= 100$ ms，访问其中一个对象还需从代理到源服务器传输约 8 ms。

- 缓存命中：$T = 2$ ms（只有客户到代理的一个往返）。
- 缓存未命中：$T = 2 + 100 + 8 = 110$ ms。
- 若命中率 $p = 0.4$：$T_{\text{平均}} = 0.4 \times 2 + 0.6 \times 110 = 0.8 + 66 = 66.8$ ms。

算例二（带宽与冲突域，经典代理服务器题）

某机构局域网内的主机通过一台代理服务器访问 Internet。局域网带宽 10 Mb/s，Internet 出口带宽 1.5 Mb/s，平均对象大小 100 kb，平均请求率 15 次/秒，客户到代理的 RTT $= 2$ ms，代理到源服务器 RTT $= 100$ ms。

- 不设缓存时：出口链路的到达速率 $\lambda = 15 \times 100$ kb $= 1500$ kb/s $= 1.5$ Mb/s，恰好等于出口带宽，链路利用率 $\rho = 1$，队列时延趋于无穷，实际时延极大。
- 设缓存且命中率 $p = 0.4$ 时：出口链路的到达速率降到 $(1-p)\lambda = 0.6 \times 1.5 = 0.9$ Mb/s，链路利用率 $\rho = 0.9/1.5 = 0.6 < 1$，链路不再拥塞。此时

$$T_{\text{平均}} = RTT_{cp} + (1-p)\left(RTT_{po} + \frac{L}{B_{\text{out}}}\right) = 2 + 0.6 \times \left(100 + \frac{100\text{ kb}}{1.5\text{ Mb/s}}\right)$$

其中 $\dfrac{100\text{ kb}}{1.5\text{ Mb/s}} = \dfrac{100 \times 10^3}{1.5 \times 10^6}$ s ≈ 66.7 ms，故 $T_{\text{平均}} \approx 2 + 0.6 \times 166.7 \approx$ **102** ms。

算例三（缓存命中率的下限）

在上例中，若要求平均时延不超过 50 ms，求所需的最低命中率 p。

$$2 + (1-p) \times 166.7 \le 50 \implies 1 - p \le 0.288 \implies p \ge 0.712$$

即命中率至少要达到约 **71.2%**。

代理服务器计算的两个要点

1. 命中时只付客户到代理的时延；未命中时要付「客户到代理」+「代理到源服务器」两个时延。
2. 代理服务器主要是通过降低出口链路的利用率来消除排队时延：$\rho = \lambda(1-p)L/B_{\text{out}}$，$\rho$ 越小，排队时延越小。这正是「接入 ISP 带宽不变、只加一台代理服务器就能把响应时间从几分钟降到几十毫秒」的原因。

真题考法

- 2014 年第 40 题问「访问 Web 主页不可能使用的协议」，答案是 **SMTP**；可能用到 HTTP、UDP（DNS）、ARP、PPP。
- 2015 年第 40 题给出一段 HTTP 请求报文，判断四个说法。`Connection: close` 表示不使用持续连接；`Cookie` 表示曾访问过该站点。
- 2011 年第 47 题（9 分）综合考以太网帧解析 + ARP + 持久非流水线 RTT 数（6 个）+ 路由器转发时修改的 IPv4 首部字段。
- 2020 年第 40 题把 DNS 迭代查询与 HTTP 请求合并计时（20 ms / 50 ms）。
- 2022 年第 40 题把 HTTP/1.1 持续连接与 TCP 慢开始、MSS 结合（4 RTT = 40 ms）。
- 2024 年第 40 题考非持久 HTTP/1.0 串行请求 8 个对象的 RTT 数（16）。

本章小结表

协议 / 概念	所属层次	传输层与端口	模型	关键结论
C/S 模型	应用层体系结构	—	—	客户面向用户、主动；服务器面向任务、被动；客户之间不能直接通信
P2P 模型	应用层体系结构	—	—	结点对等、可直接通信；可扩展性好；三方式：集中式 Napster / 全分布式非结构化 Gnutella / 混合式 BitTorrent
DNS	应用层	**UDP 53**（区域传送、响应超 512 B 时用 TCP 53）	C/S	域名 →IP；报文本地服务器向外查询次数 = 域名级数 + 1；主机 ↔ 本地服务器递归，本地服务器 ↔ 其他服务器迭代
FTP	应用层	**TCP 21**（控制）、**TCP 20**（主动模式数据）	C/S	控制连接全程保持、带外传输；数据连接按需建立、用完即关；主动模式服务器连客户，被动模式客户连服务器
电子邮件	应用层	发送 **SMTP/TCP 25**；读取 **POP3/TCP 110**、**IMAP/TCP 143**	C/S	发送用 SMTP（推），读取用 POP3/IMAP（拉）；邮件服务器必须 24 h 在线
邮件格式	—	—	—	首部（From/To/Cc/Subject）+ 空行 + 主体；邮件格式由 RFC 822/5322 定义，不由 SMTP 定义
MIME	应用层	—	—	解决 SMTP 只能传 7 位 ASCII 的问题；新增 `Content-Type`、`Content-Transfer-Encoding` 等首部；base64 编码膨胀约 33%
WWW	应用层	—	C/S	三要素：URL（标识）、HTTP（传送）、HTML（解析）
URL	—	—	—	协议`://`主机`:`端口`/`路径`?` 查询 `#` 片段；端口/路径可省略；片段不发给服务器

协议 / 概念	所属层次	传输层与端口	模型	关键结论
HTTP	应用层	**TCP 80**	C/S	面向事务、无连接、无状态；请求报文 = 请求行 + 首部行 + 空行 + 实体主体；响应报文 = 状态行 + 首部行 + 空行 + 实体主体
HTTP/1.0	—	TCP 80	—	非持久连接，每个对象 2 RTT；可用并行 TCP 连接加速
HTTP/1.1	—	TCP 80	—	持久连接默认；分非流水线与流水线两种
HTTP/2	—	TCP（HTTP/3 用 QUIC/UDP）	—	多路复用、二进制分帧、首部压缩、服务器推送；语义不变
Cookie	应用层	由 HTTP 承载	—	对无状态 HTTP 做状态化；`Set-Cookie` 下发、`Cookie` 回送
Web 缓存 / 代理服务器	中间系统	由 HTTP 承载	—	命中直接返回；未命中代客户取回并保存；用 `304 Not Modified` 校验有效性

核心计算速查

场景	公式
本地域名服务器迭代查询次数（无缓存）	域名级数（不含主机名）+1
本地域名服务器发出查询次数（有缓存）	0
因特网组合查询方式	主机 → 本地服务器：递归；本地服务器 → 其他：迭代
非持久 HTTP（串行）请求 1 页面 + N 对象	$2(N+1)$ RTT
持久 HTTP 非流水线请求 $1+N$ 个对象	$1+(N+1)=N+2$ RTT
持久 HTTP 流水线（窗口足够）	$1+\lceil (N+1)/W \rceil$ RTT
带代理缓存的平均时延	$RTT_{cp}+(1-p)\,RTT_{po}$
代理后出口链路利用率	$\rho=\frac{\lambda(1-p)L}{B_{\text{out}}}$

易错清单

一、网络应用模型

- **C/S** 模型中客户之间不能直接通信，这是 2019 年真题的错误选项来源。
- P2P 中「每个结点既是客户又是服务器」指角色，不表示不存在索引服务器或超级结点。
- 不要用「P2P 单个结点带宽更大」解释 P2P 分发更快，正确原因是总上传能力随结点数增加。

二、DNS

- 本地域名服务器不属于域名服务器等级结构，画图和说法题里它必须单独列出。
- 递归与迭代的判断：主机 → 本地域名服务器是递归；本地域名服务器 → 根/顶级/权限是迭代（题目另有规定时按题目）。
- 查询次数计算：`www.abc.xyz.com` 的迭代查询次数按 `abc.xyz.com` 算，是 $3+1=4$ 次，不是 5 次；有缓存时本地域名服务器向外发出 **0** 次。
- 别把主机 **H** 发给本地服务器的递归查询算进「本地服务器发出的次数」（2016 年真题的得分点）。
- 缓存命中时，主机到本地服务器的那一次查询仍然发生，只是本地服务器不再跑腿。

- **DNS** 主用 **UDP 53**；区域传送和响应超过 **512** 字节时改用 **TCP 53**（2018 年第 33 题答案就是 DNS）。
- 域名与 IP 不是一一对应，兼容多域名、多 IP、负载均衡；DNS 不存 MAC 地址。
- 权限域名服务器可能不止一个，具体由题目给出的域名逐级判断。

三、FTP

- **20** 和 **21** 都是服务器端口；客户端端口由系统随机分配，不可能是 20（2017 年第 40 题的错误项）。
- 主动模式：服务器用 **20** 端口主动连客户；被动模式：服务器随机开端口并告知客户，由客户发起连接。所以「FTP 数据连接一定用 20」错。
- 控制连接在整个会话期间保持；数据连接每次传送按需建立、传完即关（2023 年第 47 题第 (1) 问）。
- 「控制连接用 TCP、数据连接用 UDP」是错的——两条连接都建立在 **TCP** 之上（2009 年第 40 题考的就是这个）。
- 带外控制指控制信息走独立连接，因此数据传送期间仍能发控制命令。
- FTP 有状态（记住登录状态与工作目录），HTTP 无状态。

四、电子邮件

- 发送用 **SMTP**（**25**），读取用 **POP3**（**110**）/ **IMAP**（**143**）；「服务器 → 用户代理」绝不使用 SMTP。
- 2012 年第 40 题的三个阶段是 **SMTP**、**SMTP**、**POP3**，顺序不要颠倒。
- **SMTP** 只能传 **7** 位 **ASCII**；JPEG、MPEG、EXE 必须先经 **MIME** 编码（2018 年第 40 题答案是「ASCII 文本」）。
- **MIME** 不修改 **SMTP**，只是在发送前编码、接收后解码；`Content-Type` 说类型，`Content-Transfer-Encoding` 说编码方式。
- 邮件格式由 **RFC 822/5322** 定义，不是 **SMTP** 定义的；首部与主体之间必须有空行。
- POP3 的两种方式是下载并删除与下载并保留；POP3 不允许在服务器上管理邮件（建文件夹、分类），IMAP 才允许。
- 不能把邮件直接推给收件人的 **UA**：对方主机不一定开机、也不一定跑邮件服务器程序。
- 网页邮件（浏览器收发）时，UA 与服务器之间用 **HTTP**，服务器之间仍用 **SMTP**。
- 邮件相关协议全部基于 **TCP**，没有一个用 UDP。

五、WWW 与 HTTP

- **HTTP** 无连接指不先建 HTTP 连接，但仍需 **TCP** 三次握手，别混淆。
- **HTTP** 无状态指服务器不记历史；Cookie 是对它做状态化的技术。
- 请求报文与响应报文都必须以空行结束首部；请求报文有实体主体时（`POST`）才带内容。
- **200/301/302/304/400/403/404/500** 的语义要背熟；304 不带实体主体。
- `Connection: close` 表示非持久连接；出现它就说明不使用持续连接（2015 年第 40 题 C 选项）。
- 非持久 = 每对象 **2 RTT**；持久非流水线 = 建连 **1 RTT** + 每对象 **1 RTT**。对象数要算上 HTML 本身。
- 注意题干计时起点：从「建立 TCP 连接」开始要算建连 RTT，从「发出 HTTP 请求」开始则不算（2011 年第 47 题第 (3) 问 = 6 RTT）。
- 流水线是「同一条连接上连续发多个请求」；并行 **TCP** 是「开多条连接」。两者不是一回事。
- **URL** 的片段（`#` 之后）不发送给服务器；查询串（`?` 之后）会发送。
- 代理服务器命中时只付客户到代理的时延，未命中才付到源服务器的时延；它通过降低出口链路利用率消除排队时延。